

ADVANCE PROGRAMMING WITH JAVA

(Affiliated To Pokhara University)

Table Of Contents

Chapter 1: Basics of Programming in Java.....	7
Historical Background Of Java:	8
Versions:.....	8
Feature Of Java (Benefits) :	9
A Simple Java Program/Basic Syntax.....	10
Java Architecture.....	11
JVM Architecture.....	11
Data types And variables :	13
Local Variables:.....	13
Instance variables:	13
Class/static variables:	14
Primitive datatypes:.....	16
Assignment and Initialization And Operator :	18
String and Control Flow.....	21
Control Loop :	25
Class Methods	25
Array:	27
Java Collections.....	31
Some Important Terms:.....	34
Framework.....	34
Java Virtual Machine(JVM).....	34
IDE(Integrated Development Environment)	35
Java Runtime Environment(JRE)	35
Java Development Kit(JDK)	35
Chapter 2: Object Oriented Programming In Java.....	36
Introduction to object oriented programming in java.....	37
Building User defined class , Package :	37
Inheritance.....	39
Abstraction	42
Interfaces:	44
Access Protection Mechanism	45
Default Access Modifier - No Keyword.....	45
Private Access Modifier – Private	46

Public Access Modifier - Public	47
Protected Access Modifier - Protected	48
Inner class.....	48
Reflection Class(API):	50
Polymorphism:	55
Upcasting and Downcasting:.....	57
Wrapper Class:	59
Chapter 3: Exception Handling and Streams I/O.....	62
Exception Handling:.....	63
Java Exception Hierarchy:	63
Types of Java Exceptions:.....	63
Java Exception Keywords:	64
Java Exception Handling Examples:.....	65
Difference between throw and throws:	67
Tips on Handling Exception:.....	68
Debugging Techniques:.....	71
Java Custom Exception:	73
Streams I/O:.....	74
OutputStream vs InputStream:	76
Some classes of <i>OutputStream</i> and <i>InputStream</i> from package <i>java.io.*</i> :	77
FileOutputStream and FileInputStream:.....	77
FileWriter and FileReader:	80
BufferedOutputStream and BufferedInputStream.....	81
BufferedWriter and BufferedReader:	83
Java Console Class:	84
Data Streams:.....	86
Object Streams:.....	88
File Handling:.....	91
File Operations in Java:	92
Chapter 4: Applet and Application	95
Introduction:	96
Two types of Applets:	96
Applet Architecture:.....	96
Hierarchy of Applet:.....	97

Lifecycle of Applet:	98
Displaying Graphics in Applet:.....	100
Repaint() method / requesting repainting:.....	102
The HTML APPLET Tag:	103
getDocumentBase() and getCodeBase():	105
showStatus():	106
Communication between Applets:	106
SandBox Security Model:	114
Chapter 5: Event , Handling Events and AWT and Swing.....	116
AWT(Abstract Window Toolkit):.....	118
AWT Hierarchy:.....	118
Components:	118
Container:.....	119
How AWT is platform dependent?	119
Event Handling:.....	122
Delegation Event Model:	122
Semantic and Low-Level Events in the AWT:	124
Some basic components in AWT:.....	129
How to close AWT Window in Java ?.....	160
Java Swing:	162
Lightweight vs Heavyweight GUI Components:	162
Note:.....	163
Hierarchy of Java Swing Classess:.....	163
setDefaultCloseOperation() Method:	174
Layout Management:.....	174
BorderLayout:.....	175
FlowLayout :.....	176
GridLayout:	178
GridBagLayout:	180
BoxLayout:	183
CardLayout:	191
Old Questions Solution:	196
MouseListener Interface:.....	203
MouseMotionListener Interface:.....	204

KeyListener Interface:.....	205
Look yourself remaining Swing Components:.....	207
JavaFx.....	208
JavaFX Flow (How things work)	208
JavaFx Controls & Layout Layer	208
JavaFx vs Swing (do yourself).....	211
Chapter 6: Graphics and Images /Animation / Multimedia	212
Introduction:	213
Color Control:	215
Manipulating Fonts:	218
Font Metrics:	220
Drawing Arcs:	221
Drawing Polygon and Polyline:.....	223
Note:.....	232
Chapter 7: Network Programming.....	233
Introduction:.....	234
Networking Terminologies:	236
java.net package:	237
Exception in <i>java.net</i> :.....	238
Inet Address:	238
Socket Programming:.....	241
Socket Programming with TCP socket :	241
URL class :	251
URLConnection class:.....	254
Socket Programming using UDP	256
RMI(Remote Methods Invocation)	262
Chapter 8: JDBC	269
Introduction :	270
Why should use JDBC?.....	270
Some SQL statements and clauses:.....	270
JDBC-Drivers.....	272
How to connect Java Application to any database ?	273
DriverManager class	286
Prepared Statement:.....	286

ResultSet and ResultSetMetaData:.....	289
Java Login Application:	295
Scrollable and Updatable ResultSets.....	298
ResultSet vs RowSet	299
Java Regular Expression: do yourself.....	301
Chapter 9: Servlet and Java Server Page(Prefer class note....)	302
Chapter 10: Advanced Topics in Java	303
Overview Of ORM.....	303
Hibernate	303
Web Framework Introduction	304
Basics of Spring Boot.....	304
Concurrency and Multithreading in Java	305
Thread States and Life cycle:	305
Design Patterns.....	306
Singleton Pattern.....	306
Factory Method.....	307
Abstract Factory Pattern (do yourself.....)	308
How to setup environment?	309
Java Environment Setup.....	309
How to create project in NetBeans?.....	309
JavaFx Environment Setup(Window)	309

Chapter 1: Basics of Programming in Java

Some old Questions:

1. What are the most powerful features of java? Describe how it is a platform neutral language by a diagram.
2. List the advantages of java programming language. Why is it also known as platform independent language.
3. Explain the intermediate Working principal of java programming language that makes it runnable on any platform i.e., portable.
4. What do you mean by Java System Overview? Explain.
5. Is JVM is machine independent? Explain the use of JVM. How JVM makes Java machine independent?
6. How does java support re-usability? Provide examples.
7. Justify the statement, "Java is designed for distributed application". Explain the different types of Java variables.
8. Discuss the features of Java Programming. Why Java is popular than C/C++?
9. Short Notes:
 - i. Wrapper Class
 - ii. JIT compile
 - iii. Assignment vs Initialization
 - iv. Java Virtual Machine(JVM)
 - v. JRE
 - vi. History of Java
10. Some Array related Questions:
 - i. Write Java Program to find missing numbers. For example, given array be {1,4,5,7} where missing numbers are 2,3,6. Program must be generic and array must be runtime initialized.
 - ii. Write a Java Program to sort Names of students in an alphabetical order.
 - iii. Write a Java Program for following problem.

23	24	33
44	2	100
65	11	22

- a. Count frequency of odd and even numbers
- b. Display transpose matrix
- c. Interchange Matrix Diagonals
- d. Print prime numbers only

What is Java?

- **Java** is a general-purpose computer-programming language that is concurrent, class based , object-oriented, and specifically designed to have as few implementation dependencies as possible.
- Java is a high-level programming language originally developed by Sun Microsystems and released in 1996.
- Java runs on a variety of platforms, such as Windows, Mac OS, and the various versions of UNIX.

Historical Background Of Java:

- In June 1991 , Java was first initiated as a Java language project by James Gosling , Mike Sheridan and Patrick Naughton designed for interactive television , but it was too advanced for the digital cable television industry at that time.
- The language was initially called **Oak** after an oak tree that stood outside Gosling's office.
- The name **Java** is named after Green , from Java coffee.
- Gosling designed Java with a **C/C++** style syntax that system and application programmers would find familiar.
- Java was released for public in 1996 (Java 1.0 [J2SE] as core component of Sun Microsystem) by Sun Microsystem. It promised '*Write Once, Run Anywhere*' (WORA), providing no-cost run-times on popular platforms.
- With the advancement of Java and its widespread popularity, multiple configurations were built to suit various types of platforms. For example: J2EE for Enterprise Applications, J2ME for Mobile Applications.
- The new J2 versions were renamed as Java SE, Java EE, and Java ME respectively.

Versions:

As of March 2019, Java 8 is supported; and both Java 8 ,11, 17 and 21 as Long Term Support (LTS) versions. Major release versions of Java, along with their release dates:

- JDK 1.0 (January 23, 1996)
- JDK 1.1 (February 19, 1997)
- J2SE 1.2 (December 8, 1998)
- J2SE 1.3 (May 8, 2000)
- J2SE 1.4 (February 6, 2002)
- J2SE 5.0 (September 30, 2004)
- Java SE 6 (December 11, 2006)
- Java SE 7 (July 28, 2011)
- Java SE 8 (LTS) (March 18, 2014)
- Java SE 9 (September 21, 2017)
- Java SE 10 (March 20, 2018)
- Java SE 11(LTS) (September 25, 2018)
- Java SE 12 (March 19, 2019)
- Java SE 13 (September 2019)
- Java SE 14 (March 2020)
- Java SE 15 (September 2020)
- Java SE 16 (March 2021)

- Java SE 17(LTS) (September 2021)
- Java SE 18 (March 2022)
- Java SE 19 (September 2022)
- Java SE 20 (March 2023)
- Java SE 21 (LTS) (September 2023)
- Java SE 22 (March 19, 2024)
- Java SE 23 (September 17, 2024)
- Java SE 24 (March 18, 2025)
- Java SE 25 (September 16, 2025)
- Java SE 26 (March 17, 2026)

Feature Of Java (Benefits) :

- **Simple** – Java was designed to be easy to use, write, compile, debug, and learn than other programming languages. Java is much simpler than C++ because Java uses automatic memory allocation and garbage collection. Java has removed many complicate and rarely used features, for example explicit pointers , operator overloading etc.
- **Object-Oriented** – In Java everything is an Object. Object oriented programming is associated with concepts like class, object, inheritance, encapsulation, abstraction, polymorphism, etc. which allows you to create modular programs and reusable code. You can declare classes, create objects inside classes, and interact between two objects.
- **Platform-Independent** – As we know that C, C++, etc. which are compiled into platform specifics machines while Java is the “write once , run anywhere language” . A platform is **software** or **hardware** environment where program runs. Java provides a software-based environment; has two types of component **Runtime Environment** and **API**(Application Programming Interface). Java code can be run on multiple platforms , for example Windows , Linux, Sun Solaris , Mac ,etc. Java code is compiled by compiler and converted into **byte code**.
- **Distributed** – Java has great networking capability, it is designed to make distributed computing easy with the networking capability that is inherently integrated into it. RMI and EJB are used for creating distributed applications. This feature makes us to access files by calling the methods from any machine on the internet.
- **Secure** – Java is the first programming language to include security an integral part of the design. Java Virtual Machine has a unique identifier that identifies the bytecode and verifies it before running it. Authentication techniques are based on public-key encryption.
- **Robust** – Java is the most robust programming language , i.e. most reliable. Java makes an effort to eliminate error prone situations by emphasizing mainly on compile time error checking and runtime checking. There is automatic garbage collection in Java which runs in **Java Virtual Machine(JVM)** to get rid of objects that are not being used by Java application anymore.
- **Multithreaded** – A thread is like a separate program, running concurrently. It is possible to write programs that can perform many tasks simultaneously. This design feature allows the developers to construct interactive applications that can run smoothly. The main advantage of multithreading is that it doesn't occupy memory for each thread. It shares common memory area. Threads are important for multi-media, web application, etc.

- **Interpreted** – Java byte code is translated on the fly to native machine instructions and is not stored anywhere. The development process is more rapid and analytical since the linking is an incremental and light-weight process.
- **Portable** – Java is portable because it facilitates you to carry the bytecode to any platforms. It does not require to implementation.
- **Architecture-neutral** – Because there are no implementation dependent features , for example the size of primitives type is fixed. In C programming int data type occupies 2-bytes of memory for 32-bit architecture and 4-bytes of memory for 64-bit architecture. However, it occupies 4-bytes for both 32-bit and 64-bit architecture in Java.
- **High performance** – Java is faster than traditional interpreted programming languages because Java bytecode is “close” to native code. It is little bit slower than compiled language(C,C++,etc)
- **Dynamic** – It supports dynamic loading of classes. It means classes are loaded on demand. It supports functions from its native languages.(C,C++, etc)

A Simple Java Program/Basic Syntax

The traditional **Hello World** program can be written in Java as:

Example 1: Simple/Sample Java Program

```
/*File name : HelloWorld.java*/
public class HelloWorld{
    public static void main(String[] args){
        System.out.println("Hello Java World!");
    }
}
```

Explained (How Java Works?):

1. (To **Create Java Program**) First of all, open your text editor(Notepad/ NetBeans/ Eclipse/IntelliJ). Create file and save java source code files with name ending with the **.java** extension indicating that the file (Helloworld.java) contains Java source code. Also remember that the public class name must be as same as your file name.
2. **javac** is used to **compile** java program (like *javac HelloWorld.java*); it first compiled java source code into bytecode and then produces **HelloWorld.class** file. The keyword **public** denotes that a method can be called from code in other classes, or that a class may be used by classes outside the class hierarchy. The keyword **static** in front of a method indicates a static method, which is associated only with the particular class and not with any specific instance of that class. The keyword **void** indicates that the main method does not return any value to the caller. “**System.out.println(“ ”)**; statement print user statement.
3. To execute application JVM launched and load .class file(HelloWorld.class) by class loader and then processed for byte code verification. After successful verification initialized in different memory block in JVM memory are. Understand architecture of Java Execution process and JVM architecture.

Java Architecture

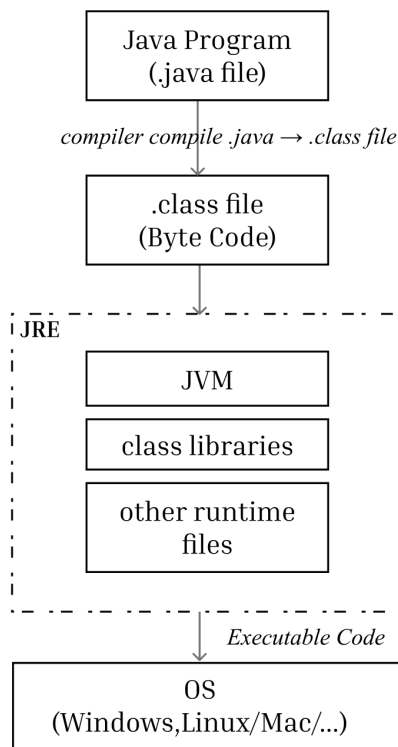


Figure : Java architecture

class libraries:

Class libraries are prewritten collections of classes and interfaces that provide ready-made functionality. Don't need to write everything from scratch. Few are listed below: Example:

1. java.lang (auto-imported) provides basic classes used in almost every program
 - String
 - Math
 - System
 - Thread
 - Object,
2. java.util provides utility classes and data structures
 - Scanner
 - Date
 - ArrayList
 - List
 - Random,
3. java.io provides input/output operations
 - File
 - FileReader
 - BufferedReader
 - PrintWriter
 - InputStream,

other runtime files:

Supporting files to run java programs used by JRE. Few of them are listed below:

1. configuration files
 - java.security
 - logging.properties
 - net.properties
2. Native Method Libraries - Platform-specific files used when Java calls OS-level code
 - .dll (windows)
 - .so (linux)
 - .dylib (Mac)
3. Resource File
 - Images
 - Properties files
 - XML / JSON
 - Text files

JVM Architecture

JVM (Java Virtual Machine) is a part of java runtime environment. It converts Java bytecode into machine language. It is known as the interpreter or the core of Java programming language because it executes Java programming. JVM is also platform-dependent and performs many functions, including memory management and security.

Interpreter:

The JVM does NOT directly understand bytecode as machine code. Interpreter reads bytecode, translates it line-by-line, and executes it immediately. The interpreter is slower and repeats translation for frequently used code. However, it has fast startup. Interpreter execution flow:

Bytecode → Interpreter → Machine code → CPU executes

JIT compiler:

Just-In-Time (JIT) compiler improves runtime performance by translating code (bytecode) while a program is running, rather than before execution. It detects frequently executed code (hot spots), converts them into native machine code, and then executes. It also stores for reuse, enabling faster execution than the interpreter. However, it requires additional memory to store compiled native code. JIT compiler execution flow:

Bytecode → Interpreter (initial) → Hotspot detected → JIT compiles → Native machine code (fast execution)

Components of JIT compiler: Parsing → Profiling → Compiling → Caching

Garbage Collector:

Garbage collector (GC) in java is an automatic memory management process to free occupied memory by object that is no longer in use by the application, free heap block. This mechanism is managed by JVM , developer don't need to free memory likewise in C/C++.

GC fundamental **working** process:

1. **Marking:** Identify reachable objects from GC roots(like local variables and static variables). Objects that are in use are marked as “live”.
2. **Sweeping:** Objects that are not marked, are considered as garbage and swept away, means reclaims or free occupied memory.
3. **Compacting:** Rearrange memory to prevent memory fragmentation. Some collectors move remaining live objects to contiguous memory block, results faster and easier to locate objects.

Example:

```
Student s1 = new Student();
Student s2 = new Student();
s1=null;
```

Results:

*s1 is not reachable and removed by GC.
s2 still referenced and not removed.*

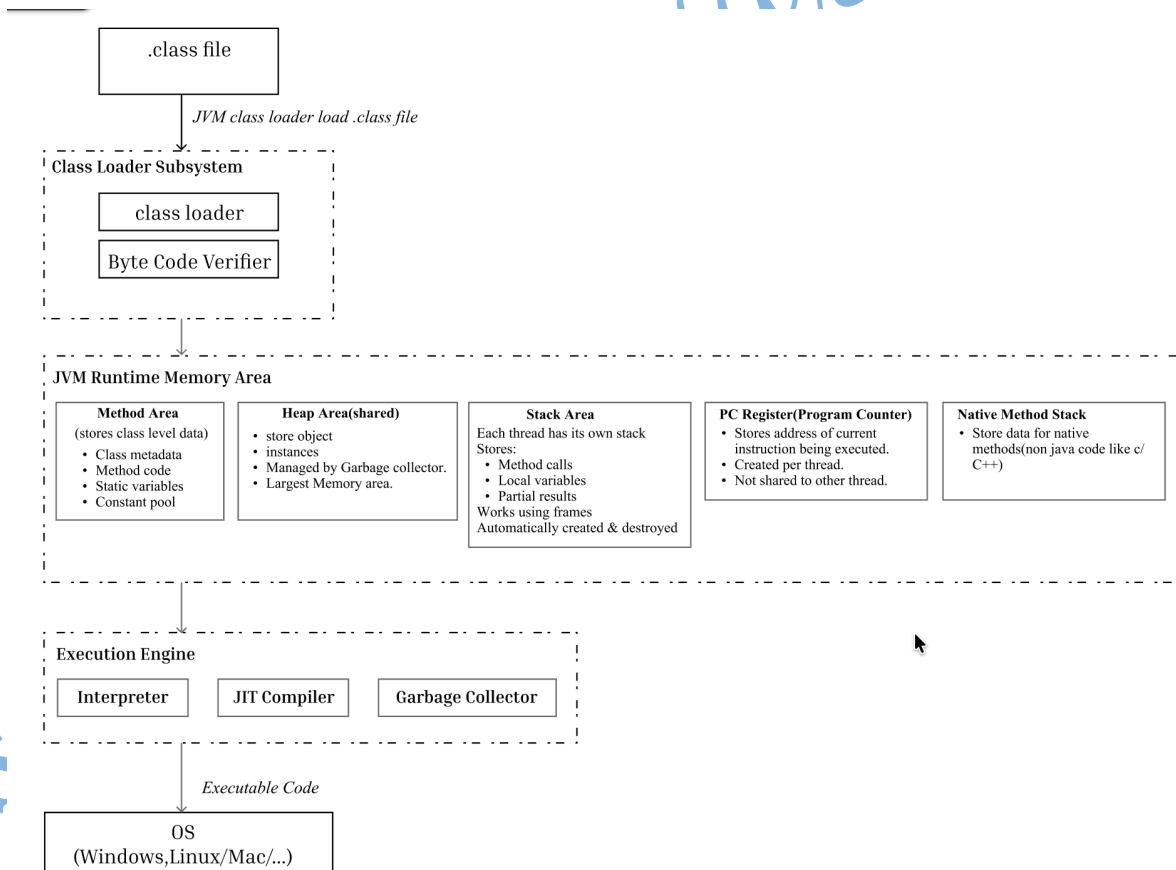


Figure : Java Virtual Machine(JVM) architecture

Data types And variables :

Variables:

Variables are containers for storing data values. Each variable in Java has a specific type, which determines the size and layout of the variable's memory; the range of values that can be stored within that memory; and the set of operations that can be applied to the variable.

Syntax:

data_type variable_name = value;

There are three kinds of variables in Java –

- Local variables
- Instance variables
- Class/Static variables

Local Variables:

- Local variables are declared in methods, constructors, or blocks.
- Local variables are created when the method, constructor or block is entered and the variable will be destroyed once it exits the method, constructor, or block.
- Access modifiers cannot be used for local variables.
- Local variables are visible only within the declared method, constructor, or block.
- There is no default value for local variables, so local variables should be declared and an initial value should be assigned before the first use.

Example 2: local variables

```
public class TestLocalVariables {  
    public void dogAge() {  
        int age = 0; // local variable defined - assigned  
        age = age + 7; //re-assigned  
        System.out.println("Puppy age is : " + age);  
    }  
    public static void main(String[] args) {  
        Test test = new Test();  
        test.dogAge ();  
    }  
}
```

Instance variables:

- Instance variables are declared in a class, but outside a method, constructor or any block.
- Instance variables are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.
- Instance variables hold values that must be referenced by more than one method, constructor or block, or essential parts of an object's state that must be present throughout the class.
- The instance variables are visible for all methods, constructors and block in the class.

- Instance variables have default values. For numbers, the default value is 0, for Booleans it is false, and for object references it is null. Values can be assigned during the declaration or within the constructor.
- Instance variables can be accessed directly by calling the variable name inside the class.

Example:

```
import java.io.*;
public class Employee {
    // name- instance variable is visible for any child class.
    public String name;
    // salary variable is visible in Employee class only.
    private double salary;
    // The name variable is assigned in the constructor.
    public Employee (String empName) {
        name = empName;
    }
    // The salary variable is assigned.
    public void setSalary(double empSal) {
        salary = empSal;
    }
    // This method prints the employee details.
    public void printEmp() {
        System.out.println("name : " + name );
        System.out.println("salary : " + salary);
    }
    public static void main(String args[]) {
        Employee empOne = new Employee("Ram");
        empOne.setSalary(1000);
        empOne.printEmp();
    }
}
```

Class/static variables:

- Class variables also known as **static** variables are declared with the static keyword in a class, but outside a method, constructor or a block.
- There would only be one copy of each class variable per class, regardless of how many objects are created from it.
- Static variables are rarely used other than being declared as constants. Constants are variables that are declared as public/private, final, and static. Constant variables never change from their initial value.
- Static variables are stored in the static memory. It is rare to use static variables other than declared final and used as either public or private constants.
- Static variables are created when the program starts and destroyed when the program stops.
- Visibility is similar to instance variables. However, most static variables are declared public since they must be available for users of the class.

- Default values are same as instance variables.
- Static variables can be accessed by calling with the class name *ClassName.VariableName*.
- When declaring class variables as public static final, then variable names (constants) are all in upper case. If the static variables are not public and final, the naming syntax is the same as instance and local variables.

Example 1:

```
import java.io.*;
public class Employee {
    // salary variable is a private static variable
    private static double salary;
    // DEPARTMENT is a constant
    public static final String DEPARTMENT = "Development ";
    public static void main(String args[]) {
        salary = 1000;
        System.out.println(DEPARTMENT + "average salary:" + salary);
    }
}
```

Output:

Development average salary:1000

Example 2:

```
package basicconcept;
public class Counter {
    int instanceCounter =0;
    static int staticCounter = 0;
    void incrementCounter(){
        instanceCounter++;
        staticCounter++;
    }
    void displayCounter(){
        System.out.println("Instance counter value:"+instanceCounter);
        System.out.println("Static counter value:"+staticCounter);
    }
    public static void main(String[] args){
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter c3 = new Counter();

        System.out.println("Show c1 related values:");
        c1.incrementCounter();
        c1.displayCounter();

        System.out.println("Show c2 related values:");
        c2.incrementCounter();
        c2.displayCounter();
    }
}
```

```

        System.out.println("Show c3 related values:");
        c3.incrementCounter();
        c3.displayCounter();
    }
}

```

Output:

```

Show c1 related values:
Instance counter value:1
Static counter value:1
Show c2 related values:
Instance counter value:1
Static counter value:2
Show c3 related values:
Instance counter value:1
Static counter value:3

```

Primitive datatypes:

Data types are specially assigned to define how given data value be understood by compiler (for example: "I have 5 rupees" is an information, but 5 is data (**integer type**). So, to represent this data in programming language we will use datatype and variable name. **Like, `int rupees = 5;`**). Primitive datatypes are predefined by the language and named by a keyword. There are eight primitive datatypes supported by Java.

byte

- Byte data type is an 8-bit signed two's complement integer
- Minimum value is -128 (-2^7)
- Maximum value is 127 (inclusive) ($2^7 - 1$)
- Default value is 0
- Byte data type is used to save space in large arrays, mainly in place of integers, since a byte is four times smaller than an integer.
- Size of **byte** data type is 1 byte.
- Example: byte a = 100, byte b = -50

short

- Short data type is a 16-bit signed two's complement integer
- Minimum value is -32,768 (-2^{15})
- Maximum value is 32,767 (inclusive) ($2^{15} - 1$)
- Short data type can also be used to save memory as byte data type. A short is 2 times smaller than an integer
- Size of **short** data type is 2 bytes.
- Default value is 0.
- Example: short s = 10000, short r = -20000

int

- Int data type is a 32-bit signed two's complement integer.
- Minimum value is - 2,147,483,648 (-2^{31})
- Maximum value is 2,147,483,647 (inclusive) ($2^{31} - 1$)
- Integer is generally used as the default data type for integral values unless there is a concern about memory.

- Size of **int** data type is 4 bytes.
- The default value is 0
- Example: int a = 100000, int b = -200000 **long**
- Long data type is a 64-bit signed two's complement integer
- Minimum value is -9,223,372,036,854,775,808(-2^{63})
- Maximum value is 9,223,372,036,854,775,807 (inclusive)($2^{63} - 1$) • This type is used when a wider range than int is needed
- Size of **long** data type is 8 bytes.
- Default value is 0L
- Example: long a = 100000L, long b = -200000L

float

- Float data type is a single-precision 32-bit IEEE 754 floating point
- Float is mainly used to save memory in large arrays of floating point numbers
- Default value is 0.0f
- Float data type is never used for precise values such as currency
- Size of **float** data type is 4 bytes.
- Example: float f1 = 234.5f

double

- double data type is a double-precision 64-bit IEEE 754 floating point
- This data type is generally used as the default data type for decimal values, generally the default choice
- Double data type should never be used for precise values such as currency
- Size of **double** data type is 8 bytes.
- Default value is 0.0d
- Example: double d1 = 123.4

boolean

- boolean data type represents one bit of information
- There are only two possible values: true and false
- This data type is used for simple flags that track true/false conditions
- Size of **boolean** data type is 1 byte.
- Default value is false
- Example: boolean one = true

char

- char data type is a single 16-bit Unicode character
- Minimum value is '\u0000' (or 0)
- Maximum value is '\uffff' (or 65,535 inclusive) • Char data type is used to store any character • Size of **char** data type is 2 bytes.
- Example: char letterA = 'A'

Reference Datatypes

- Reference variables are created using defined constructors of the classes. They are used to access objects. These variables are declared to be of a specific type that cannot be changed. For example, Employee, Puppy, etc.
- Class objects and various type of array variables come under reference datatype.
- Default value of any reference variable is null.

- A reference variable can be used to refer any object of the declared type or any compatible type.
- Example: `Animal animal = new Animal("giraffe");`

Note: Remember, java allow String as a class to hold/store string value(a sequence of characters).

Assignment and Initialization And Operator :

Java provides a rich set of operators to manipulate variables. We can divide all the Java operators into the following groups –

The Arithmetic Operators

Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra. The following table lists the arithmetic operators –

Let $A=10$ and $B=20$

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator.	$A + B$ will give 30
- (Subtraction)	Subtracts right-hand operand from left-hand operand.	$A - B$ will give - 10
* (Multiplication)	Multiplies values on either side of the operator.	$A * B$ will give 200
/ (Division)	Divides left-hand operand by right-hand operand.	B / A will give 2
% (Modulus)	Divides left-hand operand by right-hand operand and returns remainder.	$B \% A$ will give 0
++ (Increment)	Increases the value of operand by 1.	$B++$ gives 21
-- (Decrement)	Decreases the value of operand by 1.	$B--$ gives 19

The Relational Operators:

Operator	Description	Example
== (equal to)	Checks if the values of two operands are equal or not, if yes then condition becomes true.	$(A == B)$ is not true.
!= (not equal to)	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	$(A != B)$ is true.
> (greater than)	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	$(A > B)$ is not true.

< (less than)	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>= (greater than or equal to)	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<= (less than or equal to)	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

The Bitwise Operators:

Operator	Description	Example
& (bitwise and)	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12 which is 0000 1100
(bitwise or)	Binary OR Operator copies a bit if it exists in either operand.	(A B) will give 61 which is 0011 1101
^ (bitwise XOR)	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49 which is 0011 0001
~ (bitwise compliment)	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~A) will give -61 which is 1100 0011 in 2's complement form due to a signed binary number.
<< (left shift)	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240 which is 1111 0000
>> (right shift)	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15 which is 1111
>>> (zero fill right shift)	Shift right zero fill operator. The left operands value is moved right by the number of bits specified by the right operand and shifted values are filled up with zeros.	A >>> 2 will give 15 which is 0000 1111

The Logical Operators:

Operator	Description	Example
&& (logical and)	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false
(logical or)	Called Logical OR Operator. If any of the two operands are non-zero, then the condition becomes true.	(A B) is true
! (logical not)	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is true

The Assignment Operators:

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand.	$C = A + B$ will assign value of $A + B$ into C
+=	Add AND assignment operator. It adds right operand to the left operand and assign the result to left operand.	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator. It subtracts right operand from the left operand and assign the result to left operand.	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator. It multiplies right operand with the left operand and assign the result to left operand.	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator. It divides left operand with the right operand and assign the result to left operand.	$C /= A$ is equivalent to $C = C / A$
%=	Modulus AND assignment operator. It takes modulus using two operands and assign the result to left operand.	$C \% = A$ is equivalent to $C = C \% A$
<<=	Left shift AND assignment operator.	$C <<= 2$ is same as $C = C << 2$
>>=	Right shift AND assignment operator.	$C >>= 2$ is same as $C = C >> 2$
&=	Bitwise AND assignment operator.	$C \&= 2$ is same as $C = C \& 2$
^=	bitwise exclusive OR and assignment operator.	$C \wedge= 2$ is same as $C = C \wedge 2$
=	bitwise inclusive OR and assignment operator.	$C = 2$ is same as $C = C 2$

Conditional Operator:

Conditional operator is also known as the **ternary operator**. This operator consists of three operands and is used to evaluate Boolean expressions. The goal of the operator is to decide, which value should be assigned to the variable.

Syntax:

variable x = (expression) ? value_if_true : value_if_false;

Example:

```
public class Test {  
    public static void main(String args[]) {  
        int a, b;  
        a = 10;  
        b = (a == 1) ? 20: 30;  
        System.out.println( "Value of b is : " + b );  
        b = (a == 10) ? 20: 30;  
        System.out.println( "Value of b is : " + b );  
    }  
}
```

Output:

Value of b is :30

Value of b is :20

instanceof Operator:

This operator is used only for object reference variables. The operator checks whether the object is of a particular type (class type or interface type). instanceof operator is written as –

Syntax:

(Object reference variable) instanceof (class/interface type)

Example:

```
public class Test {  
    public static void main(String args[]) {  
        String name = "James";  
        // following will return true since name is type of String  
        boolean result = name instanceof String;  
        System.out.println( result );  
    }  
}
```

Output: *true*

String and Control Flow

Strings, which are widely used in Java programming, are a sequence of characters. In Java programming language, strings are treated as objects (Immutable).

Syntax: String string_name = "string_value";

Whenever it encounters a string literal in your code, the compiler creates a String object with its value. As with any other object, you can create String objects by using the new keyword and a constructor.

Example 1:

```

public class StringDemo {
    public static void main(String args[]) {
        String firstName="Ram";
        String secondName="Bahadur";

        String lastName="Shrestha";

        System.out.println(firstName+" "+secondName+" "+lastName);

        char[] helloArray = { 'h', 'e', 'l', 'l', 'o', '.' };
        String helloString = new String(helloArray);

        System.out.println( helloString );

    }
}

```

Example 2:

//String length()

```

public class StringDemo {
    public static void main(String args[]) {
        String palindrome = "Dot saw I was Tod";
        int len = palindrome.length();

        System.out.println( "String Length is : " + len );

    }
}

```

Output: String Length is: 17

String Some Methods:

charAt() Method: This method returns the character located at the String's specified index. The string indexes start from zero.

Syntax: `public char charAt(int index)`

Example:

```

public class Test {
    public static void main(String args[]) {
        String s = "This is java class";
        char result = s.charAt(8);
        System.out.println("The character is"+ result);
    }
}

```

Output: The character is j.

compareTo() Method: This method compares two strings lexicographically. The comparison is based on the Unicode value of each character in the strings.

Example:

```

public class Test {
    public static void main(String args[]) {
        String str1 = "Strings";
        String str2 = new String("Strings");
        String str3 = new String("Integers are not immutable");
        int result = str1.compareTo( str2 );
        System.out.println(result);
        result = str2.compareTo( str3 );
        System.out.println(result);
    }
}

```

Output:0

10

copyValueOf() Method: This method returns a String that represents the character sequence in the array specified.

Example:

```
public class Test {
    public static void main(String args[]) {
        char[] Str1 = {'h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd'};
        String Str2 = "";
        Str2 = Str2.copyValueOf( Str1 );
        System.out.println("Returned String: " + Str2);
    }
}
```

Note: Also can use **copyValueOf(data, offset, count)** to return specific characters.

getChars() Method: This method copies characters from this string into the destination character array.

Example:

```
import java.io.*;
public class Test {
    public static void main(String args[])
    {
        String Str1 = new String("Welcome to Java class.");
        char[] Str2 = new char[7];
        try {
            Str1.getChars(2, 9, Str2, 0);
            System.out.print("Copied Value = " );
            System.out.println(Str2 );
        }catch( Exception ex) {
            System.out.println("Raised exception...");
        }
    }
}
```

toString() Method: This method returns itself a string.

Example:

```
import java.io.*;
public class Test {
    public static void main(String args[]) {
        String Str = new String("Welcome to Java Class");
        System.out.print("Return Value :");
        System.out.println(Str.toString());
    }
}
```

toLowerCase() Method:

Example:

```
import java.io.*;
public class Test {
    public static void main(String args[]) {
        String Str = new String("Welcome to Java class.");
        System.out.print("Return Value :");
        System.out.println(Str.toLowerCase());
    }
}
```

More On compareTo() method:

Return Value	Meaning
0	Objects are equal
<0	Current Object is smaller
>0	Current object is greater

Example1:

```
package basicconcept;
public class CompareToExample {
    public static void main(String[] args) {
        String a = "Apple";
        String b = "Banana";
        System.out.println(a.compareTo(b));
    }
}
```

Output: -1 => "Apple" comes before "Banana" alphabetically

Let's change comparison:

```
package basicconcept;
public class CompareToExample {
```

```

public static void main(String[] args) {
    String a = "Apple";
    String b = "Banana";
    System.out.println(b.compareTo(a));
}
}

```

Output: 1 => " Banana " comes before " Apple " alphabetically

Example 2: More complex Comparable example

```

package basicconcept;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
class CompareSalary implements Comparable<CompareSalary>{
    private String name;
    private int age;
    private double salary;
    CompareSalary(String name,int age,double salary){
        this.name = name;
        this.age = age;
        this.salary = salary;
    }
    @Override
    public int compareTo(CompareSalary e){
        return Double.compare(this.salary , e.salary);
        //for descending => return Double.compare(e.salary,this.salary)
    }
    @Override
    public String toString() {
        return name + " - " + salary;
    }
}
public class EmployeeSalaryComparision{
    public static void main(String[] args) {
        List<CompareSalary> list = new ArrayList<>();
        list.add(new CompareSalary("Ram",22,50000));
        list.add(new CompareSalary("Hari",24,51000));
        list.add(new CompareSalary("Shyam",25,49000));

        Collections.sort(list);

        System.out.println(list); //call toString() method
    }
}

```

Output:

[Shyam - 49000.0, Ram - 50000.0, Hari - 51000.0]

Other methods are:

- concat()
- contains()
- trim()

- `formats()`
- `chars()` etc (**Practice yourself**)

Control Loop :

Example 1: <i>for</i> loop <pre>public class ArrayMethods { public static void main(String[] args) { for(int i=5;i>=1;i--){ for(int j=1;j<=i;j++){ System.out.print(j + " "); } System.out.printf("\n"); } } }</pre> Output: 1 2 3 4 5 1 2 3 4 1 2 3 1 2 1	Example 2: <i>foreach</i> OR <i>enhanced for</i> loop <pre>public class Test { public static void main(String args[]) { int [] numbers = {10, 20, 30, 40, 50}; for(int x : numbers) { System.out.print(x); System.out.print(","); } System.out.print("\n"); String [] names = {"Ram", "Shyam", "Hari", "Lacy"}; for(String name : names) { System.out.print(name); System.out.print(","); } } }</pre> Output: 10,20,30,40,50, Ram,Shyam,Hari,Lacy,
Example 3: <i>while</i> loop <pre>public class Test { public static void main(String args[]) { int x = 10; while(x < 20) { System.out.print("value of x : " + x); x++; System.out.print("\n"); } } }</pre>	Example 4: <i>do-while</i> loop <pre>public class Test { public static void main(String args[]) { int x = 10; do { System.out.print("value of x : " + x); x++; System.out.print("\n"); }while(x < 20); } }</pre>

Class Methods

A Java method is a collection of statements that are grouped together to perform an operation. Java only support methods not functions.

Syntax:

```
modifier returnType methodName(parameters lists){
    //method body
}
```

Example 1: Program to find smaller number passing parameters. Return value.

```
public class ExampleMinNumber {
    public static void main(String[] args) {
        int a = 11;
        int b = 6;
        int c = minFunction(a, b);
        System.out.println("Minimum Value = " + c);
    }
    /** returns the minimum of two numbers */
    public static int minFunction(int n1, int n2) {
        int min;
        if (n1 > n2){
            min = n2;
        }else {
            min = n1;
        }
        return min;
    }
}
```

Example 2: Method Overloading

```
public class ExampleOverloading {
    public static void main(String[] args) {
        int a = 11;
        int b = 6;
        double c = 7.3;
        double d = 9.4;
        int result1 = minFunction(a, b);
        // same function name with different parameters
        double result2 = minFunction(c, d);
        System.out.println("Minimum Value = " + result1);
        System.out.println("Minimum Value = " + result2);
    }
    // for integer
    public static int minFunction(int n1, int n2) {
        int min;
        if (n1 > n2){
            min = n2;
        }else{
            min = n1;
        }
        return min;
    }
    // for double
    public static double minFunction(double n1, double n2) {
        double min;
        if (n1 > n2){
            min = n2;
        }else{
            min = n1;
        }
        return min;
    }
}
```

Example 3: Using command -line arguments

```
public class CommandLine {
    public static void main(String args[]) {
        for(int i = 0; i<args.length; i++) {
            System.out.println("args[" + i + "]: " + args[i]);
        }
    }
}
```

Example 4:

```
public class MyClass {
    static void myMethod(String fname) {
        System.out.println(fname + " Prasad");
    }
    public static void main(String[] args) {
        myMethod("Ram");
        myMethod("Hari");
        myMethod("Shyam");
    }
}
```

Array:

Java provides a data structure, the **array**, which stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

Syntax:

`datatype[] array_name; //preferred way`

OR

`datatype array_name[]; //work but not preferred way`

1. **Compile Time array:** Array size is fixed i.e. size is hardcoded and known before the program runs(during compilation). These sorts of array executed faster, simpler, cannot change size later, memory blocks are allocated before execution.

`int[] points={2,3,44,55,1} OR int[5] points = {2,3,44,55,1} //compile-time array initialization`

2. **Run-time array:** Values are assigned to array after it is created, usually during execution not at declaration. i.e. values are assigned after memory allocation.

`int[] points = new int[4]; //run-time array initialization that hold/store four elements`

Types of Arrays:

1. Single-Dimensional Array (1D Array)
2. Multi-Dimensional Array (Most commonly 2D)
3. Jagged Array (Ragged Array)

Example 1: Can execute individual element using respective index.(1D array)

```
public class MyClass {
    public static void main(String[] args) {
        String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
        System.out.println(cars[0]);
        System.out.println(cars[1]);
        System.out.println(cars[2]);
    }
}
```

Example 2: Using new keyword

```
public class MyClass {
    public static void main(String[] args) {
        String [] cars=new String[3]; //using new keyword
        System.out.println("length :"+ cars.length);
        Cars[0]= "Volvo";
        Cars[1]= "BMW";
        Cars[2]= "Ford";
        for(int i=0;i<cars.length;i++){
            System.out.println(cars[i]);
            System.out.println(",");
        }
    }
}
```

Example 3: Print all elements of array.

```
public class TestArray {
    public static void main(String[] args) {
        double[] myList = {1.9, 2.9, 3.4, 3.5};
        // Print all the array elements
        for(int i = 0; i < myList.length; i++) {
            System.out.println(myList[i] + " ");
        }
        // Summing all elements
    }
}
```

Example 4: foreach loop OR enhanced for loop

```
public class TestArray {
    public static void main(String[] args) {
        double[] myList = {1.9, 2.9, 3.4, 3.5};
        // Print all the array elements
        for (double element: myList) {
            System.out.println(element);
        }
    }
}
```

<pre> double total = 0; for (int i = 0; i < myList.length; i++) { total += myList[i]; } System.out.println("Total is " + total); // Finding the largest element double max = myList[0]; for (int i = 1; i < myList.length; i++) { if (myList[i] > max) max = myList[i]; } System.out.println("Max is " + max); } } </pre> Output: 1.9 2.9 3.4 3.5 Total is 11.7 Max is 3.5	Example 5: Multidimensional array - A multidimensional array is an array containing one or more arrays. <pre> public class MyClass { public static void main(String[] args) { int[][] myNumbers = { {1, 2, 3, 4}, {5, 6, 7} }; int x = myNumbers[1][2]; System.out.println(x); } } </pre>
--	---

Example 6: <pre> public class MyClass { public static void main(String[] args) { int[][] myNumbers = { {1, 2, 3, 4}, {5, 6, 7} }; for (int i = 0; i < myNumbers.length; ++i) { for (int j = 0; j < myNumbers[i].length; ++j) { System.out.println(myNumbers[i][j]); } } } } </pre>	Example 7: Passing array to method <pre> public class TestArray { public static void testArray(int[] array){ for(int i=0;i<array.length;i++) { System.out.println(array[i]); } } public static void main(String[] args) { int[] number={1,2,3,4}; testArray(number); } } </pre>
Example 8: Return value <pre> import java.util.Arrays; public class PassingArray { public static void main(String[] args) { int[] a = { 3, 4, 5, 6, 7 }; int[] r = reverseArray(a); System.out.println(Arrays.toString(a)); System.out.println(Arrays.toString(r)); } private static int[] reverseArray(int[] b) { int[] c = new int[b.length]; for (int i=b.length-1, j=0; i>=0; i--, j++) { c[j] = b[i]; } return c; } } </pre>	Example 9: <pre> import java.util.Arrays; public class ArrayMethods { public static void main(String[] args) { int[] a = {5, 2, 4, 3, 1}; Arrays.sort(a); System.out.println(Arrays.toString(a)); Arrays.fill(a, 8); System.out.println(Arrays.toString(a)); int[] b = Arrays.copyOf(a, 5); if(Arrays.equals(a, b)) { System.out.println("Arrays a, b are equal"); } else { System.out.println("Arrays a, b are not equal"); } } } </pre> Output: [1, 2, 3, 4, 5] [8, 8, 8, 8, 8] Arrays a, b are equal Note:

	• The sort() method sorts the integers in an ascending order. • The fill() method assigns the specified integer value to each element of the array. • The copyOf() method copies the specified number of elements to a new array. • The equals() method compares the two arrays.
Example 10: <pre>import java.util.Arrays; public class ArrayMethods { public static void main(String[] args) { String[] planets = { "Mercury", "Venus", "Mars", "Earth", "Jupiter", "Saturn", "Uranus", "Neptune", "Pluto" }; Arrays.sort(planets); String p = "Earth"; int r = Arrays.binarySearch(planets, p); String msg; if (r >= 0) { msg = String.format("%s was found at position %d of the " + "sorted array", p, r); } else { msg = p + " was not found"; } System.out.println(msg); } }</pre> Output: Earth was found at position 0 of the sorted array Note: • The binarySearch() method is called. The first parameter is the array name, the second the element we are looking for. If the element is found, the return value is greater or equal to zero. In such a case, it is the index of the element in the sorted array.	Example 11: ArrayList in Java <pre>import java.io.*; import java.util.*; public class arrayli { public static void main(String[] args) throws IOException{ // size of ArrayList int n = 5; //declaring ArrayList with initial size n ArrayList<Integer> arrli = new ArrayList<Integer>(n); // Appending the new element at the end of the list for (int i=1; i<=n; i++) { arrli.add(i); } // Printing elements System.out.println(arrli); // Remove element at index 3 arrli.remove(3); // Displaying ArrayList after deletion System.out.println(arrli); // Printing elements one by one for (int i=0; i<arrli.size(); i++) System.out.print(arrli.get(i)+" "); } }</pre> Output: [1, 2, 3, 4, 5] [1, 2, 3, 5] 1 2 3 5 Note: • ArrayList.size() returns the numbers of elements of this list.

Example 12: 2D array

```
package basicconcept;
import java.util.Scanner;
public class MutiDimensionalArrayExample {
    public static void main(String[] args) {
        //use Scanner class to take input from user
        Scanner sc = new Scanner(System.in);
        // Take array size i.e. user input
        System.out.print("Enter number of rows: ");
        int rows = sc.nextInt(); //for rows

        System.out.print("Enter number of columns: ");
        int cols = sc.nextInt(); //for columns
        // Create matrix i.e. define 2D array with new keyword
```

```

int[][] matrix = new int[rows][cols];
// User Input array elements
System.out.println("Enter matrix elements:");
for(int i = 0; i < rows; i++){
    for(int j = 0; j < cols; j++){
        System.out.printf("Enter [%d][%d] element:",i,j);
        matrix[i][j] = sc.nextInt();
    }
}
// initialize counter to count even and odd elements
int evenCount = 0;
int oddCount = 0;
//Now count even and odd elements from 2D array
for(int i = 0; i < rows; i++){
    for(int j = 0; j < cols; j++){
        if(matrix[i][j] % 2 == 0){
            evenCount++;
        } else {
            oddCount++;
        }
    }
}
/*Create two separate arrays evenArr and oddArr to store even and
odd elements from original array->matrix respectively. */
int[] evenArr = new int[evenCount];
int[] oddArr = new int[oddCount];
// initialize above arrays with particular index and assign elements to particular array(Fill arrays).
int eIndex = 0;
int oIndex = 0;
for(int i = 0; i < rows; i++){
    for(int j = 0; j < cols; j++){
        if(matrix[i][j] % 2 == 0){
            evenArr[eIndex++] = matrix[i][j];
        } else {
            oddArr[oIndex++] = matrix[i][j];
        }
    }
}
// Display original array -> matrix
System.out.println("\nMatrix:");
for(int i = 0; i < rows; i++){
    for(int j = 0; j < cols; j++){
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
//Display even elements of evenArr
System.out.println("\nEven Elements:");
for(int num : evenArr){
    System.out.print(num + " ");
}
// Display odd elements of oddArr

```

```

        System.out.println("\nOdd Elements:");
        for(int num : oddArr){
            System.out.print(num + " ");
        }
        System.out.println("");
        System.out.println("");
        sc.close();
    }
}

```

Output:

Enter number of rows: 2
 Enter number of columns: 2
 Enter matrix elements:
 Enter [0][0] element:2
 Enter [0][1] element:3
 Enter [1][0] element:111
 Enter [1][1] element:32

Matrix:

2 3
 111 32

Even Elements:

2 32

Odd Elements:

3 111

Java Collections

Collections is a utility class that provides static methods, while the **Java Collections Framework (JCF)** is a complete set of interfaces and classes used to store and manipulate groups of objects. The JCF provides interfaces, concrete implementation classes (reusable data structures), and polymorphic algorithms to reduce programming effort and increase performance.

The primary interfaces are Collection (root interface for lists, sets and queues) and Map (part of framework but does not extend the Collection interface). Which means JCF foundation is built upon core interfaces that define abstract data types.

(JCF Hierarchy)

- **List:** An ordered collection (sequence) that allows duplicate elements and positional access via a zero-based index.
- **Implementations:** ArrayList (resizable array, good for random access), LinkedList (doubly linked list, good for insertions/deletions), Vector (synchronized ArrayList), Stack (LIFO).
- **Set:** A set is a collection that contains unique elements, disallowing duplicates. It models the mathematical set abstraction.

Implementations: `HashSet` (uses a hash table, offers fast performance, no guaranteed order), `LinkedHashSet` (maintains insertion order), `TreeSet` (stores elements in sorted order).

- **Queue:** A collection used to hold elements prior to processing, typically in a First-In-First-Out (FIFO) manner.

Implementations: `PriorityQueue`, `ArrayDeque`, `LinkedList`.

- **Map :** An object that stores key-value pairs, where each key must be unique. It does not inherit from the `Collection` interface.

Implementations: `HashMap` (hash table based, no guaranteed order), `LinkedHashMap` (maintains key insertion order), `TreeMap` (keys are sorted).

The *Collections* Utility Class:

The *Collections* class, located in the *java.util* package, provides static methods for performing common operations on collection objects. Key methods.

- *sort (List<T> list):* Sorts the elements of a List in ascending order.
- *binarySearch (List<T> list, T key):* searches for a specific element in a sorted List.
- *reverse (List<T> list):* Reverses the order of elements in a List.
- *min (Collection<T> coll) / max (Collection<T> coll):* Finds the minimum/maximum element in a collection.
- *shuffle (List<T> list):* arrange elements in random order.

(Check other methods of Collections , yourself)

Example 1: combined example(Collections and JCF)

```
package basicconcept;
import java.util.List;
import java.util.ArrayList;
import java.util.Collections;
public class CollectionsExample {
    public static void main(String[] args) {

        List<Integer> list = new ArrayList<>();
        //add() method to add value
        list.add(30);
        list.add(10);
        list.add(20);

        Collections.sort(list);    //[10, 20, 30]
        System.out.println("Sorted To Ascending :"+list);

        Collections.reverse(list);    //[30, 20, 10]
        System.out.println("Reversed list :"+list);

        System.out.println(Collections.max(list)); // 30
        System.out.println(Collections.min(list)); // 10
    }
}
```

```

        Collections.shuffle(list);    // random order

        System.out.println("Shuffled list:"+list);
    }
}

```

Output:

Sorted To Ascending:[10, 20, 30]
 Reversed list:[30, 20, 10]
 30
 10
 Shuffled list: [30, 10, 20]

Example 2: To store different types of data using **Object** -> most common .

File Name: *StoreDifferentTypes.java*

```

package basicconcept;
import java.util.ArrayList;
import java.util.List;
public class StoreDifferentTypes {
    public static void main(String[] args) {
        List<Object> list = new ArrayList<>();
        list.add("Ram");    // String
        list.add(25);       // Integer
        list.add(50000.50); // Double
        list.add(true);     // Boolean
        for(Object obj : list){
            System.out.println(obj);
        }
    }
}

```

Output:

Ram
 25
 50000.5
 true

Example 3: Using wrapper class / custom object

File name: *StoreUsingCustomObject.java*

```

package basicconcept;
import java.util.List;
import java.util.ArrayList;

```

```

class StudentInfo{
    String name;
    int roll;
    String address;
    StudentInfo(String name,int roll,String address){
        this.name = name;
    }
}

```

```

        this.roll = roll;
        this.address = address;
    }
    @Override
    /*
        1.Method from Object class
        2.Automatically called when printing object
    */
    public String toString(){
        return ("Roll Number:"+roll+" Name:"+name+" Address:"+address);
    }
}
public class StoreUsingCustomObject {
    public static void main(String[] args) {
        List<StudentInfo> studentList = new ArrayList<>();
        studentList.add(new StudentInfo("Ram",3,"Kathmandu"));
        studentList.add(new StudentInfo("Ganesh",34,"Sanepa"));

        System.out.println(studentList);
        /*
            1.Calls toString() on ArrayList
            2.ArrayList loops through elements
            3.Calls each object's toString()
        */
    }
}

```

Output:

[Roll Number:3 Name:Ram Address:Kathmandu, Roll Number:34 Name:Ganesh Address:Sanepa]

Some Important Terms:

Framework

A framework, or software framework, is a platform for developing software applications. It provides a foundation on which software developers can build programs for a specific platform. For example, a framework may include predefined classes and functions that can be used to process input, manage hardware devices, and interact with system software. This streamlines the development process since programmers don't need to reinvent the wheel each time they develop a new application.

A framework is similar to an application programming interface (API), though technically a framework includes an API. As the name suggests, a framework serves as a foundation for programming, while an API provides access to the elements supported by the framework. A framework may also include code libraries, a compiler, and other programs used in the software development process.

Java Virtual Machine(JVM)

It is a part of java runtime environment. It converts Java bytecode into machines language. It is known as the interpreter or the core of Java programming language because it executes Java programming. JVM is also platform-dependent and performs many functions, including memory management and security.

IDE(Integrated Development Environment)

Software for building applications that combines common developer tools into a single graphical user interface (GUI).

Java Runtime Environment(JRE)

JRE, is a software layer that runs on top of a computer's operating system software and provides the *class libraries* and other resources that a specific Java program needs to run. The JRE combines Java code created using the JDK with the necessary libraries required to run it on a JVM and then creates an instance of the JVM that executes the resulting program.

Java Development Kit(JDK)

The JDK is a development environment for building applications, applets, and components using the Java programming language. Includes useful tools for developing and testing programs written in the Java programming language and running on java platform. i.e. it is a bundle of software development tools and supporting libraries combined with the Java Runtime Environment (JRE) and Java Virtual Machine (JVM).

Chapter 2: Object Oriented Programming In Java

Some old Questions:

1. What is constructor, Explain with example?
2. Write a function that takes an array of integers as an argument and returns sum of even numbers in that array.
3. Explain multiple inheritance . How is it possible to achieve multiple inheritance in java ?
4. What is interface ? WAP to show interface implementation in java.
5. In what condition is the use of “this” operator mandatory? Explain with suitable program.
6. Why Reflection is regarded as a powerful aspect in java? WAP to demonstrate your answer.
7. Why multiple inheritance is not supported in Java. Provide a simple code example to support yours answer.
8. What is singleton class ? What is the difference between abstract class and interface ?
9. Define Inheritance and Composition. With the help of a suitable program illustrate the difference between them.
10. What is method overloading? Can you override a private or static method in java ? Explain with an example.
11. Differentiate between package and interface in Java. Provide examples for each of them.
12. What is reflection ? Why do we need reflection? Explain with sample code.
13. Discuss about the concept of instance variable hiding in Java. Illustrate with a code example. Define Polymorphism. How should we implement polymorphism in java? Explain with Example.
14. Create a class MyClass in a package MyPack. Import newly created class to another class.
15. Explain with example: static block, static variable and static method. Why main method is always static and public in Java?
16. What are the main uses of super, finalize and this keyword?
17. How is interface different from abstract class? Explain the use of interface to achieve multiple inheritances.
18. What are the uses of abstract keyword? Explain with a suitable program.
19. Create a class Employee with id, name, post and salary. Create a parameterized constructor to initialize the instance variables. Override the String method to display the employee details.
20. Mention the scope of all modifiers (private, default, protected and public). Write suitable program to illustrate the concept.
21. Short notes:
 - i. Reflection API.
 - ii. Interface
 - iii. Inner Classes.
 - iv. Abstract class
 - v. Runtime Polymorphism(Dynamic Dispatching)
 - vi. Casting abstract class

Introduction to object oriented programming in java

- In Object-Oriented Programming, a program is decomposed into a number of entities called objects.
- Data and the functions that operate on that data are combined into that object. OOP mainly focus on data rather than procedure.
- It considers data as critical element in the program development and does not allow it to flow freely around the system.
- It ties data more closely to the functions that operate on it and protects it from accidental modification from outside functions.

Basic Characteristics of Object-Oriented Languages :

- **Classes** – A class may be defined as a collection of similar objects . In other words , it is a general name for all similar objects. For example ,mango ,apple ,banana all may be described under the common name fruits
- **Data abstraction an encapsulation** – Wrapping of data and functions into a single unit is known as data encapsulation. Data and functions of object are encapsulated using class. The act of representing essential features without including background details or explanations is called abstraction.
- **Inheritance** – Inheritance is the process by which objects of one class acquires the properties of objects of another class. Inheritance allows us to create classes which are derived from other classes, so that they automatically include their “parents” members, plus their own .
- **Reusability** – Object---oriented programming uses concept of reusability. Reusability implies the reuse of existing code in another program without modification to it . The concept of inheritance provides an idea of reusability in OOP. Once a class has been written, created and debugged, it can be used in another program. A programmer can use an existing class without modifying it, add additional features and capabilities to it. This is done by deriving a new class from existing one. The new class will inherit the capabilities of the old one but is free to add new features of its own.
- **Polymorphism** – The property of object-oriented programming **polymorphism** is ability to take more than one form in different instances. For example, same function name can be used for different purposes.

Building User defined class , Package :

All class defined by users are called user defined class. User defined classes are defined as per require of user and with their own name. Example: **class MyClass {}**

Packages are used in Java in order to prevent naming conflicts, to control access, to make searching/locating and usage of classes, interfaces, enumerations and annotations easier, etc.

Some of the existing packages in Java are –

- **java.lang** – bundles the fundamental classes
- **java.io** – classes for input , output functions are bundled in this package

Programmers can define their own packages to bundle group of classes/interfaces, etc. It is a good practice to group related classes implemented by you so that a programmer can easily determine that the classes, interfaces, enumerations, and annotations are related.

Creating Package:

While creating a package, you should choose a name for the package and include a **package** statement along with that name at the top of every source file that contains the classes, interfaces, enumerations, and annotation types that you want to include in the package. The package statement should be the first line in the source file. There can be only one package statement in each source file, and it applies to all types in the file. To compile the Java programs with package statements, as shown bellow:

```
javac -d Destination_folder file_name.java
```

Then a folder with the given package name is created in the specified destination, and the compiled class files will be placed in that folder.

Steps: (using Netbeans IDE)

1. Create a project (if exist doesn't matter) , then will have source package folder their.
2. Right click on source package folder ; click on new and then click on java packages.
3. Now create user defined classes as needed.

There are method to access these classes through main method.(shown in example below)

Examples: *This is the first method to accessing the packages.*

<pre>/*package helloworld and file name is HelloWorld.java with main method*/ package helloworld; public class HelloWorld { package1.ABC obj=new package1.ABC(); obj.disp(); package1.XYZ obj1=new package1.XYZ(); obj1.disp(); } } Output: This is class ABC! This is class XYZ!</pre>	<pre>/*package package1 with file ABC.java*/ package package1; public class ABC { public void disp(){ System.out.println("This is class ABC!"); } }</pre>	<pre>/*package package1 with XYZ.java*/ package package1; public class XYZ { public void disp(){ System.out.println("This is XYZ!"); } }</pre>
--	---	--

Also, access all class from package at time.

<pre>/*package helloworld and file name is HelloWorld.java with main method*/ package helloworld; import package1.*; public class HelloWorld { ABC obj=new ABC(); obj.disp(); XYZ obj1=new XYZ(); obj1.disp(); } } Note: To access particular class from different package • import package1.ABC; • import package1.XYZ </pre>	<pre>/*package package1 with file ABC.java*/ package package1; public class ABC { public void disp(){ System.out.println("This is class ABC!"); } }</pre>	<pre>/*package package1 with XYZ.java*/ package package1; public class XYZ { public void disp(){ System.out.println("This is XYZ!"); } }</pre>
---	---	--

Inheritance

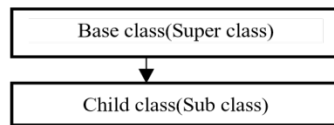
Inheritance can be defined as the process where one class acquires the properties (methods and fields) of another. With the use of inheritance, the information is made manageable in a hierarchical order. The class which inherits the properties of other is known as subclass (derived class, child class) and the class whose properties are inherited is known as superclass (base class, parent class). **extends** is the keyword used to inherit the properties of a class. The most important point in inheritance is that, the subclass object will be created and all the accessible members of superclass and its own are accessed by subclass object. Following is the syntax of extends keyword.

Syntax:

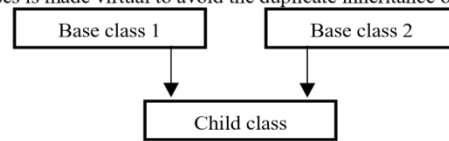
```
class SuperClass{  
    .....  
    .....  
}  
class SubClass extends SuperClass{  
    .....  
    .....  
}
```

Types of Inheritance:

Single Inheritance: In this type of inheritance, a single sub class is derived from a single base class.

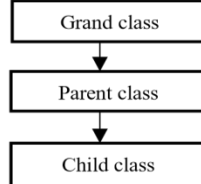


Multiple Inheritance: In multiple inheritances, there is more than one base class from which the child class is derived; results duplication of data. Virtual Base Class is provided for this, in which, one of the base classes is made virtual to avoid the duplicate inheritance of data.

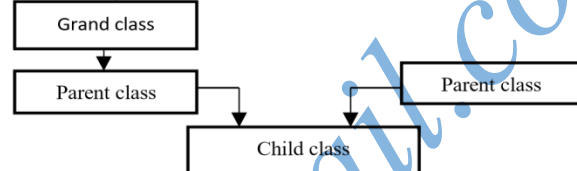


Multiple Inheritance does not directly support by Java. But supported when with Interfaces. Different classes may have variables of same name that may be contradicted and may create confusion in JVM that resulting in errors.

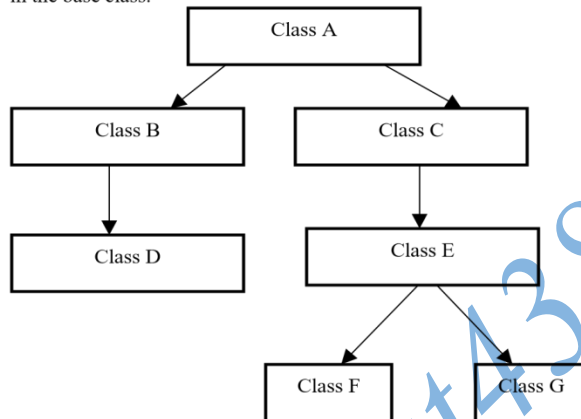
Multi-Level Inheritance: In multilevel inheritance, a class derives another class, and from the derived class, another sub class is derived.



Hybrid Inheritance: There may occur situations where we need to apply two or more types of inheritance in the design of a program. For such cases, hybrid inheritance is used.



Hierarchy Inheritance: If more than one class is inherited from the base class, it's known as hierarchical inheritance. In hierarchical inheritance, all features that are common in child classes are included in the base class.



Super Keyword: The super() keyword is used to reference objects for the immediate parent class. A subclass inherits the accessible data fields and methods from its superclass, but the constructors of the superclass are not inherited in the subclass. They can only be invoked from constructors of the subclass(es) using the keyword super.

Example:

```

class Superclass {
int num =150;
  
```

Example 1: Calculate Area of Rectangle and Triangle using single Inheritance.

```

class Rectangle{
    int first_parameter;
    int second_parameter;
    Rectangle(int first_parameter,int second_parameter){
        this.first_parameter=first_parameter;
        this.second_parameter=second_parameter;
    }
    public int areaOfRect(){
        return first_parameter*second_parameter;
    }
}
class Triangle extends Rectangle{
    Triangle(int x,int y){
        super(x,y);
    }
    public float areaOfTriangle(){
        float area;
        area=(first_parameter*second_parameter)/2;
        return area;
    }
}
class RectangleTriangle{
    public static void main(String[] args){
        Triangle obj=new Triangle(8,4);
        System.out.println("The area of
Rectangle:"+obj.areaOfRect());
        System.out.println("The area of
Triangle:"+obj.areaOfTriangle());
    }
}
  
```

Output:

The area of Rectangle:32

```

}
//Child class, subclass or derived class
class Subclass extends Superclass {
    /* The variable called num which is in the
    superclass * is also present in the subclass */
    int num =160;
    void printNumber(){
        System.out.println(num);
    }
    public static void
    main(String args[]){
        Subclass obj=new Subclass();
        obj.printNumber();
    }
}
  
```

Output: The output of this will be: 160

The area of Triangle:16.0

Example 2:

```

class Calculation {
    int z;
    public void addition(int x, int y) {
        z = x + y;
        System.out.println("The sum of the given
numbers:"+z);
    }
    public void Subtraction(int x, int y) {
        z = x - y;
        System.out.println("The difference between the
given numbers:"+z);
    }
}
public class My_Calculation extends
Calculation {
    public void multiplication(int x, int y) {
        z = x * y;
        System.out.println("The product of the given
numbers:"+z);
    }
    public static void main(String args[]) {
        int a = 20, b = 10;
        My_Calculation demo = new My_Calculation();
        demo.addition(a, b);
        demo.Subtraction(a, b);
        demo.multiplication(a, b);
    }
}

```

Output:

The sum of the given numbers:30
The difference between the given numbers:10
The product of the given numbers:200

Example 3: Single Inheritance taking input from user using Scanner class.

```

package My_Calculation;
import java.util.Scanner;
class Calculation {
    int z;
    public void addition(int x, int y) {
        z = x + y;
        System.out.println("The sum of the given
numbers:"+z);
    }
    public void Subtraction(int x, int y) {
        z = x - y;
        System.out.println("The difference between the
given numbers:"+z);
    }
}
public class My_Calculation extends Calculation {
    public void multiplication(int x, int y) {
        z = x * y;
        System.out.println("The product of the given
numbers:"+z);
    }
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter the first number : ");
        int a=sc.nextInt();
        System.out.println("Enter the second number : ");
        int b=sc.nextInt();
        My_Calculation demo = new My_Calculation();
        demo.addition(a, b);
        demo.Subtraction(a, b);
        demo.multiplication(a, b);
    }
}

```

Output:

Enter the first number : 3
Enter the second number : 5
The sum of the given numbers:8
The difference between the given numbers:-2
The product of the given numbers:15

Final Keyword:

The **final** keyword in java is used to restrict the user. The java final keyword can be used in many contexts. Final can be:

- variable
- method
- class

Java final variable: If you make any variable as final, you cannot change the value of final variable(It will be constant). Example 1: Changing final variables value leads compile time error. <pre> class Bike90{ final int speedlimit=90; <i>//final variable</i> void run(){ speedlimit=400; } public static void main(String args[]){ Bike90 obj=new Bike90(); obj.run(); } } </pre>	Example 2:Without Changing final variables value <pre> class Bike90{ final int speedlimit=90; <i>//final variable</i> void run(){ System.out.println(speedlimit); } public static void main(String args[]){ Bike90 obj=new Bike90(); obj.run(); } } </pre>
Java final method: If you make any method as final, you cannot override it. Example 3: can not be overridden function run(); means compile time errors. <pre> class Bike{ final void run(){ System.out.println("running"); } } class Honda extends Bike{ void run(){ System.out.println("running safely with 100kmph"); } } public static void main(String args[]){ Honda honda= new Honda(); honda.run(); } </pre>	Is final method inherited? Yes, final method is inherited but you cannot override it. Example 4: <pre> class Bike{ final void run(){ System.out.println("running..."); } } class Honda extends Bike{ public static void main(String args[]){ Honda honda= new Honda(); honda.run(); new honda().run(); } } </pre> Output: running... running...
Java final class: If you make any class as final, you cannot extend it. Example 5: class <i>Bike</i> can not be extended to <i>Honda1</i> class. <pre> final class Bike{ } class Honda1 extends Bike{ void run(){ System.out.println("running safely with 100kmph"); } } public static void main(String args[]){ Honda1 honda= new Honda(); honda.run(); } </pre>	Example 6: Can we initialize blank final variable <pre> class Bike10{ final int speedlimit; <i>//blank final variable</i> Bike10(){ speedlimit=70; System.out.println(speedlimit); } public static void main(String args[]){ new Bike10(); } } </pre>

Abstraction

The act of representing essential features without including background details or explanations is called abstraction. Classes use the concept of abstraction. They encapsulate all the properties of

the object to be created. As classes use the concept of abstraction, they are also called as abstract data types. When I drive my car, for example, I need to know the speed of the car and the direction it is going. Because the car is using an automatic transmission, I do not need to know the revolutions per minute (RPMs) of the engine. There are two ways for abstraction:

- Abstract classes and method
- Interfaces

Abstract classes and method:

A class which contains the **abstract** keyword in its declaration is known as abstract class.

- Abstract classes may or may not contain *abstract methods*.
- But, if a class has at least one abstract method, then the class **must** be declared abstract.
- If a class is declared abstract, it cannot be instantiated (never allow to create object).
- To use an abstract class, you have to inherit it from another class, provide implementations to the abstract methods in it.
- If you inherit an abstract class, you have to provide implementations to all the abstract methods in it. The methods that are contained by particular classes and are shared too ; also signify the abstraction properties of OOP i.e. Abstraction , we need to define that methods as abstract methods.
- **abstract** keyword is used to declare the method as abstract.
- You have to place the **abstract** keyword before the method name in the method declaration.
- An abstract method contains a method signature, but no method body.
- Instead of curly braces, an abstract method will have a semi colon (;) at the end.

Example 1:

```
abstract class Vehicle{
    abstract void start();
    void sum(){
        int a=2, b=3;
        int sum=a+b;
        System.out.println("The sum is "+sum);
    }
}
class cars extends Vehicle{
    void start(){
        System.out.println("Start with key");
    }
}
class scooter extends Vehicle{
    void start(){
        System.out.println("Start with kick");
    }
}
public static void main(String[] args){
    scooter st=new scooter();
    st.start();
    cars cr=new cars();
    cr.start();
}
```

Example 2:

```
abstract class Employee{
    private String name;
    private String address;
    private int number;
    public abstract double computePay();
    public String getName(){
        name="Ram";
        return name;
    }
}
class Salary extends Employee {
    private double salary=10000;
    public double computePay() {
        System.out.println("Computing salary pay for getName());
        return salary/52;
    }
}
public static void main(String[] args){
    Salary sal=new Salary();
    System.out.println(sal.computePay());
}
} Output:
Computing salary pay for Ram
```


<pre>cr.sum(); }}</pre> Output: Start with kick Start with key The sum is 5	192.30769230769232
---	--------------------

Interfaces:

An interface is a reference type in Java. It is similar to class. It is a collection of abstract methods. A class implements an interface, thereby inheriting the abstract methods of the interface. Along with abstract methods, an interface may also contain constants, default methods, static methods, and nested types. Method bodies exist only for default methods and static methods.

An interface is similar to a class in the following ways –

- An interface can contain any number of methods.
- An interface is written in a file with a **.java** extension, with the name of the interface matching the name of the file.
- The byte code of an interface appears in a **.class** file.
- Interfaces appear in packages, and their corresponding bytecode file must be in a directory structure that matches the package name.

However, an interface is different from a class in several ways, including –

- You cannot instantiate an interface.

- An interface does not contain any constructors.
- All of the methods in an interface are abstract.
- An interface cannot contain instance fields. The only fields that can appear in an interface must be declared both static and final.
- An interface is not extended by a class; it is implemented by a class.
- An interface can extend multiple interfaces.

Example 1:

```

interface first{
    void display();
}
class second implements first{
    public void display(){
        System.out.println("This is first interface.");
    }
    public static void main(String[] args){
        second sec=new second();
        sec.display();
    }
}

```

Example 2: Multiple Inheritance

```

interface first{
    void display();
}
interface second {
    default void disp(){
        System.out.println("This is second interface!");
    }
}
class third implements first,second{
    public void display(){
        System.out.println("This is first interface.");
    }
    public static void main(String[] args){
        third sec=new third();
        sec.display();
        sec.disp();
    }
}

```

Output:

```

This is first interface.
This is second interface!

```

Access Protection Mechanism

Java provides a number of access modifiers to set access levels for classes, variables, methods, and constructors. The four access levels are –

- Visible to the package, the default. No modifiers are needed.
- Visible to the class only (private).
- Visible to the world (public).
- Visible to the package and all subclasses (within same package or different package) -> (protected).

Default Access Modifier - No Keyword

- Default access modifier means we do not explicitly declare an access modifier for a class, field, method, etc.
- A variable or method declared without any access control modifier is available to any other class in the same package. The fields in an interface are implicitly public static final and the methods in an interface are by default public.

Example: default modifier only have result within the class. <pre> class Demo{ <i>//default class</i> int x=5; int Sum(){ int sum=x+5; return sum; } public static void main(String[] args){ Demo d=new Demo(); System.out.println("The sum is"+d.Sum()); } }</pre>	Example: In this example, we have created two packages <i>TestAccess</i> and <i>myPack</i>. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package. <pre> package myPack; class firstClass { void msg(){ System.out.println("Hello"); } } package testaccess; import myPack.*; public class TestAccess { public static void main(String args[]){ firstClass obj=new firstClass(); obj.msg(); } }</pre> Output: Compile time error
---	--

Private Access Modifier – Private

- Methods, variables, and constructors that are declared private can only be accessed within the declared class itself.
- Private access modifier is the most restrictive access level. Class and interfaces cannot be private.
- Variables that are declared private can be accessed outside the class, if public getter methods are present in the class.
- Using the private modifier is the main way that an object encapsulates itself and hides data from the outside world.

Example: In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is compile time error.

If you try to execute private data member and private methods from another class, result Compile Time Error. <pre> class A{ private int data=40; private void msg(){ System.out.println("Hello java"); } } public class Simple{ public static void main(String args[]){ A obj=new A(); System.out.println(obj.data); <i>//Compile Time Error</i> obj.msg(); <i>//Compile Time Error</i> } }</pre>	If you make any class constructor private, you cannot create the instance of that class from outside the class. <pre> class A{ private A(){ <i>//private constructor</i> } void msg(){ System.out.println("Hello java"); } } public class Simple{ public static void main(String args[]){ A obj=new A(); <i>//Compile Time Error</i> } }</pre>
--	---

Example: access private *data* and *msg()* with in the class

```

class TestAccess{
    private int data=40;
    private void msg(){
        System.out.println("Hello java");
    }
    public static void main(String args[]){
        TestAccess obj=new TestAccess();
        System.out.println(obj.data);
        obj.msg();
    }
}

```

Output:

```

40
Hello java

```

Example: accessing private *data* using inheritance

```

package testaccess;
class T{
    private int data=40;
    void msg(){
        System.out.println("Hello java");
    }
    int msg1(){
        return data;
    }
}
public class TestAccess extends T{
    public static void main(String args[]){
        TestAccess obj=new TestAccess();
        System.out.println(obj.msg1());
        obj.msg();
    }
}

```

Output:

```

40
Hello java

```

Public Access Modifier - Public

- A class, method, constructor, interface, etc. declared public can be accessed from any other class. Therefore, fields, methods, blocks declared inside a public class can be accessed from any class belonging to the Java Universe.
- However, if the public class we are trying to access is in a different package, then the public class still needs to be imported. Because of class inheritance, all public methods and variables of a class are inherited by its subclasses.

Example: Package **myPack** is imported to package **testaccess** to access the class **firstClass**.

```

package myPack;
public class firstClass {
    public void msg(){
        System.out.println("This is how packages are imported.");
    }
}
package testaccess;
import myPack.*;
public class TestAccess {
    public static void main(String args[]){
        firstClass obj=new firstClass();
        obj.msg();
    }
}

```

Protected Access Modifier - Protected

- Variables, methods, and constructors, which are declared protected in a superclass can be accessed only by the subclasses in other package or any class within the package of the protected members' class.
- The protected access modifier cannot be applied to class and interfaces. Methods, fields can be declared protected, however methods and fields in a interface cannot be declared protected.
- Protected access gives the subclass a chance to use the helper method or variable, while preventing a nonrelated class from trying to use it.

Example:

```
package myPack;
public class firstClass {
    protected int data=60;
    protected void msg(){
//protected
        System.out.println("Hello");
    }
    void msg12(){
        System.out.println("Hello "+data);}
    public void msg1(){
        System.out.println("Hello
"+data);}
    }
package
testaccess;
import
myPack.*;
public class TestAccess extends FirstClass{
    public static void main(String args[]){
        TestAccess obj=new TestAccess();
        firstClass o=new firstClass();
        //o.msg12(); //also try it, uncomment
        o.msg1();
        obj.msg();
    }
}
Output:
Hello 60
Hello
```

Inner class

Inner classes are a security mechanism in Java. We know a class cannot be associated with the access modifier **private**, but if we have the class as a member of other class, then the inner class can be made private. And this is also used to access the private members of a class.

Inner classes are of three types depending on how and where you define them. They are –

- Inner Class
- Method-local Inner Class
- Anonymous Inner Class

Example 1: Inner class

```

class Outer_Demo {
    int num;
    // inner class
    private class Inner_Demo {
        public void print() {
            System.out.println("This is an inner class");
        }
    }
    // Accessing the inner class from the method
    within
    void display_Inner() {
        Inner_Demo inner = new Inner_Demo();
        inner.print();
    }
}
public class My_class {
    public static void main(String args[]) {
        // Instantiating the outer class
        Outer_Demo outer = new
        Outer_Demo();
        // Accessing the display_Inner()
        method.
        outer.display_Inner();
    }
}
Output:
This is an inner class

```

Method-local Inner Class: In Java, we can write a class within a method and this will be a local type. Like local variables, the scope of the inner class is restricted within the method. A method-local inner class can be instantiated only within the method where the inner class is defined.

Example : The following program shows how to use a method-local inner class.

```

public class Outerclass {
    // instance method of the outer class
    void my_Method() {
        int num = 23;
        // method-local inner class
        class MethodInner_Demo
        {
            public void print() {
                System.out.println("This is method inner class ");
                System.out.println(num);
            }
        } // end of inner class
        // Accessing the inner class
        MethodInner_Demo inner = new
        MethodInner_Demo();
        inner.print();
    }
    public static void main(String args[]) {
        Outerclass outer = new Outerclass();
        outer.my_Method();
    }
}

```

Example 2: The following program shows how to access the private members of a class using inner class.

```

class Outer_Demo {
    // private variable of the outer class
    private int num = 175; // inner class
    class Inner_Demo {
        public int getNum() {
            System.out.println("This is the getnum method of
            the inner class:");
            return num;
        }
    }
}
public class My_class {
    public static void main(String args[]) {
        // Instantiating the outer class
        Outer_Demo outer = new Outer_Demo();
        //also can Instantiating the inner class
        Outer_Demo.Inner_Demo inner = outer.new
        Inner_Demo();
        System.out.println(inner.getNum());
    }
}
Output: This is the getnum()
method of the inner class:
175

```

Anonymous Inner Class: An inner class declared without a class name is known as an **anonymous inner class**. In case of anonymous inner classes, we declare and instantiate them at the same time. Generally, they are used whenever you need to override the method of a class or an interface.

Example: The following program shows how to override the method of a class using anonymous inner class.

```

abstract class AnonymousInner {
    public abstract void mymethod();
}
public class Outer_class {
    public static void main(String args[]) {
        AnonymousInner inner = new AnonymousInner() {
            public void mymethod() {
                System.out.println("This is an example of
                anonymous inner class");
            }
        };
        inner.mymethod();
    }
}
Output:
This is an example of anonymous inner class

```

<pre>} Output: This is method inner class 23</pre>	
Anonymous Inner Class as Argument Example: The following program shows how to pass an anonymous inner class as a method argument. <pre>// interface interface Message { String greet(); } public class My_class { // method which accepts the object of interface Message public void displayMessage(Message m) { System.out.println(m.greet() + ", This is an example of anonymous inner class as an argument"); } public static void main(String args[]) { // Instantiating the class My_class obj = new My_class(); // Passing an anonymous inner class as an argument obj.displayMessage(new Message() { public String greet() { return "Hello"; } }); } }</pre> Output: Hello, This is an example of anonymous inner class as an argument	Static Nested Class: A static inner class is a nested class which is a static member of the outer class. It can be accessed without instantiating the outer class, using other static members. Just like static members, a static nested class does not have access to the instance variables and methods of the outer class. Example: The following program shows how to use a static nested class. <pre>public class Outer { static class Nested_Demo { public void my_method() { System.out.println("This is my nested class"); } } public static void main(String args[]) { Outer.Nested_Demo nested = new Outer.Nested_Demo(); nested.my_method(); } }</pre> Output: This is my nested class

Reflection Class(API):

Java Reflection is a process of examining or modifying the run time behavior of a class at run time. i.e. Reflection allows us to inspect and manipulate classes, interfaces, constructors, methods, and fields at run time. The *java.lang.Class* class provides many methods that can be used to get metadata, examine and change the run time behavior of a class.

The *java.lang.Class* performs mainly two tasks:

1. Provides methods to get the metadata of class at run time.
2. Provides methods to examine and change the run time behavior of a class

Java reflection should always be used with caution. While the reflection provides a lot of advantages, it has some disadvantages too.

Advantages of Reflection Class:

- Inspection of interfaces, classes, methods, and fields during runtime without using their names during the compile time.
- Also makes possible to call methods, instantiate a class or to set the value of fields.

- Helps in the creation of Visual Development Environments and class browsers which provides aid to the developers to write the correct code.

Disadvantages of Reflection Class:

- Using Reflection , one can break the principles of encapsulation.
- Possible to access the private methods and fields.(Access Protection Challenge, that may cause leakage of important data to the outside world.

Commonly used methods of Class class:

Method	Description
public String getName()	Return the class name
public static Class.forName(String className) throws ClassNotFoundException	Loads the class and returns the reference of Class class.
public Object newInstance() throws InstantiationException, IllegalAccessException	Creates new instance.
public boolean isInterface	Checks if it is interface.
public boolean isArray()	Checks if it is array.
public boolean isPrimitive()	Checks if it is primitive
public Class getSuperclass()	Returns the superclass class reference.
public Field[] getDeclaredFields() throws SecurityException	Returns the total number of fields of this class.
public Method[] getDeclaredMethods() throws SecurityException	Returns the total number of methods of this class
public Constructor[] getDeclaredConstructors() throws SecurityException	Returns the total number of constructors of this class.
public Method getDeclaredMethod(String name, Class[] parameterTypes) throws NoSuchMethodException, SecurityException	Returns the method class instance

Example 1:

- forName()** method of Class class: used to load the class dynamically, returns the instance of Class class
- getClass()** method of Object class: Returns the instance of Class class. It should be used if you know the type.
- The **.class** syntax : If a type is available , but there is no instance, then it is possible to obtain a Class by appending “.class” to the name of the type. It can be used for primitive data types also.

/*ReflectionExample.java*/

```
import java.lang.Class;
import java.lang.reflect.Modifier;
abstract class Vehicle{
    abstract public void Start();
}
class Car extends Vehicle{
    public void Start(){
        System.out.println("Car start with key.");
    }
}
public class ReflectionExample {
    void displayName(Object obj){
        Class cs = obj.getClass(); // create an object of class
```

```

System.out.println("Class name: "+cs.getName()); // return class name
//get the access modifier of the class
int modifier = cs.getModifiers();
String mod = Modifier.toString(modifier); //convert the access modifier to string
System.out.println("Access Modifier:"+mod);

//get superClass of Car
Class s_c = cs.getSuperclass();
System.out.println("Super class name:"+s_c.getName());
}
public static void main(String[] args) {
    Car cr = new Car();
    ReflectionExample rf = new ReflectionExample();
    rf.displayName(cr);

    System.out.println("Following output result class name.");
    Class c = boolean.class;
    System.out.println(c.getName());

    Class c1 = String.class;
    System.out.println(c1.getName());

    Class c2 = ReflectionExample.class;
    System.out.println("Main class : "+c2.getName());
}
}

```

Output:

```

Class name: Car
Access Modifier: // result empty because of default class
Super class name:Vehicle
Following output result class name.
boolean
java.lang.String
Main class : ReflectionExample

```

Example 2: Method class, Field class and Constructors

```

import java.lang.Class;
import java.lang.reflect.Modifier;
import java.lang.reflect.Field;
import java.lang.reflect.Method;
abstract class Vehicle{
    public void Start(){
        System.out.println("Car start with key.");
    }
    abstract public void Engine();
}
class Car extends Vehicle{

```



```

public String model;
public void Engine(){
    System.out.println("Petrol Engine");
}
public void CarType(){
    System.out.println("Mercedes");
}
}
public class ReflectionExample {
    public static void main(String[] args) {
        try {
            Car cr = new Car();
            Class cls = cr.getClass(); //create an object of class using getClass()

            //using object i.e. cls of Class to get all the declared methods of Car
            Method[] mthds = cls.getDeclaredMethods();
            //now get methods name and access modifier
            for(Method md: mthds){
                //get names of methods
                System.out.println("Method Name:"+md.getName());
                //get access modifier
                int modifier = md.getModifiers();
                String modf = Modifier.toString(modifier);
                System.out.println("Access Modifier"+modf);
                //get returnTypes of a method
                System.out.println("Return Type:"+md.getReturnType());
                System.out.println();
            }
            //access and set the model field
            Field fld = cls.getField("model");
            fld.set(cr,"Mercedes 0010");
            //get the value of field
            String modelValue = (String) fld.get(cr);
            System.out.println("Model: "+modelValue);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Output:

Method Name:CarType
 Access Modifierpublic
 Return Type:void

Method Name:Engine
 Access Modifierpublic

Return Type:void

Model: Mercedes 0010

Example 3: getDeclaredConstructors() and getParameterCount()

```
import java.lang.Class;
import java.lang.reflect.Modifier;
import java.lang.reflect.Constructor;
class Car{
    //public constructor without parameter
    public Car(){
        System.out.println("I am default constructor.");
    }
    //private constructor with a single parameter
    private Car(String name){

    }
}
public class ReflectionExample {
    public static void main(String[] args) {
        try {
            Car cr = new Car();
            Class cls = cr.getClass(); //create an object of class using getClass()

            //using object i.e. cls of Class to get all the declared constructor of Car
            Constructor[] constructors = cls.getDeclaredConstructors();
            //now get constructor name and access modifier
            for(Constructor ctor: constructors){
                //get names of constructor
                System.out.println("Constructor Name:"+ctor.getName());
                //get access modifier
                int constr = ctor.getModifiers();
                String cst = Modifier.toString(constr);
                System.out.println("Access Modifier:"+cst);
                //get parameter number of a constructor
                System.out.println("Parameters: "+ctor.getParameterCount());
                System.out.println();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

I am default constructor.
Constructor Name:Car
Access Modifier:public
Parameters: 0

Constructor Name:Car
Access Modifier:private
Parameters: 1

Example 4: isInterface(),isArray() and isPrimitive()

```
import java.lang.Class;
class Car{
}
interface Vehicle{
}
public class ReflectionExample {
    public static void main(String[] args) {
        try {
            Class c = Class.forName("Car");
            System.out.println("IS Car Interface ? "+c.isInterface()); //check for interface

            Class c1 = Class.forName("Vehicle");
            System.out.println("IS Vehicle Interface ? "+c1.isInterface()); // check for interface

            int[] arr = {3,4,5,6,8};
            System.out.println("Is arr an array ? "+arr.getClass().isArray()); // check for array

            System.out.println("Is Car class primitive ? "+c.isPrimitive()); // check class is primitive or not
            Class c2 = int.class; //class to primitive value
            System.out.println("Is Car class primitive? "+c2.isPrimitive()); // check class is primitive or not
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
IS Car Interface ? false
IS Vehicle Interface ? true
Is arr an array ? true
Is Car class primitive ? false
Is Car class primitive? true
```

Polymorphism:

Easy to understand, your existence at different place or can say with different role like; at college as a student, at mall as a customer, at office / job as employee and at home as son/daughter.

So, here polymorphism means “many forms”, and it occurs when we have many classes that are related to each other by inheritance. Polymorphism allows objects of different classes to be treated as instance of a common superclass, enabling a single action to be performed in different ways. That means to allows a child class to share the information and behavior of its parent class while also incorporating its own functionality.

Two mechanisms / types of Polymorphism:

i. Compile-Time Polymorphism (Static. Binding)

Method call is resolved by compiler at compile time. Achieved mainly through method overloading, where multiple methods with the same name exist but differ in parameter lists.

Example 1: Overload method “addition()” of AdditionOperations class.

Filename: MethodOverloadingExample.java

```
package basicconcept;
class AdditionOperations{
    int addition(){
        int a=5,b=45;
        return (a+b);
    }
    int addition(int a,int b){
        return a+b;
    }
    static int addition(int a,int b, int c){
        return a+b+c;
    }
    double addition(int a,int b,double c){
        return a+b+c;
    }
}
public class MethodOverloadingExample {
    public static void main(String[] args) {
        AdditionOperations ao= new AdditionOperations();
        System.out.println(ao.addition());
        System.out.println(ao.addition(20,30));
        System.out.println(ao.addition(6, 2, 2.4));
        System.out.println(ao.addition(5,6,45));
    }
}
```

Note: You can't overload method if parameter type and number of parameter is same. In above example **double addition(int a,int b)** , **double addition(int a,int b,int c)** -> not allowed

ii. Run-Time Polymorphism (Dynamic Binding/Method Dispatch)

Method call is resolved at runtime rather than compile time and achieved using method overriding. Method overriding occurs when subclass have its own implementation of a method i.e., already defined in its superclass. This method must have the same name, parameters and type. Also, can incorporate its own functionality.

Example 1:

```
package basicconcept;
class ParentClass{
    void addition(){
        int a = 4;
        int b = 5;
        System.out.println("Sum: "+(a+b));
    }
    String message(){
        String msg = "Hello all child classes.";
    }
}
```

```

        return msg;
    }
}
public class MethodOverridingExample extends ParentClass{
    @Override
    void addition(){
        System.out.println("Sum of Two Numbers:"+(34+40));
    }
    public static void main(String[] args) {
        MethodOverridingExample me = new MethodOverridingExample();
        me.addition();
        System.out.println("Message:"+me.message());
    }
}

```

Example 2: Dog and Cat inherit and override sound() methods with different behavior. Dog barks and Cat meows but share common method sound from Animal class i.e. parent class => polymorphism.

```

package basicconcept;
class Animal {
    void sound(){
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    @Override
    void sound(){
        System.out.println("Dog barks");
    }
}
class Cat extends Animal {
    @Override
    void sound(){
        System.out.println("Cat meows");
    }
}
public class MethodOverridingExample {
    public static void main(String[] args) {
        Animal a; //reference type Animal -> upcasting concept

        a = new Dog();
        a.sound(); // Dog version

        a = new Cat();
        a.sound(); // Cat version
    }
}

```

Note: final, static and private methods cannot be overridden.

Upcasting and Downcasting:

Upcasting and Downcasting are key concepts behind polymorphism in java. Where casting means converting one type of reference into another.

Upcasting convert(automatic) a child class object into a parent class reference. Helps to generalized method i.e. write one method, handle many types. Upcasting is important to achieve runtime polymorphism, loose coupling and code reusability by allowing a parent reference to refer child Objects.

Example: Dog class inherit Animal class with sound () property/method, executed using upcasting concepts that reference parent class to child class. i.e., share common method.

```
package basicconcept;
class Animal {
    void sound(){
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    @Override
    void sound(){
        System.out.println("Dog barks");
    }
    void animalType(){
        System.out.println("Dog is mamal.");
    }
}
public class MethodOverridingExample {
    public static void main(String[] args) {
        Animal a=new Dog(); //parent(Animal) referenced to child(Dog)=>upcasting
        a.sound(); //exist in Animal and referenced to Dog too
        //a.animalType(); //results compile time error -> not exist in Animal and can't be referenced by Dog
    }
}
```

Note: In **upcasting**, you can only access methods defined in the **parent class**, even if the object is child.

Downcasting convert a parent reference back to a child type manually(explicitly) and may cause runtime exceptions if not handled properly. Downcasting is used to access child-specific methods when needed, making the system flexible yet specific when required.

Example: Explicit casting upcasting to downcasting.

```
package basicconcept;
class Animal {
    void sound(){
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    @Override
    void sound(){
        System.out.println("Dog barks");
    }
    void animalType(){
        System.out.println("Dog is mamal.");
    }
}
```

```

    }
}
public class MethodOverridingExample {
    public static void main(String[] args) {
        //    upcasting
        Animal a=new Dog(); //parent(Animal) referenced to child(Dog)=>upcasting
        a.sound();
        //    downcasting
        Dog d=(Dog)a; //casting assure that this animal 'a' is a Dog.
        d.animalType(); //now can execute all methods under child class i.e., Dog class
    }
}

```

Wrapper Class:

A wrapper class wraps (enclosed) around a data type and gives it an object appearance. Wrapper classes are final and immutable. Autoboxing and Unboxing concepts are used in wrapper class.

Autoboxing is a procedure of converting a primitive value into an object of the corresponding wrapper class. Methods include primarily methods like `valueOf()`, `parseInt()`, `toString()`, and many more. The Java compiler applies autoboxing when a primitive value is:

- Passed as a parameter to a method that expects an object of the corresponding wrapper class.
- Assigned to a variable of the corresponding wrapper class.

Examples:

```
int a = 5;
```

```
Integer b = Integer.valueOf(a); //wrap up to Integer object
```

Unboxing is a procedure of converting an object of a wrapper type to its corresponding primitive value.

Examples:

```
int c = b; //unwrapping object to primitive data types
```

Primitive Data types and their Corresponding Wrapper class:

Primitive Data Type	Wrapper Class
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

Example:

```

public class WrapperClassExample {
    public static void main(String[] args) {
        String str = "5";
        String str1 = "6";
        System.out.println("Lets sum :"+(str+str1)); //It will concatenate
        //now wrap up these string value to integer
        //this is also called autoboxing
        Integer num1 = Integer.parseInt(str);
        Integer num2 = Integer.parseInt(str1);
        System.out.println("Sum will be : "+(num1+num2));

        //unboxing
        String num1Str = num1.toString();
        String num2Str = num2.toString();
        System.out.println("It will again result as:"+num1Str+num2Str);
    }
}

```

Output:

Lets sum :56

Sum will be : 11

It will again result as:56

Exam Question Solution: Create a class Employee with id, name, post and salary. Create a parameterized constructor to initialize the instance variables. Override the String method to display the employee details.

/*Employee.java*/

```

abstract class InfoDisplay{
    abstract String display();
}

public class Employee extends InfoDisplay{
    int id;
    String name;
    String post;
    double salary;
    public Employee(int id,String name,String post, double salary){
        this.id = id;
        this.name = name;
        this.post = post;
        this.salary = salary;
    }

    @Override
    public String display(){
        String info = "";
        info += "Employee id:"+id;
    }
}

```



```
info += "\nEmployee name: "+name;
info += "\nEmployee post: "+post;
info += "\nEmployee salary: "+salary;
return info;
}
public static void main(String[] args) {
    Employee emp = new Employee(1,"Ram Shrestha","Manager",40000);
    Employee emp1 = new Employee(2,"Rajesh Chaudhary","Technician",30000);
    System.out.println("Employee detail:");
    System.out.println(emp.display());
    System.out.println(emp1.display());
}
}
```

Chapter 3: Exception Handling and Streams I/O

Some Old Questions:

1. Create a program to write "Test on file Handling" inside a file "abc.txt".
2. What do you mean by exception? How exception handling is different from error handling? Explain.
3. Define exception and explain why it is necessary to handle exceptions? WAP to write bytes of data to a file called "exams.txt" and reading it until end of file is obtained, using proper exception handling.
4. Explain the user defined Exceptions with suitable java code.
5. Explain various stream classes available in java. Write a program using any stream for writing test to a file.
6. What are exception? Write a program to input an integer from keyboard and print it on the console. Fire an exception if the input is other than integer using try and catch blocks.
7. Why do we need to serialize an object? Write a program to read number from the console using the Input Stream. Also, apply the method of object serialization in your program.
8. How can you create your own exception in java? Explain with program.
9. Why is exception handling essential? Differentiate between checked and unchecked exception with relevant examples.
10. What is the usage of exception handling? Define with example.
11. Create a program to write "Hello World" in a file abc.txt.
12. Write a program to store object of a class student into a file "student.dat" also read the object from same file and display the state of objects on the screen. Handle possible exceptions.
13. Differentiate error with exception. Write a program to create your own exception class in java.
14. Define Exception handling with Example.
15. What is exception handling? Explain. Write a program to handle Arithmetic Exception.
16. What are the different types of streams supported by java? Explain. A data file "emp.txt" contains name address and salary of 30 employees. Write a program to display only those records who are from "Kathmandu".
17. Explain about FileReader and Buffered writer class. How do you create own exception subclasses? Explain with an example.
18. Write a program to read content from file "abc.txt" and store it in "xyz.txt".

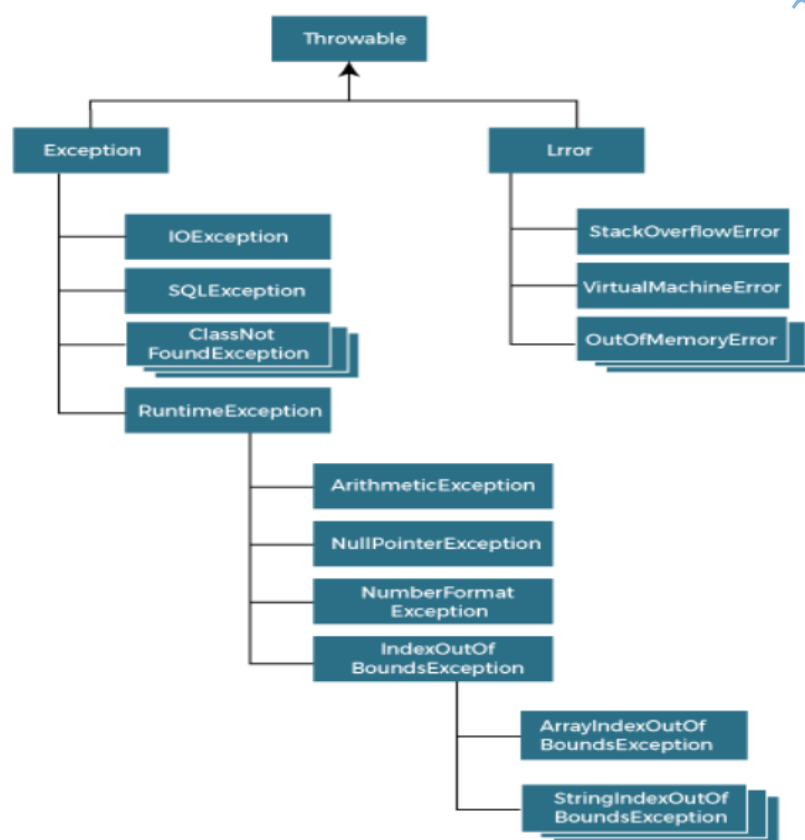
Exception Handling:

Exception means an abnormal condition. Also an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Exception Handling is a mechanism to handle the runtime errors, so that the normal flow of the application can be maintained. Exception handling enables applications to resolve (or handle) exceptions. Help to write robust and fault-tolerant programs that can deal with problems and continue executing or terminate gracefully.

Java Exception Hierarchy:

All Java exception classes inherit directly or indirectly from class **Exception**, forming an inheritance hierarchy. Figure(a) below shows a portion of the inheritance hierarchy for class **Throwable** (a subclass of **Object**), which is the superclass of class **Exception**.



Figure(a): Throwable's inheritance hierarchy

Only **Throwable** objects can be used with the exception-handling mechanism. Two subclasses of **Throwable** class: **Exception** and **Error**. Class **Exception** and its subclasses – **Runtime / Unchecked Exception** (package `java.lang`) and **Checked Exception** (package `java.io`).

Types of Java Exceptions:

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

- i. **The compiler and Checked Exception(Implicit Exception):**

- The compiler checks each method call and method declaration to determine whether the method throws a checked exception.
- If so, the compiler verifies that the checked exception is caught or is declared in a throws clause, known as catch-or-declare requirement.
- All classes that inherit from class Exception but not directly or indirectly from class **RuntimeException** are considered to be **checked** exceptions.
- Such exceptions are typically caused by conditions that are not under the control of the program.
- **Example:** In file processing, program can't open a file if it does not exist (FileNotFoundException), IOException, SQLException, etc.

ii. **RuntimeException and Unchecked Exception(Explicit Exception):** Exception types that occur at runtime and are direct or indirect subclasses of **RuntimeException**(package java.lang). Typically caused by defects in your program's code.

Examples:

a. **ArithmeticException:** Divide any number by zero.

```
int num = 3000/0; // ArithmeticException
```

b. **NullPointerException:** Have a null value in any variable.

```
String str = null;
```

```
System.out.println(str.length); // NullPointerException
```

c. **NumberFormatException:**

```
String str = "Hello Java Developer";
```

```
int strValue = Integer.parseInt(str); // NumberFormatException
```

d. **ArrayIndexOutOfBoundsException:** This occurs when an array exceeds its size.

```
int arr[] = new int[8];
```

```
arr[11] = 40; // ArrayIndexOutOfBoundsException
```

iii. **Error:** Errors typically are unrecoverable, so your program should not even attempt to deal with them. **Examples:** OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Keywords:

Used to handle the exception.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.

finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Java Exception Handling Examples:

Example 1: *ArithmeticException and InputMismatchException*

In this example you will handle divide by zero exception and input mismatch exception. **ArithmeticException** occurs when numerator divide by **zero** where **InputMismatchException** occurs when Scanner method **nextInt()** receives a string that does not represent a valid integer. Also , uses multiple catch block.

```

/* ExceptionHandling.java */
package exceptionhandling; //package
import Examples.*; //imported all classes with in Example package
public class ExceptionHandling {
    public static void main(String[] args) {
        NumberDivision nd = new NumberDivision();
        nd.OperationHere();
    }
}

/* NumberDivision.java */
package Examples; //package name where NumberDivision class exist
import java.util.InputMismatchException; //For user input type mis-match
import java.util.Scanner; //For user input to system
public class NumberDivision {
    public static int quotientValue(int first_num,int second_num){
        return first_num / second_num;
    }
    Scanner scn = new Scanner(System.in);
    public void OperationHere(){
        try{
            System.out.println("Enter First Number:");
            int f_num = scn.nextInt();
            System.out.println("Enter Second Number:");
            int s_num = scn.nextInt();
            int result = quotientValue(f_num,s_num);
            System.out.println("Result:"+result);
        }catch(InputMismatchException ie){
            System.err.printf("%nException occur becasue of string:%s%n", ie);
            System.out.println("Must enter integer. Try again.");
        }catch(ArithmeticException ae){
            System.out.println("Its Arithmetic Exception:"+ae.getMessage());
            ae.printStackTrace();
            System.out.println("Division by zero is invalid . Try again with non-zero number.");
        }
    }
}

```

```

    }
}

```

Example 2: *ArrayIndexOutOfBoundsException*

```

public class ArrayIndexOutOfBoundsExceptionExample {
    public static void main(String[] args) {
        byte[] arr = {5,6,7,30,40,89}; //array of type byte
        int[] arr1 = {30,20,44,56,88}; //array of type int

        try{
            int sum = arr[0]+arr1[12]; // arr1[12] exceed length-1 of array
            System.out.print(sum);
            throw new ArrayIndexOutOfBoundsException();
        }catch(ArrayIndexOutOfBoundsException aio){
            System.out.println("Array Index Out Of Bounds:"+aio.getMessage());
        }
    }
}

```

Example 3: Also can use *Exception class* to handle any sort of exception.

*/*UsingExceptionClass.java*/*

```

import java.util.Scanner;

public class UsingExceptionClass {
    void profitCalc(double[] cf){

        double[] returnProfit = new double[cf.length];
        try{
            for(int i=0;i<cf.length;i++){
                returnProfit[i] = cf[i] + (cf[i]*0.06);
            }

            System.out.printf("\nPrice after 6%% profit:\t\tProfit amount:");
            for(int i=0;i<returnProfit.length;i++){
                System.out.format("\n%f\t\t%f",returnProfit[i],(returnProfit[i]-cf[i]));
            }
        }catch(Exception e){
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] args) {
        UsingExceptionClass obj = new UsingExceptionClass();

        int arrayLength;
        Scanner scn = new Scanner(System.in);
        System.out.println("Enter size of array:");
        double[] costOfFruit;
        try{
            arrayLength = scn.nextInt();

```

```

costOfFruit = new double[arrayLength];
System.out.println("Enter elements of array:");
for(int i=0; i<arrayLength; i++){
    costOfFruit[i] = scn.nextInt();
}
obj.profitCalc(costOfFruit);
} catch (Exception e) {
    System.out.println("Sorry:" + e.getMessage());
}
}
}

```

Difference between throw and throws:

throw	throws
- used to throw an exception explicitly in the code, inside the function or the block of code.	- used in the method signature to declare an exception which might be thrown by function while the execution of code.
- Only propagate unchecked exception.	- We can declare both checked and unchecked exceptions.
- The throw keyword is followed by an instance of Exception to be thrown.	- The throws keyword is followed by class names of Exceptions to be thrown.
- throw is used within the method.	- throws is used with the method signature.
- Allowed to throw only one exception at a time i.e. cannot throw multiple exceptions.	- Can declare multiple exceptions using throws that can be thrown by the method. Eg: IOException, SQLException
Example : <pre> /*ExceptionUsingThrows.java*/ import java.util.Scanner; public class ExceptionUsingThrow { public static int quotient(int numerator, int denominator){ if(denominator > 0){ return numerator/denominator; }else{ throw new ArithmeticException("Denominator less than zero(0). Results Arithmetic exception:"); } } public static void main(String[] args) { //try-with resource try(Scanner scn = new Scanner(System.in));{ System.out.println("Enter numerator:"); int n = scn.nextInt(); System.out.println("Enter denominator:"); int d = scn.nextInt(); System.out.println(quotient(n,d)); } } } </pre>	Example : <pre> /*ExceptionUsingThrows.java*/ import java.util.Scanner; public class ExceptionUsingThrow { public static int quotient(int numerator, int denominator) throws ArithmeticException{ return numerator/denominator; } public static void main(String[] args) { //try-with resource try(Scanner scn = new Scanner(System.in));{ System.out.println("Enter numerator:"); int n = scn.nextInt(); System.out.println("Enter denominator:"); int d = scn.nextInt(); System.out.println(quotient(n,d)); } catch (ArithmeticException e){ System.out.println("Not divisible by /zero:"); } } } </pre>

<pre> }catch(ArithmeticException e){ System.out.println(e.getMessage()+"Not divisible by /zero:"); } } } </pre>	
---	--

Tips on Handling Exception:

1. Never use try block without catch or finally block

```

try{
    //write statement that can throw exception
}

```

This yields syntax error . Which means exception must be handled . So, prefer standard exception handling process. You can use finally block with or instead of catch block.

```

try{
    //write statement that can throw exception
} catch(exception_type_name e){
    //catch exception , response with message
    //used to fix exception / error occur
}

```

2. Use finally block instead of catch blocks if not going to handle exception

```

try{
    //write statement that can throw exception
} finally{
    //exceute statement written with in finally block
    //doesn't care about either exception handled of or not
}

```

3. Never throw any exception from finally block

```

try{
    //write statement that can throw exception
} catch(exception_type_name e){
    //catch exception , response with message
    //used to fix exception / error occur
} finally{
    throw new ArithmeticException(); //never do this
}

```

Second exception will execute and first exception will be lost forever.

4. Never swallow the exception in catch block

```

try{
    //write statement that can throw exception
} catch(IOException e){
    return null;
}

```



```
}
```

Exception must be handled with meaningful description or re-throw the exception. Returning null totally swallows the exception, and lose cause of error forever.

5. Never catch Throwable class rather catch subclasses .

```
try{  
    //write statement that can throw exception  
}catch(Throwable twa){  
    //do not do this  
}
```

It will catch all exceptions; will also catch all errors. Errors(OutOfMemoryError, StackOverflowError) are thrown by JVM to indicate serious problems, that are not handled by an application.

6. Remember “Throw early catch late” principle

Famous principle about Exception Handling; which means throw an exception as soon as possible and catch it late as much as possible. You should wait until you have all the information to handle it properly.

7. Always clean up resources in a finally block or use a try-with-resource statement

- After all statements within the try block will get executed, you can directly close resource.
- But what if one or more methods which might throw an exception.
- Which means you might not reach the end of the try block. And as a result, you will not close the resources.
- Therefore, let's use finally block or try use a try-with-resource statement to clean up resource.

Example 1: using finally block

```
import java.io.IOException;  
import java.io.InputStream;  
import java.io.FileInputStream;  
import java.io.File;  
  
public class FinallyToReleaseResource {  
    public static void main(String[] args) {  
        FileInputStream fin = null;  
        try{  
            File file = new File("./testFile.txt");  
            fin = new FileInputStream(file);  
            int size = fin.available();  
  
            for(int i=0;i<size; i++){  
                System.out.print(fin.read()+ " ");  
            }  
        }catch(IOException e){  
            System.out.println("There exist exception:"+e.getMessage());  
        }
```

```

    }finally{
        if(fin != null){
            try{
                fin.close();
            }catch(IOException e){
                System.err.print(e);
            }
        }
    }
}
}
}

```

Example 2: try-with-resource

```

import java.io.IOException;
import java.io.InputStream;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.File;

public class AutomaticallyCloseResource {
    public static void main(String[] args) {
        File file = new File("./testFile.txt");
        try(FileInputStream fin = new FileInputStream(file)){
            int data = 0;
            while((data = fin.read()) != -1){
                System.out.print(data+",");
            }
        }catch(FileNotFoundException fe){
            System.err.println("Not found !!!");
        }catch(IOException ie){
            System.err.println("Sorry, i/o problem");
        }
    }
}

```

8. Always terminate the thread which it is interrupted

```

while (true) {
    try {
        Thread.sleep(100000);
    } catch (InterruptedException e) {
        break;
    }
}

```

9. Use template methods for repeated try-catch

- Should not use similar catch block many more time like 20's in program.
- This will increase code duplicity which helps nothing.

- So, use template methods for such cases.

10. Document all exceptions in the application with Javadoc

- Make it a practice to Javadoc all exceptions which a piece of code may throw at runtime.
- Also try to include possible course of action, user should follow in case these exception occur.

Debugging Techniques:

Debugging means the technique of monitoring software/application source code, finding and removing any potential errors, bugs, or defects that causes programs behavior unexpected. Every developer faces bugs and errors in their implementation phase. So, for successful execution i.e. for expected outcome written program must be correct and error free including software development phases.

Debugging sounds simple like identify the source of the bug, correct the problem or fixed it. However, it is a tough and crucial task; what if various sub-systems, modules, sub-modules are tightly coupled.

So, to resolve defect in the program or application, developer follows some debugging techniques. Also, there is no fixed procedure to fix all bugs. So, they resolve bugs using their own methods sometime.

The three most known and used techniques are:

i. Trace-based debugging:

- Traditional, most common and based on the concept of breakpoints.
- A breakpoint is a pausing or stopping point added to the program to examine the state of program execution up until that point.
- **Conditional breakpoints** – breakpoint with a specified condition where the thread will stop at the targeted line when the specified condition is true.
- **Exception breakpoints** – At debugging if any exception occur, exception breakpoints is a solution. **Eg:** To resolve Null Pointer problem, *NullPointerException* is used.
- **Watchpoint** – is a breakpoint set up on a field or variable, where the targeted field or variable is accessed or changed, the execution of the program will get stop and then you can debug.

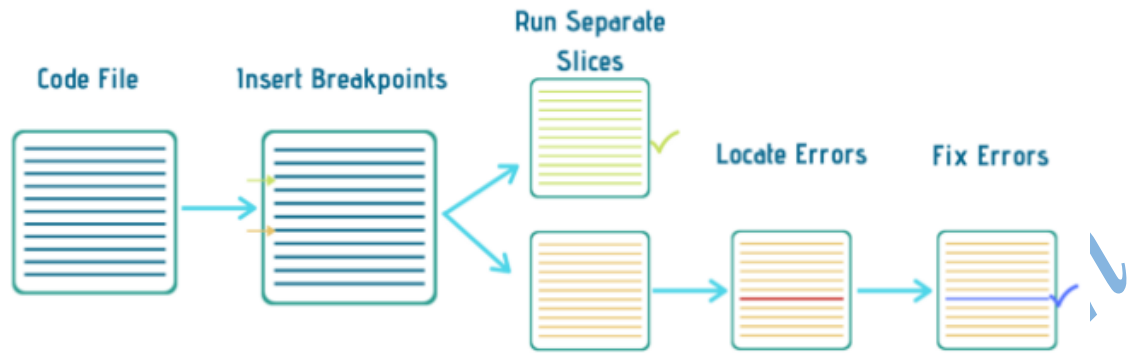


Figure: Trace-based debugging technique

ii. **Spectrum-based debugging:**

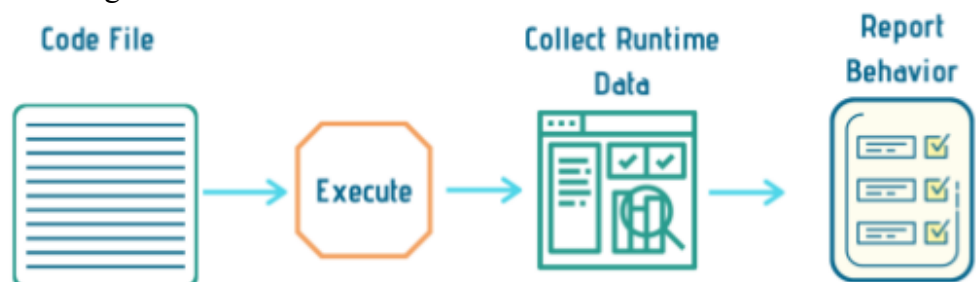
- Also known as spectrum-based fault localization (SFL), debugging is done by monitoring the statements included in a particular execution tree i.e. debugging a range of a particular type of statements.
- This debugging is achieved by identifying the active part of the program during its run.
- The program spectrum is a collection of runtime information that gives a view of the dynamic behavior of the program.
- Different types of program spectra exist, such as block hit/miss and function hit/miss, and are used to pinpoint exact sections of running code for specific or abstract input.



Figure: Spectrum-based debugging

iii. **Delta debugging:**

- Delta debugging is to minimize automated test cases.
- It takes test cases with possible error causes and prepare an error report.
- Minimal cases are selected from error report based on their high probability of producing the error.
- The minimum test cases will regenerate the same error and thus help the developer locate the bug behind it.



Java Custom Exception:

Creating our own Exception is known as custom exception or user-defined exception i.e. are used to customize the exception according to user need. So, custom exceptions are derived classes of the Exception class.

We use custom exceptions to catch and provide specific treatment to subset of existing java exceptions. Also useful for the application users or the developers to understand the exact problem, defining.

To create custom exception, you need to extend Exception class that belongs to *java.lang* package.

Example 1: Check whether Nepalese citizen is eligible for Nepali citizenship or not.

```
import java.util.Scanner;
class CheckCitizenAge extends ArithmeticException{
    CheckCitizenAge(String str){
        super(str);
    }
}
public class CustomExceptionForCitizenship {
    static void validateAge(int ag) throws CheckCitizenAge{
        if(ag<16){
            throw new CheckCitizenAge("First parameter not meet.Wait to 16.");
        }else{
            System.out.println("Eligible for citizenship.");
        }
    }
    public static void main(String[] args) {
        //try-with resource
        try(Scanner scn = new Scanner(System.in)){
            System.out.println("Enter age:");
            int age = scn.nextInt();

            validateAge(age);
        }catch(CheckCitizenAge e){
            System.out.println("Not Eligible for Nepalese Citizenship."+e.getMessage());
        } catch(Exception e){
            System.out.println("Null pointer occur when input value is not a number: "+e.getMessage());
        }
    }
}
```

Explanation:

In above example *CheckCitizenAge* is a custom exception that inheritance *java.lang ArithmeticException* class. Here we call the constructor of *ArithmeticException* class from *CheckCitizenAge* class using *super()* keyword.

Inside the method *validateAge()*, exception condition checked, and if the exception occurs, *throw* block throw exception through *CheckCitizenAge* exception class(i.e. custom exception) to *ArithmeticException* then *try...catch* block handles the exception.

Here, you see two catch block, first one is for *Custom Exception(CheckCitizenAge)* and second one handle *NullPointerException* in case input is not a number.

Example 2:

```
import java.util.ArrayList;
import java.util.Arrays;

class CustomException extends RuntimeException{
    public CustomException(String notify){
        super(notify);
    }
}

public class CustomExceptionExample {
    ArrayList<String> fruitItems = new ArrayList<>(Arrays.asList("Apple","Banana","Pineapple"));

    public void checkFruitItem(String fruitName) throws CustomException{
        if(fruitItems.contains(fruitName)){
            throw new CustomException(fruitName + " already exist in list.");
        }else{
            fruitItems.add(fruitName);
            System.out.println(fruitName + " is added to the List.");
        }
    }

    public static void main(String[] args) {
        CustomExceptionExample obj = new CustomExceptionExample();
        try{
            obj.checkFruitItem("Grapes");
            obj.checkFruitItem("Banana");
        }catch(CustomException e){
            System.out.println("Exception after adding:"+e.getMessage());
        }
    }
}
```

Streams I/O:

You store data in variables and arrays is temporary, it's lost when a local variable goes out of scope or when the program terminates. For long-term retention of data, even after the programs that create the data terminate, computers use files. Computers store files on secondary storage devices(hard disks, flash drives, DVDs and more) and data maintained in files is persistent data.

Java views each file as a sequential stream of bytes i.e. a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow. A java program processing a stream of bytes simply receives an indication from the operating system when it reaches the end of the stream. Java I/O is used to process the input and produce the output. So, Java uses the concept of stream to make I/O operation fast.

Java programs perform stream-based processing with classes and interfaces from package **java.io** and the sub-packages of **java.nio**.

Note: Java NIO APIs also include classes and interfaces that implement so-called channel-based architecture for high-performance I/O.

Byte-based Streams	Character-based Streams
• Output and input data in its binary format – a char is two bytes, an int is four bytes, a double is eight bytes etc. • File created using byte-based streams are binary files. • Binary files are read by programs that understand the file's specific content and its ordering. • A numeric value in a binary file can be used in calculations.	• Output and input data as a sequence of characters in which every character is two bytes(so, number of bytes depends on numbers of characters). • Files created using character-based streams are text files. • Text files can be read by text editors. (I/O can be performed with class Scanner and Formatter. • But 5 in character-based streams is character and can't do calculations.

When a Java program begins executing, it creates three stream objects that are associated with devices:

1. **System.out:** Standard output stream. Object normally enables a program to output character data to the screen.
2. **System.in:** Standard input stream. Object normally enables a program to input bytes from the keyboard.
3. **System.err:** Standard error stream. Object normally enables a program to output character-based error messages to the screen.

Code to print output and an error message to the console.

```
System.out.println("Just print an line.");
System.err.println("Error message be here.");
```

Code to get input from console.

```
int a = System.in.read(); //returns ASCII code of 1st character
System.out.println((char)a); //will print the character
```

Example:

```
import java.io.*;
class StandardInputOutputExample{
    public static void main(String[] args) {
        System.err.println("You can display error here !!!");
        try{
            System.out.println("Enter byte number.");
            int i = System.in.read();
            System.out.println(i);
        }catch(IOException e){
            e.printStackTrace();
        }
    }
}
```

OutputStream vs InputStream:

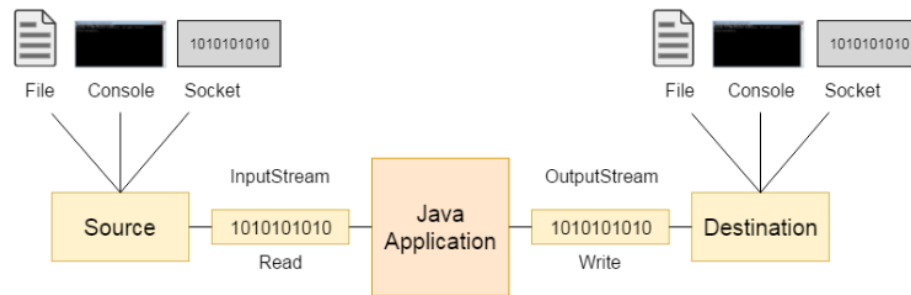


Figure: Working of Java OutputStream and InputStream

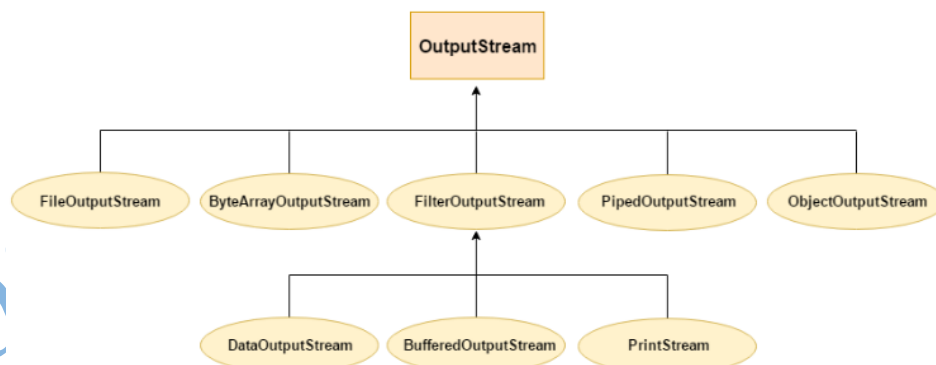
OutputStream:

Uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket. OutputStream class is an abstract class. It is the superclass of all classes representing an output stream of bytes. An output stream accepts output bytes and sends them to some sink.

Methods of OutputStream:

Method	Description
1) public void write(int)throws IOException	is used to write a byte to the current output stream.
2) public void write(byte[])throws IOException	is used to write an array of byte to the current output stream.
3) public void flush()throws IOException	flushes the current output stream.
4) public void close()throws IOException	is used to close the current output stream.

OutputStream Hierarchy:



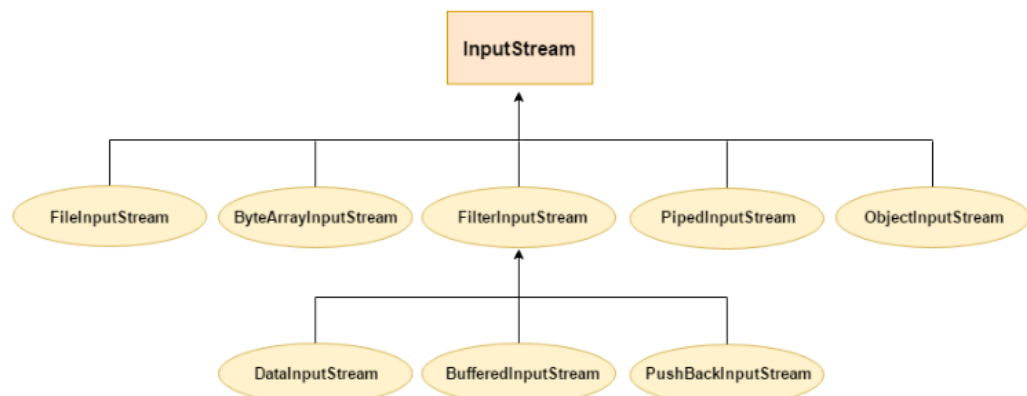
InputStream:

Uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket. InputStream class is an abstract class. It is the superclass of all classes representing an input stream of bytes.

Methods of InputStream:

Method	Description
1) public abstract int read()throws IOException	reads the next byte of data from the input stream. It returns -1 at the end of the file.
2) public int available()throws IOException	returns an estimate of the number of bytes that can be read from the current input stream.
3) public void close()throws IOException	is used to close the current input stream.

InputStream Hierarchy:



Some classes of *OutputStream* and *InputStream* from package *java.io.** :

FileOutputStream and FileInputStream:

FileOutputStream is an output stream used for writing data(primitive value) to a file. **FileOutputStream** can write byte-oriented as well as character-oriented data . but recommended to use **FileWriter** class.

Syntax:

OutputStream object_name = new **FileOutputStream**("file_location/file_name");

OR

FileOutputStream object_name = new **FileOutputStream**("file_location/file_name");

FileOutputStream class methods:

Method	Description
protected void finalize()	It is used to clean up the connection with the file output stream.
void write(byte[] ary)	It is used to write ary.length bytes from the byte array to the file output stream.
void write(byte[] ary, int off, int len)	It is used to write len bytes from the byte array starting at offset off to the file output stream.
void write(int b)	It is used to write the specified byte to the file output stream.
FileChannel getChannel()	It is used to return the file channel object associated with the file output stream.
FileDescriptor getFD()	It is used to return the file descriptor associated with the stream.
void close()	It is used to closes the file output stream.

FileInputStream class obtains input bytes from a file . i.e. Used for reading byte oriented data(streams of raw bytes) such as image data, audio, video etc. Also read character-stream data , but recommended to use **FileReader** class.

Syntax:

InputStream object_name = new **FileInputStream**("file_location/file_name");

OR

FileInputStream object_name = new **FileInputStream** ("file_location/file_name");

Example 1: Writing to file(hello.txt) and reading single byte from file (hello.txt).(byte-oriented)

/*FileOutputStreamExample.java*/

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
class FileInputOutputStreamExample{
    public static void main(String[] args) {
        //Writing string data to fileOut.txt (byte-oriented)
        try{
            FileOutputStream fout=new FileOutputStream("./hello.txt");
            fout.write(67); //writing data to fileOut.txt
            fout.close(); //closing file stream
        }catch(IOException e){
            System.out.println(e);
        }finally{
            System.out.println("Check file(hello.txt).");
        }
        //Reading file(fileOut.txt) content (using byte-oriented format)
        try{
            FileInputStream fin = new FileInputStream("./hello.txt");
            int rf=fin.read(); //read individual byte
```

```

        System.out.print((char)rf);
    fin.close();
} catch(IOException e){
    System.out.println("File(hello.txt) not found."+e.getMessage());
}
}
}

```

Example 2: Writing string to file(byte-oriented) and reading all characters from file(byte-oriented).

/*FileInputStreamExample.java*/

```

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
class FileInputStreamExample {
    public static void main(String[] args) {
        //Writing string data to fileOut.txt (byte-oriented)
        try{
            FileOutputStream fout=new FileOutputStream("./fileOut.txt");
            String str = "Lets do something. Related to java streams i/o.";
            byte bt[] = str.getBytes(); //convert string to byte
            fout.write(bt); //writing data to fileOut.txt
            fout.close(); //closing file stream
        } catch(IOException e){
            System.out.println(e.getMessage());
        } finally{
            System.out.println("Check file.");
        }

        //Reading file(fileOut.txt) content (using byte-oriented format)
        try{
            FileInputStream fin = new FileInputStream("./fileOut.txt");
            int rf=0;
            while((rf = fin.read()) !=-1){
                System.out.print((char)rf);
            }
            fin.close();
        } catch(IOException e){
            System.out.println("File to read not found."+e.getMessage());
        }
    }
}

```

FileWriter and FileReader:

FileWriter class is used to write character-oriented data to a file and also used for file handling in java. Don't need to convert string into byte array because it provides method to write string directly.

FileReader class is used to read data from the file. Returns data in byte format like `FileInputStream` class. It is character-oriented class which is used for file handling in java.

Example: Write to file and read from file using `FileWriter` and `FileReader` respectively.

`/*exceptionHandlingExample1.java*/`

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
class exceptionHandlingExample1 {
    public static void main(String[] args) {
        //Writing string data to fileToWrite.txt (character-oriented)
        try{
            FileWriter fwr=new FileWriter("./fileToWrite.txt");
            String str = "Do you know , i don't need to be converted.(to byte)";
            fwr.write(str); //writing data to fileToWrite.txt
            fwr.close(); //closing file stream
            System.out.println("Check file(fileToWrite.txt).");
        } catch(IOException e){
            System.out.println(e);
        }
        //Reading file(fileToWrite.txt) content (using byte-oriented format)
        try{
            FileReader frd = new FileReader("./fileToWrite.txt");
            int rf=frd.read(); //read individual byte
            System.out.println("Below is individual Data:");
            System.out.print((char)rf);
            System.out.printf("\n");
            frd.close();
            // Read all content of file(fileToWrite.txt)
            FileReader fr = new FileReader("./fileToWrite.txt");
            int rac = 0;
            System.out.println("All content of file:");
            while((rac = fr.read()) != -1){
                System.out.print((char)rac);
            }
            fr.close();
        } catch(IOException e){
            System.out.println("File(hello.txt) not found."+e.getMessage());
        }
    }
}
```

}

BufferedOutputStream and BufferedInputStream

BufferedOutputStream class is used for buffering an output stream, uses buffer to store data and add more efficiency than to write data directly into a stream.

BufferedOutputStream class methods:

Method	Description
void write(int b)	It writes the specified byte to the buffered output stream.
void write(byte[] b, int off, int len)	It write the bytes from the specified byte-input stream into a specified byte array, starting with the given offset
void flush()	It flushes the buffered output stream.

BufferedInputStream class is used to read information from stream. Internally uses buffer mechanism to make the performance fast. Buffer help to read after last byte.

BufferedInputStream class methods:

Method	Description
int available()	It returns an estimate number of bytes that can be read from the input stream without blocking by the next invocation method for the input stream.
int read()	It read the next byte of data from the input stream.
int read(byte[] b, int off, int ln)	It read the bytes from the specified byte-input stream into a specified byte array, starting with the given offset.
void close()	It closes the input stream and releases any of the system resources associated with the stream.
void reset()	It repositions the stream at a position the mark method was last called on this input stream.
void mark(int readlimit)	It sees the general contract of the mark method for the input stream.
long skip(long x)	It skips over and discards x bytes of data from the input stream.
boolean markSupported()	It tests for the input stream to support the mark and reset methods.

Example: BufferedOutputStream and BufferedInputStream.

```
import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;
```

```

import java.io.File;
import java.io.FileInputStream;

public class BufferedStreamExample {
    public static void main(String[] args){
        //write file using FileOutputStream and BufferedOutputStream
        try{
            FileOutputStream fout = new FileOutputStream("./fileBuffer.txt");
            BufferedOutputStream bout = new BufferedOutputStream(fout);
            String str = "Hello , Let's explore Nepal.";
            byte byt[] = str.getBytes();
            bout.write(byt);
            bout.flush();

            String st = "Heaven is myth, Nepal is Real. ";
            byte bt[] = st.getBytes();
            bout.write(bt);
            bout.flush();

            fout.close();
            System.out.println("Written Success....");
        } catch(IOException e){
            e.printStackTrace();
        }
        //reading file content using FileInputStream and BufferedInputStream
        try{
            File file = new File("./fileBuffer.txt");
            FileInputStream fin = new FileInputStream(file);
            BufferedInputStream bin = new BufferedInputStream(fin);

            System.out.println("Total available data Before read:"+bin.available());

            int data = bin.read();
            System.out.println((char)data); //read first byte-value(H from fileBuffer.txt)

            int dat = bin.read();
            System.out.println((char)dat); //read second byte-value(e from fileBuffer.txt)

            int dt = bin.read();
            System.out.println((char)dt); //read third byte-value(l from fileBuffer.txt)

            int dtt = bin.read();
            System.out.println((char)dtt); //read fourth byte-value(l from fileBuffer.txt)

            System.out.println("Total available data After read:"+bin.available());
            System.out.println("Remaining file content:");

            int content = 0;

```

```

        while((content = bin.read()) != -1){
            System.out.print((char)content);
        }
        fin.close();
    } catch(IOException e){
        e.printStackTrace();
    }
}
}
}

```

BufferedWriter and BufferedReader:

BufferedWriter class is used to provide buffering for Writer instances. Makes performance fast and inherits Writer class. Writes text to a character-output stream, buffering characters are used for providing the efficient writing of single arrays, characters, and strings.

Example: BufferedWriter class

*/*BufferedWriterExample.java*/*

```

import java.io.BufferedWriter;
import java.io.FileWriter;
public class BufferedWriterExample {
    public static void main(String[] args) throws Exception {
        FileWriter fw = new FileWriter("./fileWrite.txt");
        BufferedWriter bw = new BufferedWriter(fw);
        bw.write("This is Buffered Writer class");
        bw.close();
        System.out.println("Written success....");
    }
}

```

BufferedReader class is used to read the text from a character-based input stream. Can be used to read data line by line by **readLine()** method. It inherits Reader class.

BufferedReader class methods:

Method	Description
int read()	It is used for reading a single character.
int read(char[] cbuf, int off, int len)	It is used for reading characters into a portion of an array .
boolean markSupported()	It is used to test the input stream support for the mark and reset method.
String readLine()	It is used for reading a line of text.
boolean ready()	It is used to test whether the input stream is ready to be read.
long skip(long n)	It is used for skipping the characters.
void reset()	It repositions the stream at a position the mark method was last called on this input stream.
void mark(int readAheadLimit)	It is used for marking the present position in a stream.
void close()	It closes the input stream and releases any of the system resources associated with the stream.

Example: BufferedReader class

*/*BufferedReaderExample.java*/*

```
import java.io.BufferedReader;
import java.io.FileReader;

public class BufferedReaderExample {
    public static void main(String[] args) throws Exception {
        FileReader fr = new FileReader("./fileWrite.txt");
        BufferedReader br = new BufferedReader(fr);
        int i;
        while((i=br.read())!=-1){
            System.out.print((char)i);
        }
        br.close();
    }
}
```

Java Console Class:

Console Class is used to get input from console. It provides methods to read texts and passwords. The **java.io.Console** class is attached with system console internally. If you read password using Console class, it will not be displayed to the user.

Methods:

Method	Description
Reader reader()	It is used to retrieve the reader object associated with the console
String readLine()	It is used to read a single line of text from the console.
String readLine(String fmt, Object... args)	It provides a formatted prompt then reads the single line of text from the console.
char[] readPassword()	It is used to read password that is not being displayed on the console.
char[] readPassword(String fmt, Object... args)	It provides a formatted prompt then reads the password that is not being displayed on the console.
Console format(String fmt, Object... args)	It is used to write a formatted string to the console output stream.
Console printf(String format, Object... args)	It is used to write a string to the console output stream.
PrintWriter writer()	It is used to retrieve the PrintWriter object associated with the console.
void flush()	It is used to flushes the console.

Example 1: Reading name from console.

```
/* ConsoleClassExample1.java */
import java.io.Console;
public class ConsoleClassExample1 {
    public static void main(String[] args) {
        Console cnl = System.console();
        System.out.println("Enter name:");
        String name = cnl.readLine();
        System.out.println("Namaste "+name+" Sir.");
    }
}
```

Example 2: Read password

```
/* ConsoleClassExample2.java */
import java.io.Console;
public class ConsoleClassExample2 {
    public static void main(String[] args) {
        Console cnl = System.console();
        System.out.println("Enter password:");
        char[] pass = cnl.readPassword();
        System.out.println("Password not visible(encrypted): "+pass.toString());
    }
}
```

Example 3:

```

import java.io.Console;
public class ConsoleClassExample {
    public static void main(String[] args) {
        Console cnl = null;
        //format: %1s -> first string format specifier, %2$10s -> second string format specifier with only 10
        character.
        String fmt = "%1s %2$10s";
        String alpha = null;
        try{
            cnl = System.console();
            if(cnl != null){
                alpha = cnl.readLine(fmt,"Enter","Alphabets:");
                System.out.println("Alphabets Entered:"+alpha);
            }
        }catch(Exception ex){
            ex.printStackTrace();
        }
    }
}

```

Data Streams:

Data streams support binary I/O of primitive data type values(Boolean, char, byte, short, int, long, float, and double) as well as String values. All data streams implement either the **DataInput** interface or the **DataOutput** interface. Where **DataInputStream** and **DataOutputStream** are mostly widely-used implementations of these interface.

The **DataStreams** example demonstrates data streams by writing out a set of data records, and then reading them in again. Each record consists of three values to an item on an invoice, as shown in the following table:

Order in record	Data type	Data description	Output Method	Input Method	Sample Value
1	double	Item price	DataOutputStream.writeDouble	DataInputStream.readDouble	19.99
2	int	Unit count	DataOutputStream.writeInt	DataInputStream.readInt	12
3	String	Item description	DataOutputStream.writeUTF	DataInputStream.readUTF	"Java T-Shirt"

The **writeUTF()** method writes out String values in a modified form of **UTF-8**. This is a variable-width character encoding that only needs a single byte for common Western characters.

Example: Writing primitive data to file(*student.txt*) using DataOutput.

```
import java.io.File;
import java.io.FileOutputStream;
import java.io.FileInputStream;
public class DataInputOutputExample {
    public static void main(String[] args) {
        try{
            FileOutputStream fout = new
FileOutputStream("./student.txt");
            DataOutputStream dot = new
DataOutputStream(fout);
            dot.writeInt(90);
            dot.writeUTF("Hello world.");
            dot.close();
        } catch (Exception e){
            e.printStackTrace();
        }
    }
}
```

Example: Reading data/records from file(*student.txt*) using DataInput .

```
import java.io.DataInputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileInputStream;
public class DataInputOutputExample {
    public static void main(String[] args) {
        FileInputStream fin = new
FileInputStream("./student.txt");
        DataInputStream din =
new DataInputStream(fin);
        int num = din.readInt();
        String str = din.readUTF();
        System.out.println(num);
        System.out.println(str);
        din.close();
    } catch (Exception e){
        e.printStackTrace();
    }
}
```

Example 2: Writing item record to file (invoice.txt) using DataOutput.

```
import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.DataOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class DataInputOutputStream {
    public static void main(String[] args) {
        final String file_name = "./invoice.txt";

        final double[] prices =
{20.9,90.8,40.7,78.4,87.78};
        final int[] quantity = {4,5,8,9,2};
        final String[] title =
{"Shirt","Pant","Sneaker","Boot","Purse"};
        final String[] description =
{"Shirt","Pant","Sneakers","Purse","Belt"};
        try{
            FileOutputStream file = new
FileOutputStream(file_name);
            DataOutputStream data = new
DataOutputStream(new BufferedOutputStream(file));
```

Example 2: Reading record from file(invoice.txt) using DataInput.

```
import java.io.BufferedInputStream;
import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class DataInputOutputStream {
    public static void main(String[] args) {
        final String file_name = "./invoice.txt";

        double price;
        int unit;
        String ttl;
        String desc;
        double total = 0.0;
        try{
            FileInputStream file = new
FileInputStream(file_name);
            DataInputStream in = new
DataInputStream(new BufferedInputStream(file));

            while(in != null){
                price = in.readDouble();
```

<pre> for(int i=0; i<prices.length;i++){ data.writeDouble(prices[i]); data.writeInt(quantity[i]); data.writeUTF(title[i]); data.writeUTF(description[i]); } data.close(); System.out.println("Success...."); } catch(IOException e){ e.printStackTrace(); } } } </pre>	<pre> unit = in.readInt(); ttl = in.readUTF(); desc = in.readUTF(); total = unit*price; System.out.format("You ordered %d "+"quantity of %s at \$%.2f with title %s.Total Cost: %.2f%n",unit,desc,price,ttl,total); } // in.close(); file.close(); } catch(IOException e){ System.out.println("End of file reached."); } } } </pre>
---	---

Object Streams:

As data streams support I/O of primitive data types, object streams support I/O of objects. Generally object stream is used to Serialize objects of class using standard classes, but not all . **ObjectInputStream** and **ObjectOutputStream** are object stream classes. These classes implement **ObjectInput** and **ObjectOutput**, which are sub-interfaces of **DataInput** and **DataOutput**. That means all the primitive data I/O methods covered in **DataStreams** are also implemented in object streams. So an object stream can contain a mixture of primitive and object values.

Serialization is a mechanism of writing the state of an object into a byte-stream. **Deserialization** is the reverse process where the byte stream is used to recreate the actual Java object in memory. And used to persist the object:

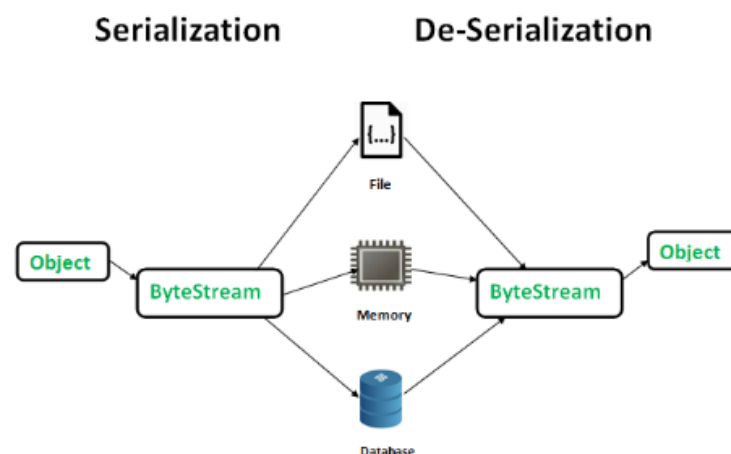


Figure: Serialization and De-Serialization

Let us consider two JVM running at different place. Where JVM one is executing an object and same object to be executed on JVM two as well as. Now the only way to transfer this object from one JVM to another JVM is either follow **serialization** or **externalization**. Here we follow **Serialization** , where object converted into byte-stream and can be stored in either **Memory**(i.e.

RAM which is temporary) OR Database OR File. This byte-stream is transferred to JVM then De-serialized to retrieve original object.

Advantages:

- Communication(Used for marshaling) – traveling the state of an object on the network. Which means multiple computer system to design , share and execute objects simultaneously over network.
- It is easy to use and customize.
- Persistence – to save and store state object in a database . So , it can be retrieved later.
- Cross JVM Synchronization (JVM independent) – Serialization works across different JVMs that might be running on different architecture.

Disadvantages:

- Except simple fields changing to the object have possible that java cannot deserialize the object correctly even if the serialization ID not changed.
- Large objects also significantly increase the memory requirements of your application.

To serialize an object, parent class should implement **Serializable** interface(**java.io.Serializable**). The **writeObject()** method of **ObjectOutputStream** class used for serializing an object and **readObject()** method of **ObjectInputStream** class to de-serialize an object.

Example 1: We are going to serialize Object of class *UserDetail* . The **writeObject()** method of *ObjectOutputStream* class provides serialization.

Note: Program file(*UserRegistration.java*) and file (*user_registration.txt*) are in same folder/directory.

```
/* UserRegistration .java*/
```

```
import java.io.OutputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.io.Serializable;
```

```
class UserDetail implements Serializable{
    int id;
    String userName;
    String userAddress;
    String userEmail;
    //parameterized constructor
    public UserDetail(int id,String userName,String userAddres,String userEmail){
        this.id = id;
        this.userName = userName;
        this.userAddress = userAddres;
        this.userEmail = userEmail;
    }
}

public class UserRegistration {
    public static void main(String[] args) {
```

```

//instance of UserDetails class with arguments
UserDetail ud1 = new UserDetails(1,"Ram","Attariya","ram@gmail.com");
String fileName = "./user_registration.txt"; //file name with location

try {
    //open file in writing mode , byte-oriented
    OutputStream os = new FileOutputStream(fileName);
    //to stream object to file
    ObjectOutputStream objs = new ObjectOutputStream(os);
    //serialize object using writeObject() method
    objs.writeObject(ud1);
    objs.flush();
    objs.close();
    os.close();

    System.out.println("Object written successfully to "+ fileName);
} catch (IOException e) {
    System.out.println("Exception occur: "+e.getMessage());
}
}
}

```

Example 2: We are going to de-serialize Object of class *UserDetail* . The *readObject()* method of *ObjectInputStream* class provides serialization. Reading data of user detail using *UserDetail* class object.

Note: Program file(*ReadUserRegistration.java*) and file (*user_registration.txt*) are in same folder/directory.

/*ReadUserRegistration.java*/

```

import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

public class ReadUserRegistration {
    public static void main(String[] args) throws ClassNotFoundException {
        String fileName = "./user_registration.txt";

        try {
            //read file in byte oriented
            FileInputStream fin = new FileInputStream(fileName);
            //to read object through byte-stream
            ObjectInputStream ois = new ObjectInputStream(fin);
            //de-serialize using readObject()
            UserDetails readUser = (UserDetails)ois.readObject();

            System.out.println("ID:\t\t\tName:\t\t\tAddress:\t\t\tEmail:");
            System.out.println(readUser.id+"\t\t\t"+readUser.userName+"\t\t\t"+readUser.userAddress+"\t\t\t"+readUser.userEmail);
        }
    }
}

```

```

        ois.close(); //closing object streaming
        fin.close(); // closing file
    } catch (IOException e) {
        System.out.println("Exception: "+e.getMessage());
    }
}
}

```

- Static data member in a class will not be serialized because static is the part of class not object.
static String user_order_item = "Pant"; //won't be serialized
- Array or collection objects must be serializable. If any object is not serializable, serialization will be failed.
- If you don't want to serialize any data member of a class, you can mark it as transient.
transient int id;
- The serialization process at runtime associates an id with each Serializable class which is known as serialVersionUID. Used to verify the sender and receiver of the serialized object.
private static final long serialVersionUID = 1234132441L;

File Handling:

File class from **java.io** package is an abstract representation of file and directory pathname. File handling in java implies reading from and writing data to a file. To use **File** class, you need to create an object of the class and specify the filename or directory name.

Syntax:

File objName = new File("file_name.extension");

Note: Java uses the concept of a stream for I/O operation on a file , which we discussed on above topics(**Stream I/O**).

Java File Methods:

Method	Type	Description
canRead()	Boolean	It tests whether the file is readable or not
canWrite()	Boolean	It tests whether the file is writable or not
createNewFile()	Boolean	This method creates an empty file
delete()	Boolean	Deletes a file
exists()	Boolean	It tests whether the file exists
getName()	String	Returns the name of the file
getAbsolutePath()	String	Returns the absolute pathname of the file
length()	Long	Returns the size of the file in bytes
list()	String[]	Returns an array of the files in the directory
mkdir()	Boolean	Creates a directory

File Operations in Java:

1. Create a File – it create file and return true if file was successfully created , and false if the file already exists.

Example :

/*FileHandling.java*/

```
import java.io.File;
import java.io.IOException;
public class FileHandling {
    public static void main(String[] args){
        String path = "";
        try{
            File file = new File("./FileTo.txt");
            if(file.createNewFile()){
                System.out.println("New File created.");
            }else{
                System.out.println("Already exists.");
            }
        }catch(IOException e){
            e.printStackTrace();
        }
    }
}
```

2. Get File Information – Lets use various methods to get file information.

Example:

/*FileHandling.java*/

```
import java.io.File;
public class FileHandling {
    public static void main(String[] args){
        try{
```



```

        while(scanData.hasNextLine()){
            String fileContent = scanData.nextLine();
            System.out.println(fileContent);
        }
        scanData.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
}
/*FileTo.txt*/

```

Hello world. Welcome to Java Class.
 What's your interested subject?
 Let's discuss informally .

5. Delete file: delete() method in order to delete a file.

Example:

```

import java.io.File;
public class FileHandling {
    public static void main(String[] args){
        File file = new File("./FileTo.txt");
        if(file.delete()){    //delete file if exists
            System.out.println(file.getName()+" deleted.");
        }else{
            System.out.println("Failed to delete file!!!");
        }
    }
}
}

```

Note: Run given programs to observe output and try to understand problem. Also , all above programs are written in basic text editor and run through command prompt i.e. without using any IDEs.

Assignment:

1. Error Handling

Chapter 4: Applet and Application

Some Old Questions:

1. What is an Applet ? Explain life cycle of Applet.
2. State the life cycle of an applet. Can it be changed into application? If yes, justify your answer.
3. Discuss the tags used in applet.
4. Define Applet and explain its lifecycle . Create a simple Applet and demonstrate how we can run a Java Applet.
5. Create an applet that plays a sample audio file. Also create a GUI that allows users to play, stop or replay the audio. Add a functionality to automatically stop the audio if the applet is inactive.
6. Discuss about the Sandbox Security model with regard to Java Applet.
7. Explain how to display a picture in an applet.
8. How can a HTML file pass data to an applet? Explain with relevant code tags.
9. Write an applet program with three **textfields** with the following names: “number1”, “number2” and “result”. When the user clicks a “sum” button then the sum of the two numbers in “number1” and “number2” should be displayed in the text field named “result”.
10. Write an Applet program to illustrate the message sharing between HTML and Applets.
11. Explain Applet architecture . How can you convert applet to application? Explain with suitable example.
12. How is applet different from normal java program? List out the steps for converting applet into application.
13. What does security restriction means in applet ? List out the attributes of applet tag and explain them.

Introduction:

Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side. However, almost all browser support only JavaScript for dynamic content interaction, which means Java Applet deprecated and not supported. **Java applet** were introduced in the first version of the Java language, released in 1995.

Java Applet is a parasite program that need host to be executed, which means depends on other program to run. Also, applet is not standalone application that means applet doesn't have main method to execute.

An applet must be a subclass of the *java.applet.Applet* class. The Applet class provides the standard interface between the applet and the browser environment. In addition, *java.applet* package also defines three interfaces: *AppletContext*, *AudioClip*, and *AppletStub*.

There are two ways to run an applet:

- i. By html file.
- ii. By appletViewer tool(for testing purpose)

Two types of Applets:

The first are those based directly on the Applet class. Use the abstract Window Toolkit(AWT) to provide the graphic user interface(or use no GUI at all). This style of applet has been available since Java was first created.

The second type of applets are those based on the Swing class JApplet. Uses Swing classes to provide the GUI. Swing offers a richer and often easier-to-use user interface than does the AWT. Thus, Swing-based applets are now the most popular. JApplet inherits Applet, all the features of Applet are also available in JApplet

Advantages :

- Works at client side so less response time.
- Secured
- It can be executed by browsers running under many platforms(i.e. platform independent).
- Applets creates and edit graphics on client-side.
- Provide functionality to import resources such as images, audio clips based on Url's.

Disadvantages:

- Plugin is required at client browser to execute applet.
- Applet itself cannot run or modify any application on the local system.
- Applets has no access to client-side resources such as files, OS etc

Applet Architecture:

Java provides both console based programming language environment and window based programming environment. Where, an applet is a window-based program. So, its architecture is different from the so-called normal, console-based programs.

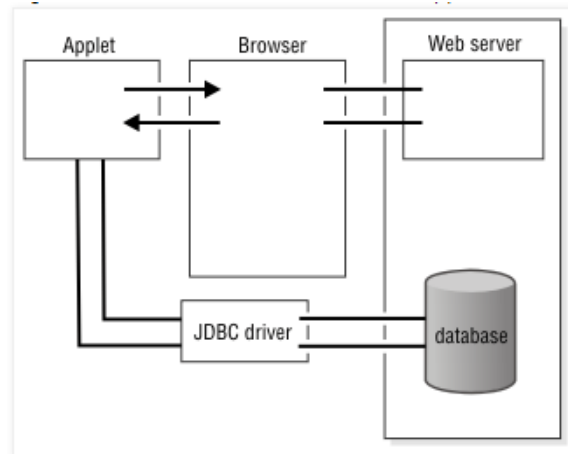
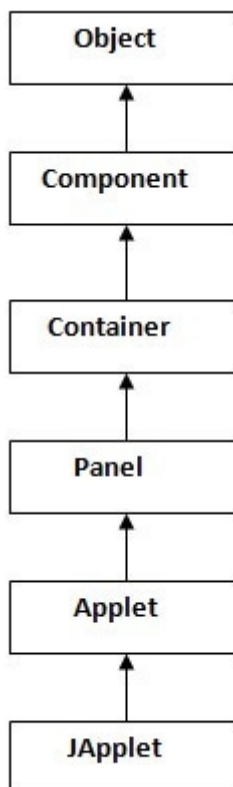


Figure: Runtime architecture for a Java Applet

First, applets are event driven; applets wait until an event occurs. The run-time system notifies the applet about an event by calling an event handler that has been provided by the applet. The user initiates interaction with an applet; these interactions are sent to the applet as events to which the applet must respond.

The database server is connected to the JDBC driver, which is connected to the applet. The applet is also connected to a browser, which is connected to a web server that communicates with the database. So explain architecture of Java Applet from above figure.

Hierarchy of Applet:



Lifecycle of Applet:

- i. Applet is initialized.
- ii. Applet is started.
- iii. Applet is painted.
- iv. Applet is stopped.
- v. Applet is destroyed.

There are 4 lifecycle methods(*init()*, *start()*, *stop()*, *destroy()*) provided by *java.applet.Applet* class and 1 lifecycle method(*paint(Graphics g)*) by *java.awt.Component* class .

- i. **public void init():** Used to initialize the Applet. Invoked only once.
- ii. **public void start():** Invoked after the init() method or browser is maximized. It is used to start/resume the Applet.
- iii. **public void stop():** Used to stop the Applet. It is invoked when Applet is stop or browser is minimized. Suspends execution.
- iv. **public void destroy():** Used to destroy the Applet. It is invoked only once. Called when applet is terminated, and is the last method. Perform shutdown activities.
- v. **public void paint(Graphics g):** Used to paint the Applet. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.

Example 1: Simple applet example with applet lifecycle methods , executed by html file.

Create an applet and compile it. After that create an html file and place the applet code in html file. Now click the html file.

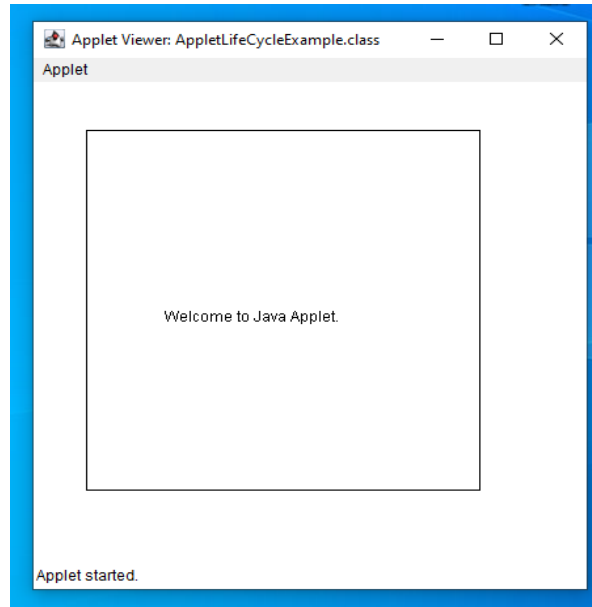
```
/*AppletLifeCycleExample.java*/
import java.applet.Applet;
import java.awt.Graphics;
public class AppletLifeCycleExample extends Applet {
    public void init(){
        System.out.println("Applet initialized.....");
    }
    public void start(){
        System.out.println("Applet Started.....");
    }
    public void stop(){
        System.out.println("Applet Stopped.....");
    }
    public void destroy(){
        System.out.println("Applet Destroyed.....");
    }
    public void paint(Graphics g){
        g.drawRect(40, 40, 300, 300);
        g.drawString("Welcome to Java Applet.", 100, 200);
    }
}
/*ByHtmlFile.html*/
<html>
    <body>
```

```

        <Applet code="AppletLifeCycleExample.class" width="400" height="400" name="app1">
        </Applet>
    </body>
</html>

```

Output:



Example 2: Simple applet example with applet lifecycle methods, executed by appletviewer tool.

```

/*AppletLifeCycleExample.java*/
import java.applet.Applet;
import java.awt.Graphics;
public class AppletLifeCycleExample extends Applet {
    public void init(){
        System.out.println("Applet initialized.....");
    }
    public void start(){
        System.out.println("Applet Started.....");
    }
    public void stop(){
        System.out.println("Applet Stopped.....");
    }
    public void destroy(){
        System.out.println("Applet Destroyed.....");
    }
    public void paint(Graphics g){
        g.drawRect(40, 40, 300, 300);
        g.drawString("Welcome to Java Applet.", 100, 200);
    }
}
/*
    <Applet code="AppletLifeCycleExample.class" width="400" height="400" name="app1">

```

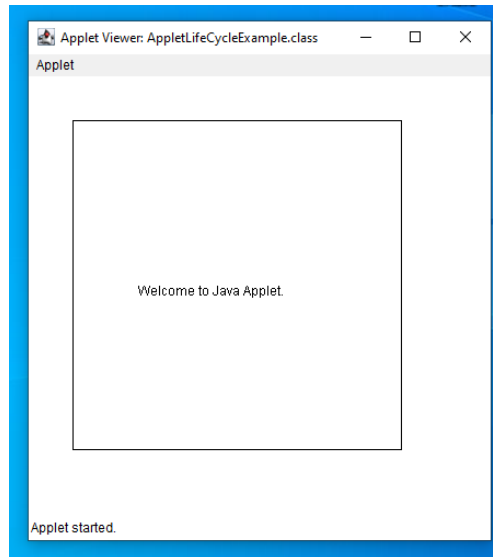
```
</Applet>
*/
```

To execute:

```
C:\Users\Dell\Desktop\Programming in Java\projects\Applets_Program> javac AppletLifeCycleExample.java
```

```
C:\Users\Dell\Desktop\Programming in Java\projects\Applets_Program> appletviewer AppletLifeCycleExample.java
```

Output:



Displaying Graphics in Applet:

java.awt.Graphics class provides different graphical methods for graphic programming in java.

1. **public abstract void drawString(String str, int x, int y):** is used to draw the specified string. Where x and y be the starting point of string to be painted on the panel.
2. **public void drawRect(int x, int y, int width, int height):** draws a rectangle with the specified width and height.
3. **public abstract void fillRect(int x, int y, int width, int height):** is used to fill rectangle with the default color and specified width and height.
4. **public abstract void drawOval(int x, int y, int width, int height):** is used to draw oval with the specified width and height.
5. **public abstract void fillOval(int x, int y, int width, int height):** is used to fill oval with the default color and specified width and height.
6. **public abstract void drawLine(int x1, int y1, int x2, int y2):** is used to draw line between the points(x1, y1) and (x2, y2).
7. **public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer):** is used draw the specified image.
8. **public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used draw a circular or elliptical arc.

9. **public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used to fill a circular or elliptical arc.
10. **public abstract void setColor(Color c):** is used to set the graphics current color to the specified color.
11. **public abstract void setFont(Font font):** is used to set the graphics current font to the specified font.

Example 3: Some drawing using Graphics methods.

```

/*SomeDrawing.java*/
import java.applet.Applet;
import java.awt.Image;
import java.awt.Graphics;
import java.awt.Color;
public class SomeDrawing extends Applet{
    Image img;
    public void init(){
        img = getImage(getCodeBase(),"img1.jpg");
        System.out.println("Image loaded....");
    }
    public void paint(Graphics g){
        /*paint an image on applet panel,60px from left,100px from top with
        300px width and 260px height */
        g.drawImage(img, 60, 100,300,260, this);
        //Let's draw smiley face
        //g.drawOval(460, 100, 200, 200); //face(uncomment if need border drawing)
        g.setColor(new Color(255,172,150)); // filled face
        g.fillOval(460, 100, 200, 200);

        g.setColor(Color.gray);
        //g.drawOval(490, 156, 40, 22); //left eye(uncomment if need border drawing)
        g.fillOval(490, 156, 40, 22); //filled eye

        //g.drawOval(584, 156, 40, 22); // right eye(uncomment if need border drawing)
        g.fillOval(584, 156, 40, 22); //filled eye

        g.setColor(Color.black);
        g.drawLine(560, 166, 560, 222); // nose
        g.drawArc(524, 240, 70, 20, 0, -180); //mouth

        //Give them heading
        g.drawString("We draw Image",60,80); //heading for image
        g.drawString("This is Smiley face.",470,80); // heading for smiley face
    }
}
/*ByHtmlFile.html*/
<html>

```

```

<body>
    <Applet code="SomeDrawing.class" width="800" height="600" name="app1">
    </Applet>
</body>
</html>

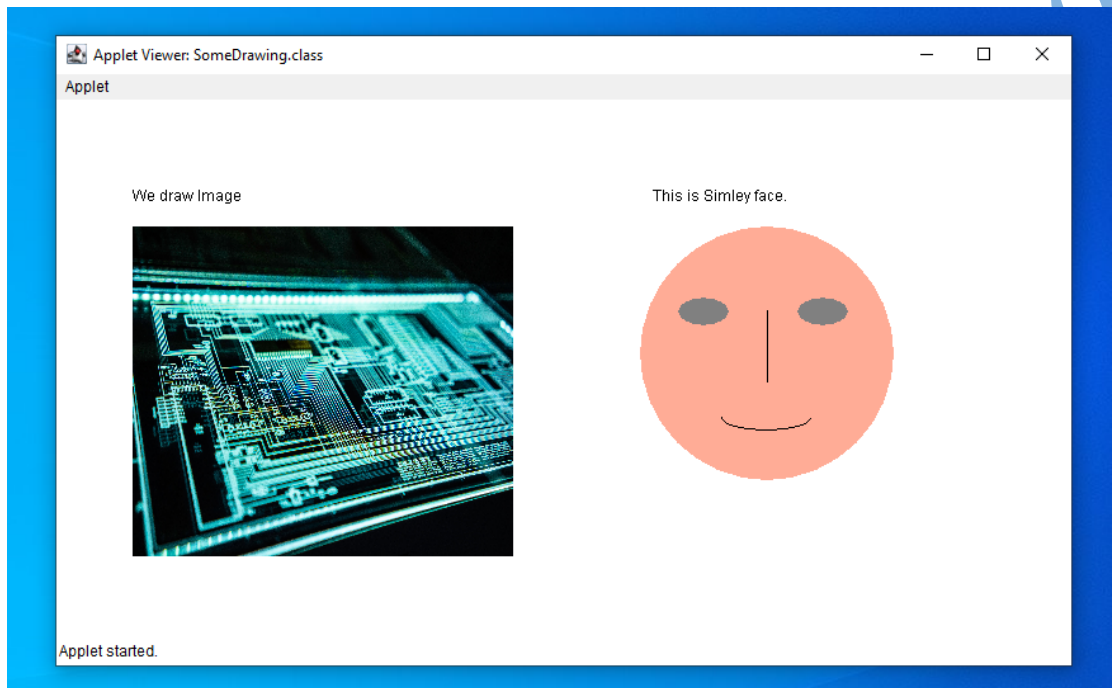
```

To execute :

C:\Users\Dell\Desktop\Programming in Java\projects\Applets_Program> javac SomeDrawing.java

C:\Users\Dell\Desktop\Programming in Java\projects\Applets_Program> appletviewer ByHtmlFile.html

Output:



Repaint() method / requesting repainting:

When applet needs to update the information displayed in its window / also refreshes the view(component) , it simply calls **repaint()**. The **repaint()** method is defined by the AWT. It causes the AWT run-time system to execute a call to your applet's **update()** method, which, in its default implementation; calls **paint()**.

The simplest versions of repaint() are shown below:

- i. ***void repaint()*** – causes the entire window to be repainted.
- ii. ***void repaint(int left, int top, int width, int height)*** – specifies a region that will be repainted. The coordinates of the upper-left corner of the region are specified by left and top, and the width and height of the region are passed in width and height. Dimensions are specified in pixels.
- iii. ***Void repaint(long maxDelay)*** – *maxDelay* specifies the maximum number of milliseconds that can elapse before update() is called. Repaint entire window after some delay.

- iv. *Void repaint(long maxDelay, int x, int y, int width, int height)* – repaint defined region after some delay.

The HTML APPLET Tag:

```
< APPLET
  [CODEBASE = codebaseURL]
  CODE = appletFile
  [ALT = alternateText]
  [NAME = appletInstanceName]
  WIDTH = pixels
  HEIGHT = pixels
  [ALIGN = alignment]
  [VSPACE = pixels]
  [HSPACE = pixels]
>
  [< PARAM NAME = AttributeName VALUE = AttributeValue>]
  [< PARAM NAME = AttributeName2 VALUE = AttributeValue>]
  ...
</APPLET>
```

- **CODEBASE** – is an optional attribute that specifies the base URL of the applet code.
- **CODE** – required attribute that gives the name of the file containing your compiled applet (.class) file. This file is relative to the code base URL of the applet.
- **ALT** – is an optional attribute used to specify a short alternative text message that should be displayed if the browser understands the APPLET tag but can't currently run Java applets.
- **NAME** – optional attribute used to specify a name for the applet instance. Which helps to order applets on the same page and communicate with them.
- **WIDTH and HEIGHT** – required attributes that give the size(in pixels) of the applet display area.
- **ALIGN** – optional attributes that specifies the alignment of the applet. Possible value(*LEFT, RIGHT, TOP, BOTTOM, MIDDLE, BASELINE, TEXTTOP, ABSMIDDLE, and ABSBOTTOM*).
- **VSPACE and HSPACE** – are optional where VSPACE specifies above and below space(in pixels) to applet and HSPACE specifies space(in pixels) on each side of the applet.
- **PARAM NAME and VALUE** – The PARAM tag specify applet arguments in an HTML page. Applets access their attributes with the *getParameter()*.

Example 4: Passing Parameters to Applets

```
/*ParameterToApplet.java*/
```

```
import java.applet.Applet;
```

```

import java.awt.Color;
import java.awt.Graphics;
public class ParameterToApplet extends Applet {
    int f_num,s_num;

    public void init(){
        setBackground(Color.yellow);
        setForeground(Color.red);
        System.out.println("Applet initialized .....");
    }
    public void start(){
        f_num = Integer.parseInt(getParameter("first-number"));
        s_num = Integer.parseInt(getParameter("second-number"));
        System.out.println(f_num+s_num);
        repaint();
    }
    public void paint(Graphics g){
        g.drawString("The sum of numbers from html as parameter is "+(f_num+s_num), 10, 100);
    }
}
/*ByHtmlFile.html*/
<html>
    <body>
        <Applet code="ParameterToApplet.class" width="300" height="260" name="app1">
            <param name="first-number" value="10">
            <param name="second-number" value="20">
        </Applet>
    </body>
</html>

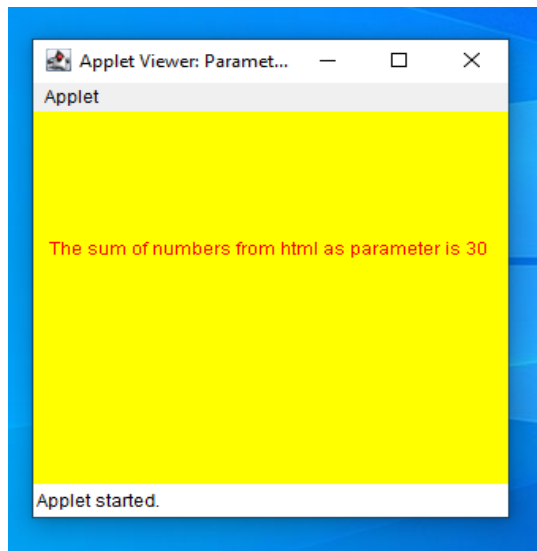
```

To execute:

C:\Users\Dell\Desktop\Programming in Java\projects\Applets_Program> javac ParameterToApplet.java

C:\Users\Dell\Desktop\Programming in Java\projects\Applets_Program> appletviewer ByHtmlFile.html

Output:



getDocumentBase() and getCodeBase():

getDocumentBase() return path of HTML file directory , and *getCodeBase()* return path of applet's class file. These directories are returned as URL objects by *getDocumentBase()* and *getCodeBase()*.

Example 5:

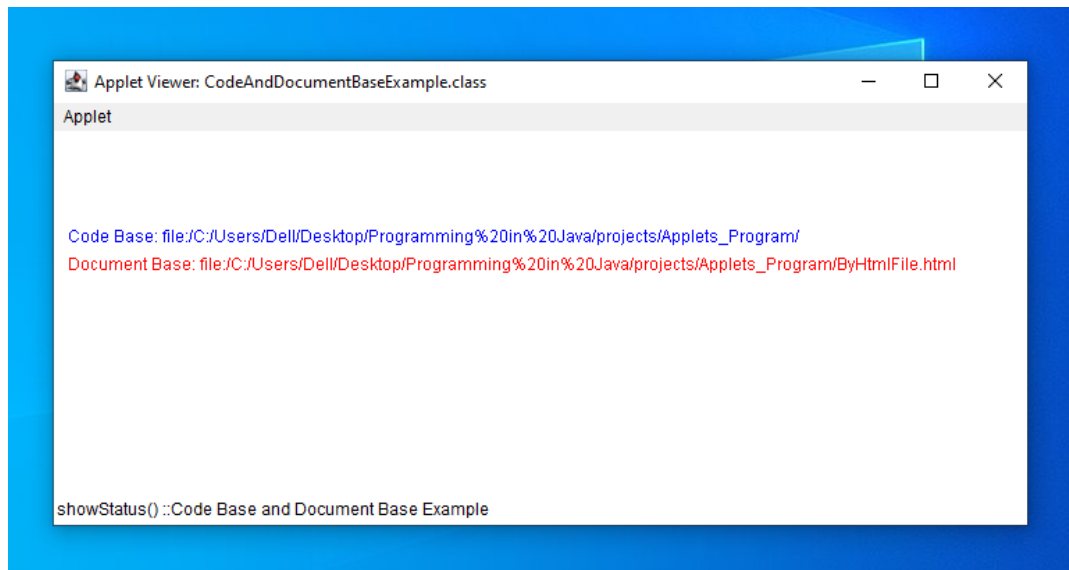
```
/* CodeAndDocumentBaseExample.java */
import java.applet.Applet;
import java.awt.Color;
import java.awt.Graphics;
import java.net.URL;
public class CodeAndDocumentBaseExample extends Applet {
    public void paint(Graphics g){
        g.setColor(Color.blue);
        URL url1 = getCodeBase();
        String baseCode = url1.toString();
        g.drawString("Code Base: "+baseCode, 10, 80);

        g.setColor(Color.red);
        URL url2 = getDocumentBase();
        String documentCode = url2.toString();
        g.drawString("Document Base: "+documentCode, 10, 100);
        showStatus("showStatus() ::Code Base and Document Base Example");
    }
}
```

/*ByHtmlFile.html*/

```
<html>
<body>
    <Applet code=" CodeAndDocumentBaseExample.class" width="700" height="260" name="app1">
    </Applet>
</body>
</html>
```

Output:



showStatus():

To display additional information in its window, applet uses **showStatus()** method. This outputs a message to the status window of the browser or applet viewer on which it is running. See example 5.

Communication between Applets:

To communicate or to transfer control from one to another applet, you must use *AppletContext* interface.

Some Methods :

- **getCodeBase()** –
- **getDocumentBase()** –
- **getAudioClip(URL url)** – loads the audio file located at url.
- **getImage(URL url)** – **getImage()** method loads the image file located at url. url must be a complete and valid URL. A call to **prepareImage()**, **MediaTracker**, or **drawImage()** forces loading to start.
- **getApplet()** / **getApplets()** – **getApplet()** method fetches the Applet from the current HTML page named name, can be applet's class name or the name provided in name attribute. Where **getApplets()** method gets all available applets.
- **showDocument()** – Where **showDocument()** method is used to transfer control. Shows url in a browser window specified by frame.

Syntax:

```
try {
    URL u = new URL (getDocumentBase(), (String) file);
    getAppletContext().showDocument (u, "_blank");
} catch (Exception e) {
}
```

Target String	Results
_blank	Show <code>url</code> new browser window with no name.
_parent	Show <code>url</code> in the parent frame of the current window.
_self	Replace current <code>url</code> with <code>url</code> (i.e., display in the current window).
_top	Show <code>url</code> in top-most frame.
name	Show <code>url</code> in new browser window named <code>name</code> .

Example 6: Communication between *MyAppletOne* and *MyAppletTwo*. Where draw image from *MyAppletOne* to *MyAppletTwo*. Change *MyAppletOne* background color to *yellow* on click of “*Change to MyAppletOne*” button from *MyAppletTwo*. Also change *MyAppletTwo* background color to *red* on click of “*Change to MyAppletTwo*” button from *MyAppletOne*.

```

/*MyAppletOne.java*/
import java.applet.Applet;
import java.applet.AppletContext;
import java.awt.Button;
import java.awt.Color;
import java.awt.event.*;
public class MyAppletOne extends Applet implements ActionListener{
    Button change;
    AppletContext acontext; //instance variable
    public void init(){
        change = new Button("Change to MyAppletTwo");
        change.setBounds(120,120,140,40);
        add(change);
        change.addActionListener(this);
        System.out.println("MyAppletOne initialized.....");
    }
    //action performed when button is clicked
    public void actionPerformed(ActionEvent ae){ //event handling(within the class)
        acontext = getAppletContext(); // returns the current AppletContext
        Applet aplt = acontext.getApplet("app2"); //return specified applet
        aplt.setBackground(Color.red); //set background color to MyAppletTwo
    }
    public void start(){
        System.out.println("MyAppletOne started.....");
    }
    public void stop(){
        System.out.println("MyAppletOne stopped.....");
    }
}

```

```

    public void destroy(){
        System.out.println("MyAppletTwo destroyed.....");
    }
}
/*MyAppletTwo.java*/
import java.applet.Applet;
import java.applet.AppletContext;
import java.awt.Image;
import java.net.URL;
import java.awt.Graphics;
import java.awt.Button;
import java.awt.Color;
import java.awt.event.*;
public class MyAppletTwo extends Applet {
    Button change;
    Image img;
    AppletContext acontext;
    public void init(){
        acontext = getAppletContext();
        Applet aplt = acontext.getApplet("app1");
        String getImg = aplt.getParameter("send_image"); //get parameter passed
        try {
            URL url = new URL(this.getCodeBase(),getImg); //get actual url of image
            img = acontext.getImage(url); //get image and assign to Image instance variable
        } catch (Exception e) {
            System.out.println("File not found."+e);
        }
        System.out.println("MyAppletTwo initialized.....");
    }

    public void start(){
        change = new Button("Change to MyAppletOne");
        change.setBounds(120,120,140,40);
        add(change);
        change.addActionListener(new ActionListener(){ //event handling (using anonymous class)
            public void actionPerformed(ActionEvent ae){
                acontext = getAppletContext();
                Applet aplt = acontext.getApplet("app1");
                aplt.setBackground(Color.yellow);
            }
        });
        System.out.println("MyAppletTwo started.....");
    }

    public void stop(){
        System.out.println("MyAppletTwo stopped.....");
    }

    public void destroy(){
        System.out.println("MyAppletTwo destroyed.....");
    }
}

```



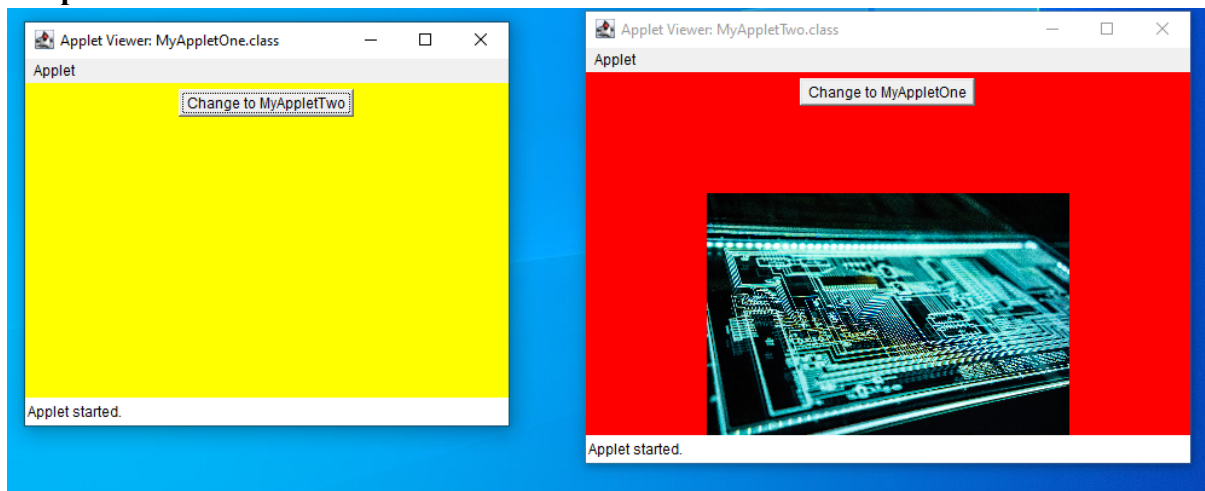
```

    }

    public void paint(Graphics g){
        g.drawImage(img, 100, 100,300,200, this);
    }
}
/*ByHtmlFile.html*/
<html>
<body>
    <Applet code="MyAppletOne.class" width="400" height="260" name="app1">
        <param name="send_image" value="img1.jpg">
    </Applet>
    <Applet code="MyAppletTwo.class" width="500" height="300" name="app2">
    </Applet>
</body>
</html>

```

Output:



Example 7: Create an applet that plays a sample audio file. Also create a GUI that allows users to play, stop or replay the audio. Add a functionality to automatically stop the audio if the applet is inactive. **Note:** Java media Framework API is rumored to support only **.au** , **.wav**, and **.aiff** audio files.

```

import java.applet.Applet;
import java.applet.AudioClip;
import java.awt.event.*;
import java.awt.*;

public class audioApplet extends Applet implements ActionListener{
    AudioClip audio1;
    Button playBtn,repeatBtn,stopBtn;
    public void init(){
        System.out.println("Initialzing an applet");
        audio1 = this.getAudioClip(this.getCodeBase(),"sample.au");
    }
}

```

```

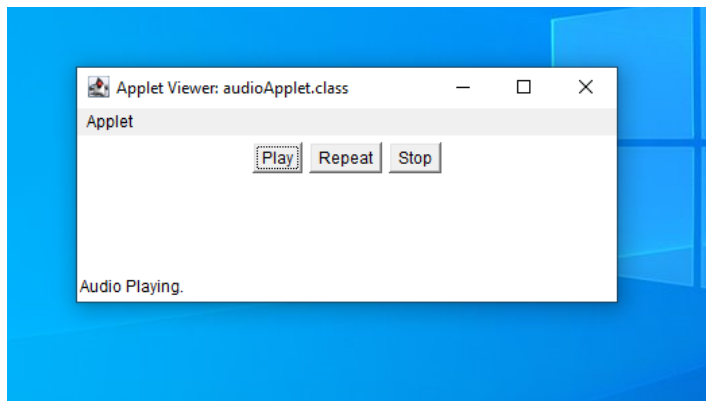
playBtn = new Button("Play");
add(playBtn);
playBtn.addActionListener(this);

repeatBtn = new Button("Repeat");
add(repeatBtn);
repeatBtn.addActionListener(this);

stopBtn = new Button("Stop");
add(stopBtn);
stopBtn.addActionListener(this);
}
public void start(){
    showStatus("Audio mode status here .");
}
public void actionPerformed(ActionEvent ae){
    if(ae.getActionCommand().equals("Play")){
        audio1.play();
        showStatus("Audio Playing.");
    }
    if(ae.getActionCommand().equals("Repeat")){
        audio1.loop();
        showStatus("Audio Repeat Mode.");
    }
    if(ae.getActionCommand().equals("Stop")){
        audio1.stop();
        showStatus("Audio Stopped");
    }
}
public void stop(){
    audio1.stop();
    System.out.println("Audio paused on inactive....");
}
}
/*ByHtmlFile.html*/
<html>
<body>
<Applet code="audioApplet.class" width="400" height="100" name="app2">
</Applet>
</body>
</html>

```

Output:



Example 8: Write an applet program with three **textfields** with the following names: “number1”, “number2” and “result”. When the user clicks a “sum” button then the sum of the two numbers in “number1” and “number2” should be displayed in the text field named “result”.

/ SumTwoNumber .java*/*

```
import java.applet.Applet;
import java.awt.Button;
import java.awt.TextField;
import java.awt.Label;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class SumTwoNumber extends Applet{
    Label fLabel,sLabel,total;
    Button SumBtn;
    TextField f_tf,s_tf,result;
    int f_num, s_num, sum;
    public void init() {

        f_tf= new TextField("number 1");
        f_tf.setBounds(40,50,120,30);

        s_tf= new TextField("number 2");
        s_tf.setBounds(40,110,120,30);

        SumBtn= new Button("SUM");
        SumBtn.setBounds(40,160,80,30);
        SumBtn.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent e) {
                f_num= Integer.parseInt(f_tf.getText());
                s_num = Integer.parseInt(s_tf.getText());
                sum = f_num + s_num;
                result.setText(String.valueOf(sum));
                showStatus("Sum of Two numbers.");
            }
        });
    }
}
```

```

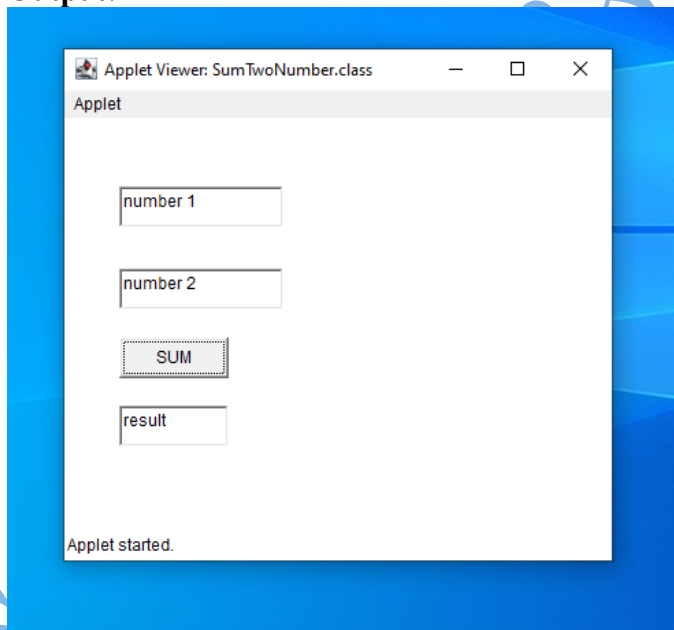
        result = new TextField("result");
        result.setBounds(40,210,80,30);

        add(SumBtn);
        add(f_tf);
        add(s_tf);
        add(result);

        setLayout(null);
    }
}
/*ByHtmlFile.html*/
<html>
    <body>
        <Applet code="SumTwoNumber.class" width="400" height="300" name="app2">
        </Applet>
    </body>
</html>

```

Output:



Example 9: Draw Nepal National Flag using applet.

```

/*NationalFlagUsingApplet.java*/
import java.applet.Applet;
import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;
public class NationalFlagUsingApplet extends Applet{

```

```

public void init() {
    System.out.println("Painting started....");
}

public void paint(Graphics g){
    g.drawRect(10,10,600,620);
    g.fillRect(40,20,10,600);

    int[] xPoints = {50,300,90,340,50,50};
    int[] yPoints = {20,240,240,500,500,20};
    g.setColor(Color.blue);
    g.fillPolygon(xPoints,yPoints,5);

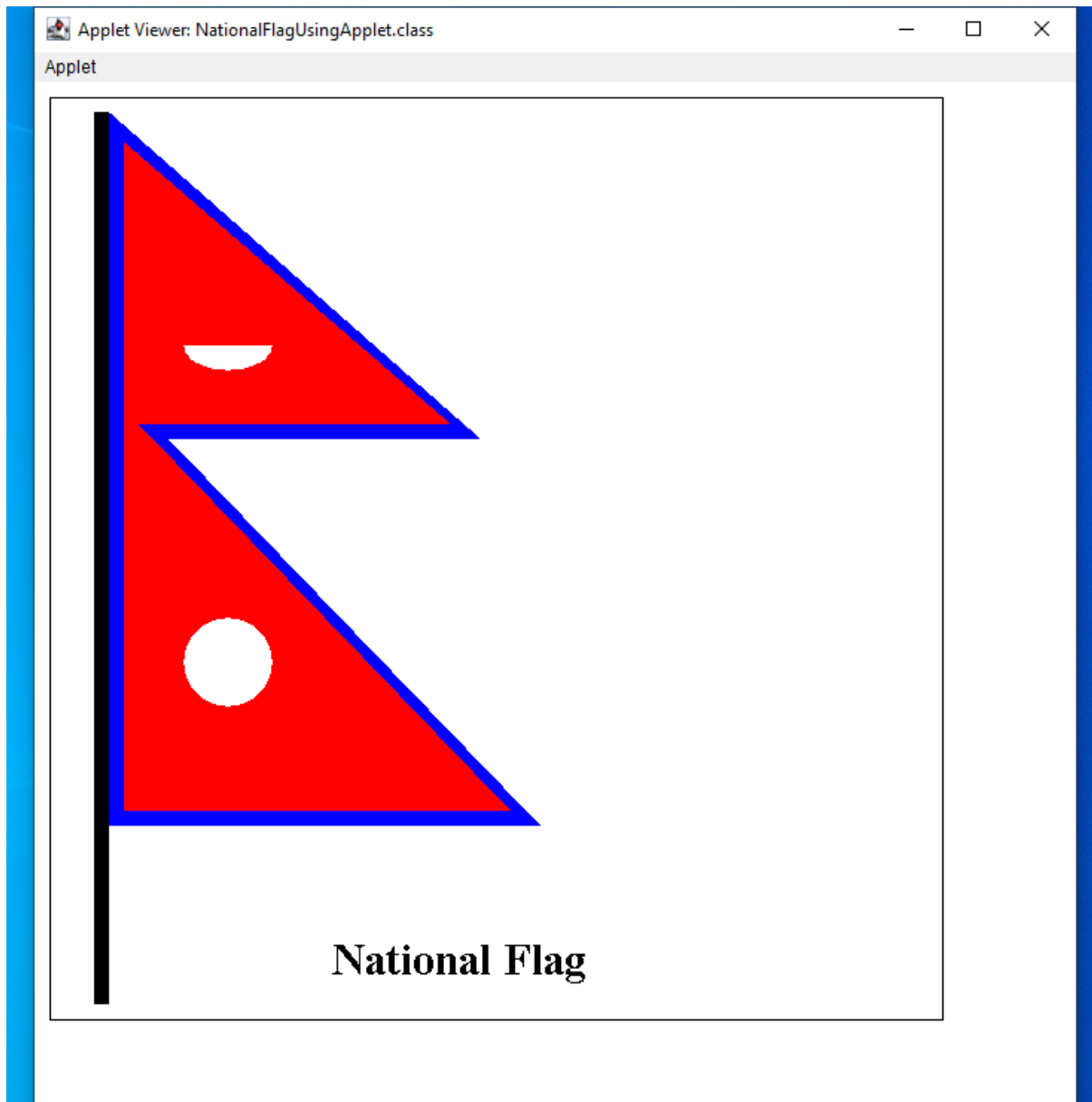
    int[] xP = {60,280,70,320,60,60};
    int[] yP = {40,230,230,490,490,40};
    g.setColor(Color.red);
    g.fillPolygon(xP,yP,5);

    g.setColor(Color.white);
    g.fillArc(100,160,60,34,0,-180);
    g.fillArc(100,360,60,60,0,360);

    g.setColor(Color.BLACK);
    g.setFont(new Font("serif",Font.BOLD,30));
    g.drawString("National Flag",200,600);
}
}
/*ByHtmlFile.html*/
<html>
    <body>
        <Applet code="NationalFlagUsingApplet.class" width="700" height="800" name="app2">
        </Applet>
    </body>
</html>

```

Output:



SandBox Security Model:

This is a security measure in the Java development environment. The sandbox is a set of rules that are used when creating an applet. It would be dangerous to allow applets, that are downloaded from the internet, to read and write files on client computer or access other system resources. In that case Java platform uses the sandbox security model .

The idea behind this model is that when you allow a program to be hosted on your computer, and you want to provide an environment where the program run, but confined within certain bounds . Which means you decide to let it have access to certain system resources, but in general, you want to make sure that the program is confined to its sandbox.

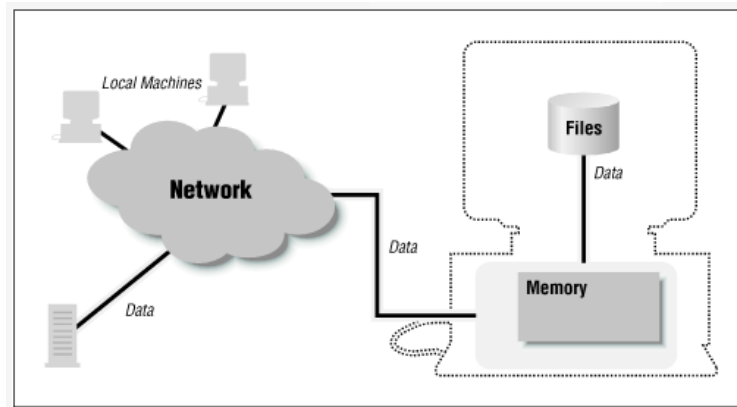


Figure: A machine has access to many resources

As seen on above figure , machine has access to many resources and they must need to be protected, and those protections form the basis of Java's security model.

Imagine a number of different-sized sandboxes in which a java program might run:

- A sandbox in which the program has access to the CPU, the screen, keyboard, and mouse, and to its own memory. This is the minimal sandbox—it contains just enough resources for a program to run.
- A sandbox in which the program has access to the CPU and its own memory as well as access to the web server from which it was loaded. Default state for the sandbox.
- A sandbox in which the program has access to the CPU, its memory, its web server, and to a set of program-specific resources (local files, local machines, etc.). A word-processing program, for example, might have access to the *docs* directory on the local filesystem, but not to any other files.
- An open sandbox, in which the program has access to whatever resources the host machine normally has access to.

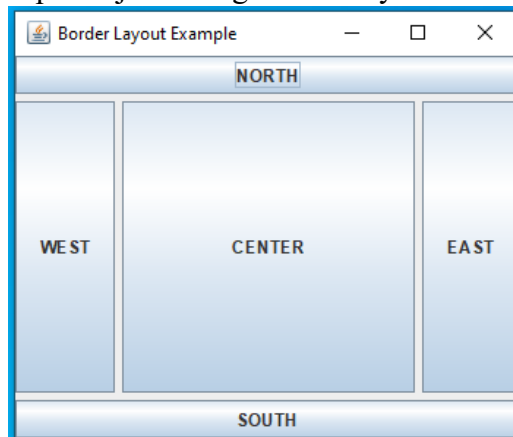
Note: Every example in this document are run under *jdk 8* version with the help of *appletviewer* tool.

Solve remaining questions yourself !!!

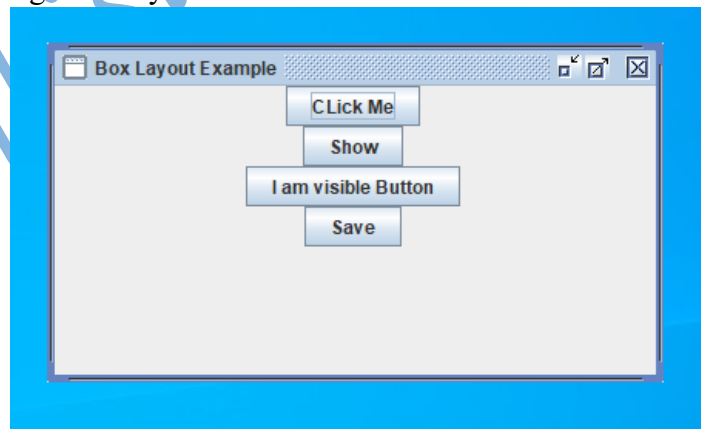
Chapter 5: Event , Handling Events and AWT and Swing

Some Old Questions:

1. What are the differences between GridLayout and GridBagLayout ? Explain FlowLayout.
2. Create a swing GUI that contains a button , and two text fields. When the button is clicked the value of first text field should be checked and display “odd number” or “even number” in the second text field.
3. What is layout manager ? State the different types of layout manager in swing. WAP to generate the following output in java using BorderLayout.



4. Create a swing GUI which contains a text field(to input radius of circle), a label and a button. When the button is clicked area of circle should be calculated and displayed in the label.
5. Create a Frame with two Text Fields , one of which should show whether the mouse pointer is inside or outside the Frame and the other text field should display the X and Y co-ordinates of the pointer when the use moves the mouse pointer inside the Frame area.
6. State the different types of layout manager is swing . WAP to generate the following output in java using Box Layout.



7. What do you mean by delegation event model? Explain various event listener interfaces with an example.
8. What advantage do java's layout managers provide over traditional windowing systems? How do you create menu in Swing ? Explain with an example.
9. What are heavyweight components in Java? Are there any alternatives to those heavyweight components? Provide a comparative illustration.

10. What is the difference between AWT and Swing package in Java? Write a sample code for creating a closable frame in the front and center of your desktop.
11. Differentiate between Panel and Frame. Differentiate between Frame and Swing.
12. Write a program to generate a Frame with two buttons “BLACK” and “BLUE”. When a button is clicked it should change the background color of the Frame to respective color.
13. What is Frame? Create a frame with following attributes: Height=400, Width=400, and Title=My Frame.
14. Create a Frame with one button and one textfield, when user clicks on the button the text entered on textfield should be changed to uppercase.
15. Explain Event Handling with various types of event available on java.
16. What is GUI programming? Write a program to show the use of BorderLayout.
17. Create a Frame which has three textfield and one button. When user clicks on button it should calculate sum of the value of first and second textfield and display on third textfield.
18. How can you create closable frames in SWING and AWT? Write a program to draw a bar chart.
19. Explain the Layout Manager in detail. Write a program to change the font of text in textfield of the Frame. The frame contains 3 checkboxes named bold, italic and plain.
20. Write a simple GUI addition application that used two input dialogs to obtain integers from the user and a message dialog to display the sum.
21. Short Notes:
 - i. Events and event classes
 - ii. Different event handling mechanisms
 - iii. Dialog Box
 - iv. Closable JFrame
 - v. Frame Layout
 - vi. Layout Management

AWT(Abstract Window Toolkit):

- AWT is an API to develop Graphical User Interface(GUI) or Windows-based applications in Java.
- AWT components are platform-dependent.
- AWT is heavy weight i.e. components are using the resources of underlying operating system.
- AWT have less components than SWING.
- The **java.awt** package provides classes for AWT components(TextField, Label , TextArea,...).
- Also **java.awt** package provides classes for Color, Graphics, Font, Event ,Layouts etc .
- Define components like **Button**, **TextField**, **Label**, **TextArea**, **CheckBbox** , **List** etc.
- AWT doesn't support pluggable look and feel.
- AWT doesn't follows MVC patterns.

AWT Hierarchy:

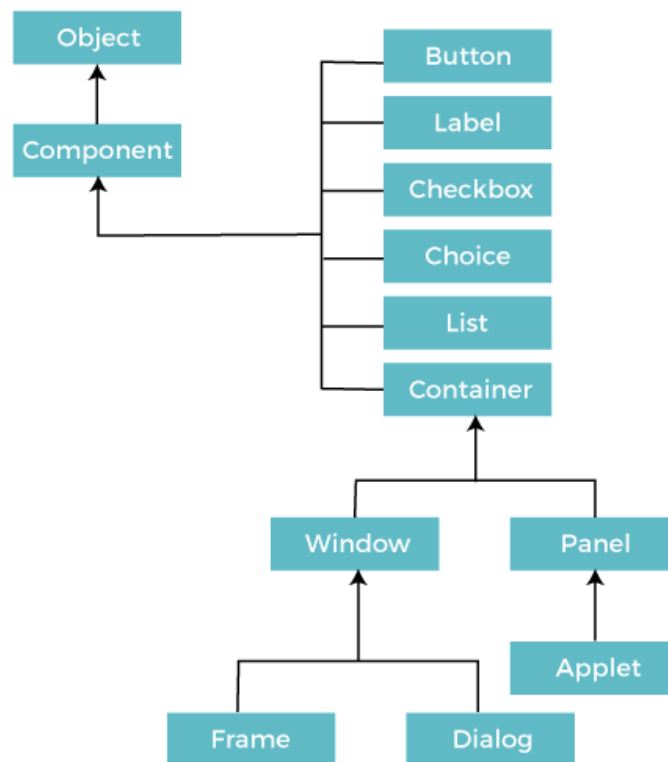


Figure: Hierarchy of Java AWT classes

Components:

Button, TextField, ScrollBar, List, Label , etc. are called components. Component is defined as an object having graphical representation that can be displayed on the screen and that can interact with users. Java AWT facilitate respective classes for each component , in order to place in a particular position on a screen. Also we need to add them to a container.

Container:

Container itself is a component in AWT , that contain another components like buttons, textfields, , labels etc. It is basically a screen where the components are placed at their specific locations. Which means it contains and controls the layout of components.

Types of Container:

- i. Window : The container that have no borders and menu bars. So , frame , dialog or another window are used for creating a window. Instance of Window class create this container.
- ii. Panel : Panel doesn't contain title bar, border or menu bar. It is generic container for holding the components. An instance of Panel class creates a container, in which we can add components. Can have other components like button, text fields, etc.
- iii. Frame : Frame contain title bar and border and can have menu bars. Most widely used container while developing an AWT application. Can have other components like panel , button, text fields, scrollbar etc.
- iv. Dialog – discussed below

Useful Methods of component class:

Method	Description
public void add(Component c)	Inserts / add a component on this container.
public void setSize(int width,int height)	Sets the size(width, height) of the container.
public void setLayout(LayoutManager m)	Defines the layout manager for the container.
public void setVisible(Boolean status)	Changes the visibility of the component, by default false. So, need true for visible.
public void setTitle(String s)	Define title to frame
public void setBounds(int x,int y, int width, int height)	Set Boundary to components where x, y means horizontal and vertical margin , and width and height define component size.

How AWT is platform dependent?

We know AWT GUI development is platform dependent i.e. components are displayed according to the view of operating system(OS). This is because , Java AWT calls the native platform (operating system) subroutine for creating API components like Button, TextField, CheckBox, etc. For example, an AWT GUI will have different look and feel for the different platforms like Windows , Linux , MAC OS , UNIX, etc. Simply, an AWT application will look like a windows application in window OS whereas it will look like a Mac application in the MAC OS.

There are two ways to create a GUI using Frame in AWT:

- i. By extending Frame class(inheritance)
- ii. By creating the object of Frame class(association)

Example 1: Using inheritance (Java Application without event handling)

```
import java.awt.Frame;
import java.awt.TextField;
import java.awt.Label;
import java.awt.Button;
class UsingInheritance extends Frame{ // inheriting awt Frame class
    TextField firstName,lastName; //instance variables of class TextField
    Label fLabel,lLabel; //instance variables of class Label
    Button btn; //instance variables of class Button
    sectionB(){
        setSize(400,300); //define frame size
        setTitle("Using Inheritance"); //define frame title

        fLabel = new Label("Enter Student First Name:"); // label name
        fLabel.setBounds(100,50,260,30); //define position and size of component i.e. Label
        add(fLabel); // add Label i.e. fLabel to frame

        firstName = new TextField(); // define input field for student name
        firstName.setBounds(100,90,160,30); //define position and size of component i.e. TextField
        add(firstName); // add TextField i.e. firstName to frame

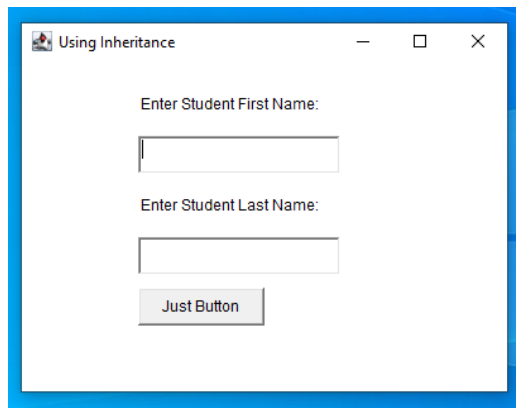
        lLabel = new Label("Enter Student Last Name:"); // label name
        lLabel.setBounds(100,130,260,30); //define position and size of component i.e. Label
        add(lLabel); // add Label i.e. lLabel to frame

        lastName = new TextField(); // define input field for student last name
        lastName.setBounds(100,170,160,30); //define position and size of component i.e. TextField
        add(lastName); // add TextField i.e. lastName to frame

        btn = new Button("Just Button"); //define button named "Just Button"
        btn.setBounds(100,210,100,30); //define position and size of Button
        add(btn); // add Button i.e. btn to frame

        setLayout(null); //set layout null which means manual setup
        setVisible(true); //set visibility to visible
    }
    public static void main(String[] args){
        new UsingInheritance (); // constructor
    }
}
```

Output:



Example 2: Using association (*Java Application without event handling*)

```
import java.awt.Frame;
import java.awt.TextField;
import java.awt.Label;
import java.awt.Button;
class UsingAssociation {
    Frame frame; // using association
    TextField firstName,lastName; //instance variables of class TextField
    Label fLabel,lLabel; //instance variables of class Label
    Button btn; //instance variables of class Button
    sectionB(){
        frame = new Frame();
        frame.setSize(400,300); //define frame size
        frame.setTitle("Using Inheritance"); //define frame title

        fLabel = new Label("Enter Student First Name:"); // label name
        fLabel.setBounds(100,50,260,30); //define position and size of component i.e. Label
        frame.add(fLabel); // add Label i.e. fLabel to frame

        firstName = new TextField(); // define input field for student name
        firstName.setBounds(100,90,160,30); //define position and size of component i.e. TextField
        frame.add(firstName); // add TextField i.e. firstName to frame

        lLabel = new Label("Enter Student Last Name:"); // label name
        lLabel.setBounds(100,130,260,30); //define position and size of component i.e. Label
        frame.add(lLabel); // add Label i.e. lLabel to frame

        lastName = new TextField(); // define input field for student last name
        lastName.setBounds(100,170,160,30); //define position and size of component i.e. TextField
        frame.add(lastName); // add TextField i.e. lastName to frame

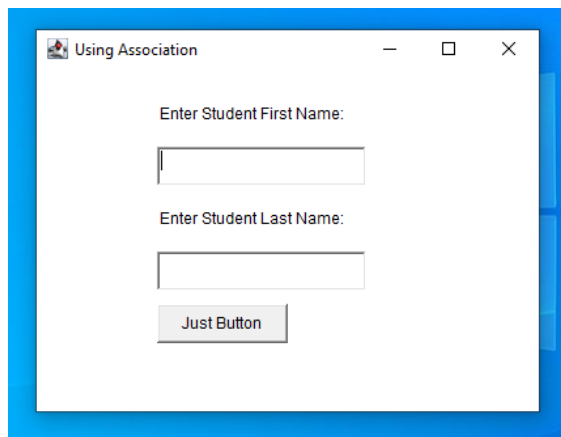
        btn = new Button("Just Button"); //define button named "Just Button"
        btn.setBounds(100,210,100,30); //define position and size of Button
        frame.add(btn); // add Button i.e. btn to frame
    }
}
```

```

        frame.setLayout(null); //set layout null which means manual setup
        frame.setVisible(true); //set visibility to visible
    }
    public static void main(String[] args){
        new UsingAssociation ();
    }
}

```

Output:



Event Handling:

Changing the state of an object is known as an **event**. For example, click on button, dragging mouse, key press, key type, key release etc. The **java.awt.event** package provides event classes and Listener interfaces for event handling.

Types of events:

- i. **Foreground Events:** Those events which require direct interaction of user, that are generated as the consequences of user interaction with the graphical components in GUI applications. **Example:** Clicking on a button, moving mouse, key type, selecting item from list, scrolling the page, check / uncheck, etc.
- ii. **Background Events:** Those events that require the interaction of internally occurred events are known as Background Events. **Example:** Operating system interrupts, hardware or software failure, timer expires, an operation completion etc.

Now, let's discuss about event handling. So, what does it mean?

Mechanism or process that controls the event and decides what should happen if an event occurs is called event handling. Where we write some related code known as event handler that is executed when an event occurs. Also Java uses the **Delegation Event Model** to handle the events.

Delegation Event Model:

Modern approach to handling events. The delegation event model defines standard and consistent mechanisms to generate and process events in GUI programming languages. The GUI stands for Graphical User Interface, where a user graphically/visually interacts with the system. The GUI programming is inherently event-driven; whenever a user initiates an activity.

Java support event processing since java 1.0 with the help of **AWT**(Abstract Window Toolkit) , which is an API used to develop the Desktop application. In Java 1.0, the AWT was based on inheritance. To catch and process GUI events for a program it should hold subclass GUI components and override **action()** and **handleEvent()** methods.

But the modern approach for event processing is based on the Delegation Model. Below image demonstrates the event processing.

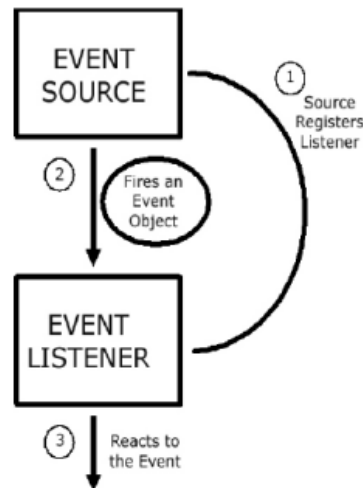


Figure: Delegation model , event processing

Delegation Event Model is based on three components:

- i. Event Source: Generates an event and forwards it to one or more listeners. Also event registration is done.
- ii. Event listeners: Waits until it receives an event. Once it receives the event , it is processed by the listener and returns it. This is also called Selection of event listener.
- iii. Event Performed(Event Handled): After receiving and selection of event listener event handling is done with the help of event handler. Also performs actual response.

Example: Let us assume , there is a button on GUI. On click of this button, button should react on click. Let see, how this could happen in Java?

```

1. Button btn = new Button("I am Button");
2. btn.addActionListener(ActionListener){
3.     public void actionPerformed(ActionEvent ae){
4.         // handler here
5.     }
6. }
```

- First define button(**line 1**)
- On click of button generate event(also registration of event) (**line 2,addActionListener**)
- Receives event by event listener(**line 2, ActionListener**)
- Perform event handling(**line 3, actionPerformed**)

Advantages of Delegation Event Model:

- Easy to learn and implement.

- Supports a clean separation between application and GUI code.
- Provides robust event handling program code which is less error-prone (strong compile-time checking).
- Flexible, can enable different types of application models for event flow and propagation.

Semantic and Low-Level Events in the AWT:

A semantic event is one that expresses what the user is doing, such as clicking button; hence, an **ActionEvent** is a semantic event. Similarly, those events that make semantic event possible known as Low-level events. In the case of button click is semantic event, also button is a mouse down, a series of mouse moves, and a mouse up are low-level events.

Semantic event classes examples: ActionEvent, AdjustmentEvent, ItemEvent, TextEvent.

Low-level event classes example: ComponentEvent, KeyEvent, MouseEvent, MouseWheelEvent, FocusEvent, WindowEvent, Container Event.

Java Event classes, Listener Interfaces, Methods, Parameter and Event Generated:

Event Classes	Listener Interfaces	Methods	Parameter	Event Generated
ActionEvent	ActionListener	actionPerformed	getActionCommand, getModifiers	Button, JComboBox, TextField, Timer
MouseEvent	MouseListener	mousePressed, mouseReleased, mouseEntered, mouseExited, mouseClicked,	getClickCount, getX, getY, getPoint, translatePoint, isPopupTrigger	Component
MouseEvent	MouseMotionListener	mouseDragged, mouseMoved,		Component
MouseWheelEvent	MouseWheelListener	mouseWheelMoved	getWheelRotation, getScrollAmount	Component
KeyEvent	KeyListener	keyPressed, keyReleased, keyTyped and	getKeyChar, getKeyCode, getKeyModifiersText, getKeyText, isActionKey	Component (Key stroke)
ItemEvent	ItemListener	itemStateChanged	getItem, getItemSelectable, getStateChange	Button, JComboBox
TextEvent	TextListener	textValueChanged		
AdjustmentEvent	AdjustmentListener	adjustmentValueChanged	getAdjustable, getAdjustmentType, getValue	JScrollBar
WindowEvent	WindowListener	windowClosing, windowOpened, windowIconified, windowDeiconified, windowClosed, windowActivated, windowDeactivated,	getWindow	Window
ComponentEvent	ComponentListener	componentMoved, componentHidden, componentResized,	getComponent	Component

		componentShown		
ContainerEvent	ContainerListener	componentAdded, componentRemoved	getChild, getContainer	Container
FocusEvent	FocusListener	focusGained, focusLost	isTemporary	Component

Note: For registering the component with the Listener, many classes provide the registration methods. For example:

Button – public void addActionListener(ActionListener a){}

MenuItem – public void addActionListener(ActionListener a){}

Checkbox – public void addItemListener(ItemListener a){}

Choice – public void addItemListener(ItemListener a){}

TextField – public void addActionListener(ActionListener a){}

– public void addTextListener(TextListener a){}

TextArea – public void addTextListener(TextListener a){}

KeyStroke – public void addKeyListener(KeyListener a){}

List – public void addActionListener(ActionListener a){}

– public void addItemListener(ItemListener a){}

We can put the event handling code into one of the following places:

1. Within class
2. Other class
3. Anonymous class

Example 3: Event Handling within class.

*/*AwtSwingExample.java*/*

package awtswingexample;

import EventHandler.EventHandlingWithinClass;

public class AwtSwingExample {

public static void main(String[] args) {

EventHandlingWithinClass eventHandler = new EventHandlingWithinClass();

}

*/*EventHandlingWithinClass.java*/*

package EventHandler; *//this is package*

import java.awt.Button; *//Button class for button*

import java.awt.Frame; *//Frame class for frame*

import java.awt.Label; *//Label class for label*

```
import java.awt.event.ActionEvent; //ActionEvent class for event action
import java.awt.event.ActionListener; //ActionListener interface to listen event
```

```
public class EventHandlingWithinClass implements ActionListener {

    Label lbl;

    Button btn;

    Frame frame;

    public EventHandlingWithinClass() { //constructor

        frame = new Frame("Within Class"); //define frame

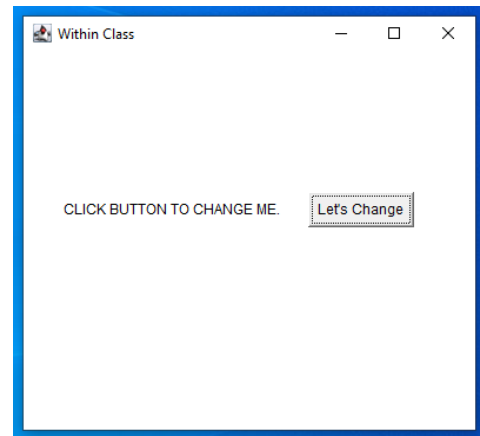
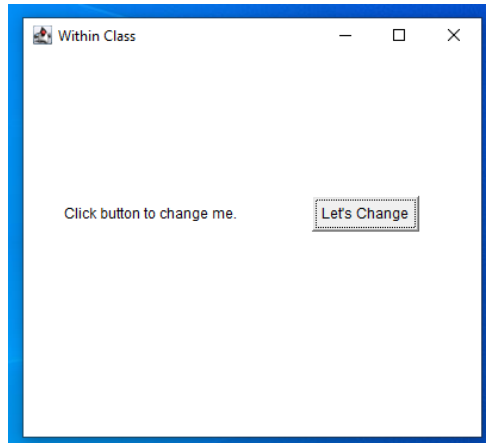
        lbl = new Label("Click button to change me."); //define label
        lbl.setBounds(40,150,200,30); //define boundary of label
        frame.add(lbl); //add label to frame

        btn = new Button("Let's Change"); //define button
        btn.setBounds(250,150,90,30); //define boundary of button
        frame.add(btn); //add label to frame

        //now event handling within class
        btn.addActionListener(this); //event registration
        frame.setSize(400,360); //define size of frame
        frame.setLayout(null); //set layout to manual
        frame.setVisible(true); //true means show all components in frame
    }

    //listen event by ActionListener and performed by actionPerformed method
    public void actionPerformed(ActionEvent ae){
        String lblText = lbl.getText(); // read label string using getText()
        lblText=lblText.toUpperCase(); // transform label string to upper case
        lbl.setText(lblText); // set transformed string on lbl
    }
}
```

Output:



Example 4: Event Handling using other class.

```
/*AwtSwingExample.java*/
package awtswingexample;
import EventHandler.EventHandlingUsingOtherClass;
public class AwtSwingExample {
    public static void main(String[] args) {
        EventHandlingUsingOtherClass eventHandler = new EventHandlingUsingOtherClass();
    }
}
```

```
/*EventHandlingUsingOtherClass.java*/
package EventHandler;
import java.awt.Button;
import java.awt.Frame;
import java.awt.Label;

public class EventHandlingUsingOtherClass {
    Label lbl;
    Button btn;
    Frame frame;
    public EventHandlingUsingOtherClass(){
        frame = new Frame("Within Class"); //define frame

        lbl = new Label("Click button to change me."); //define label
        lbl.setBounds(40,150,200,30); //define boundary of label
        frame.add(lbl); //add label to frame

        btn = new Button("Let's Change"); //define button
        btn.setBounds(250,150,90,30); //define boundary of button
        frame.add(btn); //add label to frame

        //now event handling using other class
        OutsideClass handler = new OutsideClass(this);
        btn.addActionListener(handler); //event registration

        frame.setSize(400,360); //define size of frame
        frame.setLayout(null); //set layout to manual
        frame.setVisible(true); //true means show all components in frame
    }
}
```

```

/*OutsideClass.java*/
package EventHandler;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class OutsideClass implements ActionListener {
    EventHandlerUsingOtherClass oc;
    OutsideClass(EventHandlerUsingOtherClass oc){
        this.oc = oc;
    }
    //listen event by ActionListener and performed by actionPerformed method
    public void actionPerformed(ActionEvent ae){
        String lblText = oc.lbl.getText(); // read label string using getText()
        lblText=lblText.toUpperCase(); // transform label string to upper case
        oc.lbl.setText(lblText); // set transformed string on lbl
    }
}

```

Output: As of example 3

Example 5: Event Handling using Anonymous class.

```

/*AwtSwingExample.java*/
package awtswingexample;
import EventHandler.EventHandlerUsingAnonymousClass;
public class AwtSwingExample {
    public static void main(String[] args) {
        EventHandlerUsingAnonymousClass eventHandler = new EventHandlerUsingAnonymousClass();
    }
}

```

```

/*EventHandlerUsingAnonymousClass.java*/
package EventHandler;

```

```

import java.awt.Button;
import java.awt.Frame;
import java.awt.Label;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

```

```

public class EventHandlerUsingAnonymousClass extends Frame{
    Label lbl;
    Button btn;
    public EventHandlerUsingAnonymousClass(){
        lbl = new Label("Click button to change me."); //define label
        lbl.setBounds(40,150,200,30); //define boundary of label
        add(lbl); //add label to frame
    }

```

```

        btn = new Button("Let's Change"); //define button
        btn.setBounds(250,150,90,30); //define boundary of button
        add(btn); //add label to frame
    }

```

```

    //now event handling using Anonymous class
    btn.addActionListener(new ActionListener(){
        //listen event by ActionListener and performed by actionPerformed method
        public void actionPerformed(ActionEvent ae){
            String lblText = lbl.getText(); // read label string using getText()
        }
    });
}

```

```

        lblText=lblText.toUpperCase(); // transform label string to upper case
        lbl.setText(lblText); // set transformed string on lbl
    }
}); //event registration,selection and performing
setTitle("Anonymous Class"); //set title to frame
setSize(400,360); //define size of frame
setLayout(null); //set layout to manual
setVisible(true); //true means show all components in frame
}
}

```

Output: As of example 3.

Some basic components in AWT:

- i. **Label** – The object of the label class is a component for placing text in a container. Used to display a single line of read only text. It is called a passive control as it does not create any event when it is accessed. The *java.awt.Label* class create the object of Label class.

Syntax:

Label label_name = new Label(); // constructs an empty label

Label label_name = new Label(String text); //constructs a label with the given string(default, left justified)

Label label_name = new Label(String text,int alignment); // constructs a label with the specified string and alignment

Label Fields(alignment):

1. static *int LEFT*: It specifies that the label should be left justified.
2. static *int RIGHT*: It specifies that the label should be right justified.
3. static *int CENTER*: It specifies that the label should be placed in center.

Label Class Methods:

Method name	Description
void setText(String text)	It sets the texts for label with the specified text.
void setAlignment(int alignment)	It sets the alignment for label with the specified alignment.
String getText()	It gets the text of the label
int getAlignment()	It gets the current alignment of the label.
void addNotify()	It creates the peer for the label.
AccessibleContext getAccessibleContext()	It gets the Accessible Context associated with the label.
protected String paramString()	It returns the string the state of the label.

- ii. **TextField** – The object of a TextField class is a text component that allows a user to enter a single line text and edit it. The *java.awt.TextField* class create the object of TextField class. When you enter a key in the text field(like key pressed, key released or key typed), the event is sent to TextField.

Syntax:

`TextField t_field = new TextField();` //constructs a new text field component

`TextField t_field = new TextField(String text)` //initialized text field with the given text

`TextField t_f = new TextField(int columns)` // constructs a new textfield(empty) with given number of columns.

`TextField t_f = new TextField(String txt, int columns)` // constructs a new text field with the given text and given number of columns(width).

Methods	Description
<code>void addNotify()</code>	Creates the peer of text field.
<code>Boolean echoCharIsSet()</code>	It tells whether text field has character set for echoing or not.
<code>void addActionListener(ActionListener al)</code>	Adds specified action listener to receive action events from the text field.
<code>void removeActionListener(ActionListener al)</code>	It removes specified action listener so that it doesn't receive action events anymore.
<code>ActionListener[] getActionListener()</code>	Returns array of all action listeners registered on text field.
<code>int getColumns()</code>	Fetches the number of columns in text field.
<code>Dimension getMinimumSize()</code>	It fetches the minimum dimensions for the text field.
<code>Dimension getPreferredSize()</code>	It fetches the preferred size of the text field.
<code>Protected String paramString()</code>	It Returns a string representing state of the text field.
<code>void setColumns(int columns)</code>	It sets the number of columns in text field.
<code>vgoind setEchoChar(char c)</code>	It sets the echo character for text field.
<code>void setText(String t)</code>	It sets the text presented by this text component to the specified text.

Example 6: Create a swing GUI that contains a two buttons (add and subtract) and three text fields. When the buttons are clicked sum or difference, values of first two text fields should be displayed in the third text field respectively.

*/*TextFieldExample.java*/*

```
import java.awt.Frame;
import java.awt.TextField;
import java.awt.Button;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
class TextFieldExample extends Frame implements ActionListener{
    static TextField tf1,tf2,tf3;
    Button addBtn,subBtn;
    TextFieldExample(){
        tf1 = new TextField();
        tf1.setBounds(100,50,200,30);
        add(tf1);

        tf2 = new TextField();
        tf2.setBounds(100,90,200,30);
        add(tf2);

        tf3 = new TextField();
```

```

tf3.setBounds(100,130,200,30);
add(tf3);

addBtn = new Button("Addition(+)");
addBtn.setBounds(100,170,90,30);
add(addBtn);

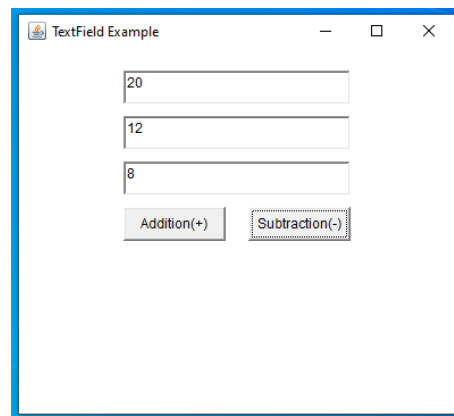
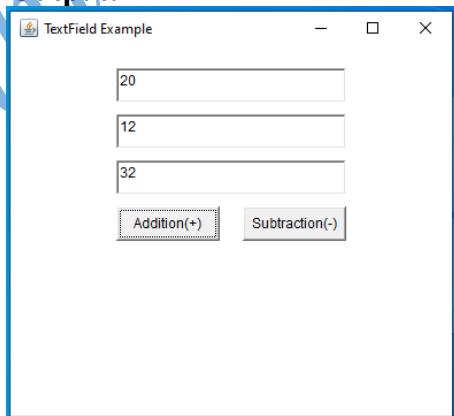
subBtn = new Button("Subtraction(-)");
subBtn.setBounds(210,170,90,30);
add(subBtn);

addBtn.addActionListener(this);
subBtn.addActionListener(this);

setSize(400,360);
setTitle("TextField Example");
setLayout(null);
setVisible(true);
}
public void actionPerformed(ActionEvent ae){
    int num1 = Integer.parseInt(tf1.getText());
    int num2 = Integer.parseInt(tf2.getText());
    int sum=0,sub=0;
    if(ae.getSource().equals(addBtn)){
        sum = num1 + num2;
        tf3.setText(String.valueOf(sum));
    }
    if(ae.getSource().equals(subBtn)){
        sub = num1 - num2;
        tf3.setText(String.valueOf(sub));
    }
}
public static void main(String[] args) {
    new TextFieldExample();
}
}

```

Output:



- iii. **Button** – basically is a control component with a label that generates an event when pushed. The *java.awt.Button* class is used to create button with label. The application result in some action when the button is pushed.

When button pressed and released, AWT sends an instance of *ActionEvent* to that button by calling *processEvent* on the button. The *processEvent* method of the button receives the all the events, then it passes an action event by calling its own method *processActionEvent*. This method passes the action event on to action listeners that are interested in the action events generated by the button. To perform an action on a button being pressed and released, the *ActionListener* interface needs to be implemented. The Java application can use the button's action command as a messaging protocol.

Syntax:

Button btn1 = new Button() // Constructs a new button with an empty string i.e. it has no label
Button btn2 = new Button("Click Me") //Constructs a new button with given String as its label

Button Class Methods:

Method	Description
void setText(String text)	It sets the string message on the button
String getText()	Fetches the String message on the button
void setLabel(String label)	Sets the label of button with the specified string
void getLabel()	Fetches the label of the button.
void addNotify()	Creates the peer of the button.
void addActionListener(ActionListener al)	It adds the specified action listener to get the action events from the button.
String getActionCommand()	It returns the command name of the action event fired by the button.
ActionListener[] getActionListeners()	It returns an array of all the action listeners registered on the button.
protected String paramString()	It returns the string which represents the state of button.
void removeActionListener(ActionListener al)	It removes the specified action listener , so that it no longer receives action events from the button.
void setActionCommand(String command)	It sets the command name for the action event given by the button.

Example 7:

```
import java.awt.Frame;
import java.awt.Button;
public class ButtonExample {
    static Frame frame;
    static Button btn1,btn2,btn3;
    public static void main(String[] args) {
        frame = new Frame();

        btn1 = new Button(); //button without label
        btn1.setBounds(50,100,40,30); // define boundary
        frame.add(btn1); // add button to frame

        btn2 = new Button(); //button without label
        btn2.setBounds(150,100,80,30);
        frame.add(btn2);
```



```

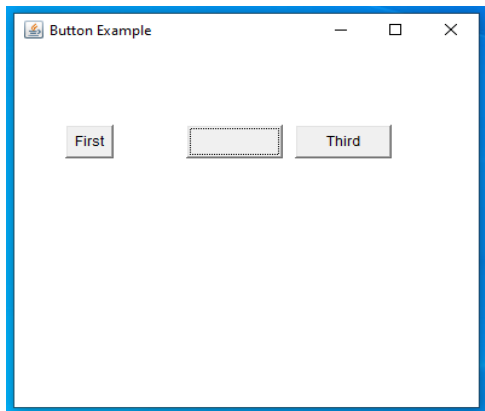
btn3 = new Button("Third");
btn3.setBounds(240,100,80,30);
frame.add(btn3);

//lets add label to button
btn1.setLabel("First");

frame.setSize(400,360);
frame.setTitle("Button Example");
frame.setLayout(null);
frame.setVisible(true);
}
}

```

Output:



- iv. **TextArea** – is a multiline region that displays text. Also allows the editing of multiple line text. IT inherits `TextComponent` class. The `java.awt.TextArea` class is used to create multiple line text displays. `TextArea` allows us to type as much text as we want and appears scroll bar automatically when text in the text area is larger than the viewable area.

Fields of TextArea Class:

- **static int SCROLLBARS_BOTH** – creates and displays both horizontal and vertical scrollbars.
- **static int SCROLLBARS_HORIZONTAL_ONLY** – creates and displays only the horizontal scrollbar.
- **static int SCROLLBARS_VERTICAL_ONLY** – creates and displays only the vertical scrollbar.
- **static int SCROLLBARS_NONE** – It doesn't create or display any scrollbar in the text area.

Syntax:

TextArea ta1 = new TextArea(); //Constructs a new and empty text area with no text in it

TextArea ta2 = new TextArea(int row, int column); //Constructs a new text area with specified number of rows and columns and empty string as text.

TextArea ta3 = new TextArea(String text); //constructs a new text area and displays the specified text in it.

TextArea ta4 = new TextArea(String text, int row , int column); // constructs a new text area with the specified text, rows and columns
TextArea ta5 = new TextArea(String text, int row, int column, int scrollbars); //construct a new text area with specified text , rows, columns and scrollbars

TextArea Class Methods:

Method name	Description
void addNotify()	It creates a peer of text area.
void append(String str)	It appends the specified text to the current text of text area.
AccessibleContext getAccessibleContext()	It returns the accessible context related to the text area
int getColumns()	It returns the number of columns of text area.
Dimension getMinimumSize()	It determines the minimum size of a text area.
Dimension getMinimumSize(int rows, int columns)	It determines the minimum size of a text area with the given number of rows and columns.
Dimension getPreferredSize()	It determines the preferred size of a text area.
Dimension preferredSize(int rows, int columns)	It determines the preferred size of a text area with given number of rows and columns.
int getRows()	It returns the number of rows of text area.
int getScrollbarVisibility()	It returns an enumerated value that indicates which scroll bars the text area uses.
void insert(String str, int pos)	It inserts the specified text at the specified position in this text area.
protected String paramString()	It returns a string representing the state of this TextArea.
void replaceRange(String str, int start, int end)	It replaces text between the indicated start and end positions with the specified replacement text.
void setColumns(int columns)	It sets the number of columns for this text area.
void setRows(int rows)	It sets the number of rows for this text area.

Example 8:

```

import java.awt.Frame;
import java.awt.TextArea;
public class TextAreaExample {
    static Frame frame;
    static TextArea ta1,ta2,ta3,ta4,ta5;
    public static void main(String[] args) {
        frame = new Frame("TextArea Example.");
        //text area with empty String
        ta1 = new TextArea();
        ta1.setBounds(50,50,200,50);
        frame.add(ta1);
        //text area with specified number of rows and columns and empty string as text.
        ta2 = new TextArea(4,6);
        ta2.setBounds(50,110,200,50);
        frame.add(ta2);
        //text area and displays the specified text in it
        ta3 = new TextArea("Description here");
        ta3.setBounds(50,170,200,50);
        frame.add(ta3);
        //text area with the specified text, rows and columns
        ta4 = new TextArea("Description here",10,20);
        ta4.setBounds(280,50,200,50);
        frame.add(ta4);
        //text area with specified text , rows, columns and scrollbars
        ta5 = new TextArea("Here...",10,20,TextArea.SCROLLBARS_VERTICAL_ONLY);
        ta5.setBounds(280,110,200,50);
        frame.add(ta5);

        frame.setSize(600,500);
        frame.setLayout(null);
        frame.setVisible(true);
    }
}

```

- v. **Checkbox** – Checkbox is used to turn an option on(true/checked) or off(false/unchecked). You can checked one , two or more option among given options. Clicking on a Checkbox changes its state from “on” to “off” or from “off” to “on”. The **Checkbox** class is used to create a checkbox. The **java.awt.Checkbox** class provides Checkbox.

Syntax:

Checkbox chk1 = new Checkbox(); *//Constructs a checkbox with no string as the label.*

Checkbox chk2 = new Checkbox(String label); *// Constructs a checkbox with given label*

Checkbox chk3 = new Checkbox(String label, Boolean state); *//constructs a checkbox with the given label and sets the given state*

Checkbox chk4 = new Checkbox(String label, Boolean state, CheckGroup group); *// Constructs a checkbox with the given label, set the given state in the specified checkbox group.*

Checkbox chk5 = new Checkbox(String label, CheckboxGroup group, Boolean state); *//Constructs a checkbox with the given label, in the given checkbox group and set to the specified state.*

Checkbox Class Methods:

S.N	Method name	Description
1.	void addItemListener(ItemListener IL)	It adds the given item listener to get the item events from the checkbox.
2.	AccessibleContext getAccessibleContext()	It fetches the accessible context of checkbox.
3.	void addNotify()	It creates the peer of checkbox.
4.	CheckboxGroup getCheckboxGroup()	It determines the group of checkbox.
5.	ItemListener[] getItemListeners()	It returns an array of the item listeners registered on checkbox.
6.	String getLabel()	It fetched the label of checkbox.
7.	T[] getListeners(Class listenerType)	It returns an array of all the objects registered as FooListeners.
8.	Object[] getSelectedObjects()	It returns an array (size 1) containing checkbox label and returns null if checkbox is not selected.
9.	boolean getState()	It returns true if the checkbox is on, else returns off.
10.	protected String paramString()	It returns a string representing the state of checkbox.
11.	protected void processEvent(AWTEvent e)	It processes the event on checkbox.
12.	protected void processItemEvent(ItemEvent e)	It process the item events occurring in the checkbox by dispatching them to registered ItemListener object.
13.	void removeItemListener(ItemListener l)	It removes the specified item listener so that the item listener doesn't receive item events from the checkbox anymore.
14.	void setCheckboxGroup(CheckboxGroup g)	It sets the checkbox's group to the given checkbox.
15.	void setLabel(String label)	It sets the checkbox's label to the string argument.
16.	void setState(boolean state)	It sets the state of checkbox to the specified state.

Example 9: Creating three checkboxes using the Checkbox(), Checkbox(String label) and Checkbox(String label, boolean state) , and adding them into the Frame using add() method.

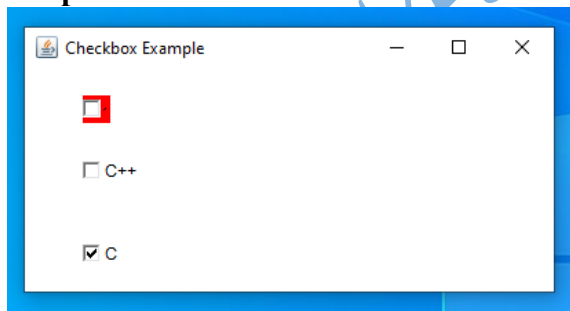
```
import java.awt.Frame;
import java.awt.Checkbox;
import java.awt.Color;
class CheckBoxFrame extends Frame{
```

```

Checkbox chk1,chk2,chk3;
CheckBoxFrame(){
    chk(); //user defined function
    setTitle("Checkbox Example");
    setSize(400,200);
    setLayout(null);
    setVisible(true);
}
public void chk(){
    chk1 = new Checkbox();
    chk2 = new Checkbox("C++");
    chk3 = new Checkbox("C",true);
    chk1.setBounds(50,50,20,20);
    chk1.setBackground(Color.red);
    chk2.setBounds(50,80,50,50);
    chk3.setBounds(50,140,60,50);
    add(chk1);
    add(chk2);
    add(chk3);
}
}
public class CheckBoxExample{
    public static void main(String[] args) {
        new CheckBoxFrame();
    }
}

```

Output:



Example 10: Write a program to change the font of text in *textfield* of the Frame. The frame contains 3 checkboxes named bold, italic and plain.

/ ChangeFontStyleExample .java*/*

```

import java.awt.Frame;
import java.awt.Checkbox;
import java.awt.TextField;
import java.awt.event.ItemListener;
import java.awt.event.ItemEvent;
import java.awt.Font;

```

```

class CheckBoxFrame extends Frame{
    TextField tft;
    Checkbox chk1,chk2,chk3;
    int mode = 0;
    CheckBoxFrame(){
        chk();
        setTitle("Change font style");
        setSize(400,200);
        setLayout(null);
        setVisible(true);
    }
    public void chk(){
        tft = new TextField("Check for font style.");
        tft.setFont(new Font("Times New Roman",Font.PLAIN,16));
        chk1 = new Checkbox("bold");
        chk2 = new Checkbox("italic");
        chk3 = new Checkbox("plain");

        tft.setBounds(50,40,220,30);
        chk1.setBounds(50,100,40,30);
        chk2.setBounds(110,100,40,30);
        chk3.setBounds(170,100,50,30);

        add(tft);
        add(chk1);
        add(chk2);
        add(chk3);
        chk1.addItemListener(new ItemListener(){
            public void itemStateChanged(ItemEvent ie){
                if(ie.getStateChange()==1){
                    mode += Font.BOLD;
                    tft.setFont(new Font("Times New Roman",mode,16));
                }else{
                    mode -= Font.BOLD;
                    tft.setFont(new Font("Times New Roman",Font.PLAIN,16));
                }
            }
        });
        chk2.addItemListener(new ItemListener(){
            public void itemStateChanged(ItemEvent ie){
                if(ie.getStateChange()==1){
                    mode +=Font.ITALIC;
                    tft.setFont(new Font("Times New Roman",mode,16));
                }else{
                    mode -= Font.ITALIC;
                    tft.setFont(new Font("Times New Roman",mode,16));
                }
            }
        });
    }
}

```

de,16));

Change font style

Check for font style.

☒ bold ☒ italic ☐ plain

- vi. **CheckboxGroup** – The *java.awt.* **CheckboxGroup**. At a time only one of the remaining check box buttons in “On” state. To create *radio* buttons in AWT . The **CheckboxGroup** class in AWT.
- Example 11:** Creating a **CheckboxGroup** and a radio button.
- ```
/*CheckboxGroupExample.java
import java.awt.Frame;
import java.awt.Checkbox;
import java.awt.CheckboxGroup;
class CheckboxGroupFrame extends Frame
{
 Checkbox chk4,chk5,chk6;
```

```
/*CheckboxGroupExample.java*/
```

```
import java.awt.Frame;
import java.awt.Checkbox;
import java.awt.CheckboxGroup;
class CheckboxGroupFrame extends Frame
{
 Checkbox chk4,chk5,chk6;
 CheckboxGroupFrame(){
 chk();
 setTitle("Checkbox Example");
 setSize(400,200);
 setLayout(null);
 setVisible(true);
 }
}
```

```

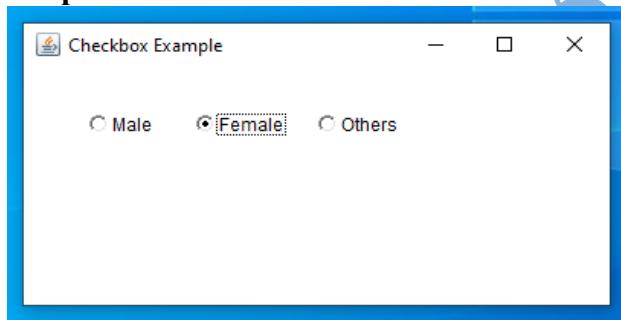
 }
 public void chk(){
 CheckboxGroup chg = new CheckboxGroup();
 chk4 = new Checkbox("Male",true,chg);
 chk4.setBounds(50,50,50,40);
 add(chk4);

 chk5 = new Checkbox("Female",true,chg);
 chk5.setBounds(120,50,60,40);
 add(chk5);

 chk6 = new Checkbox("Others",chg,true);
 chk6.setBounds(200,50,60,40);
 add(chk6);
 }
}
public class CheckboxGroupExample{
 public static void main(String[] args) {
 new CheckboxGroupFrame();
 }
}

```

### Output:



**Example 12:** Let's add ItemListener with CheckboxGroup  
**/\*CheckboxGroupExample.java\*/**

```

import java.awt.Frame;
import java.awt.Checkbox;
import java.awt.CheckboxGroup;
import java.awt.Label;
import java.awt.event.ItemListener;
import java.awt.event.ItemEvent;
class CheckboxGroupFrame extends Frame implements ItemListener{
 Checkbox chk4,chk5,chk6;
 Label lbl;
 CheckboxGroupFrame(){
 chk();
 setTitle("Checkbox Example");
 setSize(400,200);
 setLayout(null);
 setVisible(true);
 }
}

```



```

 }
 public void chk(){
 CheckboxGroup chg = new CheckboxGroup();
 chk4 = new Checkbox("Male",true,chg);
 chk4.setBounds(50,50,50,40);
 add(chk4);

 chk5 = new Checkbox("Female",true,chg);
 chk5.setBounds(120,50,60,40);
 add(chk5);

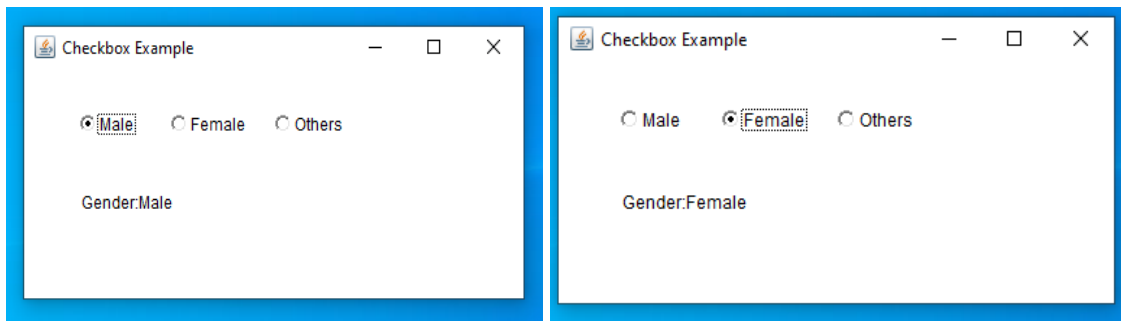
 chk6 = new Checkbox("Others",chg,true);
 chk6.setBounds(200,50,60,40);
 add(chk6);

 lbl = new Label();
 lbl.setBounds(50,110,200,30);
 add(lbl);

 chk4.addItemListener(this);
 chk5.addItemListener(this);
 chk6.addItemListener(this);
 }
 public void itemStateChanged(ItemEvent ie){
 if(ie.getSource().equals(chk4)){
 lbl.setText("Gender:"+chk4.getLabel());
 }
 if(ie.getSource().equals(chk5)){
 lbl.setText("Gender:"+chk5.getLabel());
 }
 if(ie.getSource().equals(chk6)){
 lbl.setText("Gender:"+chk6.getLabel());
 }
 }
 };
}
public class CheckboxGroupExample{
 public static void main(String[] args) {
 new CheckboxGroupFrame();
 }
}

```

**Output:**



- vii. **Choice** – The *java.awt.Choice* class provides choice to select item from list of items. Choice selected by user is shown on the top of a menu.

**Syntax:**

Choice chc = new Choice(); *//It constructs a new choice menu*

**Choice Class Methods:**

| S.N | Method name                                 | Description                                                                                |
|-----|---------------------------------------------|--------------------------------------------------------------------------------------------|
| 1.  | void add(String item)                       | It adds an item to the choice menu.                                                        |
| 2.  | void addItemListener(ItemListener l)        | It adds the item listener that receives item events from the choice menu.                  |
| 3.  | void addNotify()                            | It creates the peer of choice.                                                             |
| 4.  | AccessibleContext<br>getAccessibleContext() | It gets the accessible context related to the choice.                                      |
| 5.  | String getItem(int index)                   | It gets the item (string) at the given index position in the choice menu.                  |
| 6.  | int getItemCount()                          | It returns the number of items of the choice menu.                                         |
| 7.  | ItemListener[] getItemListeners()           | It returns an array of all item listeners registered on choice.                            |
| 8.  | T[] getListeners(Class listenerType)        | Returns an array of all the objects currently registered as FooListeners upon this Choice. |
| 9.  | int getSelectedIndex()                      | Returns the index of the currently selected item.                                          |
| 10. | String getSelectedItem()                    | Gets a representation of the current choice as a string.                                   |
| 11. | Object[] getSelectedObjects()               | Returns an array (length 1) containing the currently selected item.                        |
| 12. | void insert(String item, int index)         | Inserts the item into this choice at the specified position.                               |
| 13. | protected String paramString()              | Returns a string representing the state of this Choice menu.                               |
| 14. | protected<br>processEvent(AWTEvent e)       | void<br>It processes the event on the choice.                                              |

|     |                                               |                                                                                                                                     |
|-----|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 15. | protected void processItemEvent (ItemEvent e) | Processes item events occurring on this Choice menu by dispatching them to any registered ItemListener objects.                     |
| 16. | void remove(int position)                     | It removes an item from the choice menu at the given index position.                                                                |
| 17. | void remove(String item)                      | It removes the first occurrence of the item from choice menu.                                                                       |
| 18. | void removeAll()                              | It removes all the items from the choice menu.                                                                                      |
| 19. | void removeItemListener (ItemListener l)      | It removes the mentioned item listener. Thus it doesn't receive item events from the choice menu anymore.                           |
| 20. | void select(int pos)                          | It changes / sets the selected item in the choice menu to the item at given index position.                                         |
| 21. | void select(String str)                       | It changes / sets the selected item in the choice menu to the item whose string value is equal to string specified in the argument. |

**Example 13:** Creating a choice menu using **Choice()** constructor. Here, we add 4 items to the menu using add() method and then add the choice menu into the frame.

```
import java.awt.Frame;
import java.awt.Choice;
class ChoiceFrame {
 Frame frame;
 Choice chc;
 ChoiceFrame() {
 frame = new Frame("Choice Example");

 chc = new Choice();
 chc.setBounds(50,50,80,40);

 //adding items to choice menu
 chc.add("C");
 chc.add("Java");
 chc.add("C++");
 chc.add("Web");

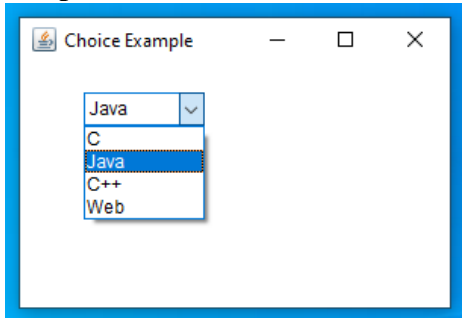
 frame.add(chc);

 frame.setSize(300,200);
 frame.setLayout(null);
 frame.setVisible(true);
 }
}

public class ChoiceExample {
 public static void main(String[] args) {
 new ChoiceFrame();
 }
}
```

```
}
}
```

### Output:



**Example 14:** Creating a choice menu with 5 items. Along with that we create a button and label. When button is clicked, selected item from choice menu is displayed on the label .

```
import java.awt.Frame;
import java.awt.Label;
import java.awt.Button;
import java.awt.Choice;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
class ChoiceFrame{
 Frame frame;
 Label lbl;
 Button btn;
 Choice chc;
 ChoiceFrame(){
 frame = new Frame("Choice Example");

 lbl = new Label("Select Item.");
 lbl.setAlignment(Label.CENTER);
 lbl.setBounds(20,40,300,30);

 chc = new Choice();
 chc.setBounds(50,80,80,40);

 btn = new Button("Show Selected Item");
 btn.setBounds(140,80,140,30);

 //adding items to choice menu
 chc.add("C");
 chc.add("Java");
 chc.add("C++");
 chc.add("Web");
 chc.add("C#");

 frame.add(chc);
 frame.add(lbl);
 frame.add(btn);
 }
}
```

```

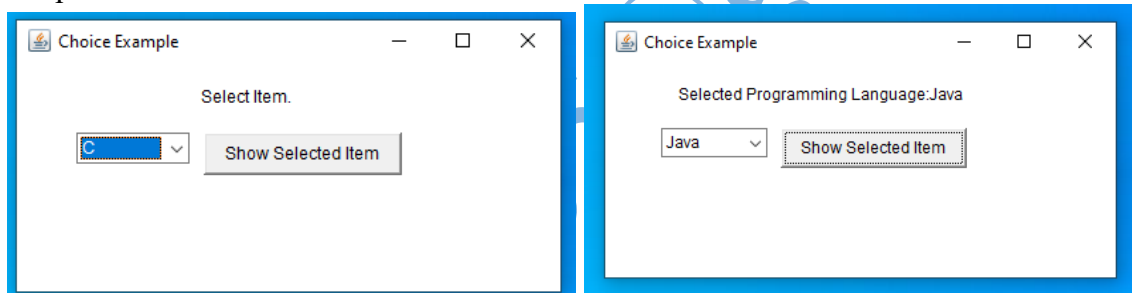
 btn.addActionListener(new ActionListener(){
 public void actionPerformed(ActionEvent ae){
 String selected_item = "Selected Programming Language:"+chc.getItem(chc.getSelectedIndex());
 lbl.setText(selected_item);
 }
 });

 frame.setSize(400,200);
 frame.setLayout(null);
 frame.setVisible(true);
 }
}

public class ChoiceExample {
 public static void main(String[] args) {
 new ChoiceFrame();
 }
}

```

Output:



- viii. **ScrollBar** – Scrollbar is a GUI component allows us to see invisible of rows and columns. The object of `java.awt.Scrollbar` class is used to add horizontal and vertical scrollbar. Also can be added to top-level container like Frame or a component like Panel.

**Some parameters are:**

- Orientation** : specify whether the scrollbar will be horizontal or vertical. Fields of `java.awt.Scrollbar` class are **static int HORIZONTAL** and **static int VERTICAL**.
- Value** : specify the starting position of the knob of scrollbar on its track.
- Minimum** : specify the minimum width of track on which scrollbar is moving.
- Maximum** : specify the maximum width of track on which scrollbar is moving.

**Syntax:**

`Scrollbar s_name = new Scrollbar();` // *Constructs a new vertical scroll bar.*

`Scrollbar s_name = new Scrollbar(int orientation);` // *Constructs a new scroll bar with the specified orientation.*

`Scrollbar s_name = new Scrollbar(int orientation, int value, int visible, int minimum, int maximum);` // *Constructs a new scroll bar with the specified orientation, initial value, visible amount, and minimum and maximum values.*

Scrollbar Class Methods:

| Sr. no. | Method name                                               | Description                                                                                                                 |
|---------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 1.      | void addAdjustmentListener (AdjustmentListener l)         | It adds the given adjustment listener to receive instances of AdjustmentEvent from the scroll bar.                          |
| 2.      | void addNotify()                                          | It creates the peer of scroll bar.                                                                                          |
| 3.      | int getBlockIncrement()                                   | It gets the block increment of the scroll bar.                                                                              |
| 4.      | int getMaximum()                                          | It gets the maximum value of the scroll bar.                                                                                |
| 5.      | int getMinimum()                                          | It gets the minimum value of the scroll bar.                                                                                |
| 6.      | int getOrientation()                                      | It returns the orientation of scroll bar.                                                                                   |
| 7.      | int getUnitIncrement()                                    | It fetches the unit increment of the scroll bar.                                                                            |
| 8.      | int getValue()                                            | It fetches the current value of scroll bar.                                                                                 |
| 9.      | int getVisibleAmount()                                    | It fetches the visible amount of scroll bar.                                                                                |
| 10.     | boolean getValueIsAdjusting()                             | It returns true if the value is in process of changing where action results are being taken by the user.                    |
| 11.     | protected String paramString()                            | It returns a string representing the state of Scroll bar.                                                                   |
| 12.     | protected void processAdjustmentEvent (AdjustmentEvent e) | It processes the adjustment event occurring on scroll bar by dispatching them to any registered AdjustmentListener objects. |
| 13.     | protected void processEvent(AWTEvent e)                   | It processes the events on the scroll bar.                                                                                  |
| 14.     | void removeAdjustmentListener (AdjustmentListener l)      | It removes the given adjustment listener. Thus it no longer receives the instances of AdjustmentEvent from the scroll bar.  |
| 15.     | void setBlockIncrement(int v)                             | It sets the block increment from scroll bar.                                                                                |
| 16.     | void setMaximum (int newMaximum)                          | It sets the maximum value of the scroll bar.                                                                                |
| 17.     | void setMinimum (int newMinimum)                          | It sets the minimum value of the scroll bar.                                                                                |
| 18.     | void setOrientation (int orientation)                     | It sets the orientation for the scroll bar.                                                                                 |
| 19.     | void setUnitIncrement(int v)                              | It sets the unit increment for the scroll bar.                                                                              |
| 20.     | void setValue (int newValue)                              | It sets the value of scroll bar with the given argument value.                                                              |
| 21.     | void setValueIsAdjusting (boolean b)                      | It sets the valueIsAdjusting property to scroll bar.                                                                        |

|     |                                                                                |                                                                                                     |
|-----|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 22. | <code>void setValues (int value, int visible, int minimum, int maximum)</code> | It sets the values of four properties for scroll bar: value, visible amount, minimum and maximum.   |
| 23. | <code>void setVisibleAmount (int newAmount)</code>                             | It sets the visible amount of the scroll bar.                                                       |
| 24. | <code>AccessibleContext<br/>getAccessibleContext()</code>                      | It gets the accessible context related to the scroll bar.                                           |
| 25. | <code>AdjustmentListener[]<br/>getAdjustmentListeners()</code>                 | It returns an array of all the adjustment listeners registered on the scroll bar.                   |
| 26. | <code>T[]<br/>getListeners(Class listenerType)</code>                          | It returns an array of all objects that are registered as FooListeners on the scroll bar currently. |

**Example 15:** Creating three scrollbar using different constructor of class Scrollbar and adding them to frame.

**/\*ScrollbarExample1.java\*/**

```
import java.awt.Frame;
import java.awt.Scrollbar;

public class ScrollbarExample1 {
 public static void main(String[] args) {
 Frame frame = new Frame("Scrollbar Example");

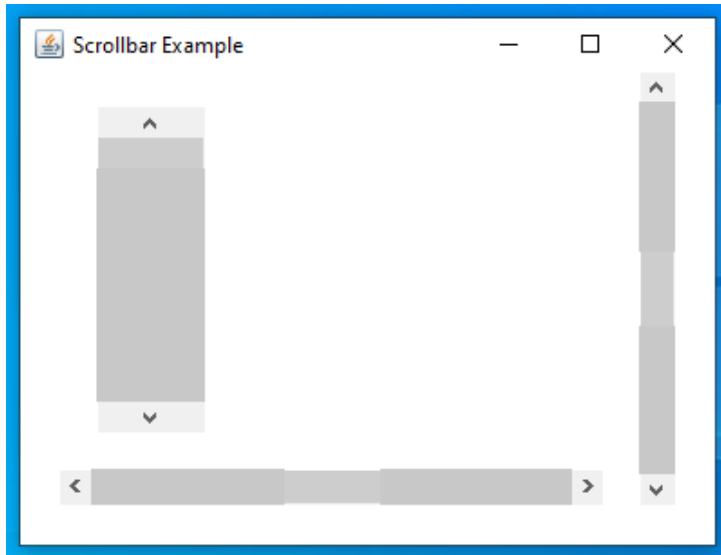
 Scrollbar scbr1 = new Scrollbar();
 scbr1.setBounds(50,50,60,180);
 frame.add(scbr1);

 Scrollbar scbr2 = new Scrollbar(Scrollbar.HORIZONTAL,40,20,0,100);
 scbr2.setBounds(30,250,300,20);
 frame.add(scbr2);

 Scrollbar scbr3 = new Scrollbar(Scrollbar.VERTICAL,40,20,0,100);
 scbr3.setBounds(350,30,20,240);
 frame.add(scbr3);

 frame.setLayout(null);
 frame.setSize(400,300);
 frame.setVisible(true);
 }
}
```

**Output:**



**Example 16:** Creating a Scrollbar , and label and adding them into the Frame. Here we are using `addAdjustmentListener()` method of Scrollbar class which receives the instances of `AdjustmentEvent` and eventually we display it in the form of Label.

**/\*ScrollbarExample2.java\*/**

```
import java.awt.Frame;
import java.awt.Scrollbar;
import java.awt.Label;
import java.awt.event.AdjustmentListener;
import java.awt.event.AdjustmentEvent;
public class ScrollbarExample2 {
 public static void main(String[] args) {
 Frame frame = new Frame("Scrollbar Example");

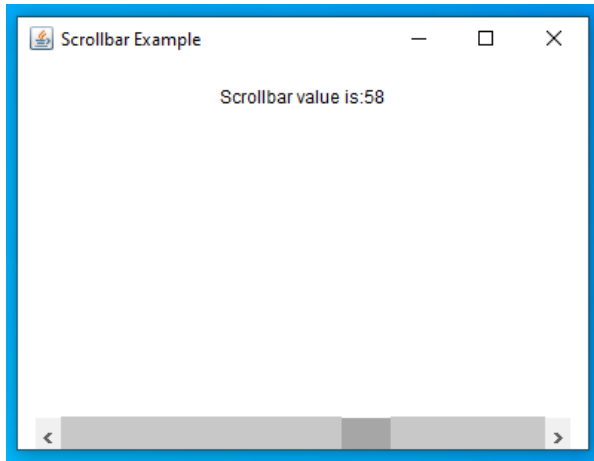
 Scrollbar scbr1 = new Scrollbar(Scrollbar.HORIZONTAL,100,10,0,100);
 scbr1.setBounds(20,270,360,30);
 frame.add(scbr1);

 Label lbl = new Label();
 lbl.setAlignment(Label.CENTER);
 lbl.setBounds(100,40,200,30);
 frame.add(lbl);

 scbr1.addAdjustmentListener(new AdjustmentListener(){
 public void adjustmentValueChanged(AdjustmentEvent ae){
 lbl.setText("Scrollbar value is:"+scbr1.getValue());
 }
 });
 frame.setLayout(null);
 frame.setSize(400,300);
 frame.setVisible(true);
 }
}
```

**Output:**





- ix. **List** – The *java.awt.List* class represents a list of text items. With the help of the list class, user can choose either one item or multiple items.

**Syntax:**

List lst1 = new List(); // constructs a new scrolling list.

List lst2 = new List(int row\_num); // It constructs a new scrolling list initialized with the given number of rows visible.

List lst3 = new List(int row\_num, Boolean multipleMode); // It constructs a new scrolling list initialized which displays the given number of rows.

**List Class Methods:**

| S.N | Method name                                 | Description                                                               |
|-----|---------------------------------------------|---------------------------------------------------------------------------|
| 1.  | void add(String item)                       | It adds the specified item into the end of scrolling list.                |
| 2.  | void add(String item, int index)            | It adds the specified item into list at the given index position.         |
| 3.  | void addActionListener(ActionListener l)    | It adds the specified action listener to receive action events from list. |
| 4.  | void addItemListener(ItemListener l)        | It adds specified item listener to receive item events from list.         |
| 5.  | void addNotify()                            | It creates peer of list.                                                  |
| 6.  | void deselect(int index)                    | It deselects the item at given index position.                            |
| 7.  | AccessibleContext<br>getAccessibleContext() | It fetches the accessible context related to the list.                    |
| 8.  | ActionListener[] getActionListeners()       | It returns an array of action listeners registered on the list.           |
| 9.  | String getItem(int index)                   | It fetches the item related to given index position.                      |
| 10. | int getItemCount()                          | It gets the count/number of items in the list.                            |
| 11. | ItemListener[] getItemListeners()           | It returns an array of item listeners registered on the list.             |
| 12. | String[] getItems()                         | It fetched the items from the list.                                       |

|     |                                                  |                                                                                                           |
|-----|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 13. | Dimension getMinimumSize()                       | It gets the minimum size of a scrolling list.                                                             |
| 14. | Dimension getMinimumSize(int rows)               | It gets the minimum size of a list with given number of rows.                                             |
| 15. | Dimension getPreferredSize()                     | It gets the preferred size of list.                                                                       |
| 16. | Dimension getPreferredSize(int rows)             | It gets the preferred size of list with given number of rows.                                             |
| 17. | int getRows()                                    | It fetches the count of visible rows in the list.                                                         |
| 18. | int getSelectedIndex()                           | It fetches the index of selected item of list.                                                            |
| 19. | int[] getSelectedIndexes()                       | It gets the selected indices of the list.                                                                 |
| 20. | String getSelectedItem()                         | It gets the selected item on the list.                                                                    |
| 21. | String[] getSelectedItems()                      | It gets the selected items on the list.                                                                   |
| 22. | Object[] getSelectedObjects()                    | It gets the selected items on scrolling list in array of objects.                                         |
| 23. | int getVisibleIndex()                            | It gets the index of an item which was made visible by method makeVisible()                               |
| 24. | void makeVisible(int index)                      | It makes the item at given index visible.                                                                 |
| 25. | boolean isIndexSelected(int index)               | It returns true if given item in the list is selected.                                                    |
| 26. | boolean isMultipleMode()                         | It returns the true if list allows multiple selections.                                                   |
| 27. | protected String paramString()                   | It returns parameter string representing state of the scrolling list.                                     |
| 28. | protected void processActionEvent(ActionEvent e) | It process the action events occurring on list by dispatching them to a registered ActionListener object. |
| 29. | protected void processEvent(AWTEvent e)          | It process the events on scrolling list.                                                                  |
| 30. | protected void processItemEvent(ItemEvent e)     | It process the item events occurring on list by dispatching them to a registered ItemListener object.     |
| 31. | void removeActionListener(ActionListener l)      | It removes specified action listener. Thus it doesn't receive further action events from the list.        |
| 32. | void removeItemListener(ItemListener l)          | It removes specified item listener. Thus it doesn't receive further action events from the list.          |
| 33. | void remove(int position)                        | It removes the item at given index position from the list.                                                |
| 34. | void remove(String item)                         | It removes the first occurrence of an item from list.                                                     |
| 35. | void removeAll()                                 | It removes all the items from the list.                                                                   |

|     |                                                         |                                                                                          |
|-----|---------------------------------------------------------|------------------------------------------------------------------------------------------|
| 36. | <code>void replaceItem(String newVal, int index)</code> | It replaces the item at the given index in list with the new string specified.           |
| 37. | <code>void select(int index)</code>                     | It selects the item at given index in the list.                                          |
| 38. | <code>void setMultipleMode(boolean b)</code>            | It sets the flag which determines whether the list will allow multiple selection or not. |
| 39. | <code>void removeNotify()</code>                        | It removes the peer of list.                                                             |

**Example 17:** Creating List with 6 items and adding into frame.

**/\*ListExample1.java\*/**

`import java.awt.Frame;`

`import java.awt.List;`

`public class ListExample1 {`

`public static void main(String[] args) {`

`Frame frame = new Frame("List Example");`

`List lst1 = new List(3); //creating the list of 3 rows but accept more`  
`lst1.setBounds(80,60,70,80);`

`lst1.add("Ram");`

`lst1.add("Shyam");`

`lst1.add("Sita");`

`lst1.add("Gita");`

`lst1.add("Hari");`

`lst1.add("Laxman");`

`frame.add(lst1);`

`frame.setLayout(null);`

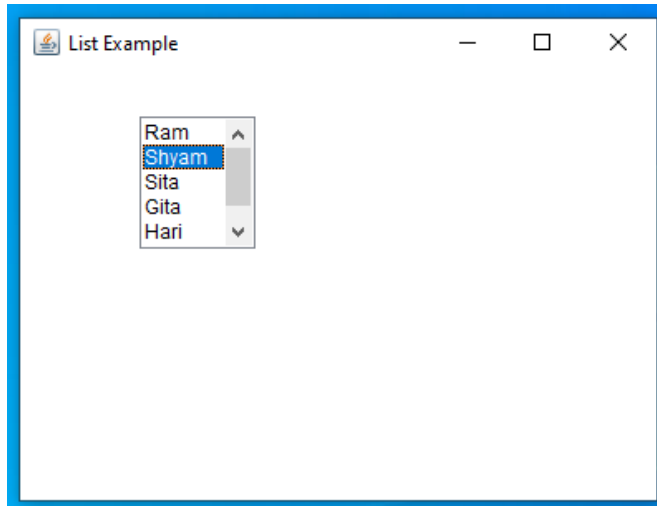
`frame.setSize(400,300);`

`frame.setVisible(true);`

`}`

`}`

Output:



**Example 18:** Creating 2 List , a Button and a label and adding them into the frame. Here , we are generating an event on the button using the ***addActionListener(ActionListener al)*** method. On clicking the button, it displays the selected Students name and their address.

**/\*ListExample2.java\*/**

```
import java.awt.Frame;
import java.awt.List;
import java.awt.Button;
import java.awt.Label;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class ListExample2 {
 static List lst1,lst2;
 static Button btn;
 static Label lbl;
 public static void main(String[] args) {
 Frame frame = new Frame("List Example");

 lst1 = new List(3); //creating first list of 3 rows but accept more
 lst1.setBounds(80,60,70,80);

 lst1.add("Ram");
 lst1.add("Shyam");
 lst1.add("Sita");
 lst1.add("Gita");
 lst1.add("Hari");
 lst1.add("Laxman");

 lst2 = new List(4); //creating second list of 4 rows but accept more
 lst2.setBounds(170,60,70,80);

 lst2.add("Kathmandu");
 lst2.add("Bhaktapur");
 lst2.add("Nepaljung");
 lst2.add("Lamki");
```

```

lst2.add("Attariya");
lst2.add("Dhangadhi");

btn = new Button("Select student address");
btn.setBounds(80,160,160,30);

lbl = new Label();
lbl.setBounds(40,200,360,30);
lbl.setAlignment(Label.CENTER);

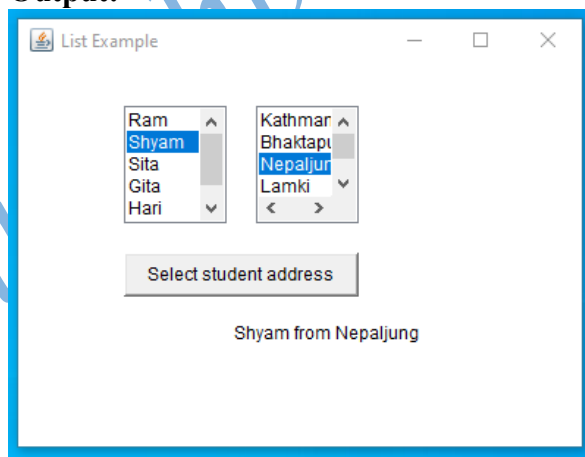
frame.add(lst1);
frame.add(lst2);
frame.add(btn);
frame.add(lbl);

btn.addActionListener(new ActionListener(){
 public void actionPerformed(ActionEvent ae){
 String info = lst1.getItem(lst1.getSelectedIndex());
 info += " from ";
 for(String fm:lst2.getSelectedItems()){
 info +=fm + " ";
 }
 lbl.setText(info);
 }
});

frame.setLayout(null);
frame.setSize(400,300);
frame.setVisible(true);
}
}

```

**Output:**



- x. **MenuItem & Menu** – Let's understand MenuItem & Menu through example.

**Example 19:** Here, we are creating MenuBar , Menu, and MenuItem using *java.awt.MenuBar* class, *java.awt.Menu* class and *java.awt.MenuItem* class respectively.

```

/*MenuExample.java*/
import java.awt.Frame;
import java.awt.MenuBar;
import java.awt.Menu;
import java.awt.MenuItem;

public class MenuExample {
 static MenuBar menubar;
 static Menu menu,submenu;
 static MenuItem mi1,mi2,mi3,mi4,mi5;
 public static void main(String[] args) {
 Frame frame = new Frame("Menu and MenuItem Example");

 menubar = new MenuBar(); //define menubar
 menu = new Menu("Menu"); // define menu
 submenu = new Menu("Soft Drink"); //define sub-menu

 mi1 = new MenuItem("Pizza"); //define menu item
 mi2 = new MenuItem("Mo:Mo");
 mi3 = new MenuItem("Chowmein");
 mi4 = new MenuItem("Coffee");
 mi5 = new MenuItem("Tea");

 menu.add(mi1); //add item to menu
 menu.add(mi2);
 menu.add(mi3);
 menu.add(submenu); // add sub-menu to menu

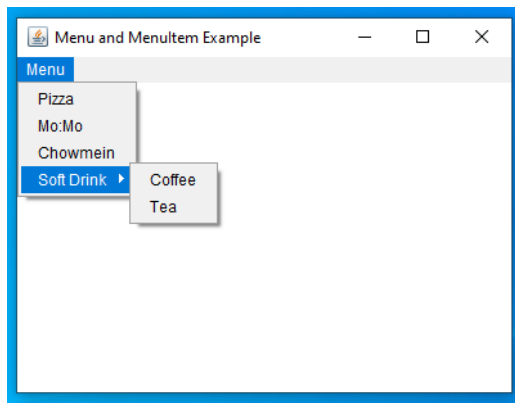
 submenu.add(mi4); // add item to sub-menu
 submenu.add(mi5);

 menubar.add(menu); //add menu to menu bar
 frame.setMenuBar(menubar); //set menubar to frame

 frame.setLayout(null);
 frame.setSize(400,300);
 frame.setVisible(true);
 }
}

```

Output:



- xi. **PopupMenu** – The *java.awt.PopupMenu* class provides Popup Menu. Popup menu represents a menu which can be dynamically popped up at a specified position within a component.

**Syntax:**

`PopupMenu ppm = new PopupMenu();` // *construct a new popup menu with an empty string*

`PopupMenu ppm = new PopupMenu(String label);` // *construct a new popup menu with an the specified name.*

PopupMenu class methods:

| S.N. | Method & Description                                                                                                           |
|------|--------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>void addNotify()</b><br>Creates the popup menu's peer.                                                                      |
| 2    | <b>AccessibleContext getAccessibleContext()</b><br>Gets the AccessibleContext associated with this PopupMenu.                  |
| 3    | <b>MenuContainer getParent()</b><br>Returns the parent container for this menu component.                                      |
| 4    | <b>void show(Component origin, int x, int y)</b><br>Shows the popup menu at the x, y position relative to an origin component. |

**Example 20:** Creating a popup menu attached to frame. Also , if you click within the area of frame popup menu will be shown. Similarly, when clicked on menu item some message shown on label.

*/\*PopupMenuExample.java\*/*

```
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
class PopupMenuExample implements ActionListener
{
```

```

Frame frame;
Label status_label;
PopupMenuExample() {
 frame = new Frame("PopupMenu Example");

 status_label = new Label();
 status_label.setBounds(20,20,300,40);
 status_label.setAlignment(Label.CENTER);
 status_label.setSize(350,50);

 PopupMenu popupmenu = new PopupMenu("Edit");

 MenuItem cut = new MenuItem("Cut");
 cut.setActionCommand("Cut");
 cut.addActionListener(this);

 MenuItem copy = new MenuItem("Copy");
 copy.setActionCommand("Copy");
 copy.addActionListener(this);

 MenuItem paste = new MenuItem("Paste");
 paste.setActionCommand("Paste");
 paste.addActionListener(this);

 popupmenu.add(cut);
 popupmenu.add(copy);
 popupmenu.add(paste);

 frame.addMouseListener(new MouseAdapter() {
 public void mouseClicked(MouseEvent e) {
 popupmenu.show(frame,e.getX(),e.getY());
 }
 });

 frame.add(popupmenu);
 frame.add(status_label);

 frame.setSize(400,400);
 frame.setLayout(null);
 frame.setVisible(true);
}

public void actionPerformed(ActionEvent ae){
 status_label.setText(ae.getActionCommand() + " MenuItem Clicked.");
}

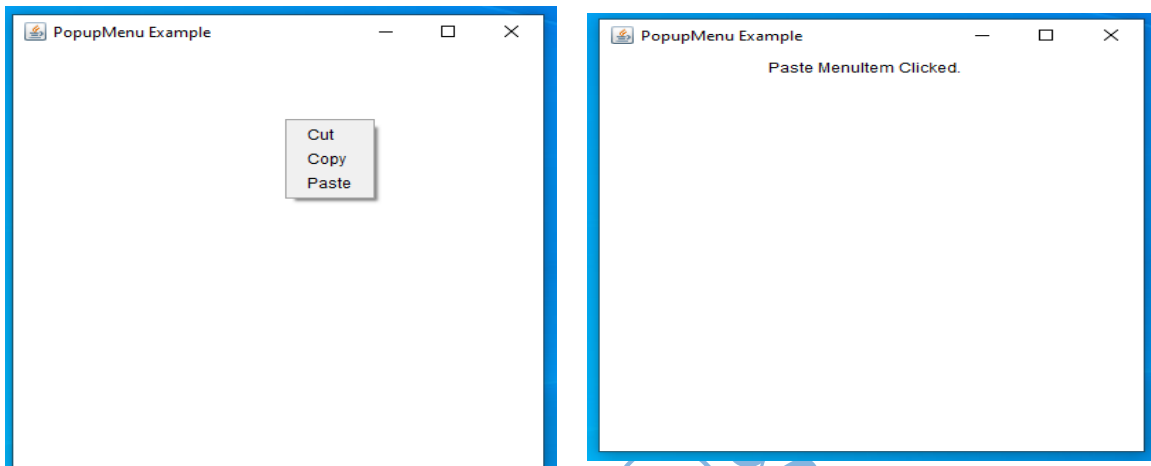
public static void main(String args[])
{
 new PopupMenuExample();
}

```



}

### Output:



- xii. Dialog – The dialog control represents a top level window with a border and a title used to take some form of input from the user . It inherits the window class. As we know Frame and Dialog both inherits Window class , but Frame has maximize and minimize buttons but Dialog doesn't have. The *java.awt.Dialog* class represents Dialog.

#### Syntax:

Dialog d1 = new Dialog(Dialog owner); // *Constructs an initially invisible , modeless Dialog with the specified owner and an empty title.*

Dialog d2 = new Dialog(Dialog owner, String title); //*Constructs an initially invisible, modeless Dialog with the specified owner Dialog and title.*

Dialog d3 = new Dialog(Dialog owner , String title, Boolean modal); //*Constructs an initially invisible Dialog with the specified owner Dialog, title, and modality.*

Dialog d4 = new Dialog(Frame owner); //*Constructs an initially invisible, modeless Dialog with the specified owner Frame and an empty title.*

Dialog d5 = new Dialog(Frame owner , String title, Boolean modal); //*Constructs an initially invisible Dialog with the specified owner Frame, title and modality .*

Dialog d6 = new Dialog(Window owner); //*Constructs an initially invisible, modeless Dialog with the specified owner Window and an empty title.*

Dialog d7 = new Dialog(Window owner, String title); //*Constructs an initially invisible, modeless Dialog with the specified owner Window and title.*

#### Dialog Class Methods:

| Methods                                  | Description                                                                 |
|------------------------------------------|-----------------------------------------------------------------------------|
| void addNotify()                         | Makes this Dialog displayable by connecting it to a native screen resource. |
| AccessibleContext getAccessibleContext() | Gets the AccessibleContext associated with this Dialog.                     |
| String getTitle()                        | Gets the title of the dialog.                                               |
| boolean isModal()                        | Indicates whether the dialog is modal.                                      |

|                                |                                                                                                                                              |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| boolean isResizable()          | Indicates whether this dialog is resizable by the user.                                                                                      |
| protected String paramString() | Returns a string representing the state of this dialog.                                                                                      |
| void setTitle(String title)    | Sets the title of the Dialog.                                                                                                                |
| void setVisible(boolean b)     | Shows or hides this Dialog depending on the value of parameter b.                                                                            |
| void toBack()                  | If this Window is visible, sends this Window to the back and may cause it to lose focus or activation if it is the focused or active Window. |

**Example 21:** Creating Dialog with Label and Button , also Dialog attached to Frame. When we click button “Close Dialog” , Dialog close from application but not from machine.

**/\*DialogExample.java\*/**

```
import java.awt.Frame;
import java.awt.Dialog;
import java.awt.FlowLayout;
import java.awt.Label;
import java.awt.Button;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class DialogExample {
 Frame frame;
 static Dialog dlg;
 Button btn;
 DialogExample(){
 frame = new Frame();

 dlg = new Dialog(frame,"Dialog Example",true);

 btn = new Button("Close Dialog");
 btn.addActionListener(new ActionListener(){
 public void actionPerformed(ActionEvent ae){
 DialogExample.dlg.setVisible(false);
 }
 });
 dlg.add(new Label("Click button to close Dialog."));
 dlg.add(btn);
 dlg.setSize(400,400);
 dlg.setLayout(new FlowLayout());
 dlg.setVisible(true);
 }
 public static void main(String[] args) {
 new DialogExample();
 }
}
```

- xiii. **Toolkit** – Toolkit class is the abstract superclass of every implementation in the Abstract Window Toolkit. Subclasses of Toolkit are used to bind various components.

**Example 22:** We use Toolkit class to get screen resolution, screen width and screen height.

*/\*ToolkitExample.java\*/*

```
import java.awt.Toolkit;
import java.awt.Dimension;
public class ToolkitExample {
 public static void main(String[] args) {
 Toolkit tkt = Toolkit.getDefaultToolkit();
 System.out.println("Screen resolution::"+tkt.getScreenResolution());
 Dimension dm = tkt.getScreenSize();
 System.out.println("Screen width::"+dm.width);
 System.out.println("Screen height::"+dm.height);
 }
}
```

**Output:**

Screen resolution::96 //1 inch = 96 pixels

Screen width::1366

Screen height::768

**Example 23:** With the help of following program, we are able to change icon of frame and when clicked on a button sounds BEEP.

*/\*ToolkitExample.java\*/*

```
import java.awt.Toolkit;
import java.awt.Frame;
import java.awt.Button;
import java.awt.Image;
import java.awt.Label;
import java.awt.event.*;
public class ToolkitExample {
 public static void main(String[] args) {
 Frame f=new Frame("Toolkit Example");

 Button b=new Button("Beep");
 b.setBounds(50,100,60,30);

 Label lbl2 = new Label("Click above button, it sounds BEEP.");
 lbl2.setBounds(15,140,200,30);

 b.addActionListener(new ActionListener(){
 public void actionPerformed(ActionEvent e){
 Toolkit.getDefaultToolkit().beep();
 }
 });

 Image icn = Toolkit.getDefaultToolkit().getImage("C://Users/Dell/Desktop/update
folder/echulo/img.jpg");
 f.setIconImage(icn);

 Label lbl1 = new Label("See icon changed before frame title.");
 }
}
```

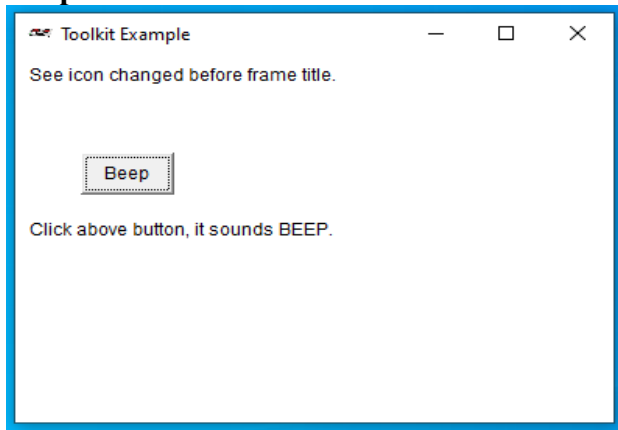
```

 lbl1.setBounds(15,35,200,20);

 f.add(b);
 f.add(lbl1);
 f.add(lbl2);
 f.setSize(400,300);
 f.setLayout(null);
 f.setVisible(true);
 }
}

```

### Output:



## How to close AWT Window in Java ?

AWT Window or Frame can be closed by calling *dispose()* or *System.exit()* inside *windowClosing()* method. The *windowClosing()* method is in *WindowListener* interface and *WindowAdapter* class.

If you use *WindowAdapter* class which implements *WindowListener* interfaces. It provides the default implementation of all the 7 methods of *WindowListener* interface.

If you implement the *WindowListener* interface, you will be forced to override all the 7 methods of *WindowListener* interface. So it is better to use *WindowAdapter* class.

### Different ways to override windowClosing() method:

- By anonymous class
- By inheriting WindowAdapter class
- By implementing WindowListener interface

**Example 24:** Using anonymous class to close AWT Window.

```
/*AwtWindowExample.java*/
```

```

import java.awt.Frame;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
public class AwtWindowExample extends Frame{
 AwtWindowExample(){
 addWindowListener(new WindowAdapter(){

```

```

 public void windowClosing(WindowEvent we){
 dispose();
 }
 });
 setTitle("Closing AWT window");
 setSize(300,200);
 setLayout(null);
 setVisible(true);
}
public static void main(String[] args) {
 new AwtWindowExample();
}
}

```

**Example 25:** Implementing the window closing functionality using the **WindowAdapter** class.

**/\*AwtWindowExample1.java\*/**

```

import java.awt.Frame;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
public class AwtWindowExample1 extends WindowAdapter{
 Frame frame;
 AwtWindowExample1(){
 frame = new Frame();
 frame.addWindowListener(this);
 frame.setTitle("Closing AWT window");
 frame.setSize(300,200);
 frame.setLayout(null);
 frame.setVisible(true);
 }
 public void windowClosing(WindowEvent we){
 frame.dispose();
 }
 public static void main(String[] args) {
 new AwtWindowExample1();
 }
}

```

**Example 26:** Implementing the **WindowListener** interface to display the functionality of closing the AWT Window.

**/\*AwtWindowExample2.java\*/**

```

import java.awt.Frame;
import java.awt.event.WindowListener;
import java.awt.event.WindowEvent;
public class AwtWindowExample implements WindowListener{

```

```

Frame frame;
AwtWindowExample(){
 frame = new Frame();
 frame.addWindowListener(this);
 frame.setTitle("Closing AWT window");
 frame.setSize(300,200);
 frame.setLayout(null);
 frame.setVisible(true);
}
public void windowActivated(WindowEvent we){}
public void windowClosed(WindowEvent we){}
public void windowClosing(WindowEvent we){
 // frame.dispose();
 System.exit(0); // can also use exit()
}
public void windowDeactivated(WindowEvent we){}
public void windowDeiconified(WindowEvent we){}
public void windowIconified(WindowEvent we){}
public void windowOpened(WindowEvent we){}
public static void main(String[] args) {
 new AwtWindowExample();
}
}

```

## Java Swing:

- Java Swing is a part of Java Foundation Classes (JFC) that is used to create window-based applications.
- It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in Java.
- Java Swing provides platform-independent and lightweight components.
- The javax.swing package provides class for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckBox, JMenu, JColorChooser, JComboBox, JOptionPane, JTable, JList, JScrollBar etc.
- Swing supports pluggable look and feel.
- Swing follows MVC pattern.

## Lightweight vs Heavyweight GUI Components:

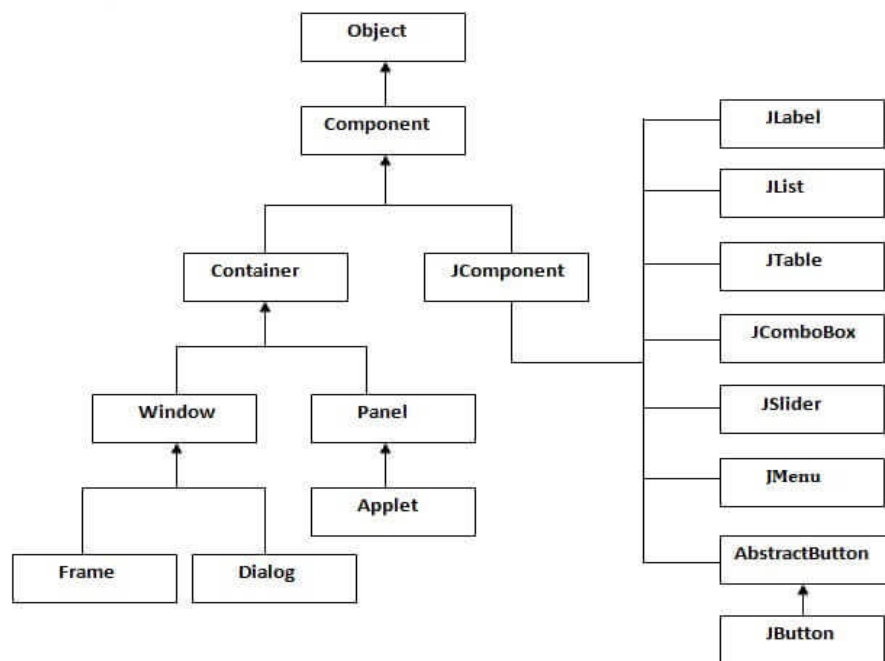
Most Swing components are lightweight components – they're written, manipulated and displayed completely in Java. AWT components are heavyweight components, because they rely on the local platform's windowing system to determine their functionality and their look-and-feel. Several Swing components are heavyweight components.

## What is JFC ?

JFC stands for Java Foundation Classes are a set of GUI components which simplify the development of desktop applications.

**Note:** We will discuss Swing components that are not provided by AWT under Java Swing. Other similar components define same like AWT components but classes to define those components are different. **Example:** `java.awt.Button` class define **Button** in **AWT** where `javax.swing.JButton` class define **Button**(i.e. **JButton**) in **Java Swing**. As we know Swing includes all AWT components and some additional components.

## Hierarchy of Java Swing Classes:



From above list of components, some of them are already discussed before in AWT. So, let's have look on different components from Java Swing.

- i. **JOptionPane** – The **JOptionPane** class(package `javax.swing`) provides prebuilt standard dialog boxes such as message dialog box, confirm dialog box and input dialog box. These are displayed by invoking static **JOptionPane** methods.

**Input Dialogs:** **JOptionPane** static method `showInputDialog()` displays an input dialog. The user types characters in the text field, then clicks **OK** or presses the Enter key to submit the String to the program. If the user clicks **Cancel**, `showInputDialog` returns null.

### Syntax:

`String var_name = JOptionPane.showInputDialog(String label);`

**Message Dialogs:** **JOptionPane** static method `showMessageDialog()` used to display a message dialog.

### Syntax:

`JOptionPane.showMessageDialog(position,message_to_display,title_bar,JOptionPane.message_dialog);`

**JOptionPane Message Dialog Constants:** All message dialog types except `PLAIN_MESSAGE` display an icon to the left of the message. A `Question_MESSAGE` icon is the default icon for an input dialog box.

| Message dialog type | Icon                                                                              | Description                                                                                                      |
|---------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| ERROR_MESSAGE       |  | Indicates an error.                                                                                              |
| INFORMATION_MESSAGE |  | Indicates an informational message.                                                                              |
| WARNING_MESSAGE     |  | Warns of a potential problem.                                                                                    |
| QUESTION_MESSAGE    |  | Poses a question. This dialog normally requires a response, such as clicking a <b>Yes</b> or a <b>No</b> button. |
| PLAIN_MESSAGE       | no icon                                                                           | A dialog that contains a message, but no icon.                                                                   |

**Confirm Dialogs:** JOptionPane static method `showConfirmDialog()` used to asks a confirming question, like yes/no/cancel.

**Syntax:**

*JOptionPane.showMessageDialog(position,message\_to\_display,title\_bar,JOptionPane.option);*

**Option Type:**

- DEFAULT\_OPTION
- YES\_NO\_OPTION
- YES\_NO\_CANCEL\_OPTION
- OK\_CANCEL\_OPTION

**Example 27:** Write a program to sum two numbers from user and display sum message using dialogs .

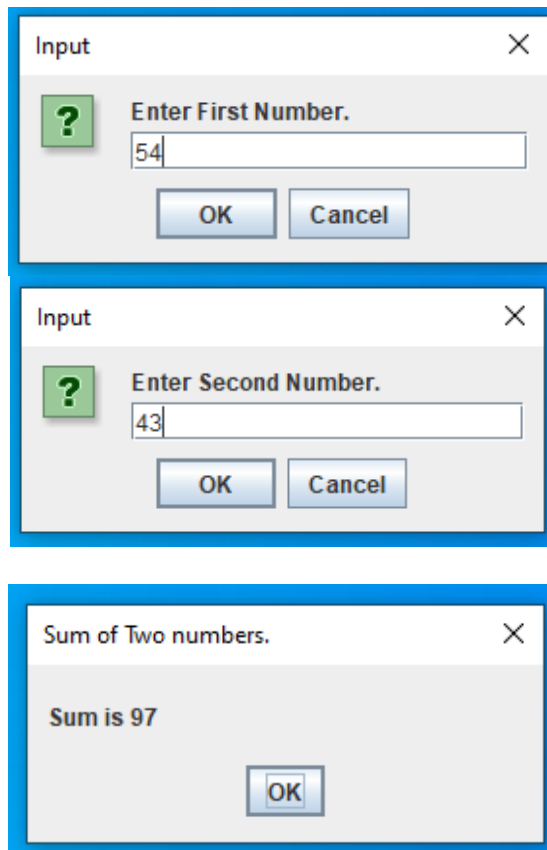
**/\*InputAndMessageDialogExample.java\*/**

```
import javax.swing.JOptionPane;

public class InputAndMessageDialogExample {
 public static void main(String[] args) {
 String first_num = JOptionPane.showInputDialog("Enter First Number.");
 String second_num = JOptionPane.showInputDialog("Enter Second Number.");
 int num1 = Integer.parseInt(first_num);
 int num2 = Integer.parseInt(second_num);
 int sum = num1 + num2;
 // If the first argument is null, the dialog box is displayed at the center of your screen
 JOptionPane.showMessageDialog(null, "Sum is "+sum,"Sum of Two numbers.",
 JOptionPane.PLAIN_MESSAGE);
 }
}
```

**Output:**





**Example 28:** Run below program that results confirm dialog when you try to close Frame or Window or Application.

```

/*ConfirmDialogExample.java*/
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
public class ConfirmDialogExample extends WindowAdapter {
 JFrame frame;
 ConfirmDialogExample(){
 frame = new JFrame("Confirm Dialog Example.");
 frame.addWindowListener(this);
 frame.setSize(360,300);
 frame.setLayout(null);
 frame.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
 frame.setVisible(true);
 }
 public void windowClosing(WindowEvent we){
 int a = JOptionPane.showConfirmDialog(frame, "Are you sure?", "Confirmation",
 JOptionPane.YES_NO_CANCEL_OPTION);
 System.out.println(a); //return 0 if YES
 if(a==JOptionPane.YES_OPTION){
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 }
 }
}

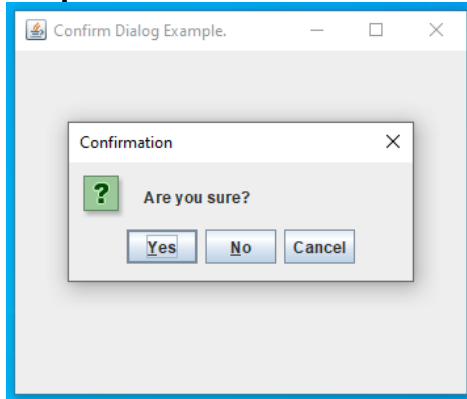
```

```

 public static void main(String[] args) {
 new ConfirmDialogExample();
 }
}

```

### Output:



ii. **JCheckBox** – The `javax.swing.JCheckBox` class defines `JCheckBox`. Let's see an example.

**Example 29:** We use three `JCheckBox`s to select the desired font style of the text displayed in a `JTextField`. When selected, one applies bold style, another applies italic style and last one applies plain style.

*/\* JCheckBoxExample.java \*/*

```

import javax.swing.JCheckBox;
import javax.swing.JFrame;
import javax.swing.JTextField;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ItemListener;
import java.awt.event.ItemEvent;

public class JCheckBoxExample extends JFrame implements ItemListener{
 JTextField tft;
 JCheckBox boldBtn,italicBtn,plainBtn;
 JCheckBoxExample(){

 tft = new JTextField("You can write something here. Select below option for result ");
 tft.setFont(new Font("Calibri Light",Font.PLAIN,14));
 add(tft);

 boldBtn = new JCheckBox("Bold");
 add(boldBtn);
 italicBtn = new JCheckBox("Italic");
 add(italicBtn);
 plainBtn = new JCheckBox("Plain");
 add(plainBtn);

 boldBtn.addItemListener(this);
 italicBtn.addItemListener(this);
 }
}

```

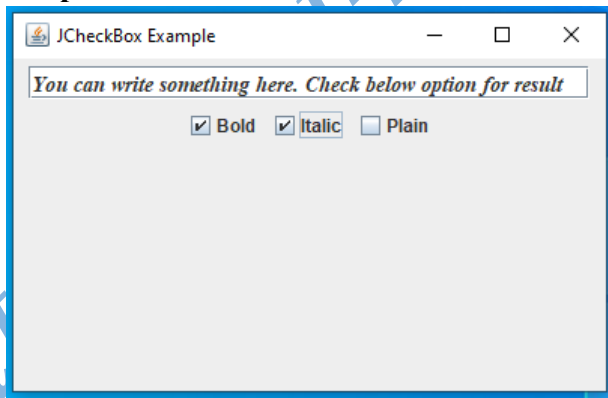
```

plainBtn.addItemListener(this);

setTitle("JCheckBox Example");
setLayout(new FlowLayout());
setSize(410,260);
setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
setVisible(true);
}
public void itemStateChanged(ItemEvent ie){
 Font font = null;
 if(boldBtn.isSelected() && italicBtn.isSelected())
 font = new Font("serif",Font.BOLD+Font.ITALIC,14);
 else if(boldBtn.isSelected())
 font = new Font("serif",Font.BOLD,14);
 else if(italicBtn.isSelected())
 font = new Font("serif",Font.ITALIC,14);
 else if(plainBtn.isSelected())
 font = new Font("Times New Roman",Font.PLAIN,14);
 else
 font = new Font("Calibri Light",Font.PLAIN,14);
 tft.setFont(font);
}
public static void main(String[] args) {
 new JCheckBoxExample();
}
}

```

### Output:



- iii. **JRadioButton** – The `JRadioButton` class is used to create a radio button. It is used to choose one option from multiple options. The `javax.swing.JRadioButton` class defines `JRadioButton`. Radio buttons normally appear as a group in which only one button can be selected at a time. Radio buttons are used to represent mutually exclusive options (i.e. multiple options in the group cannot be selected at the same time). The relationship between radio buttons is maintained by a `ButtonGroup` object (package `javax.swing`), which itself is not a GUI component.

### Syntax:

JRadioButton rbtn1 = new JRadioButton(); // creates an unselected radio button with no text.  
 JRadioButton rbtn2 = new JRadioButton(String s); //Creates an unselected radio button with specified text.  
 JRadioButton rbtn3 = new JRadioButton(String s, Boolean selected); //creates a radio button with the specified text and selected status.

**Commonly used Methods:**

| Methods                                  | Description                                          |
|------------------------------------------|------------------------------------------------------|
| void setText(String s)                   | It is used to set specified text on button.          |
| String getText()                         | It is used to return the text of the button.         |
| void setEnabled(boolean b)               | It is used to enable or disable the button.          |
| void setIcon(Icon icn)                   | It is used to set the specified Icon on the button.  |
| Icon getIcon()                           | It is used to get the Icon of the button.            |
| Void setMnemonic(int a)                  | It is used to set the mnemonic on the button.        |
| Void addActionListener(ActionListener a) | It is used to add the action listener to this object |

**Example 30:** The application here is similar as Example 29, using Radio Button.

**/\* JRadioButtonExample.java \*/**

```
import javax.swing.JRadioButton;
import javax.swing.ButtonGroup;
import javax.swing.JFrame;
import javax.swing.JTextField;
import javax.swing.ButtonGroup;

import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ItemListener;
import java.awt.event.ItemEvent;

public class JRadioButtonExample extends JFrame implements ItemListener{
 JTextField tft;
 JRadioButton boldBtn,italicBtn,boldItalicBtn,plainBtn;
 ButtonGroup radioGroup;
 JRadioButtonExample(){

 tft = new JTextField("You can write something here. Check below option for result ");
 tft.setFont(new Font("Calibri Light",Font.PLAIN,14));
 add(tft);

 boldBtn = new JRadioButton("Bold");
 add(boldBtn);
 italicBtn = new JRadioButton("Italic");
 add(italicBtn);
 boldItalicBtn = new JRadioButton("Bold & Italic");
 add(boldItalicBtn);
 plainBtn = new JRadioButton("Plain");
 add(plainBtn);

 //create logical relationship between JRadioButtons
```

```

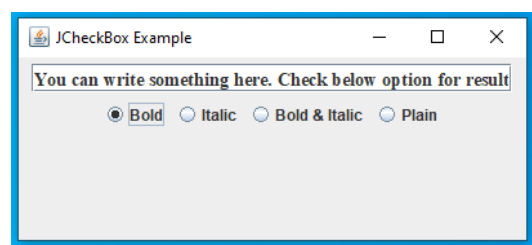
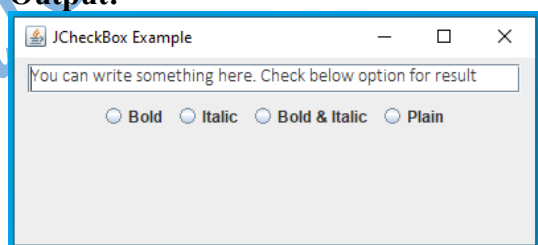
radioGroup = new ButtonGroup(); //Create ButtonGroup
radioGroup.add(boldBtn);
radioGroup.add(italicBtn);
radioGroup.add(boldItalicBtn);
radioGroup.add(plainBtn);

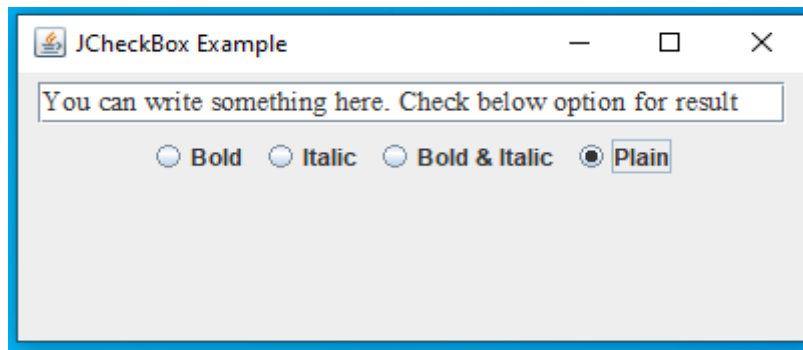
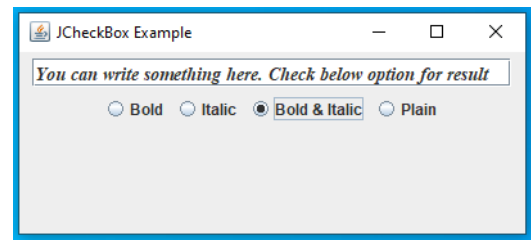
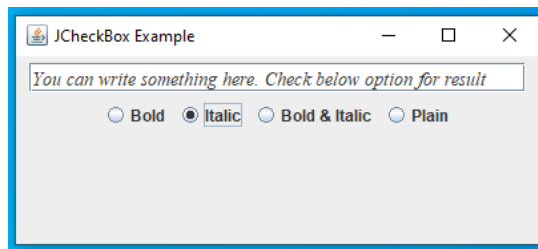
boldBtn.addItemListener(this);
italicBtn.addItemListener(this);
boldItalicBtn.addItemListener(this);
plainBtn.addItemListener(this);

setTitle("JCheckBox Example");
setLayout(new FlowLayout());
setSize(410,180);
setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
setVisible(true);
}
public void itemStateChanged(ItemEvent ie){
 Font font = null;
 if(boldItalicBtn.isSelected())
 font = new Font("serif",Font.BOLD+Font.ITALIC,14);
 else if(boldBtn.isSelected())
 font = new Font("serif",Font.BOLD,14);
 else if(italicBtn.isSelected())
 font = new Font("serif",Font.ITALIC,14);
 else if(plainBtn.isSelected())
 font = new Font("Times New Roman",Font.PLAIN,14);
 else
 font = new Font("Calibri Light",Font.PLAIN,14);
 tft.setFont(font);
}
public static void main(String[] args) {
 new JRadioButtonExample();
}
}

```

### Output:





- iv. **JComboBox** – A combo box (sometimes called a drop-down list) enables the user to select one item from a list. Combo boxes are implemented with class `JComboBox(javax.swing.JComboBox)`, which extends class `JComponent`. `JComboBox` is a generic class, like the class `ArrayList`. When you create a `JComboBox`, you specify the type of the objects that it manages – the `JComboBox` then displays a `String` representation of each object.

#### Syntax:

`JComboBox variable_name = new JComboBox();`

#### Commonly used JComboBox constructors:

| Constructor                                   | Description                                                                                        |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>JComboBox()</code>                      | Creates a <code>JComboBox</code> with a default data model.                                        |
| <code>JComboBox(Object[] items)</code>        | Creates a <code>JComboBox</code> that contains the elements in the specified array.                |
| <code>JComboBox(Vector&lt;?&gt; items)</code> | Creates a <code>JComboBox</code> that contains the elements in the specified <code>Vector</code> . |

#### Commonly used Methods:

| Methods                                                | Description                                                       |
|--------------------------------------------------------|-------------------------------------------------------------------|
| <code>void addItem(Object anObject)</code>             | Used to add an item to the item list.                             |
| <code>void removeItem(Object anObject)</code>          | Used to delete an item to the item list.                          |
| <code>void removeAllItems()</code>                     | Used to remove all the items from the list.                       |
| <code>void setEditable(boolean b)</code>               | Used to determine whether the <code>JComboBox</code> is editable. |
| <code>void addActionListener(ActionListener ae)</code> | Used to add the <code>ActionListener</code> .                     |
| <code>void addItemListener(ItemListener il)</code>     | Used to add the <code>ItemListener</code> .                       |

**Example 31:** In this example we create label, combo box and button. When you click button it shows message in label with selected combo box item .

*/\*JComboBoxExample.java\*/*

`import javax.swing.JButton;`

```

import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;

import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class JComboBoxExample {
 JFrame frame;
 JLabel lbl;
 JButton btn;
 JComboBox cmb;
 JComboBoxExample(){
 frame = new JFrame("JComboBox Example");

 lbl = new JLabel("hello");
 lbl.setHorizontalAlignment(JLabel.CENTER);
 lbl.setSize(300,30);

 String[] c_lang = {"C","C++","Java","PHP"};
 cmb = new JComboBox(c_lang);
 cmb.setEditable(true); //set combo box editable

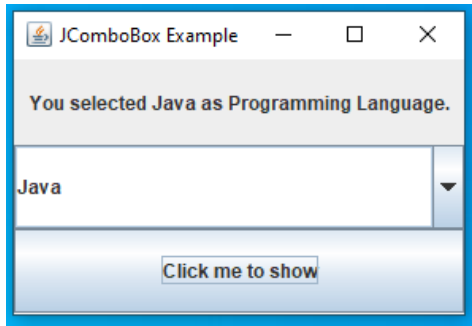
 btn = new JButton("Click me to show");
 btn.addActionListener(new ActionListener(){
 public void actionPerformed(ActionEvent ae){
 String info = "You selected "+cmb.getItemAt(cmb.getSelectedIndex())+" as Programming
Language.";
 lbl.setText(info);
 }
 });

 frame.add(lbl);
 frame.add(cmb);
 frame.add(btn);

 frame.setSize(300,200);
 frame.setLayout(new GridLayout(3,1));
 frame.setVisible(true);
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 }
 public static void main(String[] args) {
 new JComboBoxExample();
 }
}

```

**Output:**



**Example 32:** Creating application that uses a JComboBox to provides a list of four image filenames from which the user can select one image to display.

**/\* JComboBoxExample.java \*/**

```
import javax.swing.Icon;
import javax.swing.ImageIcon;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.SwingConstants;

import java.awt.FlowLayout;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;

public class JComboBoxExample {
 JFrame frame;
 JLabel lbl; //display selected icon or image
 JComboBox<String> imagesJComboBox; // holds icon names
 JComboBoxExample(){
 frame = new JFrame("JComboBox Example");

 final String[] names = {"Lily","Lotus","Marigold","Rose"};
 final String[] location =
{"images/lily.jpg","images/lotus.jpg","images/marigold.jpg","images/rose.jpg"};
 final Icon[] icons = { new ImageIcon(getClass().getResource(location[0])),
 new ImageIcon(getClass().getResource(location[1])),
 new ImageIcon(getClass().getResource(location[2])),
 new ImageIcon(getClass().getResource(location[3]))
 };
 imagesJComboBox = new JComboBox<String>(names); //set up JComboBox
 imagesJComboBox.setMaximumRowCount(3); //display three rows
 imagesJComboBox.setBounds(30,100,80,30);

 imagesJComboBox.addItemListener(new ItemListener(){
 public void itemStateChanged(ItemEvent ie){
 //determine whether item selected
 if(ie.getStateChange() == ItemEvent.SELECTED){
 lbl.setIcon(icons[imagesJComboBox.getSelectedIndex()]);
 }
 }
 })
 }
}
```



```

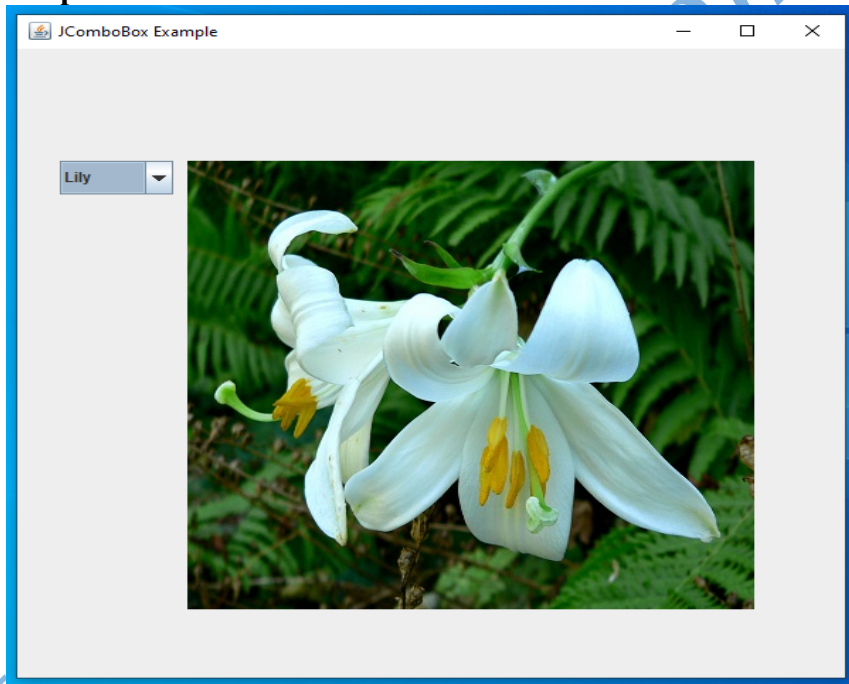
});

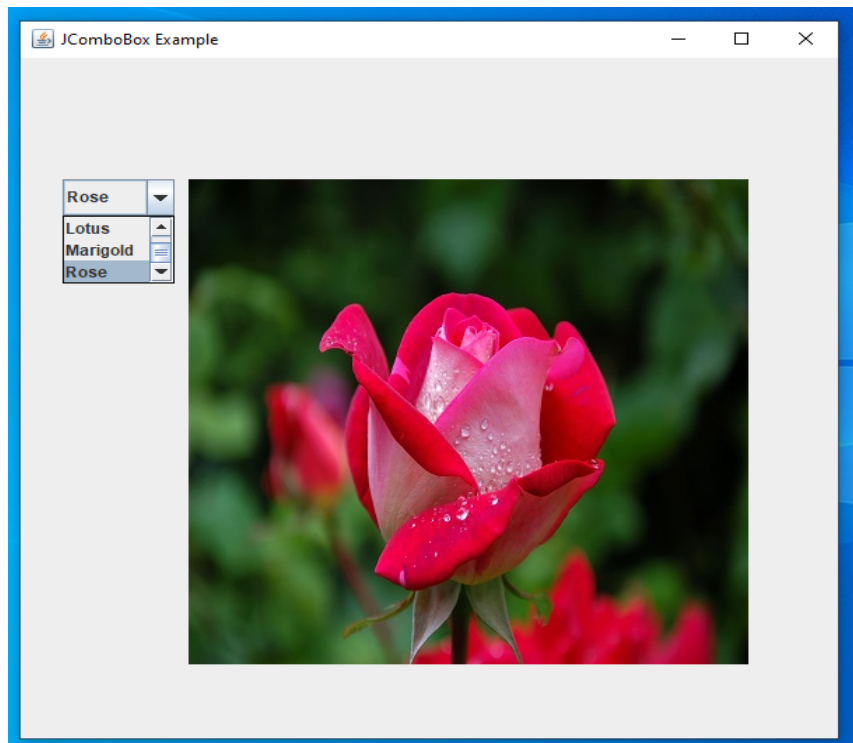
frame.add(imagesJComboBox); // add JComboBox to JFrame
lbl = new JLabel(icons[0]); //display first icon
lbl.setBounds(120,100,400,400);
frame.add(lbl); // add label to JFrame

frame.setSize(600,600);
frame.setLayout(null);
frame.setVisible(true);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
public static void main(String[] args) {
 new JComboBoxExample();
}
}

```

### Output:





**Note:** Closable Frame in Swing is performed either using `windowClosing()` method likewise AWT Or by using `setDefaultCloseOperation()` method. So, let's look after `setDefaultCloseOperation()` method where `windowClosing()` method already discussed.

### **setDefaultCloseOperation() Method:**

This method is used to specify one of several options for the close button of frame. Use one of the following constants to specify your choice:

- `JFrame.DISPOSE_ON_CLOSE` – Exit that running application only. Example: Among two frame if you exit frame2 where frame 1 still running.
- `JFrame.HIDE_ON_CLOSE` – Hide the frame, but keep the application running.
- `JFrame.EXIT_ON_CLOSE` – Dispose of the frame object, which means kill application from JVM.
- `JFrame.DO_NOTHING_ON_CLOSE` – Just Ignore the click.

Frame without `setDefaultCloseOperation()` method will have `JFrame.HIDE_ON_CLOSE` by default.

### **Layout Management:**

Layout Managers are used to arrange components in a particular manner. The Java LayoutManagers facilitates us to control the positioning and size of the components in GUI forms. LayoutManager is an interface that is implemented by all the classes of layout managers. There are the following classes that represent the layout managers.

1. `java.awt.BorderLayout`
2. `java.awt.FlowLayout`

3. java.awt.GridLayout
4. java.awt.CardLayout
5. java.awt.GridBagLayout
6. javax.swing.BoxLayout
7. javax.swing.GroupLayout
8. javax.swing.ScrollPaneLayout
9. javax.swing.SpringLayout etc.

Here, we will discuss some of them.

### **BorderLayout:**

BorderLayout is used to arrange the components in five regions: north, south, east, west, and center. Each region (area) may contain one component only. It is the default layout of a frame or window. The BorderLayout provides five constants for each region:

1. public static final int NORTH
2. public static final int SOUTH
3. public static final int EAST
4. public static final int WEST
5. public static final int CENTER

### **Constructors of BorderLayout class:**

- BorderLayout(): creates a border layout but with no gaps between the components.
- BorderLayout(int hgap, int vgap): creates a border layout with the given horizontal and vertical gaps between the components.

### **Example 33:**

```
/*BorderLayoutExample.java*/
import java.awt.BorderLayout;
import javax.swing.JFrame;
import javax.swing.JButton;
public class BorderLayoutExample
{
 JFrame frame;
 // constructor
 BorderLayoutExample()
 {
 // creating a frame
 frame = new JFrame("Border Layout Example");
 // horizontal as well as the vertical gap is 5 units.
 frame.setLayout(new BorderLayout(5,5));
 // creating the buttons
 JButton b1 = new JButton("NORTH");
 JButton b2 = new JButton("EAST");
 JButton b3 = new JButton("SOUTH");
 JButton b4 = new JButton("WEST");
 JButton b5 = new JButton("CENTER");
 }
}
```

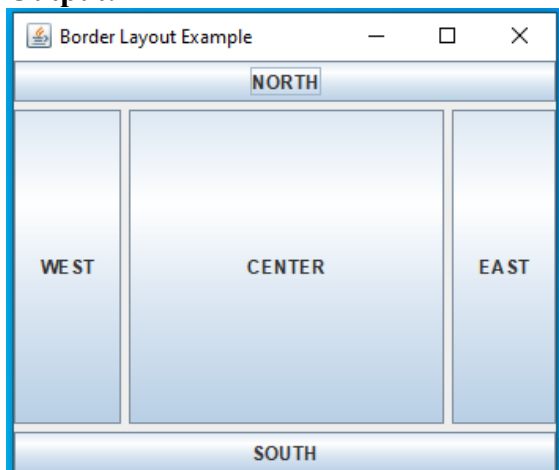
```

// adding the buttons to frame
frame.add(b1,BorderLayout.NORTH);
frame.add(b2,BorderLayout.EAST);
frame.add(b3,BorderLayout.SOUTH);
frame.add(b4,BorderLayout.WEST);
frame.add(b5,BorderLayout.CENTER);

frame.setSize(360, 300);
frame.setVisible(true);
frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
}
public static void main(String args[])
{
 new BorderLayoutExample();
}
}

```

### Output:



### FlowLayout :

The Java FlowLayout class is used to arrange the components in a line, one after another (in a flow). It is the default layout of the applet or panel.

#### Fields of FlowLayout class:

1. public static final int LEFT
2. public static final int RIGHT
3. public static final int CENTER
4. public static final int LEADING
5. public static final int TRAILING

### Constructors of FlowLayout class:

1. **FlowLayout():** creates a flow layout with centered alignment and a default 5 unit horizontal and vertical gap.
2. **FlowLayout(int align):** creates a flow layout with the given alignment and a default 5 unit horizontal and vertical gap.
3. **FlowLayout(int align, int hgap, int vgap):** creates a flow layout with the given alignment and the given horizontal and vertical gap.

### Example 34:

**/\*FlowLayoutExample.java\*/**

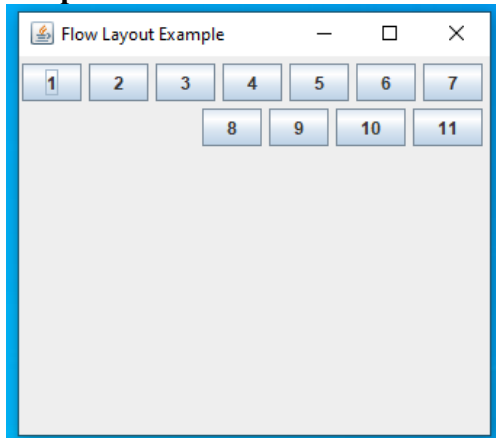
```
import java.awt.FlowLayout;
import javax.swing.JFrame;
import javax.swing.JButton;
public class FlowLayoutExample
{
 JFrame frameObj;
 // constructor
 FlowLayoutExample ()
 {
 // creating a frame object
 frameObj = new JFrame("Flow Layout Example");
 // creating the buttons
 JButton b1 = new JButton("1");
 JButton b2 = new JButton("2");
 JButton b3 = new JButton("3");
 JButton b4 = new JButton("4");
 JButton b5 = new JButton("5");
 JButton b6 = new JButton("6");
 JButton b7 = new JButton("7");
 JButton b8 = new JButton("8");
 JButton b9 = new JButton("9");
 JButton b10 = new JButton("10");
 JButton b11 = new JButton("11");
 // adding the buttons to frame
 frameObj.add(b1); frameObj.add(b2);
 frameObj.add(b3); frameObj.add(b4);
 frameObj.add(b5); frameObj.add(b6);
 frameObj.add(b7); frameObj.add(b8);
 frameObj.add(b9); frameObj.add(b10);
 frameObj.add(b11);
 // parameter less constructor is used
 // therefore, alignment is center
 // horizontal as well as the vertical gap is 5 units.
 frameObj.setLayout(new FlowLayout(FlowLayout.RIGHT));
 frameObj.setSize(340, 300);
 }
}
```

```

 frameObj.setVisible(true);
 frameObj.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
 }
 public static void main(String argsv[])
 {
 new FlowLayoutExample();
 }
}

```

### Output:



### GridLayout:

Used to arrange the components in a rectangular grid. One component is displayed in each rectangle.

Constructors of GridLayout class:

1. **GridLayout():** creates a grid layout with one column per component in a row.
2. **GridLayout(int rows, int columns):** creates a grid layout with the given rows and columns but no gaps between the components.
3. **GridLayout(int rows, int columns, int hgap, int vgap):** creates a grid layout with the given rows and columns along with given horizontal and vertical gaps

### Example 35:

*/\*GridLayoutExample.java\*/*

```

import java.awt.GridLayout;
import javax.swing.JFrame;
import javax.swing.JButton;
public class GridLayoutExample
{
 JFrame frameObj;
 GridLayoutExample ()
 {
 // creating a frame object
 frameObj = new JFrame("Grid Layout Example");
 }
}

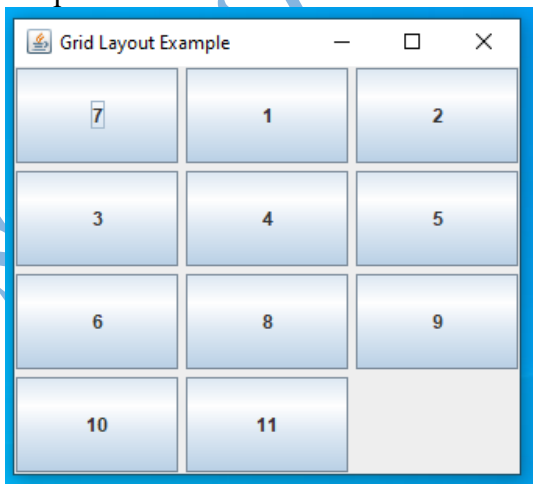
```

```

// creating the buttons
JButton b1 = new JButton("1");
JButton b2 = new JButton("2");
JButton b3 = new JButton("3");
JButton b4 = new JButton("4");
JButton b5 = new JButton("5");
JButton b6 = new JButton("6");
JButton b7 = new JButton("7");
JButton b8 = new JButton("8");
JButton b9 = new JButton("9");
JButton b10 = new JButton("10");
JButton b11 = new JButton("11");
// adding the buttons to frame
frameObj.add(b1); frameObj.add(b2);
frameObj.add(b3); frameObj.add(b4);
frameObj.add(b5); frameObj.add(b6);
frameObj.add(b7,0); //can pass indes to assign location
frameObj.add(b8);
frameObj.add(b9); frameObj.add(b10);
frameObj.add(b11);
// horizontal as well as the vertical gap is 5 units.
// 4 rows and 3 column
frameObj.setLayout(new GridLayout(4,3,5,5));
frameObj.setSize(340, 300);
frameObj.setVisible(true);
frameObj.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
}
public static void main(String argsv[])
{
 new GridLayoutExample();
}
}

```

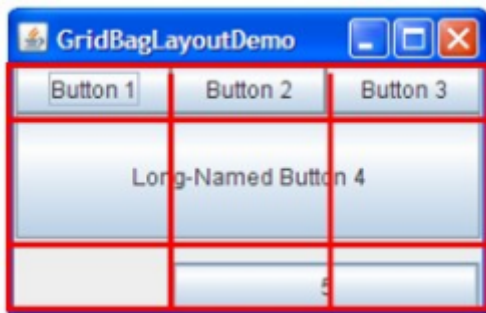
Output:



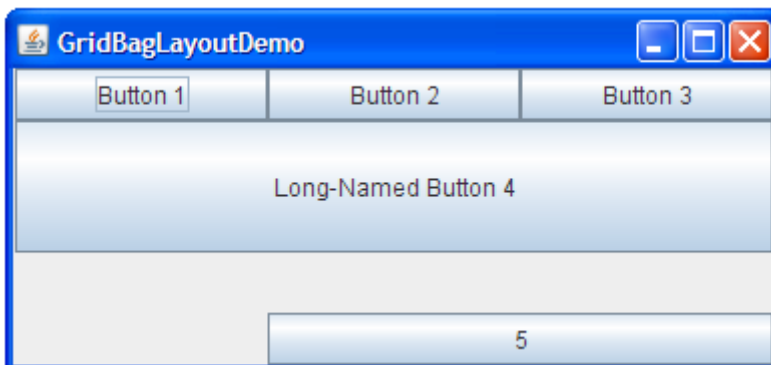
## GridBagLayout:

GridBagLayout is one of the most flexible and complex layout managers the Java platform provides. A GridBagLayout places components in a grid of rows and columns, allowing specified components to span multiple rows or columns. Not all rows necessarily have the same height. Similarly, not all columns necessarily have the same width. Essentially, GridBagLayout places components in rectangles (cells) in a grid, and then uses the components' preferred sizes to determine how big the cells should be.

**Example:** The following figure shows the grid for the preceding applet. As you can see, the grid has three rows and three columns. The button in the second row spans all the columns; the button in the third row spans the two right columns.



If you enlarge the window as shown in the following figure, you will notice that the bottom row, which contains Button 5, gets all the new vertical space. The new horizontal space is split evenly among all the columns. This resizing behavior is based on weights the program assigns to individual components in the GridBagLayout.



The way the program specifies the size and position characteristics of its components is by specifying constraints for each component. Each component associates an instance of **GridBagConstraints**. With the help of the constraints object, we arrange the component's display area on the grid.

GridBagConstraints instance variables:

1. **gridx, gridy:** Specify the row and column at the upper left of the component. The leftmost column has address `gridx=0` and the top row has address `gridy=0`.
2. **gridWidth, gridHeight:** Specify the number of columns (for `gridwidth`) or rows (for `gridheight`) in the component's display area. These constraints specify the number of cells the component uses, *not* the number of pixels it uses. The default value is 1.
3. **fill:** Used when the component's display area is larger than the component's requested size to determine whether and how to resize the component. Valid values (defined as `GridBagConstraints` constants) include **NONE** (the default), **HORIZONTAL** (make the component wide enough to fill its display area horizontally, but do not change its



height), **VERTICAL** (make the component tall enough to fill its display area vertically, but do not change its width), and **BOTH** (make the component fill its display area entirely).

4. **ipadx, ipady:** Specifies the internal padding: how much to add to the size of the component. The default value is zero.
5. **insets:** Specifies the external padding of the component -- the minimum amount of space between the component and the edges of its display area. The value is specified as an **Insets** object. By default, each component has no external padding.
6. **anchor:** Used when the component is smaller than its display area to determine where (within the area) to place the component. Valid values (defined as GridBagConstraints constants) are CENTER (the default), PAGE\_START, PAGE\_END, LINE\_START, LINE\_END, FIRST\_LINE\_START, FIRST\_LINE\_END, LAST\_LINE\_END, and LAST\_LINE\_START.
7. **weightx, weighty:** Weights are used to determine how to distribute space among columns (weightx) and among rows (weighty); this is important for specifying resizing behavior. Unless you specify at least one non-zero value for weightx or weighty, all the components clump together in the center of their container. This is because when the weight is 0.0 (the default), the GridBagLayout puts any extra space between its grid of cells and the edges of the container. Generally weights are specified with 0.0 and 1.0 as the extremes: the numbers in between are used as necessary.

**Example 36:** Following example illustrate application using GridBagLayout, where uses JButton and JFrame. Also, we have some screenshots of AWT and Swing component combination.

**/\*GridBagLayoutExample.java\*/**

```
import javax.swing.JFrame;
import javax.swing.JButton;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;

public class GridBagLayoutExample extends JFrame{
 public GridBagLayoutExample() {
 GridBagLayout grid = new GridBagLayout();
 GridBagConstraints gbc = new GridBagConstraints();

 setLayout(grid);
 setTitle("GridBag Layout Example");

 gbc.anchor = GridBagConstraints.LINE_START;
 gbc.fill = GridBagConstraints.VERTICAL;

 gbc.gridx = 0; //first column
 gbc.gridy = 0; //first row
 this.add(new JButton("Button One"), gbc);

 gbc.gridx = 1; //second column
 gbc.gridy = 0; //second row
 this.add(new JButton("Button two"), gbc);
 }
}
```

```

gbc.fill = GridBagConstraints.HORIZONTAL;
gbc.ipady = 30; //horizontal padding
gbc.ipadx = 40; //vertical padding

gbc.gridx = 0; //first column
gbc.gridy = 1; //second row
this.add(new JButton("Button Three"), gbc);

gbc.gridx = 1; //second column
gbc.gridy = 1; //second row
this.add(new JButton("Button Four"), gbc);

gbc.gridx = 0; //first column
gbc.gridy = 2; //third row

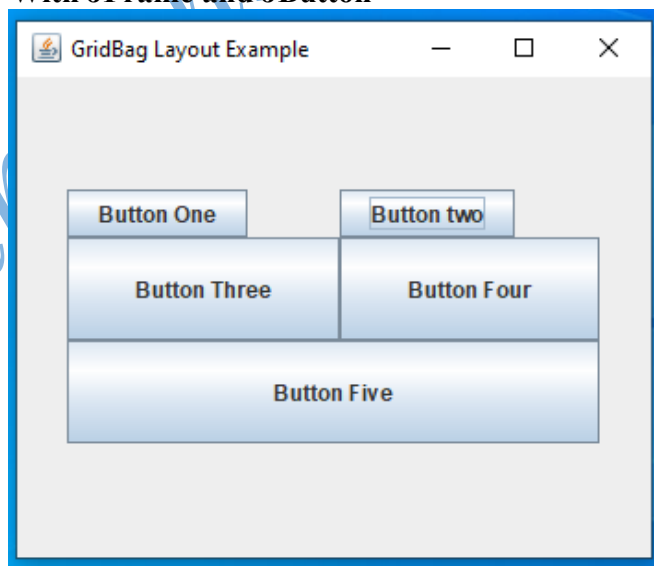
gbc.fill = GridBagConstraints.HORIZONTAL;
gbc.gridwidth = 2; // include two grid i.e columns
this.add(new JButton("Button Five"), gbc);

setSize(360, 300);
setPreferredSize(getSize());
setVisible(true);
setDefaultCloseOperation(EXIT_ON_CLOSE);
}
public static void main(String[] args) {
 new GridBagLayoutExample();
}
}

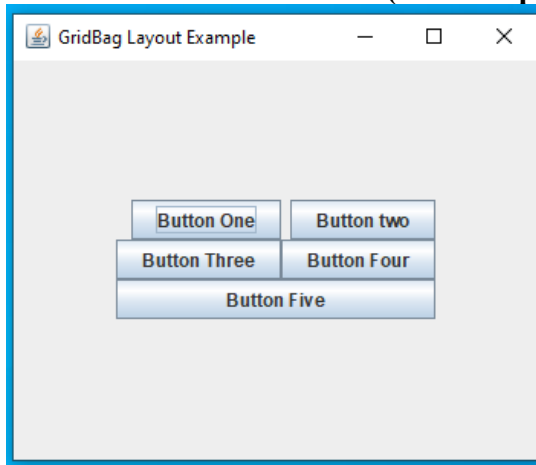
```

**Output:**

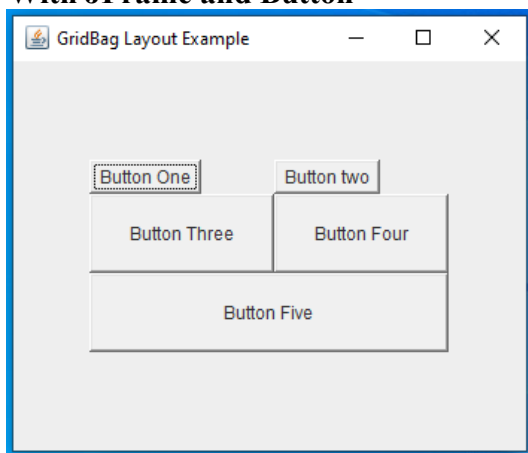
### 1. With JFrame and JButton



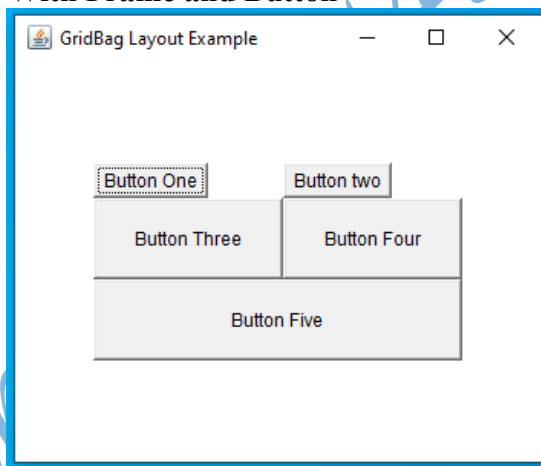
2. With JFrame and JButton. (without ipady, ipadx and with anchor(LINE\_END))



3. With JFrame and Button



4. With Frame and Button



**BoxLayout:**

BoxLayout either stacks its components on top of each other or places them in a row as your choice. You might think of it as a version of FlowLayout, but with greater functionality. The *javx.swing.BoxLayout* provides BoxLayout. Java AWT application and Java Swing application uses BoxLayout differently. As the box layout arranges components(either on top of each other or in a row), it takes the component's alignments and minimum, preferred, and maximum sizes into account.

After JDK version 1.4, you can specify the box layout's axis in a localizable way. Specify X\_AXIS(left to right) or Y\_AXIS(top to bottom) when creating the BoxLayout. Similarly, LINE\_AXIS and PAGE\_AXIS also exist. Both(LINE\_AXIS and PAGE\_AXIS) are preferred because they enable programs to adjust to languages that have different orientations.

**LINE\_AXIS:** Alignment of the components is similar to the way words are aligned in a line. If the Component Orientation property of the container is horizontal, then the components are aligned horizontally; otherwise, the components are aligned vertically.

**PAGE\_AXIS:** Alignment of the components is similar to the way text lines are put on a page. If the Component Orientation property of the container is horizontal, then components are aligned vertically; otherwise, the components are aligned horizontally.

Also it is easy to align given components with the help of `getAlignmentX()` and `getAlignmentY()` methods. For alignment of component, constant value be define as :

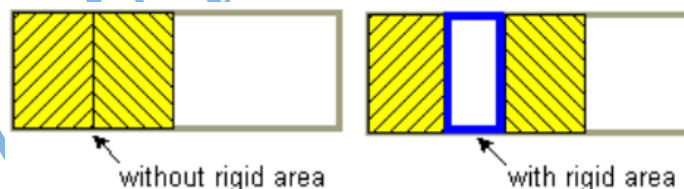
- i. **Component.LEFT\_ALIGNMENT or 0.0f :** Components Left sides should be aligned
- ii. **Component.CENTER\_ALIGNMENT or 0.5f:** Components Center should be aligned.
- iii. **Component.RIGHT\_ALIGNMENT or 1.0f:** Components Right sides should be aligned.

### Using Invisible Components as Filler:

Each component controlled by a box layout butts up against its neighboring components. For space between components, you can either add an empty border to one or both components, or insert invisible components to provide the space. Box class is used to create invisible components.

### Types of filler:

**Rigid area:** Use this when you want a fixed-size between two components. For example, to put 5 pixels between two components in a left-to-right box.

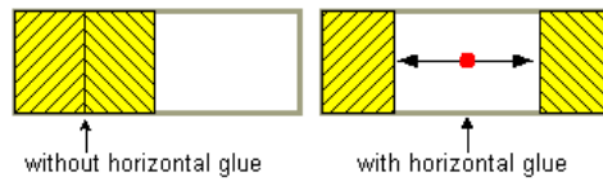


### Snippet:

```
container.add(btn1);
container.add(Box.createRigidArea(new Dimension(5,0)));
container.add(btn2);
```

**Glue:** Used to specify excess space in a layout should go. It's as a kind of elastic glue – stretchy and expandable, and take space when you pull apart the components that it is sticking to. Horizontal glue(`Box.createHorizontalGlue()`) between two components in a left-to-right box, make extra space between those components(left-to-right). Vertical

glue(**Box.createVerticalGlue()**) between two components in a top-to-bottom box, make extra space between those components (top-to-bottom).



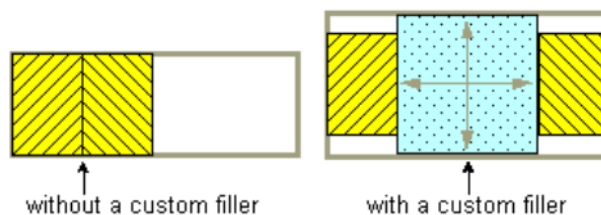
#### Snippet:

```
container.add(btn4);

container.add(Box.createHorizontalGlue());

container.add(btn5);
```

**Custom Box.Filler:** Used to specify a component with whatever minimum, preferred, and maximum sizes you want. For example, to create some filler in a left-to-right layout that puts at least 5 pixels between two components and ensures that the container has a minimum height of 100 pixels.



#### Snippet:

```
container.add(btn2);

Dimension minSize = new Dimension(5,100);

Dimension prefSize = new Dimension(5,100);

Dimension maxSize = new Dimension(Short.MAX_VALUE, 100);

container.add(new Box.filler(minSize,prefSize, maxSize));

container.add(btn3);
```

#### Creating BoxLayout Objects:

| Constructor or Method     | Description                                                                                                                                                            |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BoxLayout(Container, int) | Creates a BoxLayout instance that controls the specified Container. The integer argument specifies the axis along which the container's components should be laid out. |

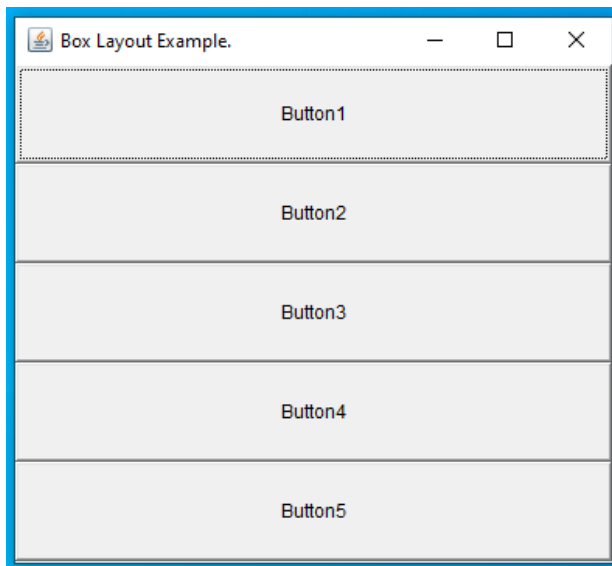
|                                                                |                                                                                                                                                                                                                                                             |
|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                | When the container has the default component orientation, <code>BoxLayout.LINE_AXIS</code> specifies that the components be laid out from left to right, and <code>BoxLayout.PAGE_AXIS</code> specifies that the components be laid out from top to bottom. |
| <code>Box(int)</code>                                          | Creates a <code>Box</code> — a container that uses a <code>BoxLayout</code> with the specified axis.                                                                                                                                                        |
| <code>static Box<br/>createHorizontalBox()<br/>(in Box)</code> | Creates a <code>Box</code> that lays out its components from left to right.                                                                                                                                                                                 |
| <code>static Box<br/>createVerticalBox()<br/>(in Box)</code>   | Creates a <code>Box</code> that lays out its components from top to bottom.                                                                                                                                                                                 |

### Example 37: AWT application using `BoxLayout` as Layout Manager.

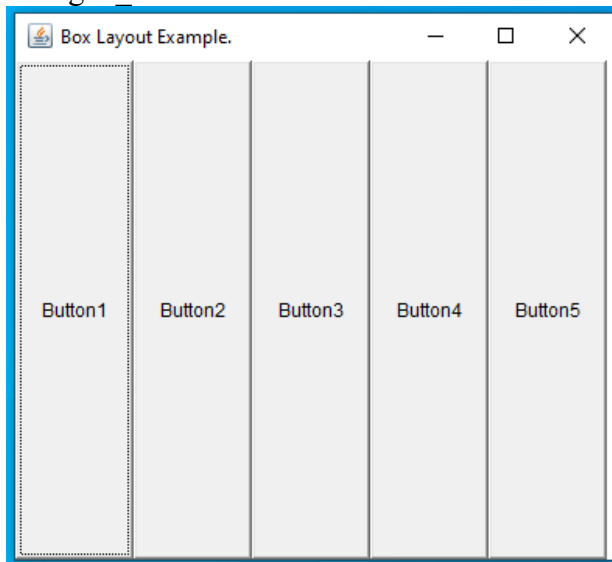
```
import javax.swing.BoxLayout;
import java.awt.Frame;
import java.awt.Button;
public class BoxLayoutExample extends Frame {
 Button btn[];
 public BoxLayoutExample(){
 btn = new Button[5];
 for(int i = 0; i<5;i++){
 btn[i] = new Button("Button" + (i+1));
 add(btn[i]);
 }
 setSize(400,360);
 setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));
 //setLayout(new BoxLayout(this, BoxLayout.X_AXIS));
 setVisible(true);
 }
 public static void main(String[] args) {
 new BoxLayoutExample();
 }
}
```

#### Output:

1. Using `Y_AXIS`



## 2. Using X\_AXIS



**Example 38:** SWING application using BorderLayout as Layout Manager. In SWING application you can't directly implement BorderLayout to Frame(JFrame).

```
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;
import java.awt.Insets;

public class BorderLayoutExample1 {
 JFrame frame;
 JButton btn[];
 JPanel panel;
 public BorderLayoutExample1(){
 JFrame.setDefaultLookAndFeelDecorated(true);
 frame = new JFrame("Box Layout Example");
```

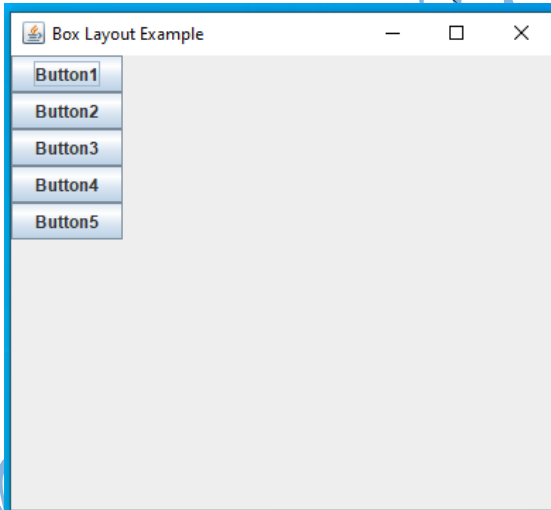
```

panel = new JPanel();
BoxLayout boxLayout = new BoxLayout(panel, BoxLayout.PAGE_AXIS);
panel.setLayout(boxLayout);
panel.setBorder(new EmptyBorder(new Insets(50,80,50,80)));
btn = new JButton[5];
for(int i = 0; i<5;i++){
 btn[i] = new JButton("Button"+(i+1));
 panel.add(btn[i]);
}
frame.add(panel);
frame.setSize(400,360);
// frame.pack();
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
public static void main(String[] args) {
 new BoxLayoutExample1();
}
}

```

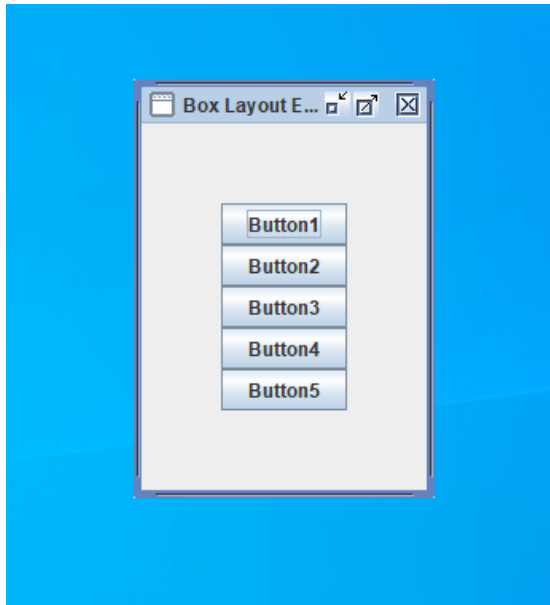
### Output:

1. Application with out Insets(top,left,bottom,right) , pack(), and setDefaultCloseOperationDecorated(true/false).



2. Application with Insets(top,left,bottom,right), pack(), and setDefaultCloseOperationDecorated(true).





**Example 39:** Java Swing application using BorderLayout as Layout Manager.

```
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFrame;

import java.awt.Component;
import java.awt.Container;
import java.awt.Dimension;
public class BorderLayoutExample1 {
 JFrame frame;
 JButton btn1, btn2, btn3, btn4;
 public BorderLayoutExample1(){
 //setDefaultLookAndFeelDecorated() method is optional
 JFrame.setDefaultLookAndFeelDecorated(true);
 frame = new JFrame("Box Layout Example");

 Container container = frame.getContentPane();
 BorderLayout boxLayout = new BorderLayout(container, BorderLayout.Y_AXIS);
 container.setLayout(boxLayout);

 btn1 = new JButton("Click Me");
 btn2 = new JButton("Show");
 btn3 = new JButton("I am visible Button");
 btn4 = new JButton("Save");

 btn1.setAlignmentX(0.5f); //can use value from 0.0f to 1.0f
 btn2.setAlignmentX(0.5f);
 //LEFT_ALIGNMENT,CENTER_ALIGNMENT,RIGHT_ALIGNMENT,TOP_ALIGNMENT,BOTTOM_ALIGNMENT
 btn3.setAlignmentX(Container.CENTER_ALIGNMENT);
```

```

btn4.setAlignmentX(Container.CENTER_ALIGNMENT);

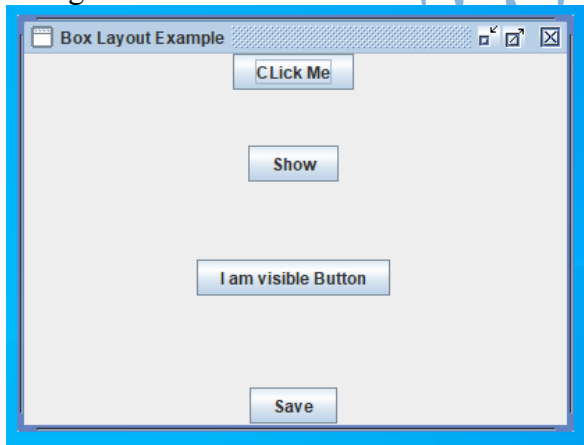
container.add(btn1);
container.add(Box.createRigidArea(new Dimension(0,40)));
container.add(btn2);
container.add(Box.createVerticalGlue());
container.add(btn3);
Dimension minSize = new Dimension(50,10);
Dimension prefSize = new Dimension(20,10);
Dimension maxSize = new Dimension(50,Short.MAX_VALUE);
container.add(new Box.Filler(minSize, prefSize, maxSize));
container.add(btn4);

frame.setSize(400,300);
frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
frame.setVisible(true);
}
public static void main(String[] args) {
 new BoxLayoutExample1();
}
}

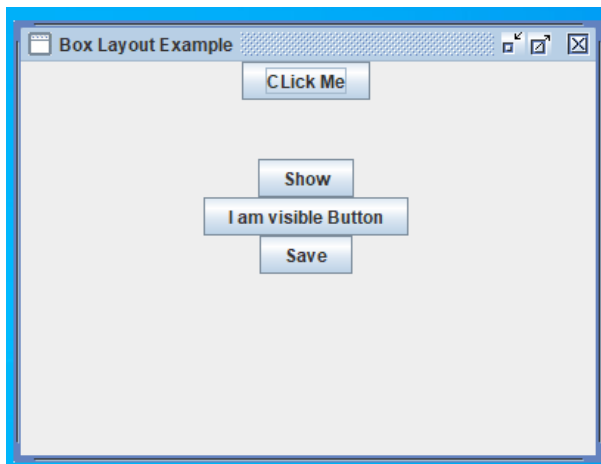
```

### Output:

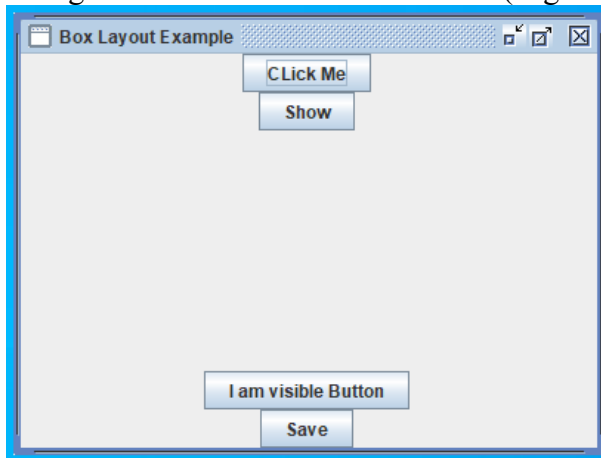
1. Using all filler:



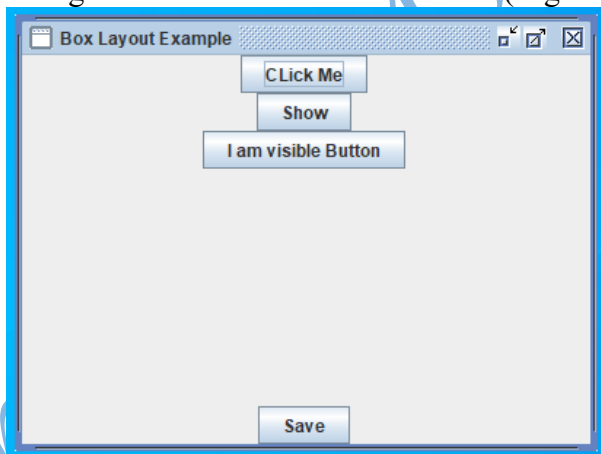
2. Using Rigid Filler: Comment other filler(Glue and Box.filler)



3. Using Glue Fill: Comment other filler (Rigid and Box.filler)



4. Using Box.filler: Comment other filler(Rigid and Glue)



### CardLayout:

The CardLayout class manages two or more components (usually JPanel instances) that share the same display space i.e. only one component is visible at a time. When using the CardLayout class, let the user choose between the components by using a combo box. Tapped Pane is alternative to CardLayout.

### Constructors of CardLayout class:

1. **CardLayout:** create a card layout with zero horizontal and vertical gap.

2. **CardLayout(int hgap, int vgap):** creates a card layout with the given horizontal and vertical gap.

#### CardLayout class Methods:

- **public void next(Container parent):** is used to flip to the next card of the given container.
- **public void previous(Container parent):** is used to flip to the previous card of the given container.
- **public void first(Container parent):** is used to flip to the first card of the given container.
- **public void last(Container parent):** is used to flip to the last card of the given container.
- **public void show(Container parent, String name):** is used to flip to the specified card with the given name.

**Example 40:** In this application, creating 4 buttons and CardLayout is used as layout manager. When user clicking the button, the next card of the container is shown after the last card, again, the first card of the container is shown upon clicking

*/\*CardLayoutExample.java\*/*

```
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

public class CardLayoutExample extends JFrame implements ActionListener {
 CardLayout crd;
 // button instance variables to hold the references of buttons
 JButton btn1, btn2, btn3, btn4;
 Container cPane;
 CardLayoutExample()
 {
 cPane = getContentPane();
 //constructor with horizontal gap 20px and vertical gap 40px
 crd = new CardLayout(20,40);
 cPane.setLayout(crd);

 //creating the buttons
 btn1 = new JButton("This is ");
 btn2 = new JButton("Programming");
 btn3 = new JButton("In");
 btn4 = new JButton("Java");

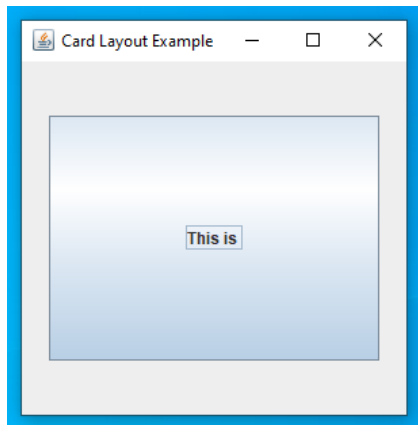
 // adding listeners to it
 btn1.addActionListener(this);
 btn2.addActionListener(this);
 btn3.addActionListener(this);
 btn4.addActionListener(this);
 }
}
```

```

cPane.add("a", btn1); //first card is the button btn1
cPane.add("b", btn2); //first card is the button btn2
cPane.add("c", btn3); //first card is the button btn3
cPane.add("d", btn4); //first card is the button btn4
}
public void actionPerformed(ActionEvent e)
{
 crd.next(cPane);
}
public static void main(String argsv[])
{
 CardLayoutExample crdl = new CardLayoutExample();
 crdl.setTitle("Card Layout Example");
 crdl.setSize(300, 300);
 crdl.setVisible(true);
 crdl.setDefaultCloseOperation(EXIT_ON_CLOSE);
}
}

```

### Output:



**Example 41:** In this application four buttons and four labels are added to two different panel. Also those panels are added to frame , where CardLayout is used as Layout Manager.

**/\*CardlayoutExample.java\*/**

```

import javax.swing.JButton;
import javax.swing.JPanel;
import javax.swing.JFrame;
import javax.swing.JLabel;

import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.Color;
import java.awt.event.*;

public class CardLayoutExample extends JFrame implements ActionListener {

```

```

CardLayout card;
JButton btn1, btn2, btn3, btn4;
JLabel lbl1, lbl2, lbl3, lbl4;
JPanel cardPanel, cardPanel1, cardPanel2, cardPanel3, cardPanel4, btnPanel;
int currentCard = 1;
CardLayoutExample()
{
 cardPanel = new JPanel();
 card = new CardLayout();
 cardPanel.setLayout(card);

 cardPanel1 = new JPanel();
 lbl1 = new JLabel("Card 1");
 cardPanel1.setBackground(Color.cyan);
 cardPanel1.add(lbl1);
 cardPanel.add(cardPanel1, "1");

 cardPanel2 = new JPanel();
 lbl2 = new JLabel("Card 2");
 cardPanel2.setBackground(Color.orange);
 cardPanel2.add(lbl2);
 cardPanel.add(cardPanel2, "2");

 cardPanel3 = new JPanel();
 lbl3 = new JLabel("Card 3");
 cardPanel3.setBackground(Color.yellow);
 cardPanel3.add(lbl3);
 cardPanel.add(cardPanel3, "3");

 cardPanel4 = new JPanel();
 lbl4 = new JLabel("Card 4");
 cardPanel4.setBackground(Color.magenta);
 cardPanel4.add(lbl4);
 cardPanel.add(cardPanel4, "4");

 btnPanel = new JPanel();
 btn1 = new JButton("First");
 btnPanel.add(btn1);
 btn2 = new JButton("Next");
 btnPanel.add(btn2);
 btn3 = new JButton("Previous");
 btnPanel.add(btn3);
 btn4 = new JButton("Last");
 btnPanel.add(btn4);

 btn1.addActionListener(this);
 btn2.addActionListener(this);
 btn3.addActionListener(this);

```

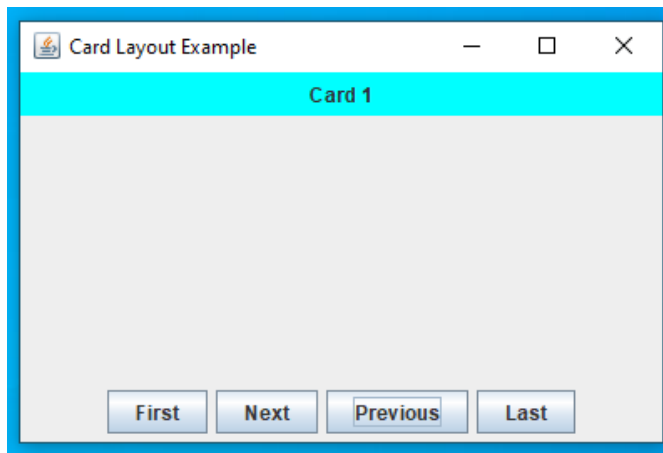
```

btn4.addActionListener(this);

getContentPane().add(cardPanel,BorderLayout.NORTH);
getContentPane().add(btnPanel,BorderLayout.SOUTH);
}
public void actionPerformed(ActionEvent ae)
{
 if(ae.getActionCommand().equals("First")){
 card.first(cardPanel);
 currentCard = 1;
 }
 if(ae.getActionCommand().equals("Last")){
 card.last(cardPanel);
 currentCard = 4;
 }
 if(ae.getActionCommand().equals("Next")){
 if(currentCard < 4){
 currentCard +=1;
 card.next(cardPanel);
 }
 }
 if(ae.getActionCommand().equals("Previous")){
 if(currentCard > 1){
 currentCard -= 1;
 card.previous(cardPanel);
 }
 }
}
public static void main(String argsv[])
{
 CardLayoutExample crdl = new CardLayoutExample();
 crdl.setTitle("Card Layout Example");
 crdl.setSize(400, 260);
 crdl.setVisible(true);
 crdl.setDefaultCloseOperation(EXIT_ON_CLOSE);
}
}

```

**Output:**



**Remaining do yourself !!!**

### **Old Questions Solution:**

1. Create a swing GUI that contains a button , and two text fields. When the button is clicked the value of first text field should be checked and display “odd number” or “even number” in the second text field.

***Solution:***

***/\*CheckNumber.java\*/***

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JTextField;
import javax.swing.border.EmptyBorder;
import javax.swing.JButton;

import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class CheckNumber {
 JFrame frame;
 JButton btn;
 JTextField ftxt,rtxt;
 JPanel panel;
 QuestionSolution(){
 frame = new JFrame("Check Given Number");

 ftxt = new JTextField();
 rtxt = new JTextField();
 btn = new JButton("Check number");
 btn.addActionListener(new ActionListener(){
 public void actionPerformed(ActionEvent ae){
 String number = ftxt.getText();
 int num = Integer.parseInt(number);
 if(num%2 == 0){
```



```

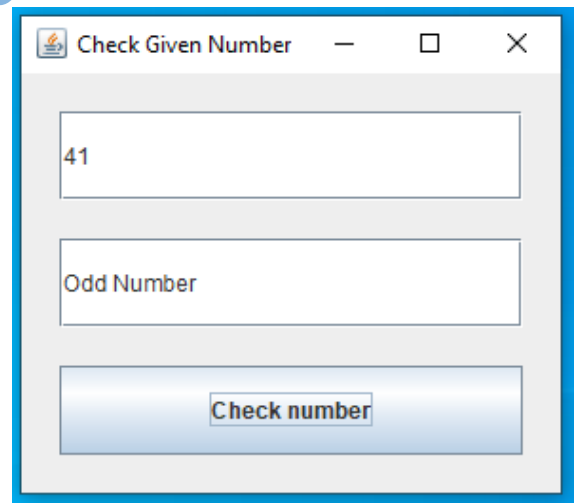
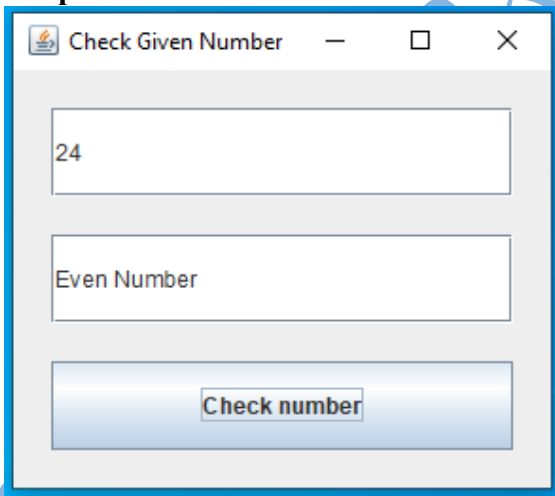
 rtxt.setText("Even Number");
 }else{
 rtxt.setText("Odd Number");
 }
 }
 });
 panel = new JPanel();
 panel.setBorder(new EmptyBorder(new Insets(20,20,20,20)));
 panel.setLayout(new GridLayout(3,1,20,20));

 panel.add(ftxt);
 panel.add(rtxt);
 panel.add(btn);

 frame.add(panel, BorderLayout.CENTER);
 frame.setSize(300,260);
 frame.setVisible(true);
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
public static void main(String[] args) {
 new CheckNumber ();
}
}

```

### Output:



2. Create a swing GUI which contains a text field(to input radius of circle), a label and a button. When the button is clicked area of circle should be calculated and displayed in the label.

### **Solution:**

**/\*AreaOfCircle.java\*/**

```

import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;

```

```

import javax.swing.border.EmptyBorder;
import javax.swing.JButton;

import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class AreaOfCircle {
 JFrame frame;
 JButton btn;
 JTextField ftxt;
 JLabel lbl;
 JPanel panel;
 AreaOfCircle(){
 frame = new JFrame("Area of circle");

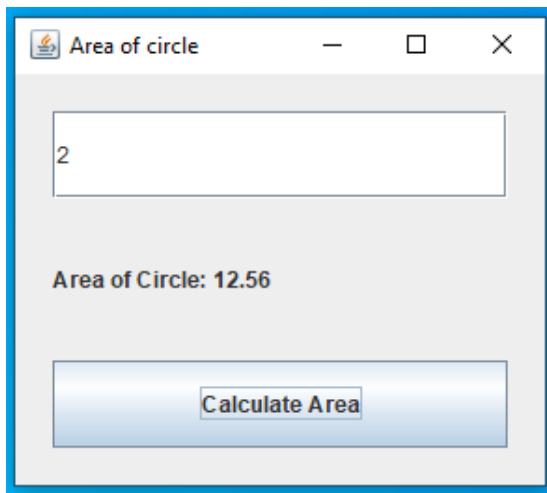
 ftxt = new JTextField();
 lbl = new JLabel();
 btn = new JButton("Calculate Area");
 btn.addActionListener(new ActionListener(){
 public void actionPerformed(ActionEvent ae){
 String number = ftxt.getText();
 int radius = Integer.parseInt(number);
 double area = 2*3.14*radius;
 lbl.setText("Area of Circle: "+area);
 }
 });
 panel = new JPanel();
 panel.setBorder(new EmptyBorder(new Insets(20,20,20,20)));
 panel.setLayout(new GridLayout(3,1,20,20));

 panel.add(ftxt);
 panel.add(lbl);
 panel.add(btn);

 frame.add(panel, BorderLayout.CENTER);
 frame.setSize(300,260);
 frame.setVisible(true);
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 }
 public static void main(String[] args) {
 new AreaOfCircle();
 }
}

```

**Output:**



3. Create a Frame with two Text Fields , one of which should show whether the mouse pointer is inside or outside the Frame and the other text field should display the X and Y co-ordinates of the pointer when the user moves the mouse pointer inside the Frame area.  
**/\*MouseListenerExample.java\*/**

```
import javax.swing.JFrame;
import javax.swing.JTextField;
```

```
import java.awt.Color;
import java.awt.event.MouseEvent;
import java.awt.event.MouseAdapter;
public class MousePointerExample {
 JFrame frame;
 JTextField pointerInOut,pCoordinates;
 MousePointerExample (){
 frame = new JFrame("Mouse Tracker");
```

```
 pointerInOut = new JTextField("Mouse outside Frame.");
 pointerInOut.setBounds(50,30,140,30);
```

```
 pCoordinates = new JTextField("Coordinate Here");
 pCoordinates.setBounds(50,70,140,30);
```

```
 frame.add(pointerInOut);
 frame.add(pCoordinates);
```

```
 //instance of MouseHandler class for event handling
```

```
 MouseHandler handler = new MouseHandler();
 frame.addMouseListener(handler);
 frame.addMouseMotionListener(handler);
```

```
 frame.setSize(300,260); //size of frame
```

```
 frame.setLayout(null); // Layout Manager null means manual
```

```
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); //close from application
```

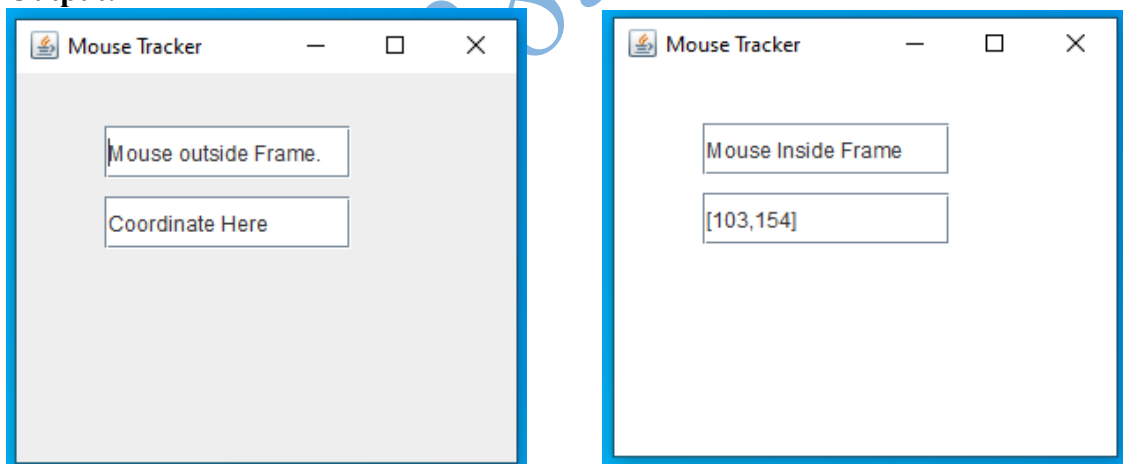
```
 frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE); // close from JVM(machine)
```

```

 frame.setVisible(true);
 }
 //inner class handling MouseListener and MouseMotionListener event using MouseAdapter class
 class MouseHandler extends MouseAdapter{
 @Override
 public void mouseEntered(MouseEvent me){
 pointerInOut.setText("Mouse Inside Frame");
 frame.getContentPane().setBackground(Color.white);
 }
 @Override
 public void mouseExited(MouseEvent me){
 pointerInOut.setText("Mouse Outside Frame");
 frame.getContentPane().setBackground(new Color(238,238,238));
 }
 @Override
 public void mouseMoved(MouseEvent me){
 pCoordinates.setText(String.format("[%d,%d]",me.getX(),me.getY()));
 }
 }
 //end of inner class
 public static void main(String[] args) {
 new MousePointerExample ();
 }
}

```

#### Output:



4. Write a program to generate a Frame with two buttons “BLACK” and “BLUE”. When a button is clicked it should change the background color of the Frame to respective color.

*/\*ChangeFrameBackground.java\*/*

```

import javax.swing.JButton;
import javax.swing.JFrame;

import java.awt.Color;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class ChangeFrameBackground {

```

```

JFrame frame;
JButton btn1,btn2;
ChangeFrameBackground (){
 frame = new JFrame("Change Background Color");

 btn1 = new JButton("BLACK");
 btn1.setBounds(100,30,140,30);
 btn1.addActionListener(new ActionListener(){
 public void actionPerformed(ActionEvent ae){
 frame.getContentPane().setBackground(Color.BLACK);
 }
 });

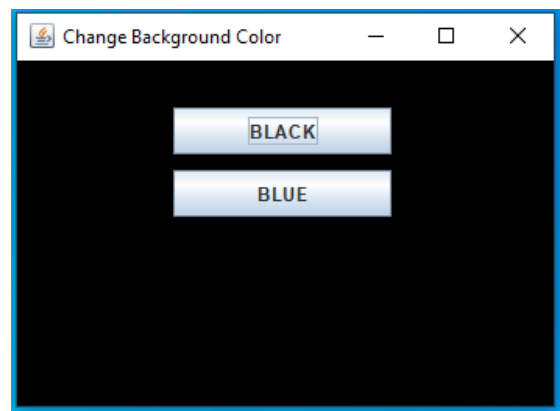
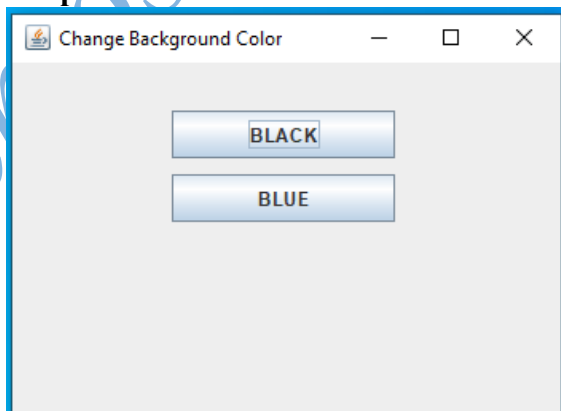
 btn2 = new JButton("BLUE");
 btn2.setBounds(100,70,140,30);
 btn2.addActionListener(new ActionListener(){
 public void actionPerformed(ActionEvent ae){
 frame.getContentPane().setBackground(Color.BLUE);
 }
 });

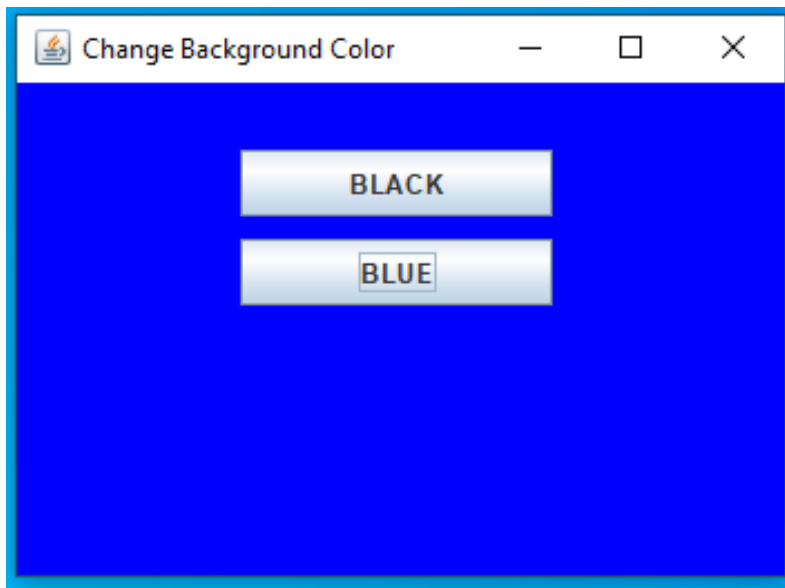
 frame.add(btn1);
 frame.add(btn2);

 frame.setSize(360,260); //size of frame
 frame.setLayout(null); // Layout Manager null means manual
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); //close from application
 frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE); // close from JVM(machine)
 frame.setVisible(true);
}
public static void main(String[] args) {
 new ChangeFrameBackground ();
}
}

```

**Output:**





5. Create a Frame with one button and one textfield, when user clicks on the button the text entered on textfield should be changed to uppercase.

**/\*ChangeToUppercase.java\*/**

```
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JTextField;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class ChangeToUppercase {
 JFrame frame;
 JTextField inpTft;
 JButton btn;
 ChangeToUppercase () {
 frame = new JFrame("Change To Uppercase");

 inpTft = new JTextField("Let's see ");
 inpTft.setBounds(80,30,180,30);

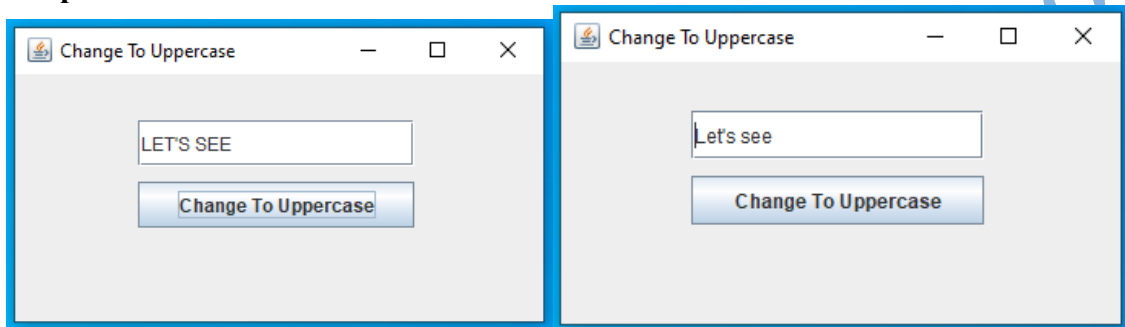
 btn = new JButton("Change To Uppercase");
 btn.setBounds(80,70,180,30);
 btn.addActionListener(new ActionListener(){
 public void actionPerformed(ActionEvent ae){
 String inp = inpTft.getText();
 inp = inp.toUpperCase();
 inpTft.setText(inp);
 }
 });
 frame.add(inpTft);
 frame.add(btn);
 frame.setSize(360,200); //size of frame
 frame.setLayout(null); // Layout Manager null means manual
```

```

 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); //close from application
 frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE); // close from JVM(machine)
 frame.setVisible(true);
 }
 public static void main(String[] args) {
 new ChangeToUppercase ();
 }
}

```

### Output:



### MouseListener Interface:

Whenever state of mouse is changed `MouseListener` is notified. The `java.awt.event.MouseListener` package define `MouseListener` interface. It is notified against `MouseEvent`.

**Methods of MouseListener interface:** There are 5 abstract methods().

1. `public abstract void mouseClicked(MouseEvent me);`
2. `public abstract void mouseEntered(MouseEvent me);`
3. `public abstract void mouseExited(MouseEvent me);`
4. `public abstract void mousePressed(MouseEvent me);`
5. `public abstract void mouseReleased(MouseEvent me);`

**Example 42:** Implementation of all `MouseListener` interface methods. Execute following program in your PC to visualize output.

**/\* MouseListenerExample.java \*/**

```

package guiconcept;
import java.awt.FlowLayout;
import java.awt.event.MouseEvent;
import java.awt.event.MouseListener;
import javax.swing.JFrame;
import javax.swing.JLabel;
public class MouseListenerExample extends JFrame implements MouseListener {
 JLabel label;
 public MouseListenerExample() {
 setTitle("MouseListener Example");
 setSize(400, 300);
 setLayout(new FlowLayout());
 label = new JLabel("Perform mouse actions here");
 }
}

```

```

 add(label);
 addMouseListener(this);
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 setVisible(true);
 }
 @Override
 public void mouseClicked(MouseEvent e) {
 label.setText("Mouse Clicked at: " + e.getX() + ", " + e.getY());
 }
 @Override
 public void mousePressed(MouseEvent e) {
 label.setText("Mouse Pressed");
 }
 @Override
 public void mouseReleased(MouseEvent e) {
 label.setText("Mouse Released");
 }
 @Override
 public void mouseEntered(MouseEvent e) {
 label.setText("Mouse Entered Window");
 }
 @Override
 public void mouseExited(MouseEvent e) {
 label.setText("Mouse Exited Window");
 }
 public static void main(String[] args) {
 new MouseListenerExample();
 }
}

```

### MouseMotionListener Interface:

Whenever user move or drag mouse , MouseMotionListener is notified. It is notified against MouseEvent. The *java.awt.event.MouseMotionListener* package provide MouseMotionListener Interface.

#### Methods of MouseMotionListener Interface:

1. public abstract void mouseDragged(MouseEvent me);
2. public abstract void mouseMoved(MouseEvent me);

**Note:** When using MouseListener and MouseMotionListener you have to override all abstract methods(5 for MouseListener and 2 for MouseMotionListener) even those all not needed. So, to use only desired abstracts methods , use **MouseAdapter** or **MouseMotionAdapter**. Better use MouseAdapter.

**Example 43:** Implementation of all MouseMotionListener interface methods. Execute following program in your PC to visualize output.

**/\* MouseMotionExample.java\*/**

```

package guiconcept;
import java.awt.FlowLayout;
import java.awt.event.MouseEvent;

```



```

import java.awt.event.MouseMotionListener;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class MouseMotionExample extends JFrame implements MouseMotionListener {
 JLabel label;
 public MouseMotionExample() {
 setTitle("MouseMotionListener Example");
 setSize(400, 300);
 setLayout(new FlowLayout());

 label = new JLabel("Move mouse here");
 add(label);
 addMouseMotionListener(this);
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 setVisible(true);
 }
 @Override
 public void mouseDragged(MouseEvent e) {
 label.setText("Mouse Dragged at: " + e.getX() + ", " + e.getY());
 }
 @Override
 public void mouseMoved(MouseEvent e) {
 label.setText("Mouse Moved at: " + e.getX() + ", " + e.getY());
 }
 public static void main(String[] args) {
 new MouseMotionExample();
 }
}

```

## KeyListener Interface:

When the state of key changes Java KeyListener is notified. It is notified against KeyEvent. The *java.awt.event.KeyListener* package provides KeyListener interface.

### Methods of KeyListener interface:

1. public abstract void keyPressed(KeyEvent ke); //Invoked when a key has been pressed
2. public abstract void keyReleased(KeyEvent ke); //Invoked when a key has been released
3. public abstract void keyTyped(KeyEvent ke); //Invoked when a key has been typed

**Example 44:** This swing application have one TextField and one Label . when you hit key it will trigger all abstract methods of KeyListener interface.

**/\*KeyStrokeExample.java\*/**

```

import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;

```

```

import java.awt.event.KeyListener;
import java.awt.event.KeyEvent;
public class KeyStrokeExample implements KeyListener{
 JFrame frame;
 JTextField inputText;
 JLabel lbl;
 KeyStrokeExample(){
 frame = new JFrame("Key stroke example");

 inputText = new JTextField();
 inputText.setBounds(50,50,200,30);

 lbl = new JLabel("Status");
 lbl.setBounds(50,90,200,30);

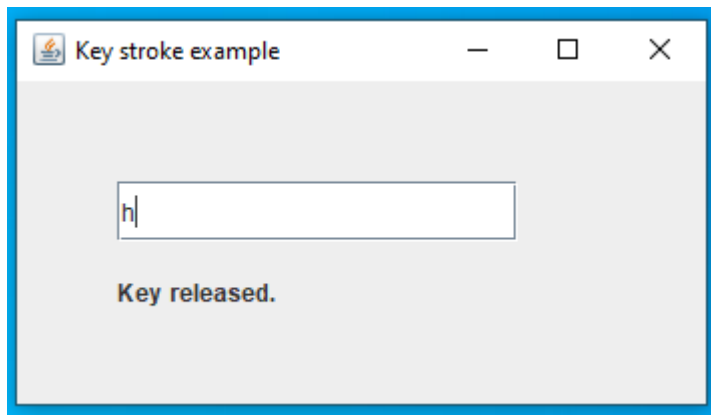
 inputText.addKeyListener(this);

 frame.add(inputText);
 frame.add(lbl);

 frame.setSize(360,200);
 frame.setLayout(null);
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
 frame.setVisible(true);
 }
 @Override
 public void keyPressed(KeyEvent ke){
 lbl.setText("Key pressed.");
 System.out.println("Pressed...");
 }
 @Override
 public void keyReleased(KeyEvent ke){
 lbl.setText("Key released.");
 System.out.println("Released...");
 }
 @Override
 public void keyTyped(KeyEvent ke){
 lbl.setText("Typing...");
 System.out.println("Typed...");
 }
 public static void main(String[] args) {
 new KeyStrokeExample();
 }
}

```

**Output:**



#### Console Output:

```
Pressed...
Typed...
Released...
```

#### Look yourself remaining Swing Components:

1. JTable
2. JList
3. JScrollBar
4. JTabbedPane
5. JSlider
6. JSpinner
7. JProgressBar
8. JFileChooser
9. JToolBar
10. JDesktopPane
11. JEditorPane
12. JToggleButton
13. JViewport
14. JScrollPane etc

## JavaFx

JavaFX is a UI toolkit for building advanced, cross-platform desktop applications in Java. JavaFX is a set of graphics and media packages that enables developers to design, create, test, debug, and deploy rich client applications that operate consistently across diverse platforms.

A fundamental feature of JavaFX is its declarative language, FXML. It allows developers to define the user interface (UI) of their applications using an XML-based syntax. This separation of UI from application logic simplifies the design and maintenance of complex interfaces, enhancing development efficiency.

JavaFX applications are structured around stages and scenes. A stage represents the main window of the application, while a scene defines the content within that window. Various nodes, such as buttons, labels, text fields, etc., can be added to the scene to create a visually appealing and interactive UI.

### JavaFX Flow (How things work)

1. You write code → (Application Layer)
2. UI created → (Scene Graph)
3. Controls added → (UI Layer)
4. Rendered → (Prism Engine)
5. Displayed → (Glass Toolkit)

### JavaFx Controls & Layout Layer

This layer provides ready-made UI components.

Controls (UI elements) – provides ready-made UI elements, are actually shown to the user for interaction and information exchange. However, the package *javafx.scene.control* provides all the necessary classes for the UI components like Button, Label, etc. Every class represents a specific UI control and defines some methods for their styling.

- Button
- Label
- TextField
- TableView, ....

Layouts (arrangement) – arrange elements on the screen. The way in which the components are to be seen on the stage. It basically organizes the scene-graph nodes. All these classes belong to *javafx.scene.layout* package. *javafx.scene.layout.Pane* class is the base class for all the built-in layout classes in JavaFX.

- VBox (vertical)
- HBox (horizontal)
- GridPane
- BorderPane
- StackPane
- FlowPane

**Example 45:** Two buttons are organized using VBox layout class on Scene. VBox arrange UI elements vertically.

```
package basicfxgui;

import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.scene.Scene;

public class VBoxLayoutExample extends Application{
 public void start(Stage stage){
 Button addBtn = new Button("Addition Calculator");
 Button caseBtn = new Button("Lower To Upper");

 VBox layout = new VBox(20,addBtn,caseBtn); //with 20px vertical gap
 Scene scene = new Scene(layout,400,400);
 stage.setScene(scene);
 stage.show();
 }
 public static void main(String[] args) {
 launch(args);
 }
}
```

**Example 46:** BorderPane layout arranges UI elements in 5 regions(Top, Left, Bottom, Right and Center).

**/\* BorderPaneLayoutExample.java \*/**

```
package basicfxgui;
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.layout.BorderPane;
import javafx.scene.control.Label;
import javafx.scene.Scene;

public class BorderPaneLayoutExample extends Application{
 public void start(Stage stage){
 BorderPane bpane = new BorderPane();
 bpane.setTop(new Label("Hello Top Level."));
 bpane.setLeft(new Label("Hello Left Level."));
 bpane.setBottom(new Label("Hello Bottom Level."));
 bpane.setRight(new Label("Hello Right Level."));
 bpane.setCenter(new Label("Hello Center Level."));

 Scene scene = new Scene(bpane,400,400);
 stage.setScene(scene);
 stage.show();
 }
 public static void main(String[] args) {
 launch(args);
 }
}
```

```
}
```

**Example 47:** GridPane Layout pane allows us to add the multiple nodes in multiple rows and columns.

**/\* GridPaneLayoutExample.java\*/**

```
package basicfxgui;
import javafx.application.Application;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.control.Button;
import javafx.scene.Scene;
public class GridPaneLayoutExample extends Application{
 public void start(Stage stage){
 Label first_name=new Label("Enter First Name");
 Label last_name=new Label("Enter Last Name");
 TextField txt1=new TextField();
 TextField txt2=new TextField();

 Button Submit=new Button ("Send");

 GridPane root=new GridPane();
 root.setHgap(20);
 root.setVgap(10);

 Scene scene = new Scene(root,400,400);
 root.addRow(0, first_name,txt1);
 root.addRow(1, last_name,txt2);
 root.addRow(2, Submit);
 stage.setScene(scene);
 stage.show();
 }
 public static void main(String[] args) {
 launch(args);
 }
}
```

**Example 48:** HBox layout arranges the nodes/UI elements in a single row.

**/\* HboxLayoutExample.java\*/**

```
package basicfxgui;
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.HBox;
import javafx.scene.Scene;
public class HboxLayoutExample extends Application{
 public void start(Stage stage){
 Button btn1 = new Button("First Button");
 Button btn2 = new Button("Second Button");
```

```
// HBox layout = new HBox(15,btn1,btn2);
// OR
HBox layout = new HBox(15);
layout.getChildren().addAll(btn1,btn2);
Scene scene = new Scene(layout,400,400);

stage.setScene(scene);
stage.show();
}
public static void main(String[] args) {
 launch();
}
}
```

## JavaFx vs Swing (do yourself)

### Note:

1. **Application** class – It is main class of any JavaFx program, provides life-cycle methods and must extend Application class.

```
@Override
public void start(Stage stage)
```

This method is automatically called after launch(). Used to build and display UI.

2. **Stage** class – represents the **main window** of the application.
3. **Scene** class – holds all UI elements. It is placed **inside the Stage**.

### Relationship

Stage (Window)



Scene (Container)



Nodes (UI Elements like Button, Text)

**launch()** – Starts the JavaFX application. Initializes JavaFX runtime . Calls start() method.

**Show()** – Used to display the Stage (window). Without show(), the window will not appear.

## Chapter 6: Graphics and Images /Animation / Multimedia

### Some old Questions:

1. Create a graphics application to display “Pokhara University” in blue color with font name: Times New Roman, type: Bold and size: 20. Also write HTML to embed the applet.
2. WAP to display the flag of Nepal, drawing a graph along with the reasonable coordinates.
3. As per a recent survey, among all the JavaScript frameworks 40% prefers React while Angular comes second with 30% developers preferring it and 20% of developers prefer Vue while remaining 10% use other frameworks. Create a pie to show case the survey information.
4. As per a poll conducted in a class of 48 students , 25 students are supportive of virtual classroom while 15 are against it and remaining 8 students chose to stay neutral. Create a pie chart to illustrate this poll result.
5. Write a program to draw a Bar chart of Number of Students giving exam of Java, C and C++.
6. Write a program to draw a Bar Chart of the total number of students appearing exam in Java. Also , provide Bar Chart for number of male and female students. **HINT:** Each Bar can be drawn using drawRect() and fillRect() method.
7. Write a Java Program to draw a 2D rectangle in green color. Next draw the flag of Nepal inside the rectangle. Write a string “MY NEPAL” below the rectangle.
8. Demonstrate various drawing methods. How do you create ,load and display image?(see chapter 4)
9. Short Notes:
  - i. Closable Frames
  - ii. Repaint method (*Read chapter 4*)



## Introduction:

Graphics are the visual representation of objects and images and are often compared with text. Graphics can be defined differently like visual representation of data and includes a variety of forms including drawings, photographs, line art, graphs, diagrams, numbers, symbols, geometric designs, maps, and engineering drawings. So, simply understand graphics as visual representation.

Computer Graphics (abbreviate as CG), an art of drawing pictures on computer screens with the help of programming that represents pictorially some meaningful information. Computer Graphics is used where a set of images needs to be manipulated or the creation of the image in the form of pixels and is drawn on the computer. Used digital photography, film, entertainment, electronic gadgets, etc. Also can be used in UI design, rendering, geometric objects, animation, and many more.

### Types of Computer Graphics:

- **Raster Graphics:** Pixels are used for an image to be drawn. Also known as a bitmap image in which a sequence of images is into smaller pixels. Basically, a bitmap indicates a large number of pixels.
- **Vector Graphics:** Mathematical formulae are used to draw different types of shapes, lines, objects, and so on.

So, in this chapter we discuss about Graphics in Java. Graphics class is the abstract class for all graphics contexts that allow an application to draw onto components that are realized on various devices, as well as onto off-screen images. Here we overview several of Java's capabilities for drawing two-dimensional shapes, controlling colors and controlling fonts, helps programmers to visually enhance their applications. Also, Java contains more sophisticated drawing capabilities as part of the Java 2D API and its successor technology JavaFX.

Java's coordinate system, which is a scheme for identifying every point on the screen. By default, the upper-left corner of a GUI component (e.g. a window) has the coordinates (0,0). x-coordinate => horizontal coordinate and y-coordinate => vertical coordinate.

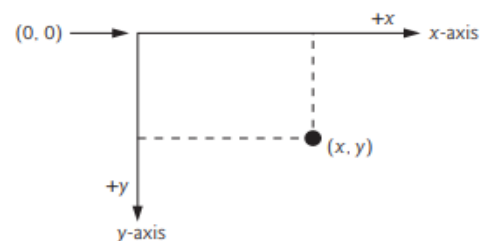


Figure below shows a portion of the class hierarchy that includes various graphics class and Java 2D API classes and interfaces.

- Class **Color** – contains methods and constants for manipulation colors.
- Class **JComponent** – contains method *paintComponent*, used to draw graphics on a component.
- Class **Font** – contains methods and constants for manipulating fonts.
- Class **FontMetrics** – contains methods for obtaining font information.
- Class **Graphics** – contains methods for drawing strings, lines, rectangle and other shapes.
- Class **Graphics2D** – extends class Graphics, used for drawing with the Java 2D API.
- Class **BasicStroke** – helps to specify the drawing characteristics of lines.
- Class **GradientPaint** and **TexturePaint** – help specify the characteristics for filling shapes with *colors* or *patterns*.



**Figure: Classes and Interfaces used from Java's original graphics capabilities and from the Java 2D API.**

**Note:** Different display monitors have different resolutions(i.e. the density of the pixels varies). This can cause graphics to appear in different sizes on different monitors or on the same monitor with different settings.

Graphics class facilitate to draw different shapes , string , images using different methods(see chapter 4). A **graphics context** enables drawing on the screen. A **Graphics object** manages a graphics context and draws pixels on the screen that represent text and other graphical objects (e.g., lines, ellipses, rectangles and other polygons). Graphics objects contain methods for drawing, font manipulation, color manipulation and the like.

Class Component is the superclass for many of the classes in package **java.awt**. Class JComponent(package **javax.swing**) , which inherits indirectly from class component, contains a **paintComponent** method that can be used to draw graphics.

## Color Control:

Class **Color** declares methods and constants for manipulating colors in a Java program. The pre-declared color constants are as bellow:

| Color constant                                    | RGB value     |
|---------------------------------------------------|---------------|
| <code>public static final Color RED</code>        | 255, 0, 0     |
| <code>public static final Color GREEN</code>      | 0, 255, 0     |
| <code>public static final Color BLUE</code>       | 0, 0, 255     |
| <code>public static final Color ORANGE</code>     | 255, 200, 0   |
| <code>public static final Color PINK</code>       | 255, 175, 175 |
| <code>public static final Color CYAN</code>       | 0, 255, 255   |
| <code>public static final Color MAGENTA</code>    | 255, 0, 255   |
| <code>public static final Color YELLOW</code>     | 255, 255, 0   |
| <code>public static final Color BLACK</code>      | 0, 0, 0       |
| <code>public static final Color WHITE</code>      | 255, 255, 255 |
| <code>public static final Color GRAY</code>       | 128, 128, 128 |
| <code>public static final Color LIGHT_GRAY</code> | 192, 192, 192 |
| <code>public static final Color DARK_GRAY</code>  | 64, 64, 64    |

**Figure: Color constants and their RGB values**

Several color methods and constructors are as follows:

| Method                                               | Description                                                                                                 |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <i>Color constructors and methods</i>                |                                                                                                             |
| <code>public Color(int r, int g, int b)</code>       | Creates a color based on red, green and blue components expressed as integers from 0 to 255.                |
| <code>public Color(float r, float g, float b)</code> | Creates a color based on red, green and blue components expressed as floating-point values from 0.0 to 1.0. |
| <code>public int getRed()</code>                     | Returns a value between 0 and 255 representing the red content.                                             |
| <code>public int getGreen()</code>                   | Returns a value between 0 and 255 representing the green content.                                           |
| <code>public int getBlue()</code>                    | Returns a value between 0 and 255 representing the blue content.                                            |
| <i>Graphics methods for manipulating Colors</i>      |                                                                                                             |
| <code>public Color getColor()</code>                 | Returns Color object representing current color for the graphics context.                                   |
| <code>public void setColor(Color c)</code>           | Sets the current color for drawing with the graphics context.                                               |

**Figure: Color methods and Color-related Graphics methods**

### Example 1:

**/\*ColorExample.java\*/**

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Color;
import java.awt.Graphics;
```

```
class ColorPanel extends JPanel{
```

```
 ColorPanel(){
 setSize(400,360);
 }
```

```
// let's draw rectangles and Strings in different colors
```

```
public void paintComponent(Graphics g){
```

```
 super.paintComponent(g);
 this.setBackground(Color.white);
```

```
//set new drawing color using integers
```

```
 g.setColor(new Color(255,0,0));
 g.fillRect(15, 25, 100, 20);
 g.drawString("Current RGB::"+g.getColor(), 130, 40);
```

```
//set new drawing color using floats
```

```
 g.setColor(new Color(0.2f,0.6f,0.3f));
 g.fillRect(15, 50, 100, 20);
 g.drawString("Current RGB::"+g.getColor(), 130, 65);
```

```
//set new drawing color using static Color objects
```

```
 g.setColor(Color.blue);
 g.fillRect(15, 75, 100, 20);
 g.drawString("Current RGB::"+g.getColor(), 130, 90);
```

```
//display individual RGB values
```

```
 Color color = Color.pink;
 g.setColor(color);
 g.fillRect(15, 100, 100, 20);
 g.drawString("RGB values::"+color.getRed()+" "+color.getGreen()+" "+color.getBlue(), 130, 115);
```

```
 g.setColor(Color.DARK_GRAY);
```

```
 g.drawString("For more follow : Deitel and Deitel book.", 15, 300);
```

```
}
```

```
}
```

```
public class ColorExample {
```

```
 public static void main(String[] args) {
```

```
 JFrame frame = new JFrame();
```

```
 ColorPanel cp = new ColorPanel();
```

```
 frame.add(cp);
```

```
 frame.setTitle("Color methods and constructor");
```

```
 frame.setSize(400,360);
```

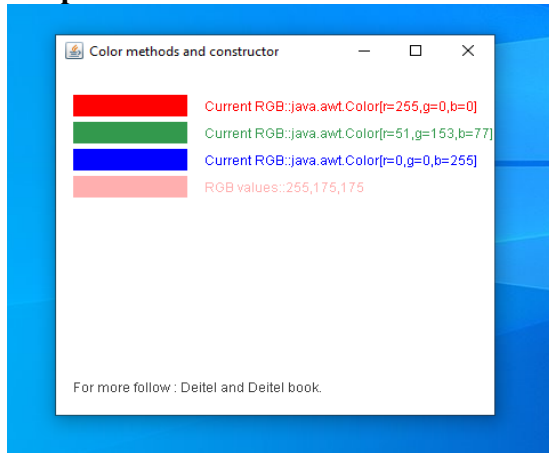
```
 frame.setLayout(null);
```

```
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
 frame.setVisible(true);
```

```
}
}
```

## Output:



## Example 2: Choosing colors with *JColorChooser*

```
/*ColorChooserExample.java*/
```

```
import javax.swing.JButton;
```

```
import javax.swing.JColorChooser;
```

```
import javax.swing.JFrame;
```

```
import javax.swing.JPanel;
```

```
import java.awt.Color;
```

```
import java.awt.event.ActionEvent;
```

```
import java.awt.event.ActionListener;
```

```
class ShowColorChooser extends JPanel{
```

```
 JButton cColor; //declaration of button (instance variable cColor)
```

```
 Color color = Color.blue; //define color (instance variable color with value blue)
```

```
 public ShowColorChooser(){ // constructor
```

```
 setSize(300,100); //set size of panel
```

```
 cColor = new JButton("Choose Color"); // define button
```

```
 add(cColor); //add button to panel
```

```
 cColor.addActionListener(new ActionListener(){ // event handling, using anonymous class
```

```
 public void actionPerformed(ActionEvent ae){ // performing action
```

```
 //showing color chooser dialog
```

```
 color = JColorChooser.showDialog(ShowColorChooser.this, "Choose a color", color);
```

```
 if(color == null){ //if not select any color from color chooser
```

```
 color = Color.blue;
```

```
 }
```

```
 setBackground(color); //set background color of panel on click of button
```

```
 }
```

```
 });
```

```
}
```

```
}
```

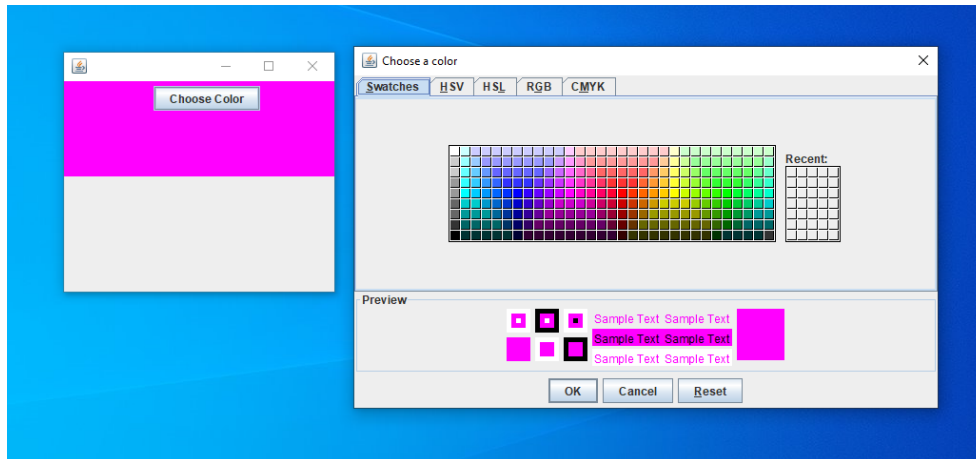
```
public class ColorChooserExample {
```

```

public static void main(String[] args) {
 JFrame frame = new JFrame(); //define frame
 frame.add(new ShowColorChooser()); //adding panel
 frame.setSize(300,260); //size of frame
 frame.setLayout(null); // layout set to null i.e. manual layout
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // exit frame from application
 frame.setVisible(true); // make frame visible
}
}

```

**Output:**



## Manipulating Fonts:

Introduces methods and constants for manipulating fonts. Class Font's constructor takes three arguments – the font name, font style and font size. Most font methods and font constants are part of class Font , are shown below:

| Method or constant                                         | Description                                                         |
|------------------------------------------------------------|---------------------------------------------------------------------|
| <i>Font constants, constructors and methods</i>            |                                                                     |
| <code>public static final int PLAIN</code>                 | A constant representing a plain font style.                         |
| <code>public static final int BOLD</code>                  | A constant representing a bold font style.                          |
| <code>public static final int ITALIC</code>                | A constant representing an italic font style.                       |
| <code>public Font(String name, int style, int size)</code> | Creates a Font object with the specified font name, style and size. |
| <code>public int getStyle()</code>                         | Returns an int indicating the current font style.                   |
| <code>public int getSize()</code>                          | Returns an int indicating the current font size.                    |
| <code>public String getName()</code>                       | Returns the current font name as a string.                          |
| <code>public String getFamily()</code>                     | Returns the font's family name as a string.                         |
| <code>public boolean isPlain()</code>                      | Returns true if the font is plain, else false.                      |
| <code>public boolean isBold()</code>                       | Returns true if the font is bold, else false.                       |
| <code>public boolean isItalic()</code>                     | Returns true if the font is italic, else false.                     |

#### Graphics methods for manipulating Fonts

**public Font** getFont()

Returns a Font object reference representing the current font.

**public void** setFont(Font f)

Sets the current font to the font, style and size specified by the Font object reference f.

### Example 3: Manipulating Fonts

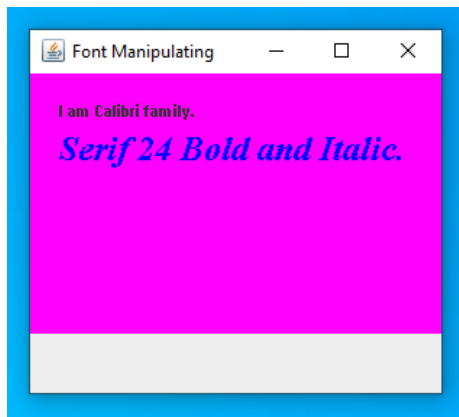
**/\* FontExample .java\*/**

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;

class FontJPanel extends JPanel{
 FontJPanel(){
 setSize(300,180);
 setBackground(Color.MAGENTA);
 }
 public void paintComponent(Graphics g){
 super.paintComponent(g); // referencing paintComponent method from JComponent
 //set font to, family: Calibri, style:bold, size:12 pixels and draw string
 g.setFont(new Font("Calibri",Font.BOLD,12));
 g.drawString("I am Calibri family.", 20, 30);
 //set font to Serif,bold and italic, 24pt and draw a string
 g.setColor(Color.blue);
 g.setFont(new Font("Serif",Font.BOLD+Font.ITALIC,24));
 g.drawString(g.getFont().getName()+" "+g.getFont().getSize()+" Bold and Italic.", 20, 60);
 }
}

public class FontExample {
 public static void main(String[] args) {
 JFrame frame = new JFrame("Font Manipulating");
 FontJPanel fp = new FontJPanel();
 frame.add(fp);
 frame.setSize(300,260);
 frame.setLayout(null);
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 frame.setVisible(true);
 }
}
```

**Output:**



## Font Metrics:

Font Metrics provide precise information about a font, such as height, descent (the amount a character dips below the baseline), ascent (the amount a character rises above the baseline) and leading (the difference between the descent of one line of text and the ascent of the line of text below it – that is, the interline spacing).



Figure: Font metrics

Some methods for obtaining font metrics are as bellow:

| Method                                                   | Description                                                     |
|----------------------------------------------------------|-----------------------------------------------------------------|
| <i>FontMetrics methods</i>                               |                                                                 |
| <code>public int getAscent()</code>                      | Returns the ascent of a font in points.                         |
| <code>public int getDescent()</code>                     | Returns the descent of a font in points.                        |
| <code>public int getLeading()</code>                     | Returns the leading of a font in points.                        |
| <code>public int getHeight()</code>                      | Returns the height of a font in points.                         |
| <i>Graphics methods for getting a Font's FontMetrics</i> |                                                                 |
| <code>public FontMetrics getFontMetrics()</code>         | Returns the FontMetrics object for the current drawing Font.    |
| <code>public FontMetrics getFontMetrics(Font f)</code>   | Returns the FontMetrics object for the specified Font argument. |

**Example 4:** Font Metrics example with methods.

**/\*FontMetricsExample.java\*/**

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Color;
import java.awt.Font;
```



```

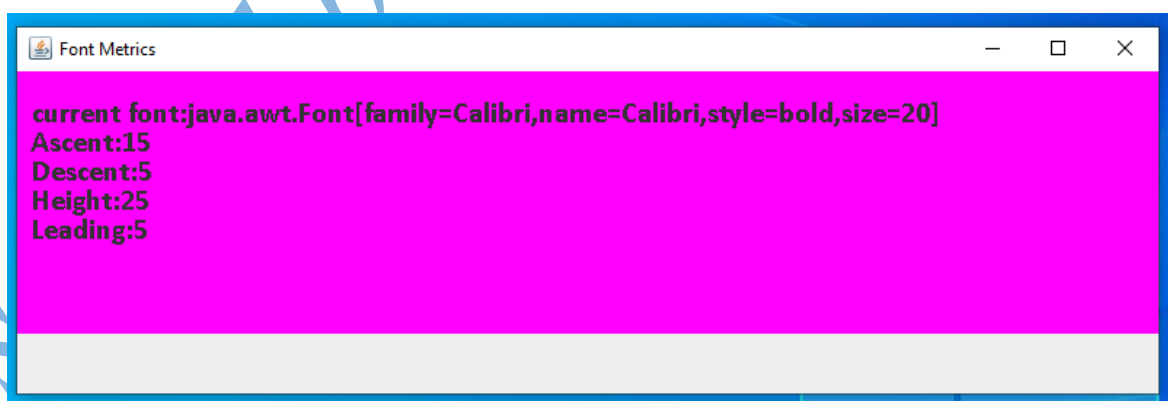
import java.awt.Graphics;
import java.awt.FontMetrics;

class FontMetricsJPanel extends JPanel{
 FontMetricsJPanel(){
 setSize(800,180);
 setBackground(Color.MAGENTA);
 }
 public void paintComponent(Graphics g){
 super.paintComponent(g);
 g.setFont(new Font("Calibri",Font.BOLD,20));
 FontMetrics metrics =g.getFontMetrics();
 g.drawString("current font:"+g.getFont(), 10, 35);
 g.drawString("Ascent:"+metrics.getAscent(), 10, 55);
 g.drawString("Descent:"+metrics.getDescent(), 10, 75);
 g.drawString("Height:"+metrics.getHeight(), 10, 95);
 g.drawString("Leading:"+metrics.getLeading(), 10, 115);
 }
}

public class FontMetricsExample {
 public static void main(String[] args) {
 JFrame frame = new JFrame("Font Metrics");
 FontMetricsJPanel fmp = new FontMetricsJPanel();
 frame.add(fmp);
 frame.setSize(800,260);
 frame.setLayout(null);
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 frame.setVisible(true);
 }
}

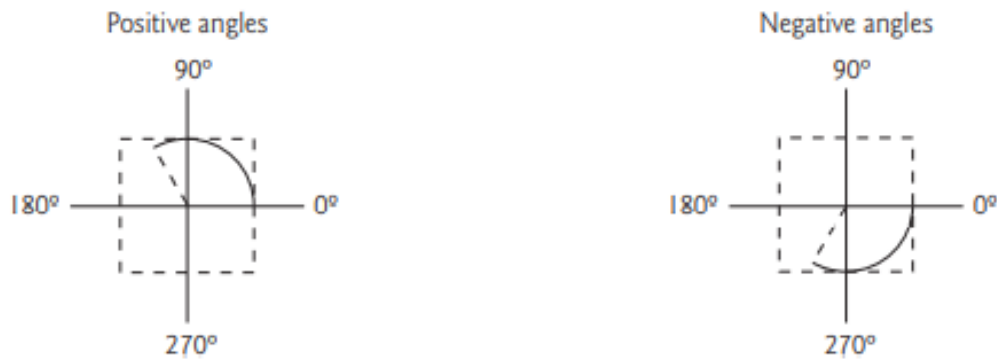
```

### Output:



### Drawing Arcs:

An arc is drawn as a portion of an oval. Arc Angles are measured in degrees. Arcs that sweep in a counterclockwise direction are measured in positive degrees and arcs sweep in a clockwise direction are measured in negative degrees.



**Figure: Positive and Negative arc angles**

| Method                                                                                              | Description                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle)</code> | Draws an arc relative to the bounding rectangle's top-left x- and y-coordinates with the specified width and height. The arc segment is drawn starting at startAngle and sweeps arcAngle degrees.                        |
| <code>public void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle)</code> | Draws a filled arc (i.e., a sector) relative to the bounding rectangle's top-left x- and y-coordinates with the specified width and height. The arc segment is drawn starting at startAngle and sweeps arcAngle degrees. |

**Figure: Graphics methods for drawing arcs**

#### Example 5: Draw different arcs

**/\* DifferentArcExample.java \*/**

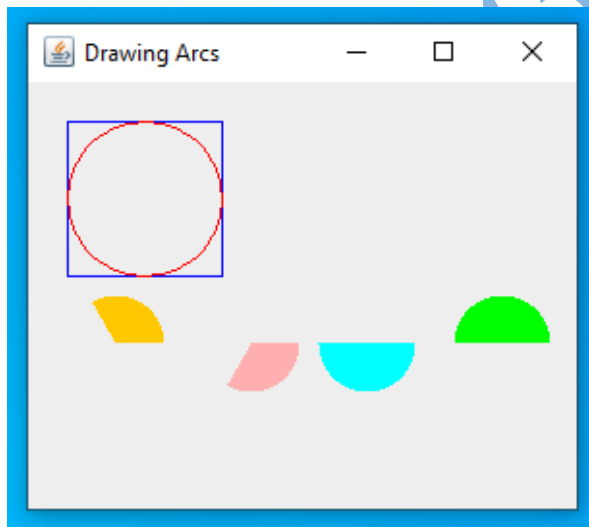
```
import java.awt.Color;
import java.awt.Graphics;
import javax.swing.JFrame;
import javax.swing.JPanel;
class ArcJPanel extends JPanel{
 ArcJPanel(){
 setSize(300,260);
 }
 public void paintComponent(Graphics g){
 super.paintComponent(g);
 //start at 0 and sweep 360 degrees
 g.setColor(Color.blue);
 g.drawRect(20, 20, 80, 80);
 g.setColor(Color.red);
 g.drawArc(20, 20, 80, 80, 0, 360);
 //start at 0 and sweep 120 degrees
```

```

g.setColor(Color.orange);
g.fillArc(20, 110, 50, 50, 0, 120);
//start at 0 and sweep -120 degrees
g.setColor(Color.pink);
g.fillArc(90, 110, 50, 50, 0, -120);
//start at 0 and sweep 180 degrees
g.setColor(Color.green);
g.fillArc(220, 110, 50, 50, 0, 180);
//start at 0 and sweep -180 degrees
g.setColor(Color.cyan);
g.fillArc(150, 110, 50, 50, 0, -180);
}
}
public class DifferentArcExample {
 public static void main(String[] args) {
 JFrame frame = new JFrame("Font Manipulating");
 ArcJPanel ajp = new ArcJPanel();
 frame.add(ajp);
 frame.setSize(300,260);
 frame.setLayout(null);
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 frame.setVisible(true);
 }
}

```

**Output:**



### **Drawing Polygon and Polylines:**

Polygons are closed multisided shapes composed of straight-line segments. Polylines are sequence of connected points.

| Method                                                                          | Description                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Graphics methods for drawing polygons</i>                                    |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <code>public void drawPolygon(int[] xPoints, int[] yPoints, int points)</code>  | Draws a polygon. The <i>x</i> -coordinate of each point is specified in the <i>xPoints</i> array and the <i>y</i> -coordinate of each point in the <i>yPoints</i> array. The last argument specifies the number of points. This method draws a <i>closed polygon</i> . If the last point is different from the first, the polygon is <i>closed</i> by a line that connects the last point to the first.               |
| <code>public void drawPolyline(int[] xPoints, int[] yPoints, int points)</code> | Draws a sequence of connected lines. The <i>x</i> -coordinate of each point is specified in the <i>xPoints</i> array and the <i>y</i> -coordinate of each point in the <i>yPoints</i> array. The last argument specifies the number of points. If the last point is different from the first, the polyline is <i>not</i> closed.                                                                                      |
| <code>public void drawPolygon(Polygon p)</code>                                 | Draws the specified polygon.                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>public void fillPolygon(int[] xPoints, int[] yPoints, int points)</code>  | Draws a <i>filled</i> polygon. The <i>x</i> -coordinate of each point is specified in the <i>xPoints</i> array and the <i>y</i> -coordinate of each point in the <i>yPoints</i> array. The last argument specifies the number of points. This method draws a <i>closed polygon</i> . If the last point is different from the first, the polygon is <i>closed</i> by a line that connects the last point to the first. |
| <code>public void fillPolygon(Polygon p)</code>                                 | Draws the specified <i>filled</i> polygon. The polygon is <i>closed</i> .                                                                                                                                                                                                                                                                                                                                             |
| <i>Polygon constructors and methods</i>                                         |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <code>public Polygon()</code>                                                   | Constructs a new polygon object. The polygon does not contain any points.                                                                                                                                                                                                                                                                                                                                             |
| <code>public Polygon(int[] xValues, int[] yValues, int numberOfPoints)</code>   | Constructs a new polygon object. The polygon has <i>numberOfPoints</i> sides, with each point consisting of an <i>x</i> -coordinate from <i>xValues</i> and a <i>y</i> -coordinate from <i>yValues</i> .                                                                                                                                                                                                              |
| <code>public void addPoint(int x, int y)</code>                                 | Adds pairs of <i>x</i> - and <i>y</i> -coordinates to the <i>Polygon</i> .                                                                                                                                                                                                                                                                                                                                            |

**Figure: Graphics methods for polygons and class Polygon methods**

#### Example 6: Drawing polygons and polylines

**/\*DrawingPolygonPolylineExample.java\*/**

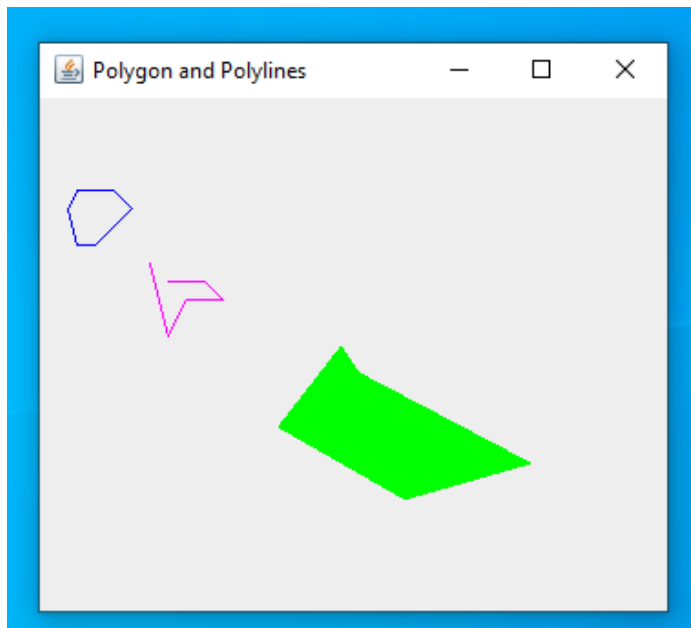
```
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Polygon;
class PolyJPanel extends JPanel{
 PolyJPanel(){
 setSize(360,320);
 }
 public void paintComponent(Graphics g){
 super.paintComponent(g);
 //draw polygon with Polygon object
 int[] x = {20,40,50,30,20,15};
```

```

int[] y = {50,50,60,80,80,60};
g.setColor(Color.blue);
Polygon poly1 = new Polygon(x,y,6);
g.drawPolygon(poly1); //can draw polygons with two arrays
//draw polylines with two arrays
int[] xP = {70,90,100,80,70,65,60};
int[] yP = {100,100,110,110,130,110,90};
g.setColor(Color.magenta);
g.drawPolyline(xP, yP, 7);
//draw filled polygon with polygon object
Polygon polygon = new Polygon();
polygon.addPoint(165, 135);
polygon.addPoint(175, 150);
polygon.addPoint(270, 200);
polygon.addPoint(200, 220);
polygon.addPoint(130, 180);
g.setColor(Color.green);
g.fillPolygon(polygon);
}
}
public class DrawingPolygonPolylineExample {
 public static void main(String[] args) {
 JFrame frame = new JFrame("Polygon and Polylines");
 PolyJPanel pjp = new PolyJPanel();
 frame.add(pjp);
 frame.setSize(360,320);
 frame.setLayout(null);
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 frame.setVisible(true);
 }
}

```

**Output:**



**Example 7:** Create a graphics application to display “Pokhara University” in blue color with font name: Times New Roman, type: Bold and size: 20.

**/\*DrawPokharaUniversity.java\*/**

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;
class DrawPokharaUniversity extends JPanel{
 DrawPokharaUniversity(){
 setSize(300,200);
 }
 public void paint(Graphics g){
 g.setColor(Color.blue);
 g.setFont(new Font("Times New Roman",Font.BOLD,20));
 g.drawString("Pokhara University", 100, 80);
 }
}
public class PokharaUniversityExample {
 public static void main(String[] args) {
 JFrame frame = new JFrame("Draw Pokhara University");
 frame.add(new DrawPokharaUniversity());
 frame.setSize(400,200);
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 frame.setLayout(null);
 frame.setVisible(true);
 }
}
```

**Output:**



**Example 8:** Write a Java Program to draw a 2D rectangle in green color. Next draw the flag of Nepal inside the rectangle. Write a string “MY NEPAL” below the rectangle.

**/\* NationalFlag .java\*/**

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Graphics;
import java.awt.Color;
import java.awt.Font;
class DrawFlag extends JPanel{
 DrawFlag(){
 setSize(700,720);
 }
 public void paintComponent(Graphics g){
 super.paintComponent(g);

 g.setColor(Color.green);
 g.drawRect(10,10,600,620);
 g.setColor(Color.black);
 g.fillRect(40,20,10,600);

 int[] xPoints = {50,300,90,340,50,50};
 int[] yPoints = {20,240,240,500,500,20};
 g.setColor(Color.blue);
 g.fillPolygon(xPoints,yPoints,5);

 int[] xP = {60,280,70,320,60,60};
 int[] yP = {40,230,230,490,490,40};
 g.setColor(Color.red);
 g.fillPolygon(xP,yP,5);

 g.setColor(Color.white);
 g.fillArc(100,160,60,34,0,-180);
```

```

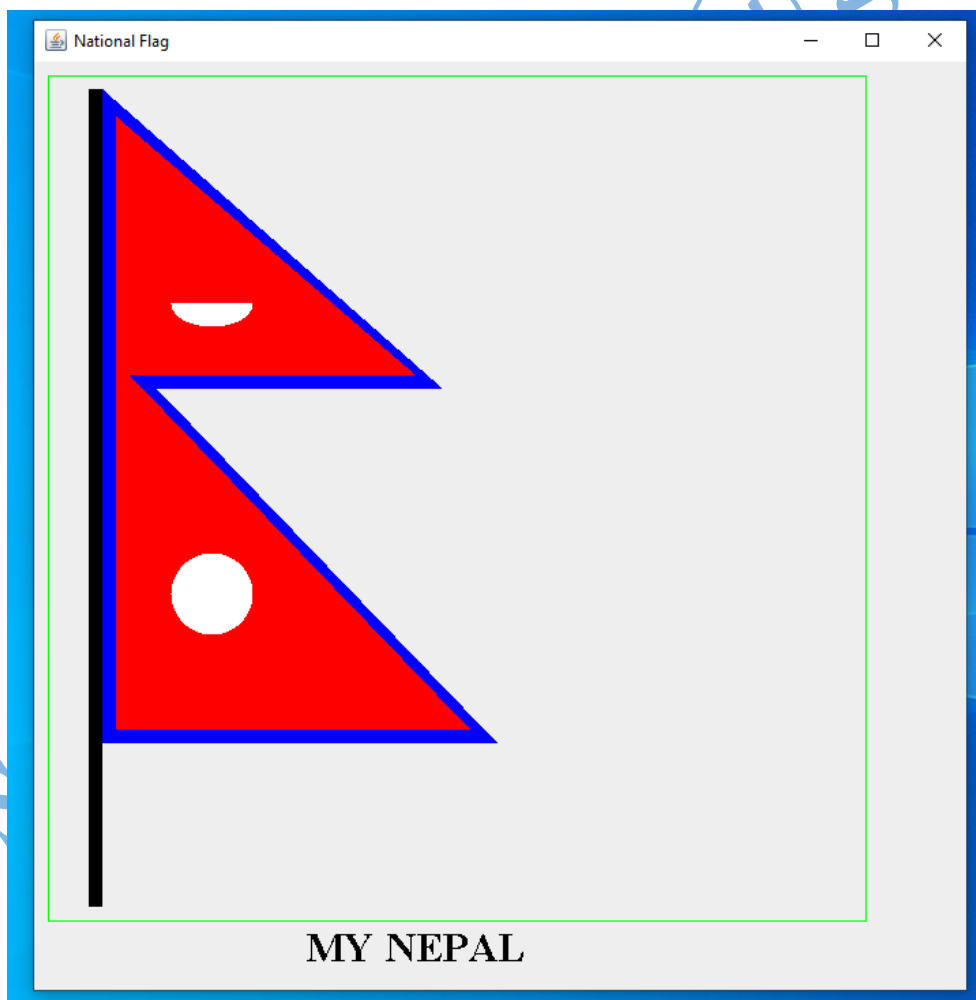
 g.fillArc(100,360,60,60,0,360);

 g.setColor(Color.BLACK);
 g.setFont(new Font("serif",Font.BOLD,30));
 g.drawString("MY NEPAL",200,660);
 }
}

public class NationalFlag {
 public static void main(String[] args) {
 JFrame frame = new JFrame();
 frame.add(new DrawFlag());
 frame.setSize(700,720);
 frame.setTitle("National Flag");
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 frame.setLayout(null);
 frame.setVisible(true);
 }
}

```

### Output:





**Example 9:** As per a poll conducted in a class of 48 students , 25 students are supportive of virtual classroom while 15 are against it and remaining 8 students chose to stay neutral. Create a pie chart to illustrate this poll result.

**/\* PieChartExample .java\*/**

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Graphics;
import java.awt.Color;

class Pie {
 double value;
 Color color;
 public Pie(double value, Color color){
 this.value = value;
 this.color = color;
 }
}

class PieChart extends JPanel{
 Pie[] pies = {new Pie(25,Color.blue),new Pie(15, Color.red),new Pie(8, Color.black)};

 public PieChart(){
 setSize(400,360);
 }
 public void paint(Graphics g){
 double total = 0.0D;
 for(int i=0;i<pies.length;i++){ //sum all values for total
 total+=pies[i].value;
 }
 double currentValue = 0.0;
 int startAngle = 0;

 for(int i=0;i<pies.length;i++){
 //total value is equal to 360 degrees. So, estimate equivalent degree to student number
 startAngle = (int)(currentValue*360/total); //set individual start angle for given data
 int endAngle = (int)(pies[i].value*360/total); // set individual end angel for given data
 g.setColor(pies[i].color); //set individual color
 g.fillArc(50,100,200,200,startAngle,endAngle);
 currentValue+=pies[i].value;
 }
 g.setColor(Color.blue);
 g.fillRect(5,10,20,20);
 g.drawString("Support Virtual Class",28,22);

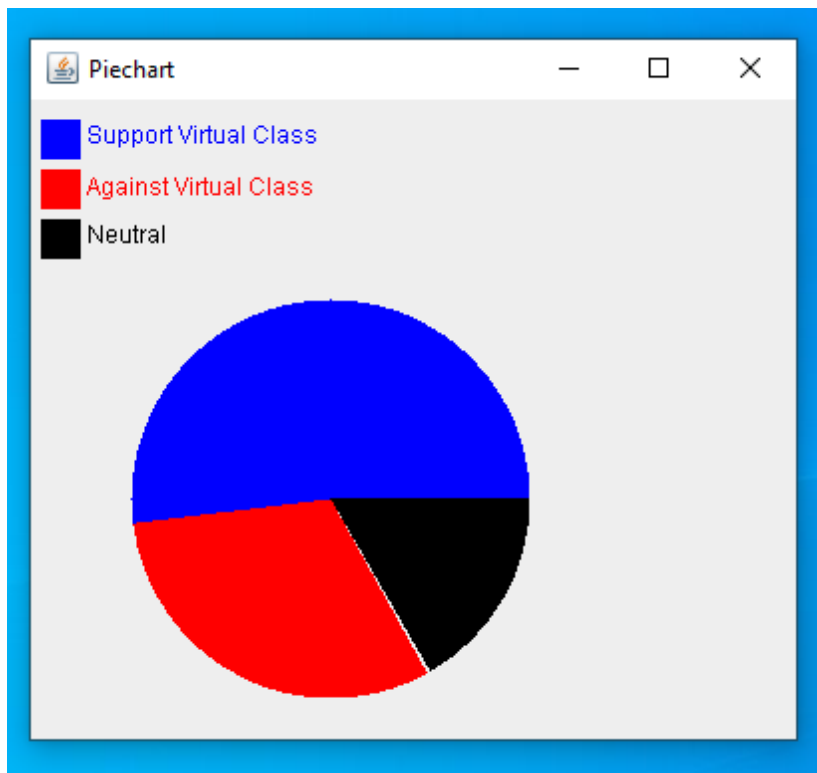
 g.setColor(Color.red);
 g.fillRect(5,35,20,20);
 g.drawString("Against Virtual Class",28,48);
 }
}
```

```

 g.setColor(Color.black);
 g.fillRect(5,60,20,20);
 g.drawString("Neutral",28,72);
 }
}
public class PieChartExample{
 public static void main(String[] args) {
 JFrame frame = new JFrame();
 frame.getContentPane().add(new PieChart());
 frame.setTitle("Piechart");
 frame.setLayout(null);
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 frame.setSize(400,360);
 frame.setVisible(true);
 }
}

```

**Output:**



**Example 10:** Write a program to draw a Bar chart of Number of Students giving exam of Java, C and C++. (**Hint:** Let us consider number of student that appear on exam of Java be 40, C be 94 and C++ be 85).

**/\* BarChartExample.java \*/**

```

import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Color;
import java.awt.Graphics;

```

```

import java.awt.Font;
class DrawBarChart extends JPanel{
 double[] values = {46,94,85}; //array of student number (double)
 String[] labels = {"Java","C","C++"}; // array of subject number(String)
 Color[] colors = {Color.green,Color.red,Color.blue}; //array of color for different bar
 int panelWidth,panelHeight,barWidth;
 int vgap=30; //top and bottom margin(vertical gap)
 int hgap=40; //left and right margin(horizontal gap)
 public DrawBarChart(){
 int maxValue=0,labelMaxValue=0;
 for(int i=0;i<values.length;i++){
 if(maxValue < values[i]){
 maxValue = (int)values[i]; //sorting max value to be drawn
 }
 if(labelMaxValue < labels[i].length()){
 labelMaxValue = labels[i].length(); //for barWidth , sorting max length string
 }
 }
 barWidth = labelMaxValue*values.length; //setting barWidth like Java =4 and 3(4*3=12px)
 panelHeight = maxValue*3+2*vgap; // dynamically assigned panel height (94*3+2*30 = 342px)
 panelWidth = barWidth*25+2*hgap; // dynamically assigned panel width (12*25+2*40 = 380px)
 System.out.println(panelWidth+" "+panelHeight); //see console
 setSize(panelWidth,panelHeight); //now seting(dynamic) panel size(380,342)
 }
 public int setHeight(){ // return height for frame
 return panelHeight;
 }
 public int setWidth(){ //return width for frame
 return panelWidth;
 }
 public void paintComponent(Graphics g){
 super.paintComponent(g);
 int initialX = 80; //first bar initial x-coordinate
 int initialY;
 for(int i=0;i<values.length;i++){
 initialY = panelHeight-3*vgap-(int)values[i]*2; // for individual y-coordinate
 g.setColor(colors[i]); //setting particular color
 g.fillRect(initialX, initialY, barWidth*4, (int)values[i]*2); //(int)values[i]*2 magnify height of bar twice
 g.setColor(Color.BLACK);
 g.drawRect(initialX, initialY, barWidth*4, (int)values[i]*2);
 g.setFont(new Font("Times New Roman",Font.BOLD,20));
 g.drawString(labels[i], initialX+4, panelHeight-64);
 initialX = initialX+(barWidth*4);
 }
 }
}
public class BarChartExample {
 public static void main(String[] args) {

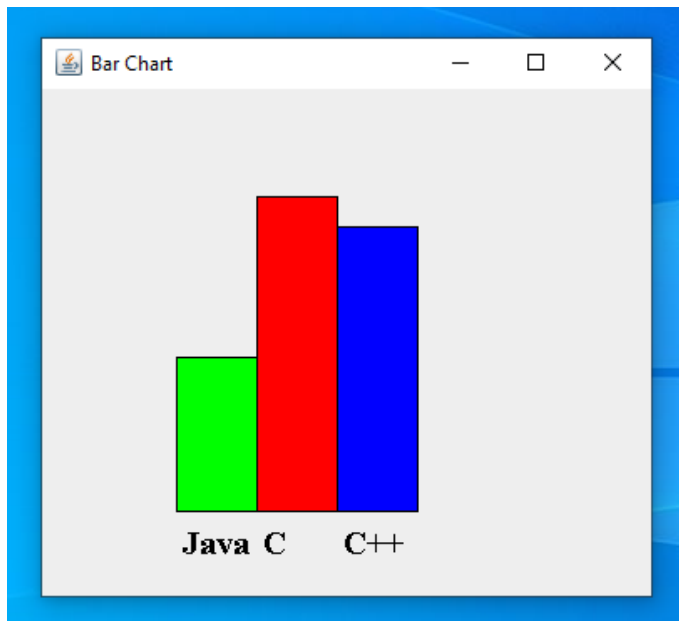
```

```

JFrame frame = new JFrame("Bar Chart");
DrawBarChart dbc = new DrawBarChart();
int w = dbc.setWidth(); // width of frame dynamically
int h = dbc.setHeight(); // height of frame dynamically
frame.add(dbc);
frame.setSize(w,h);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setLayout(null);
frame.setVisible(true);
}
}

```

**Output:**



**Note:** To draw Pie-chart and bar chart you can use java.swing.JFreeChart package for more advance chart base design. Example 9 & 10 are done using 2D shapes i.e. basic graphics properties.

**Note:**

Every example are illustrate Swing application in this chapter executed on closable frames. So , if you are asked about **closable** frames , use either **EXIT\_ON\_CLOSE** or **DISPOSE\_ON\_CLOSE** or **both** values in **setDefaultCloseOperation** method.

## Chapter 7: Network Programming

### Some old Questions:

1. Draw a diagram illustrating how client server interaction occurs in UNIX based system, with detailed methods in each step.
2. Create a TCP client/server program in which client sends an integer to the server and the server responds to client by sending square of the number sent by client.
3. Explain network programming . Compare TCP and UDP sockets.
4. Create a TCP client/server program in which client and server communicate by sending a message to each other.
5. What are the different socket types in Java? Draw a diagram with appropriate methods to describe client server interaction in java.
6. Explain the use of URL and URL connection class with a suitable example.
7. Write a TCP socket program to accept a line of string input from the client and send it to server. The server checks if the string is palindrome or not and send result back to client. Client machine displays the string received from server.
8. Discuss the process of creation of server and client sockets with Exceptions handled explicitly with a suitable example.
9. What is an URL? Write a Java Program for reading contents in a given URL.
10. Write a client program which connects with server running at port 5000 and send "I am client" message to server and also print received message from server in console.(Assume TCP/IP communication)
11. Define Socket Programming with example.
12. What is a socket? Differentiate between datagram and stream communication.
13. Write the steps to create a client server program. Provide code snippet for each steps.
14. Write a program to send " Message from Pokhara University" from client to server using java socket programming.
15. Explain InetAddress class.
16. What are some key classes defined in java to work with datagrams ? How do you get a list of IP addresses that are assigned to a network interface?
17. What is socket ? Write a program to find local IP address and Hostname of the system.
18. Short Notes:
  - i. TCP and UDP socket
  - ii. Connection Less and Connection Oriented protocol

## Introduction:

Networking means the action or process of interacting with others to exchange information and develop professional or social contacts. Also, Networking means the linking of computer to allow them to operate interactively / share resources using common communication protocols. Types of networking centralized and distributed networking.

A **distributed** network is a system where computing tasks, data, and applications are spread across multiple, interconnected nodes (computers) rather than a single central location. These nodes communicate and work together to function as a unified system, enhancing fault tolerance, scalability, and resource sharing.

Java Networking designed with the concept(notion) to connect two or more computing devices together to share the resources. Almost all communication is done at application layer, similarly java program also uses **application layer** to communicate over the network. Where Java provides a number of built-in networking capabilities that make easy to develop Internet-based and web-based applications.

**java.net** package declare fundamental networking capabilities, classes and interfaces. Java offers **stream-based** communications and **packet-based** communications. Also **java.net** package provides support for two protocols(**TCP and UDP**). Both TCP and UDP helps to specify how computer should be connected to the internet and transmit data segment through out the network. TCP and UDP belongs to Network layer of TCP/IP model.

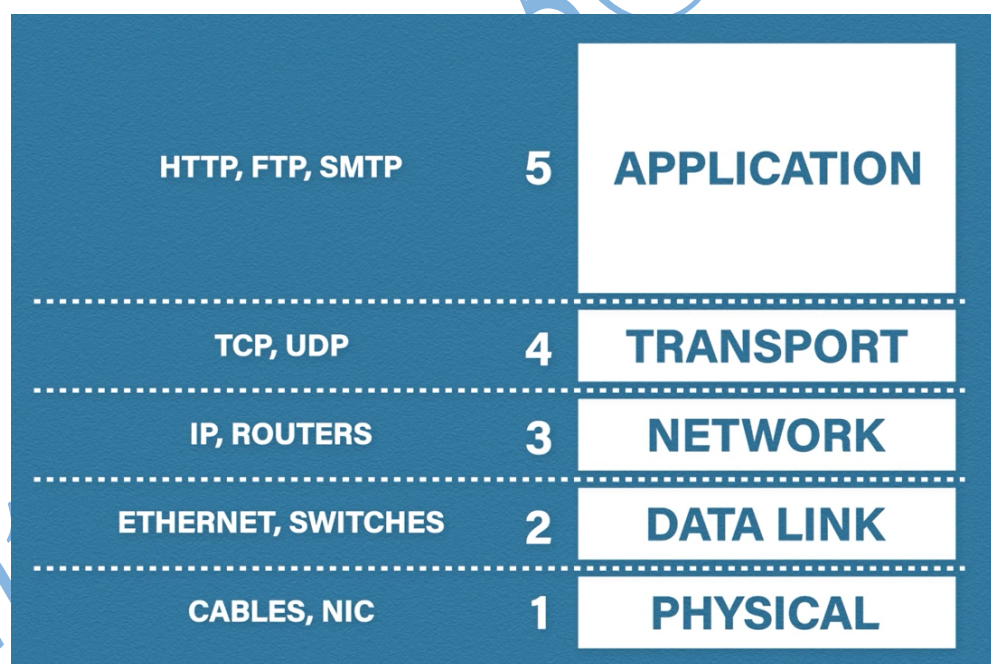


Figure: TCP/IP model

| TCP(Transmission Control Protocol)                                                                                                                                                                                                                                            | UDP(User Datagram Protocol)                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Specially designed to offer highly reliable and end-to-end byte stream .</li><li>• It is a connection-oriented protocol.</li><li>• TCP message make their way(shortest path) across the network from source to destination.</li></ul> | <ul style="list-style-type: none"><li>• Used for broadcast and multicast type of network transmission. It is unreliable than TCP. Packets were sent one by one.</li><li>• It is a connectionless protocol.</li><li>• It can send lots of packets to another.</li></ul> |

- TCP rearranges data packets in the specific order. (i.e. sequence of data)
- Retransmission of lost packets is possible.
- Uses three-handshaking. Such as SYN,SYN-ACK, and ACK.
- TCP has a (20-60) bytes variable length header.
- TCP is heavy-weight.
- Comparatively slower than UDP.
- Provides extensive error-checking mechanisms.(because of flow control and acknowledgement of data.
- Used by HTTP,HTTPs, FTP,SMTP and Telnet.

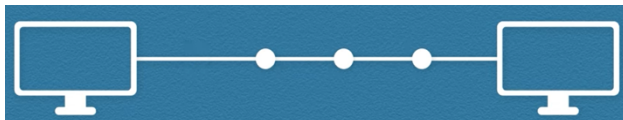


Figure: sending pieces of data

- Has no fixed order because all packets are independent or each other.(If need order application layer managed.)
- No retransmission of lost packet in UDP. Which leads corruption of file/data.
- No handshaking.
- UDP has an 8 bytes fixed-length header.
- UDP is lightweight.
- Faster, simpler , and more efficient than TCP.
- Has only basic error checking mechanism using checksums.
- UDP is used by DNS, DHCP, TFTP, SNMP, RIP, VoIP.

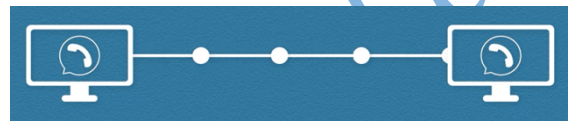


Figure: Firing data to receiver

#### Stream-based communication

- Enables applications to view networking as streams of data.
- Is a form of communication in which timing plays an important role.
- Transmission mode: Synchronous, Asynchronous and Isochronous.

#### Packet-based communication

- Transmit individual packets of information.(Commonly data images, audio and video over the internet).
- Transfer of small pieces of data across various networks.
- Packets are routed individually and not via same path .Destination have to reorder the packet to retrieve original message.

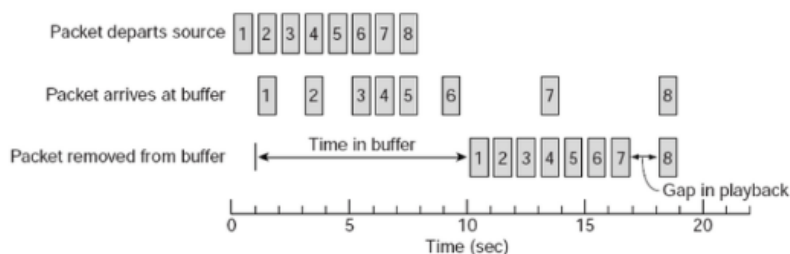
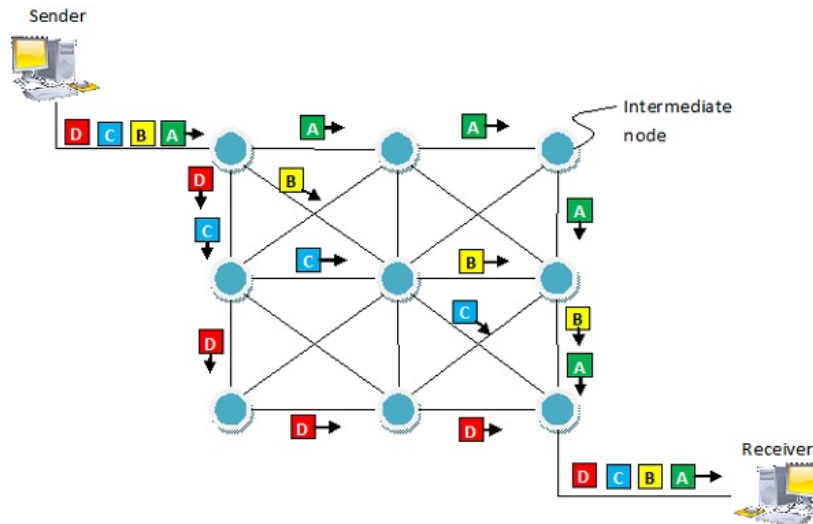


Figure: Stream based communication





**Figure: Packet Switching(Packet-based communication)**

#### Advantage of Java Networking:

1. Sharing resources
2. Centralize software management

We focus on both sides of **client/server** relationship. Client requests for some action be performed, where server performs the action and responds to the client. A common implementation of the request-response model is between web browsers and web servers. We introduce Java's **socket-based communications**, which enables applications to view networking. Example: File I/O – a program can read from a socket or write to a socket as simply as reading from a file or writing to a file.

#### Networking Terminologies:

1. **IP Address** : IP address is a unique number assigned to a node of a network . An IP address is represented by 32-bit or 128-bit unsigned number. **Example:** IPv4:**127.168.0.1**, composed of octets that range from 0 – 255; IPv6 : **2001:0db8:85a3:0000:0000:8a2e:0370:7334**.
2. **Protocol**: a set of rules basically that is followed for communication . Eg: TCP, FTP, Telnet, SMTP, POP,HTTP,HTTPS.
3. **Port Number**: Used to identify different applications. It acts as a communication endpoint between applications. Associated with the IP address(**127.168.0.1:80**) for communication between two applications. Some basic and well known ports are given following:

| Protocol Name | Types   | Port Number |
|---------------|---------|-------------|
| FTP           | TCP     | 20          |
| FTP Control   | TCP     | 21          |
| SSH           | TCP     | 22          |
| Telnet        | TCP     | 23          |
| SMTP          | TCP     | 25          |
| DNS           | TCP/UDP | 53          |
| DHCP          | UDP     | 67,68       |



|       |     |     |
|-------|-----|-----|
| TFTP  | UDP | 69  |
| HTTP  | TCP | 80  |
| HTTPs | TCP | 443 |
| POP3  | TCP | 110 |
| SNMP  | UDP | 161 |

4. **MAC Address:** Stands Media Access Control. Basically a hardware identification number which uniquely identifies each device on a network. Also a unique identifier of NIC(Network Interface Controller) . A network node can have multiple NIC but each with unique MAC address. For example, 00:0d:83::b1:c0:8e .
5. **Connection-oriented and connection-less protocol:**  
 Connection-oriented: Acknowledgment is sent by the receiver. So it is reliable but slow. Prefer hand shaking. Example, TCP(Transmission Control Protocol).  
 Connection-less protocol: Acknowledgement is not sent by the receiver. So it is not reliable but fast. Prefer no hand shaking. Example, UDP(User Datagram Protocol)
6. **Socket:** A socket in Java is one endpoint of a two-way communication link between two programs running on the network. A socket is bound to a port number so that the TCP layer can identify the application that data is destined to be sent to.

### java.net package:

The java.net package can be divided into two sections:

1. **A Low-Level API:** Deals with the abstractions of addresses i.e. networking identifiers, sockets i.e. bidirectional data communication mechanism and Interfaces i.e. network interfaces.
2. **A High Level API:** Deals with the abstraction of URIs i.e. Universal Resource Identifier, URLs i.e. Universal Resource Locator, and Connections i.e. connections to the resource pointed by URLs.

| List of classes provided by <i>java.net package</i> to deal with networking applications are:                                                                                                                                                                                                                                                                                                                                                       | List of interfaces provided by <i>java.net package</i> are:                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| i. Authenticator<br>ii. CacheRequest<br>iii. CacheResponse<br>iv. Content Handler<br>v. CookieHandler<br>vi. CookieManager<br>vii. DatagramPacket<br>viii. DatagramSocket<br>ix. DatagramSocketImpl<br>x. InterfaceAddress<br>xi. JarURLConnection<br>xii. MulticastSocket<br>xiii. InetSocketAddress<br>xiv. InetAddress<br>xv. Inet4Address<br>xvi. Inet6Address<br>xvii. IDN<br>xviii. HttpURLConnection<br>xix. HttpCookie<br>xx. NetPermission | i. ContentHandlerFactory<br>ii. CookiePolicy<br>iii. CookieStore<br>iv. DatagramSocketImplFactory<br>v. FileNameMap<br>vi. SocketOption<T><br>vii. SocketOptions<br>viii. SocketImplFactory<br>ix. URLStreamHandlerFactory<br>x. ProtocolFamily |

|         |                        |
|---------|------------------------|
| xxi.    | NetworkInterface       |
| xxii.   | PasswordAuthentication |
| xxiii.  | Proxy                  |
| xxiv.   | ProxySelector          |
| xxv.    | ServerSocket           |
| xxvi.   | Socket                 |
| xxvii.  | SocketAddress          |
| xxviii. | SocketImpl             |
| xxix.   | SocketPermission       |
| xxx.    | StandardSocketOptions  |
| xxxi.   | URI                    |
| xxxii.  | URL                    |
| xxxiii. | URLClassLoader         |
| xxxiv.  | URLConnection          |
| xxxv.   | URLDecoder             |
| xxxvi.  | URLEncoder             |
| xxxvii. | URLStreamHandler       |

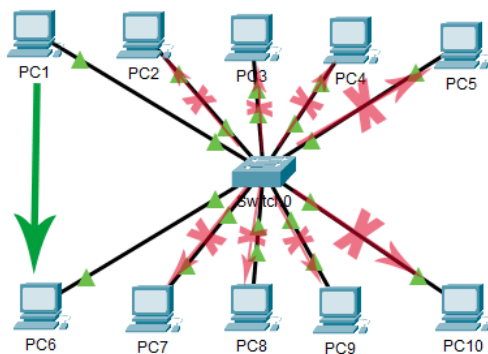
### Exception in *java.net*:

1. SocketException
2. ProtocolException
3. ConnectException
4. UnknownHostException

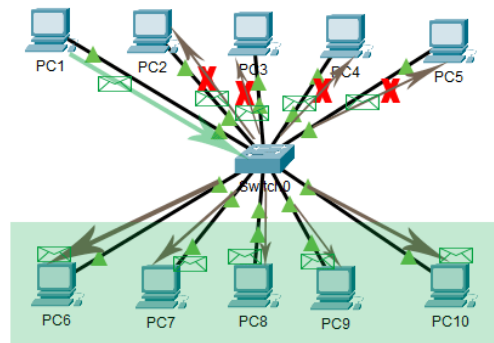
### Inet Address:

Java **InetAddress** class represents an IP address. Inet Address is used to encapsulate both the numerical IP address and the domain name for that address. Example: [www.facebook.com](http://www.facebook.com), [www.eemc.edu.np](http://www.eemc.edu.np), etc. It can handle both IPv4(32-bit) and IPv6(128-bit) addresses. An instance of **InetAddress** represents the IP address with its corresponding host name. There are two types of address: **Unicast** and **Multicast**. Moreover, **InetAddress** has a cache mechanism to store successful and unsuccessful host name resolutions.

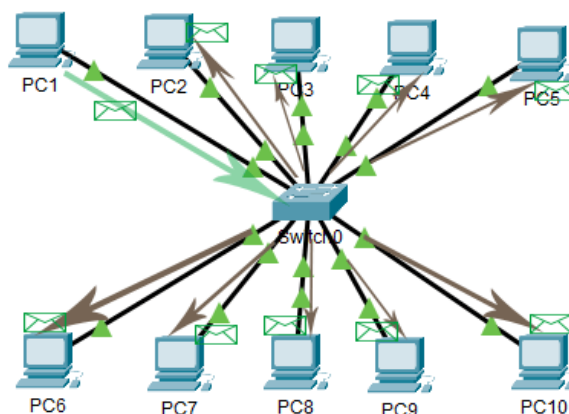
**Unicast** address represents a single device in the network. Multicast address represents a group of devices in the network. Where to represents all devices in the network known as **broadcast** address. As we know devices send or transmit information in small data packets, so packets can travel through any network paths and contains source address and destination address. Also only one device create the packet, the source address in the packet always remains a unicast address. However, the destination address in the packet can be a unicast, multicast, or a broadcast address.



**Figure: Unicast data transmission(Unicasting)**



**Figure: Multicast data transmission(Multicasting)**



**Figure: Broadcasting**

### Java InetAddress Class Factory Methods:

| Method                                                                          | Description                                                                    |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| public static InetAddress getByName(String host)<br>throws UnknownHostException | It returns the instance of InetAddress containing LocalHost IP and name.       |
| public static InetAddress getLocalHost() throws<br>UnknownHostException         | It returns the instance of InetAddress containing local host name and address. |
| public String getHostName()                                                     | It returns the host name of the IP address.                                    |
| public String.getHostAddress()                                                  | It returns the IP address in string format.                                    |

### Example 1:

```
import java.net.Inet4Address;
import java.net.Inet6Address;
import java.net.InetAddress;

class inet_address_example{
 public static void main(String[] args) {
```

```

try {
 System.out.println("For(www.facebook.com) host name and ip address :");
 InetAddress inp = InetAddress.getByName("www.facebook.com");
 System.out.println("Host Name:"+inp.getHostName());
 System.out.println("IP Address:"+inp.getHostAddress());

 System.out.println("\n");
 System.out.println("For(www.facebook.com) IPv4:");
 InetAddress ip4 = InetAddress4Address.getByName("www.facebook.com");
 System.out.println("IP4 address Detail:"+ip4);

 System.out.println("\n");
 System.out.println("For(www.facebook.com) IPv6:");
 InetAddress i6 = InetAddress6Address.getByName("www.facebook.com");
 System.out.println("Ipv6 :"+i6);

 System.out.println("\n");
 System.out.println("For(www.facebook.com) all important:");
 InetAddress ip[] = InetAddress.getAllByName("www.facebook.com");
 for(int i=0;i<ip.length;i++){
 System.out.println(ip[i]);
 }

 System.out.println("\n");
 System.out.println("For Local Host(Your PC):");
 InetAddress ia = InetAddress.getLocalHost();
 System.out.println("Local host name:"+ia.getHostName());
 System.out.println("Local Host address:"+ia.getHostAddress());
 System.out.printf("\n");

} catch (Exception e) {
 System.out.println(e);
}
}

```

### Output:

**For(www.facebook.com) host name and ip address :**

Host Name:www.facebook.com  
IP Address:157.240.198.35

**For(www.facebook.com) IPv4:**

IP4 address Detail:www.facebook.com/157.240.198.35

**For(www.facebook.com) IPv6:**

Ipv6 :www.facebook.com/157.240.198.35

**For(www.facebook.com) all important:**

www.facebook.com/157.240.198.35  
[www.facebook.com/2a03:2880:f144:82:face:b00c:0:25de](http://www.facebook.com/2a03:2880:f144:82:face:b00c:0:25de)

**For Local Host(Your PC):**

Local host name:DESKTOP-SA5E979

## Socket Programming:

An endpoint is a combination of an IP address and a port number, known as socket. **Socket** programming in Java is used for communication between the applications that are running on different JRE. Socket programming can be connection-oriented or connection-less. Also focus on both sides of the **client/server** relationship. The client requests, some action be performed, and the server performs the action and responds to the client.

Java introduce **socket-based** communications, which enable applications to view networking as – a program can read from a socket or write to a socket as simply as reading from a file or writing to a file. We will create and manipulate **stream sockets** and **datagram sockets**.

Stream sockets, a process establishes a connection to another process and data flows between the processes in continuous streams. Stream sockets are said to provide a connection-oriented service. The protocol used for transmission is the popular TCP(Transmission Control Protocol).

Datagrams sockets transmits individual packets of information. The protocol used for transmission is UDP, the User Datagram Protocol is a connectionless service. Also does not guarantee that packets arrive in any particular order. With UDP, packets can even be **lost** or **duplicated**.

**Socket** and **ServerSocket** classes are used for connection-oriented socket programming and **DatagramSocket** and **DatagramPacket** classes are used for connection-less socket programming. That means socket programming can be accomplished using either TCP socket API or UDP socket API. So, we focus on TCP socket API:

### Socket Programming with TCP socket :

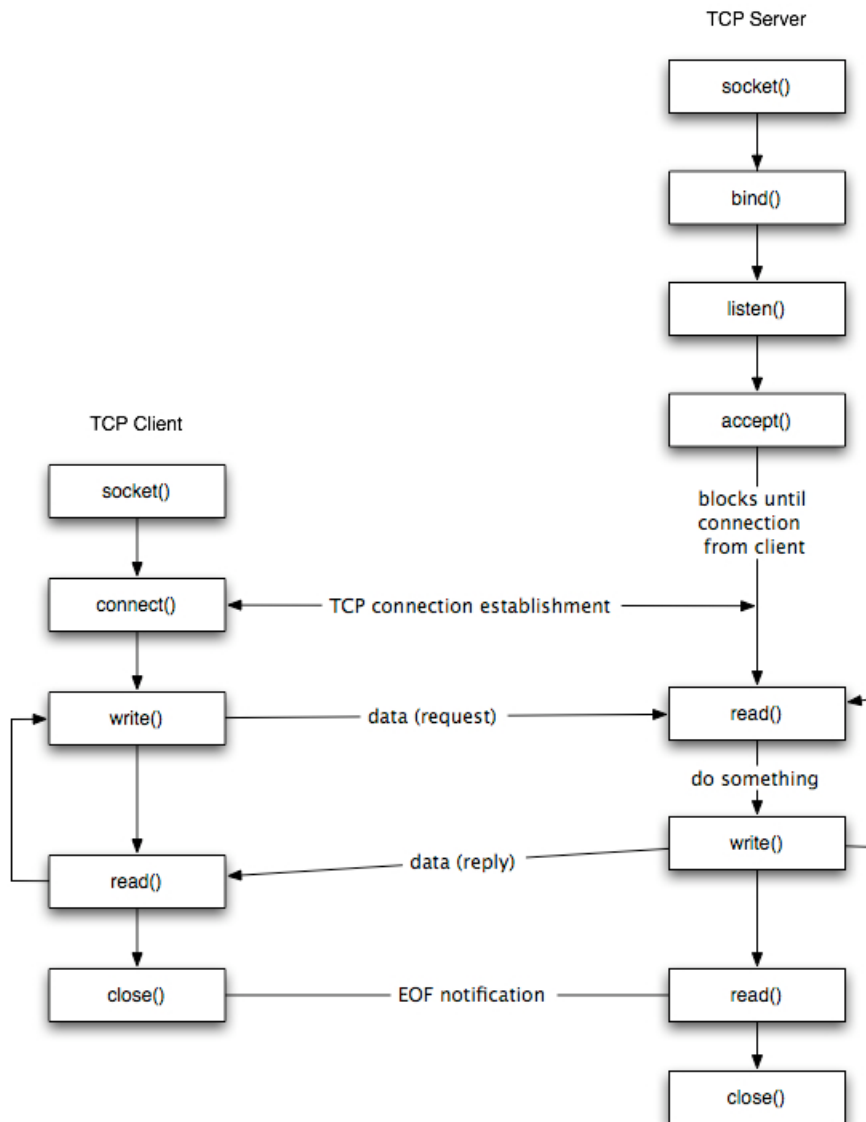
The steps for establishing a TCP socket on the client side are:

- i. Create a socket using the `socket()` function.
- ii. Connect the socket to the address of the server using the `connect()` function with port.
- iii. Send and receive data by means of the `write()` and `read()` function.
- iv. Close the connection by means of the `close()` function.

Steps for establishing a TCP socket on the server side are:

- i. Create a socket with the `socket()` function.
- ii. Bind the socket to an address using the `bind()` function.
- iii. Listen for connections with the `listen()` function.
- iv. Accept a connection with `accept()` function system call. This call typically blocks until a client connects with the server.
- v. Receive and send data by means of `read()` and `write()` functions.
- vi. Close the connection by means of the `close()` function.

The sequence of function calls for the client and a server participating in a TCP connection is as below:



**Figure: TCP client-server**

Here, we are going to make one-way (TCP socket) client and server communication. Where, client sends a message to the server, server reads the message and response as requested. We use two classes: **Socket** and **ServerSocket**. The **Socket** class is used to communicate client and server and can read and write message. The **ServerSocket** class is used at server-side, where **accept()** method of **ServerSocket** class blocks the console until the client is connected. After the successful connection of client, it returns the instance of socket at server-side.

**Socket class important methods:**

| Method                                   | Description                                         |
|------------------------------------------|-----------------------------------------------------|
| 1) public InputStream getInputStream()   | returns the InputStream attached with this socket.  |
| 2) public OutputStream getOutputStream() | returns the OutputStream attached with this socket. |
| 3) public synchronized void close()      | closes this socket                                  |

### ServerSocket class important methods:

| Method                              | Description                                                              |
|-------------------------------------|--------------------------------------------------------------------------|
| 1) public Socket accept()           | returns the socket and establish a connection between server and client. |
| 2) public synchronized void close() | closes the server socket.                                                |

### Example 1:

Create the server application, using instance of **ServerSocket** class. Also , we are using **5000**(can choose any available port number like 3001,6666,...) port number for the communication between the client and server.

For the client application, create the instance of **Socket** class. Here , need to pass the IP Address or hostname of the server and a port number. Here , we are using “localhost” / “127.0.0.1” because our server is running on same system.

Also , the accept() method waits for the client. If connects with the given port number, it returns an instance of Socket.

*/\* FirstServerExample.java \*/*

```
import java.io.DataInputStream;
import java.net.ServerSocket;
import java.net.Socket;

public class FirstServerExample{
 public static void main(String[] args) {
 try {
 ServerSocket fss = new ServerSocket(5000);
 System.out.println("Server started at 5000 port.");

 Socket socket = fss.accept();

 DataInputStream dis = new DataInputStream(socket.getInputStream());
 String str = (String)dis.readUTF();
 System.out.println("Message:"+str);
 }
 }
}
```

```

 fss.close();
 } catch (Exception e) {
 System.out.println(e);
 }
}
}
/* FirstClientExample.java */

import java.io.DataOutputStream;
import java.net.Socket;

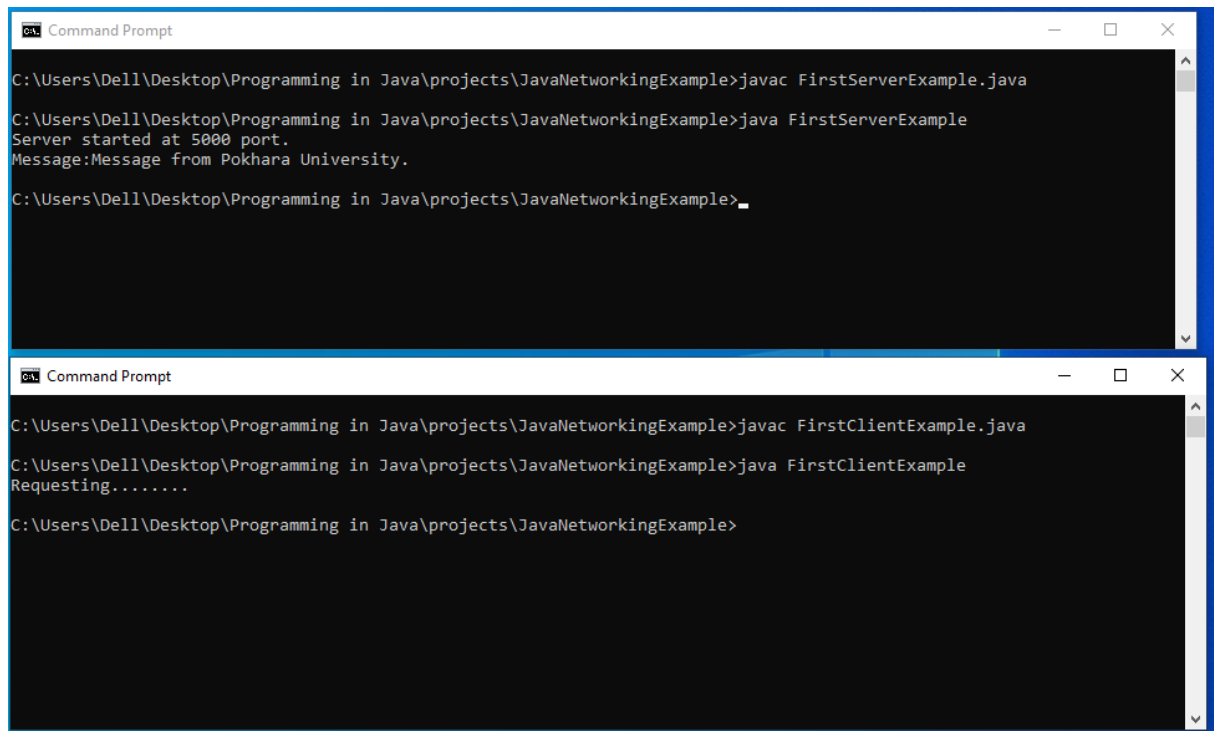
public class FirstClientExample {
 public static void main(String[] args) {
 try {
 Socket socket = new Socket("127.0.0.1",5000); //establishing connection
 DataOutputStream dout = new DataOutputStream(socket.getOutputStream());
 dout.writeUTF("Message from Pokhara University."); //sending message to server
 dout.flush();
 dout.close();
 socket.close();
 } catch (Exception e) {
 System.out.println(e);
 }
 }
}

```

**To run above program(if uses cmd or terminal):**

- i. Open two Command prompt window
- ii. Execute client(*second*) application and server(*first*) application at each command prompt.
- iii. "Message From Pokhara University" will be displayed on the server console .





```
Command Prompt
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>javac FirstServerExample.java
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>java FirstServerExample
Server started at 5000 port.
Message:Message from Pokhara University.
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>

Command Prompt
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>javac FirstClientExample.java
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>java FirstClientExample
Requesting.....
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>
```

**Example 2:** Create a TCP client/server program in which client sends an integer to the server and the server responds to client by sending square of the number sent by client.

**/\*SecondServerExample.java\*/**

```
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.UnknownHostException;

public class SecondServerExample {
 public static void main(String[] args) throws UnknownHostException {
 try{
 ServerSocket secondServer = new ServerSocket(6000);
 System.out.println("Server running at port: 6000");

 Socket secondSocket = secondServer.accept(); // connection accepted
 System.out.println("Connection request accepted....");

 DataInputStream din = new DataInputStream(secondSocket.getInputStream());

 DataOutputStream dout = new DataOutputStream(secondSocket.getOutputStream());

 int num = din.readInt();
 int squareNum = num*num;
 dout.writeInt(squareNum);
 System.out.println("Request for square of number "+num+ " responded.");
```

```

 din.close();
 dout.close();
 secondSocket.close();
 secondServer.close();
 }catch(IOException e){
 System.out.println(e);
 }
}
}
}
/* SecondClientExample.java */
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.net.Socket;
import java.util.Scanner;

public class SecondClientExample {
 public static void main(String[] args) {
 Scanner scn = new Scanner(System.in);
 try {
 Socket second_client_socket = new Socket("localhost",6000);
 System.out.println("Server Connected.");
 DataOutputStream dout = new DataOutputStream(second_client_socket.getOutputStream());

 DataInputStream din = new DataInputStream(second_client_socket.getInputStream());
 System.out.println("Enter number:");
 int number = scn.nextInt();

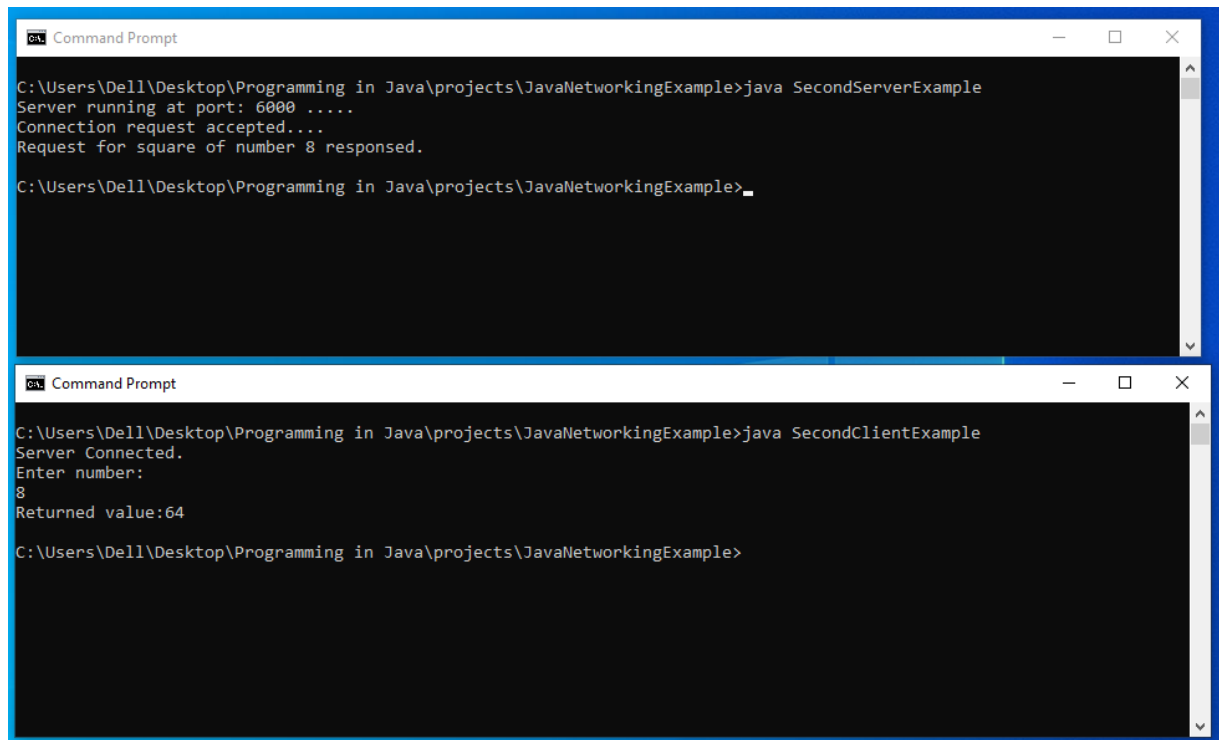
 dout.writeInt(number);

 int num_square = din.readInt();
 System.out.println("Returned value:"+num_square);

 dout.close();
 din.close();
 second_client_socket.close();
 } catch (Exception e) {
 System.out.println(e);
 }
 }
}

```

**Output:**



```
Command Prompt
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>java SecondServerExample
Server running at port: 6000
Connection request accepted...
Request for square of number 8 responded.
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>

Command Prompt
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>java SecondClientExample
Server Connected.
Enter number:
8
Returned value:64
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>
```

**Example 3:** Create a TCP client/server program in which client and server communicate by sending a message to each other.

**/\* ThirdServerExample.java \*/**

```
import java.io.BufferedReader;
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.InputStreamReader;
import java.net.ServerSocket;
import java.net.Socket;

public class ThirdServerExample {
 public static void main(String[] args) {
 int PORT = 5555;
 try{
 ServerSocket thirdServer = new ServerSocket(PORT);
 System.out.println("Server running at port "+PORT);

 Socket thirdSocket = thirdServer.accept();

 DataInputStream dins = new DataInputStream(thirdSocket.getInputStream());
 DataOutputStream dos = new DataOutputStream(thirdSocket.getOutputStream());
 BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

 String receive_client="", send_response="";
 while(!receive_client.equals("stop")){
```

```

 receive_client = dins.readUTF();
 System.out.println("From client:"+receive_client);

```

```

 send_response = br.readLine();
 dos.writeUTF(send_response);
 dos.flush();
 }
 br.close();
 dos.close();
 dins.close();
 thirdServer.close();
} catch (Exception e) {
 System.out.println(e);
}
}
}

```

**/\* ThirdClientExample.java \*/**

```

import java.io.BufferedReader;
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.InputStreamReader;
import java.net.Socket;

```

```

public class ThirdClientExample {
 public static void main(String[] args) {
 String host = "127.0.0.1";
 int PORT = 5555;
 try {
 System.out.println("Requesting");
 Socket thirdSocket = new Socket(host, PORT);
 System.out.println("Server Connected.....");

 DataOutputStream douts = new DataOutputStream(thirdSocket.getOutputStream());
 DataInputStream dins = new DataInputStream(thirdSocket.getInputStream());
 BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

 String request_server = "";
 receive_server = "";
 while (!request_server.equals("stop")) {
 request_server = br.readLine();
 douts.writeUTF(request_server);
 douts.flush();

 receive_server = dins.readUTF();
 System.out.println("From server:" + receive_server);
 }

 br.close();

```

```

 dins.close();
 douts.close();
 thirdSocket.close();
 } catch (Exception e) {
 System.out.println(e);
 }
}
}
}

```

### Output:

```

C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>java ThirdServerExample
Server running at port 5555
From client:Hello server...
Hello client....
From client:Are okay?
Yes I am Okay?
From client:Let's stop .
Okay
From client:
From client:stop

C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>

C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>java ThirdClientExample
Requesting
Server Connected.....
Hello server...
From server:Hello client....
Are okay?
From server:Yes I am Okay?
Let's stop .
From server:Okay

From server:
stop
From server:

C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>_

```

**Note:** *InputStreamReader* is a bridge from byte streams to character streams. It reads bytes and decodes them into characters using a specified charset.

**Example 4:** Write a TCP socket program to accept a line of string input from the client and send it to server. The server checks if the string is palindrome or not and send result back to client. Client machine displays the string received from server.

```

/* FourthServerExample.java */
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.net.ServerSocket;
import java.net.Socket;
public class FourthServerExample {
 public static void main(String[] args) {
 final int PORT = 3001;
 try {

```

```

ServerSocket fourthServer = new ServerSocket(PORT);
System.out.println("Server running at port "+PORT);
Socket fourthSocket = fourthServer.accept(); //connection accepted
System.out.println("Connection success....");

DataInputStream din = new DataInputStream(fourthSocket.getInputStream());
DataOutputStream dout = new DataOutputStream(fourthSocket.getOutputStream());

String clientInput = din.readUTF(); //read data from client

String msg = "", palindrome = "";
int len = clientInput.length();
for(int i=len-1; i>=0;--i){
 palindrome = palindrome + clientInput.charAt(i);
}
if(clientInput.toLowerCase().equals(palindrome.toLowerCase())){
 msg = "Given String is Palindrome.";
}else{
 msg = "Given String is not Palindrome.";
}
dout.writeUTF(msg); //writing to client

din.close();
dout.close();
fourthSocket.close();
fourthServer.close();
} catch (Exception e) {
 System.out.println(e);
}
}
}

/* FourthClientExample.java */
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.net.Socket;
import java.util.Scanner;
public class FourthClientExample {
 public static void main(String[] args) {
 final int PORT = 3001;
 try(Scanner scn = new Scanner(System.in)){
 Socket fourthSocket = new Socket("127.0.0.1",PORT); //request for connection
 System.out.println("Requesting connection");
 DataOutputStream dout = new DataOutputStream(fourthSocket.getOutputStream());
 DataInputStream din = new DataInputStream(fourthSocket.getInputStream());

 System.out.println("Enter String:");
 String cInput = scn.nextLine(); //user input
 dout.writeUTF(cInput); //writing to server

```

```

String result = din.readUTF(); //reading data from server
System.out.println(result);

din.close();
dout.close();
fourthSocket.close();
} catch (Exception e) {
 System.out.println(e);
}
}
}

```

**Output:**

```

C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>javac FourthServerExample.java
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>java FourthServerExample
Server running at port 3001
Connection success...
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>

C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>javac FourthClientExample.java
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>java FourthClientExample
Requesting connection
Enter String:
Mom
Given String is Palindrome.
C:\Users\Dell\Desktop\Programming in Java\projects\JavaNetworkingExample>

```

## URL class :

If you wish to access any content over internet you will see something basically like ***https://xyz.com/index.html*** OR something else including ***https://.....*** , called URLs. URL is an acronym for **Uniform Resource Locator** and is a reference (an address) to a resource on the internet. Used to access HTML pages from the Web.

A URL contains many information:

1. Protocol: https, http, ftp , tcp, .....etc.
2. Server name or IP Address or resource name: For example facebook.com, google.cm, or 127.0.0.1, .....
3. Port number: If we write http//localhost:8080, 8080 is the port number. If port number is not mentioned in the URL, it returns -1.
4. File name or directory name: [http://localhost:8080/php\\_example/index.php](http://localhost:8080/php_example/index.php), where php\_example is directory name and index.php file name.

### Constructors of Java URL class:

1. URL(String spec): Create an instance of a URL from the specified String representation.
2. URL(String protocol, String host, int port, String file): Creates an instance of a URL from the given protocol, host, port number, and file.
3. URL(String protocol, String host, int port, String file, URLStreamHandler handler): Creates an instance of a URL from the given protocol, host, port number, file, and handler.
4. URL(String protocol, String host, String file): Creates an instance of a URL from the given protocol name, host name, and file name.
5. URL(URL context, String spec): Creates an instance of a URL by parsing the given specified context.
6. URL(URL context, String spec, URLStreamHandler handler): Creates an instance of a URL by parsing the given spec with the specified handler within a given context.

### Methods of Java URL class:

| Method                                | Description                                                             |
|---------------------------------------|-------------------------------------------------------------------------|
| public String getProtocol()           | it returns the protocol of the URL.                                     |
| public String getHost()               | it returns the host name of the URL.                                    |
| public String getPort()               | it returns the Port Number of the URL.                                  |
| public String getFile()               | it returns the file name of the URL.                                    |
| public String getAuthority()          | it returns the authority of the URL.                                    |
| public String toString()              | it returns the string representation of the URL.                        |
| public String getQuery()              | it returns the query string of the URL.                                 |
| public String getDefaultPort()        | it returns the default port of the URL.                                 |
| public URLConnection openConnection() | it returns the instance of URLConnection i.e. associated with this URL. |
| public boolean equals(Object obj)     | it compares the URL with the given object.                              |
| public Object getContent()            | it returns the content of the URL.                                      |
| public String getRef()                | it returns the anchor or reference of the URL.                          |
| public URI toURI()                    | it returns a URI of the URL.                                            |

### Example:

```
import java.net.URL;
```



```

public class UrlClassExample {
 public static void main(String[] args) {
 try {
 URL url = new URL("https://en.wikipedia.org/wiki/Java_(programming_language)");
 System.out.println("Protocol:"+url.getProtocol());
 System.out.println("Host Name:"+url.getHost());
 System.out.println("Port Number:"+url.getPort());
 System.out.println("Default Port Number:"+url.getDefaultPort());
 System.out.println("File Name:"+url.getFile());
 System.out.println("Path:"+url.getPath());
 System.out.println("Query String:"+url.getQuery());
 } catch (Exception e) {
 e.printStackTrace();
 }
 }
}

```

### Output:

```

Protocol:https
Host Name:en.wikipedia.org
Port Number:-1
Default Port Number:443
File Name:/wiki/Java_(programming_language)
Path:/wiki/Java_(programming_language)
Query String:null

```

**Example :** Write a Java Program for reading contents in a given URL.

```

import java.io.BufferedReader;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.net.URL;
import java.net.URLConnection;

public class UrlClassExample {
 public static void main(String[] args) {
 try {
 URL url = new URL("https://en.wikipedia.org/wiki/Java_(programming_language)");
 BufferedReader ins = new BufferedReader(new InputStreamReader(url.openStream()));
 String i;
 while((i=ins.readLine())!=null){
 System.out.print(i);
 }
 } catch (Exception e) {
 e.printStackTrace();
 }
 }
}

```

## URLConnection class:

Java URLConnection class represents a communication link between the URL and the application. It can be used to read and write data to the specified resource referred by the URL.

URL has two components:

1. The protocol required to access the resource.
2. The location of the resource.

The two subclasses HttpURLConnection and JarURLConnection makes the connection between the client Java program and URL resource on the internet. Once a connection is established and the Java program has an URLConnection object, we can use it to read or write or get further information like content, length, etc.

## Constructors:

Protected URLConnection(URL url) – It constructs a URL connection to the specified URL.

URLConnection Class Methods:

There are many methods , here we listed some of them.

| Method                            | Description                                                                                                            |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------|
| void connect()                    | It opens a communications link to the resource referenced by this URL, if not been established.                        |
| boolean getAllowUserInteraction() | It returns the value of the allowUserInteraction field for the object.                                                 |
| int getConnectionTimeout()        | It returns setting for connect timeout.                                                                                |
| Object getContent()               | It retrives the contents of the URL connection.                                                                        |
| int getContentLength()            | It returns the value of the content-length header field.                                                               |
| long getDate()                    | It returns the value of the date header field.                                                                         |
| long getExpiration()              | It returns the value of the expires header files.                                                                      |
| String getHeaderField(int n)      | It returns the value of nth header field.                                                                              |
| InputStream getInputStream()      | It returns an input stream that reads from the open condition.                                                         |
| long getLastModified()            | It returns the value of the last-modified header field.                                                                |
| OutputStream getOutputStream()    | It returns an output stream that writes to the connection.                                                             |
| Permission getPermission()        | It returns a permission object representing the permission necessary to make the connection represented by the object. |
| int getReadTimeout()              | It returns setting for read timeout.                                                                                   |
| Boolean getUseCaches()            | It returns the value of the URLConnection's useCaches field.                                                           |

**Example:** Displaying source code of a webpage

```
import java.io.InputStream;
import java.net.URL;
import java.net.URLConnection;

public class UrlClassExample {
 public static void main(String[] args) {
 try {
 URL url = new URL("https://en.wikipedia.org/wiki/Java_(programming_language)");
 URLConnection urlconnect = url.openConnection();
 InputStream ins = urlconnect.getInputStream();
```

```

 int i;
 while((i=ins.read())!=-1){
 System.out.print((char)i);
 }
 } catch (Exception e) {
 e.printStackTrace();
 }
}
}

```

**Example :** How do you get a list of IP addresses that are assigned to a network interface?

```

import java.net.InterfaceAddress;
import java.net.NetworkInterface;
import java.net.SocketException;
import java.net.UnknownHostException;
import java.util.ArrayList;
import java.util.Collections;

public class NetworkInterfaceExample {
 public static void main(String[] args) throws SocketException, UnknownHostException{
 ArrayList<NetworkInterface> interfaces = Collections.list(NetworkInterface.getNetworkInterfaces());
 for(NetworkInterface nif : interfaces){
 for(InterfaceAddress iadd : nif.getInterfaceAddresses()){
 System.out.println("\t"+iadd.getAddress());
 }
 }
 }
}

```

**Example :** Reading html content using openConnection() method and connect() method.

```

import java.net.URL;
import java.net.URLConnection;
import java.io.IOException;
import java.io.InputStream;
public class UrlTest{
 public static void main(String[] args){
 try{
 String new_url = " https://www.mcvtc.org.np/aboutUs";
 URL url=new URL(new_url);
 URLConnection urlcon=url.openConnection(); // returns the instance of URLConnection
 InputStream is=urlcon.getInputStream(); //connection to remote object
 int i;
 while((i=is.read())!=-1){
 System.out.print((char)i);
 }
 }catch(IOException e){
 System.out.println(e);
 }
 }
}

```

```
}
}
}
```

## Socket Programming using UDP

As we know UDP (User Datagram protocol) is Connection-less protocol . Connectionless transmission with datagrams is more like the mail is carried through postal service . If your message is small enough to fit in envelope, no need to worry about sequence. If your message is large and not fit in one envelope, you break it into separate pieces with sequentially numbered envelopes. The letters could arrive in order or out of order.

Here, **DatagramSocket** and **DatagramPacket** classes are used for connection-less socket programming.

**Datagram** – when collections of information sent from one device to another device through active network , called Datagram. Also, there is no guarantee that sent datagram will reach to the targeted device safely and completely. It may get damaged or lost in between. Similarly, the receiving device never know if received datagram is damaged or not.

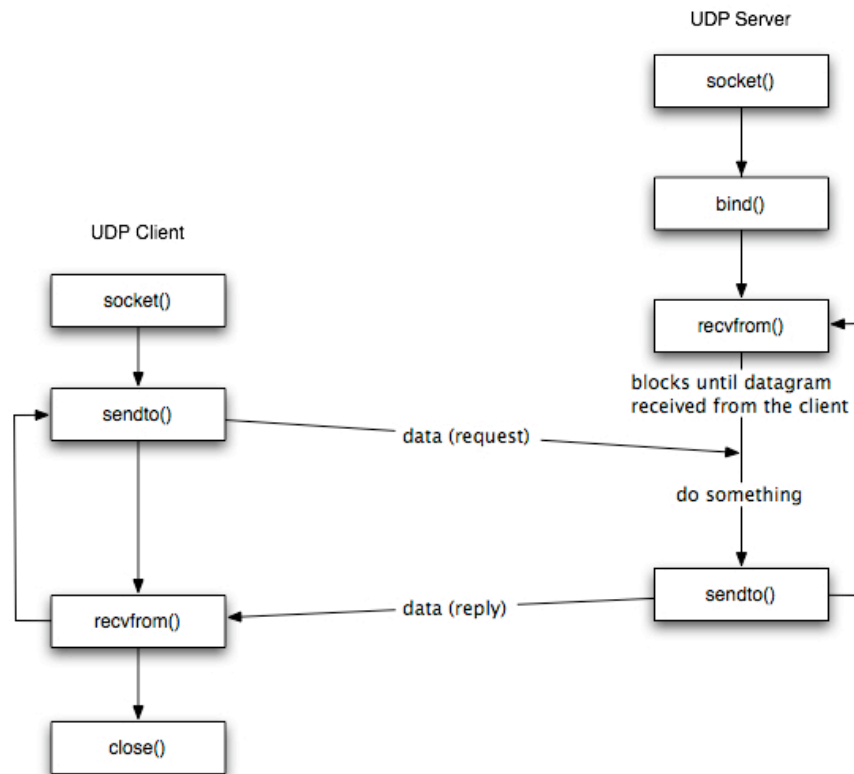
### Constructors of DatagramSocket class:

- **DatagramSocket() throws SocketException:** creates a datagram socket and binds it with the available Port Number on the localhost machine.
- **DatagramSocket(int port) throws SocketException:** creates a datagram socket and binds it with the given Port Number.
- **DatagramSocket(int port, InetAddress address) throws SocketException:** creates a datagram socket and binds it with the specified Port Number and host address.

**DatagramPacket** – is a message that can be sent or received. It is a data container.

### Constructors of DatagramPacket class:

- **DataPacket(byte[] data, int length):** creates a datagram packet and used to receive the packets.
- **DataPacket(byte[] data, int length, InetAddress address, int port):** creates a datagram packet and used to receive the packets.



**Figure: UDP client-server Architecture**

**Example 1:** One way communication using UDP socket programming (Client Request Received by Server)

**/\*UdpServer.java\*/**

```

import java.net.SocketException;
import java.net.DatagramSocket;
import java.net.DatagramPacket;

import java.io.IOException;
public class UdpServer {
 public static void main(String[] args) throws SocketException{
 try{
 DatagramSocket ds = new DatagramSocket(5000);
 System.out.println("Connection Established....");
 byte[] dataFromClient = new byte[1024];
 DatagramPacket dataPacket = new DatagramPacket(dataFromClient,1024);
 ds.receive(dataPacket);
 String data = new String(dataPacket.getData(),0,dataPacket.getLength());
 System.out.println(data);
 }catch(IOException ie){
 System.out.println(ie.getMessage());
 }
 }
}

```

**/\*UdpClient.java\*/**

```
import java.net.SocketException;
import java.net.DatagramSocket;
import java.net.DatagramPacket;
import java.net.InetAddress;
import java.net.UnknownHostException;
import java.io.IOException;
public class UdpClient{
 public static void main(String[] args) throws UnknownHostException{
 try{
 int port = 5000;
 DatagramSocket ds = new DatagramSocket();
 System.out.println("Waiting for connection....");
 String msg = "Hello Developer";
 InetAddress address = InetAddress.getByName("127.0.0.1");
 DatagramPacket dataP = new DatagramPacket(msg.getBytes(),msg.length(),address,port);
 ds.send(dataP);
 }catch(SocketException se){
 System.out.println(se.getMessage());
 }catch(IOException ie){
 System.out.println(ie.getMessage());
 }
 }
}
```

**Example 2:** Two way communication using UDP socket programming(Client Request Received and Responded by Server)

**/\*UdpClient.java\*/**

```
import java.net.SocketException;
import java.net.DatagramSocket;
import java.net.DatagramPacket;
import java.net.InetAddress;
import java.net.UnknownHostException;
import java.io.IOException;
public class UdpClient{
 public static void main(String[] args) throws UnknownHostException{
 try{
 int port = 5000;
 DatagramSocket ds = new DatagramSocket();
 System.out.println("Waiting for connection....");
 String msg = "Hello Developer";
 InetAddress address = InetAddress.getByName("127.0.0.1");
 DatagramPacket dataP = new DatagramPacket(msg.getBytes(),msg.length(),address,port);
 ds.send(dataP);

 byte[] dataFromServer = new byte[1024];
 DatagramPacket dataFrom = new DatagramPacket(dataFromServer,1024);
 ds.receive(dataFrom);
 }
 }
}
```

```

String data = new String(dataFrom.getData(),0,dataFrom.getLength());
System.out.println(data);

}catch(SocketException se){
 System.out.println(se.getMessage());
}catch(IOException ie){
 System.out.println(ie.getMessage());
}
}
}

/* UdpServer.java */

import java.net.SocketException;
import java.net.DatagramSocket;
import java.net.DatagramPacket;
import java.io.IOException;
public class UdpServer {
 public static void main(String[] args) throws SocketException{
 try{
 DatagramSocket ds = new DatagramSocket(5000);
 System.out.println("Connection Established....");
 byte[] dataFromClient = new byte[1024]; //this is buffer to store received message / data
 DatagramPacket dataPacket = new DatagramPacket(dataFromClient,1024);
 ds.receive(dataPacket);
 String data = new String(dataPacket.getData(),0,dataPacket.getLength());
 System.out.println(data);
 //server response to client by sending response "Hello Client"
 String response = "Hello Client";
 byte[] sendData = response.getBytes();
 DatagramPacket res = new
 DatagramPacket(sendData,sendData.length,dataPacket.getAddress(),dataPacket.getPort());
 ds.send(res);
 }catch(IOException ie){
 System.out.println(ie.getMessage());
 }
 }
}

```

### **What must do in Socket Programming(Low-Level)**

- Design communication protocol(need to know protocols and process to connect)
- Convert objects to bytes
- Send / Receive OR Write/Read messages
- Parse responses
- Handle failures manually

Results, complex, error-prone and not object-oriented . However, socket programming is powerful.

## Java Mail API (Application Programming Interface)

(API – set of rules and protocols that allows different applications / software to communicate with each other. Defines the methods and data formats that applications can use to request and exchange information .)

Java **Mail API** is used to compose, write and read electronic message(emails) . It provides protocol-independent and platform-independent environment / framework for sending and receiving mails. The **javax.mail** and **javax.activation** packages contains the core classes of java mail API and are old packages or can use **jakarta.mail** api. Java Mail API have more use like registering the user(sending toast message/notification), forgot password(sending password to the users email id), sending notifications for important updates etc.

### Some Protocols Used in Java Mail API:

- SMTP (Simple Mail Transfer Protocol) – It provides a mechanism to deliver (sending and receiving) the email. You can use different SMTP server like Apache James Server, postcast server, email server etc or host provided server. If you use the SMTP server provided by the host provider, authentication is required for sending and receiving emails. PORT 25 by default for communication and for secure communication over TLS (Transport Layer Security) 587.
- POP (Post Office Protocol, also known as POP3) – Used to receive the email. Provides support for single mail box. You can't preview your email; this protocol will download your received email on local device and remove from email server. Port 110 – default port.
- IMAP (Internet Message Access Protocol) – This is an advance protocol for receiving messages. Support for multiple mail box for each user. Allow user to preview, download, manage emails. Port 143 – default port
- MIME (Multiple Internet Mail Extension)
- NNTP (Network News Transfer Protocol)

**Note:** First you need to download either javax.mail

([https://javaee.github.io/javamail/#Download\\_JavaMail\\_Release](https://javaee.github.io/javamail/#Download_JavaMail_Release)) or jakarta.mail. Here we will use **javax.mail** package and activation package

package(<https://repo1.maven.org/maven2/javax/activation/activation/1.1.1/>). Then add that jar file to existing project (Select Libraries -> click Add JAR/Folder and then select downloaded jar file).

Example: Send email From [santosh\\_bist63989@gmail.com](mailto:santosh_bist63989@gmail.com) to [santoshbist5424@gmail.com](mailto:santoshbist5424@gmail.com) using javax.mail and activation packages.

```
package emailexamples; // Package name where your class is stored
// Import core Java Mail classes
import javax.mail.*;
import javax.mail.internet.*;
// Import Properties class to configure SMTP settings
import java.util.Properties;
public class SendEmailToCustomer {
```



```

public static void main(String[] args) {
 // Sender valid email address (must match the account used to generate App Password)
 final String from = "santosh_bist63989@gmail.com";
 // App Password (16-character valid Gmail app password, no spaces)
 final String password = "uhaxsxdceahchylr";
 // Receiver email address
 String to = "santoshbist5424@gmail.com";
 // Create Properties object to hold SMTP configuration
 Properties props = new Properties();
 // SMTP server host (Gmail SMTP server)
 props.put("mail.smtp.host", "smtp.gmail.com");
 // SMTP port number (587 = TLS)
 props.put("mail.smtp.port", "587");
 // Enable authentication (required for Gmail)
 props.put("mail.smtp.auth", "true");
 // Enable STARTTLS encryption (secure connection)
 props.put("mail.smtp.starttls.enable", "true");
 // Create mail session with authentication
 Session session = Session.getInstance(props,
 new Authenticator() {
 // Override method to provide email & password
 protected PasswordAuthentication getPasswordAuthentication() {
 // Return login credentials
 return new PasswordAuthentication(from, password);
 }
 });
 try {
 // Create a new email message object
 Message message = new MimeMessage(session);
 // Set sender email address
 message.setFrom(new InternetAddress(from));
 // Set recipient email address (TO field)
 message.setRecipients(
 Message.RecipientType.TO,
 InternetAddress.parse(to)
);
 // Set subject of email
 message.setSubject("Greeting Mail");
 // Set body (text content) of email
 message.setText("Hello,How you doing?");
 // Send the email using SMTP transport
 Transport.send(message);
 // Print success message in console
 System.out.println("Message Sent Successfully!");
 } catch (MessagingException e) {
 // Print error details if sending fails
 e.printStackTrace();
 }
}
}

```

# RMI(Remote Methods Invocation )

## Introduction

RMI stands for Remote Method Invocation as an API defined under *java.rmi* package in Java Programming Language. RMI API are used for distributed application in Java. Using RMI allows an object running in one java virtual machine (JVM) to invoke methods on an object running in another java virtual machine (JVM) , as if it were a local call. It provides remote communication between java programs. RMI also uses two objects **stub** and **skeleton** for remote communication between the applications with the **remote object**. Objects with methods that can be invoked across different JVM are called **remote object** , also called the **server object**.

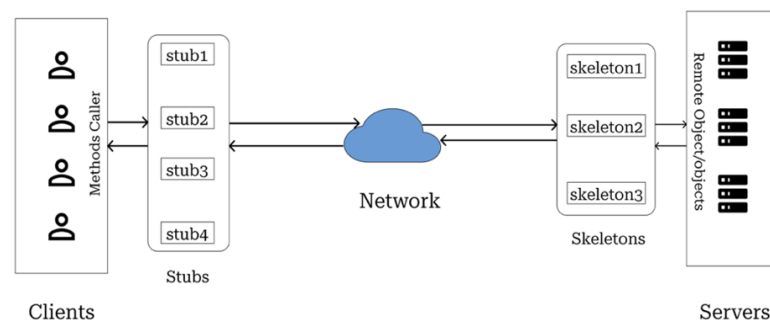


Figure : RMI Architecture

Also , can represent RMI Architecture as bellow:

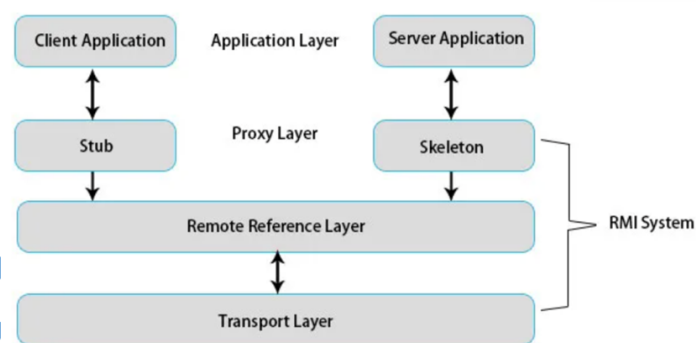


Figure: RMI Architecture

### Clients Role:

- Requests services from a remote object.
- Invoke methods using a stub(proxy)
- Client think's as calling a local object.

**Stub:** Stub is an object that acts as a gateway for the client side. All the outgoing requests are routed through it; stub forwards requests to a remote object via RMI infrastructure through *skeleton*. It resides at the client side and represents the remote object and acts as a proxy for the

remote object. It has the same methods as a remote object. When the caller invokes method on the stub object, it does the following tasks:

- i. Initiates a connection with remote Virtual Machine (JVM)
- ii. Writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM)
- iii. Waits for the result
- iv. Reads (Un-marshals) the return value or exception
- v. Finally, returns the value to the caller

**Skeleton(logical-component):** The skeleton is an object, acts as a gateway for the server-side object. All incoming requests from the stub are routed through it. The responsibility of the skeleton object is to send parameters to method implementation and send the return values back to the client. When the skeleton receives the incoming request, it does the following tasks:

- i. It reads the parameter for the remote method (Un-marshals)
- ii. It invokes the method on the actual remote object
- iii. It writes and transmits(marshals) the result to the caller

**Remote Object (Server):** Actual implementation of the remote service. Remote object is an object whose methods can be invoked from another JVM through a network using RMI.

**Proxy:** local object that represents the remote object. Hides network communication from the client. Acts as an intermediary between client code and remote service.

**Marshalling:** Process of converting method parameters and data objects into a format suitable for network transmission. Objects and complex data structures must be converted into bytes .

**Example:**

add (5,4)

converted into like: {method: "add", param1: 5, param2: 4}

**Unmarshalling:** Reverse process of Marshalling – converting received byte data back into usable.

**Application layer:** Also known as the “Business logic layer” or “Service layer” is a component of software architecture that exist between the presentation layer (user interface) and the data layer. Application layer encapsulates the business logic and rules of an application but organize/arrange flow of data between the presentation layer and the data layer with implementation of application specific functionality. Here, java program on the client-side communicates with the java program on the server-side.

**Proxy layer:** This layer contains the client stub and server skeleton objects. Actually , it acts as a gateway to communicate between clients and servers , providing various functionalities such as caching, filtering, security, and load balancing. Proxy layer intercepts clients requests and forward to the appropriate destination servers , and vice versa.

**Remote Reference Layer:** Manage session during the method call . i.e. It manages the references made by the client to the remote server object. Also responsible for handling duplicated objects.

**Transport Layer:** Responsible for setting up communication between the two machines. TCP / IP protocol is used for connection in this layer . The actual transportation of data is performed through this layer.

## Creating and Executing RMI Applications

Follow following 6 steps to create and run the RMI application.

- i. Create the remote interface
- ii. Implement remote interface (i.e. through class)
- iii. Compile the implementation class and create the stub and skeleton objects
- iv. Start the registry service
- v. Create and start the remote / server application
- vi. Create and start the client application

**Example:** RMI application with single service.

**/\*RemoteMethods.java\*/**

```
package RmiExample;
import java.rmi.Remote;
import java.rmi.RemoteException;
public interface RemoteMethods extends Remote{
 int multiplication(int a,int b) throws RemoteException;
}
```

**/\*CalculationImplementation.java\*/**

```
//remote object
package RmiExample;
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
public class CalculationImplementation extends UnicastRemoteObject implements RemoteMethods{
 CalculationImplementation() throws RemoteException{
 super();
 }
 @Override
 public int multiplication(int a,int b){
 return a*b;
 }
}
```

**/\*RmiServerCalculation.java\*/**

```
package RmiExample;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
public class RmiServerCalculation {
 public static void main(String[] args) {
 try{
 Registry rg = LocateRegistry.createRegistry(5005);
 CalculationImplementation cil = new CalculationImplementation();
 rg.rebind("New Server", cil);
 }
 }
}
```

```

 System.out.println("Server running....");
 }catch(Exception e){
 e.printStackTrace();
 }
}
}
}
/*RmiClient.java*/
package RmiExample;
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
public class RmiClient {
 public static void main(String[] args) {
 try{
 Registry rg = LocateRegistry.getRegistry("localhost",5005);
 RemoteMethods cil = (RemoteMethods)rg.lookup("New Server");
 int result = cil.multiplication(4, 3);
 System.out.println("Result::"+result);
 }catch(Exception e){
 e.printStackTrace();
 }
 }
}

```

**Example:** server provides multiple services.

```

/*ReferenceInterface.java*/
package RmiApplication;
import java.rmi.Remote;
import java.rmi.RemoteException;
//interface interact with both client and server
public interface ReferenceInterface extends Remote{
 //All following methods are abstract and throws RemoteException
 //These methods exists between stub and skeleton objects
 public int add(int x,int y) throws RemoteException;
 public int toSquare(int x)throws RemoteException;
 public int product(int x,int y) throws RemoteException;
 public String printOddEven(int x) throws RemoteException;
 public String printPrime(int x) throws RemoteException;
 public String printFactorial(int x) throws RemoteException;
}

```

```

/*Server.java*/
package RmiApplication;
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import I
import java.rmi.registry.LocateRegistry;
//inherit UnicastRemoteObject class and ReferenceInterface interface
//UnicastRemoteObject class define communication between single server and single client
//Also help to export remote objects

```

```

public class Server extends UnicastRemoteObject implements ReferenceInterface{
 //default constructor that handle RemoteException
 public Server() throws RemoteException{
 }
 public static void main(String[] args) throws RemoteException{
 try{
 //register server at assigned port i.e. 3000
 Registry rg = LocateRegistry.createRegistry(3000);
 Server obj = new Server();
 //bind remote object to the registry,
 //here obj is an remote object and binded as "server_response"
 //client can now look up this remote object("server_response") using the registry's lookup() method.
 rg.rebind("server_response", obj);
 System.out.println("Server is ready....");
 }catch(RemoteException ae){
 System.out.println(ae.getCause());
 }
 }
 //Here, we override all abstract methods of interface ReferenceInterface
 @Override
 public int add(int a,int b) throws RemoteException {
 return (a+b);
 }
 @Override
 public int toSquare(int a) throws RemoteException {
 return(a*a);
 }
 @Override
 public int product(int a, int b) throws RemoteException{
 return(a*b);
 }
 @Override
 public String printOddEven(int a) throws RemoteException{
 String result = "";
 if(a%2==0){
 result += "Given number is Even.";
 }else{
 result += "Given number is Odd.";
 }
 return(result);
 }
 @Override
 public String printPrime(int a) throws RemoteException{
 int i, flag = 0;
 String result = "";
 if (a == 0 || a == 1)
 flag = 1;
 for (i = 2; i <= a / 2; ++i) {
 if (a % i == 0) {
 flag = 1;
 break;
 }
 }
 }
}

```

```

 }
 if(flag==0){
 result += "Given number is Prime";
 }else{
 result += "Given number is not a Prime Number";
 }
 return(result);
}
@Override
public String printFactorial(int a) throws RemoteException{
 String result = "";
 int n=a;
 for(int i=n-1;i>0;i--){
 n=n*i;
 }
 result += "Factorial of "+a+" is "+n;
 return(result);
}
}

```

**/\*Client.java\*/**

```

package RmiApplication;
import java.util.Scanner;
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.RemoteException;
public class Client {
 public Client() throws RemoteException{
 }
 public static void main(String[] args) {
 try(Scanner scn = new Scanner(System.in)){
 //used to access server running at socket (ip:127.0.0.1 and port: 3000)
 Registry registry = LocateRegistry.getRegistry("127.0.0.1",3000);
 ReferenceInterface stub = (ReferenceInterface)registry.lookup("server_response");
 int first_number,second_number,choice,num;
 while(true){
 System.out.println("*****");
 System.out.println("\tSelect Your Request");
 System.out.println("\t1.Add two number.");
 System.out.println("\t2.Square given number.");
 System.out.println("\t3.Product of given numbers.");
 System.out.println("\t4.Identify Given number is odd or even.");
 System.out.println("\t5.Identify Given number is prime or not.");
 System.out.println("\t6.Calculate factorial of given number.");
 System.out.println("\t7.Exit");
 System.out.println("*****");
 System.out.println("Enter your choice:");
 choice = scn.nextInt();
 switch(choice){
 case 1:
 System.out.println("Enter First Number:");
 first_number= scn.nextInt();

```

```

 System.out.println("Enter Second Number:");
 second_number = scn.nextInt();
 System.out.println("Addition of Two number is:"+stub.add(first_number,second_number));
 break;
 case 2:
 System.out.println("Enter Number to square:");
 num= scn.nextInt();
 System.out.println("Square is:"+stub.toSquare(num));
 break;
 case 3:
 System.out.println("Enter First Number:");
 first_number= scn.nextInt();
 System.out.println("Enter Second Number:");
 second_number = scn.nextInt();
 System.out.println("Product of two number is:"+stub.product(first_number,second_number));
 break;
 case 4:
 System.out.println("Enter number:");
 num = scn.nextInt();
 System.out.println(stub.printOddEven(num));
 break;
 case 5:
 System.out.println("Enter number:");
 num = scn.nextInt();
 System.out.println(stub.printPrime(num));
 break;
 case 6:
 System.out.println("Enter number:");
 num = scn.nextInt();
 System.out.println(stub.printFactorial(num));
 break;
 case 7:
 System.exit(0);
 break;
 default:
 System.out.println("Enter valid choice:");
 break;
 }
}
} catch (Exception e){
 e.printStackTrace();
 System.out.println(e.getCause());
}
}
}

```

**CORBA(Common Object Request Broker Architecture) (do yourself)**



## Chapter 8: JDBC

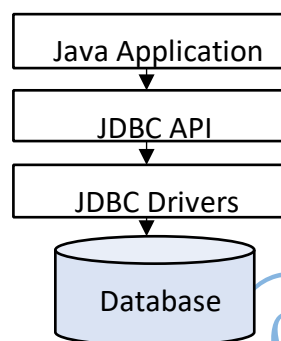
### Some old Questions:

1. Briefly describe the JDBC-ODBC types of bridge and driver in java.
2. What is JDBC? Draw JDBC architecture and describe different types of JDBC Drivers with their advantages and disadvantages.
3. What are the difference between JDBC and ODBC ? What are the steps to connect to the database in Java.
4. Write a program to display only those records whose salary is more than 25000 from a table that contains id, name, post and salary of some employees.
5. A database “testdb” contains a table “Student” with fields id, name and address. Write a program that asks user to enter the information and save them in the database. The program should prompt the user to press y/n to either continue or exit the program after every successful entry.
6. WAP to insert data of any student with schemas (name, age, semester, university, status). By default, Sql-connector is to be used (if comfortable with any database connector , free to describe and use accordingly).
7. Differentiate between simple statements and prepared statements. Provide an example to illustrate their differences.
8. Write a simple Java Program to connect database and read data in the table named “Student” which have four fields named “id”, “name”, “address”,and “DOB”. Assuming the “id” field as simple number, display the data of the students with id less than 100.
9. What is a **ResultSetMetadata**? Provide a simple example code to illustrate its usage.
10. Discuss about the different types of database drivers. Why JDBC is required in Java?
11. What is the benefit of using Prepared Statement in java? What is JDBC database connection pool(step to connect database) ? How to setup in Java?
12. A database “testdb” contains a table “employee” with some records having id, name, post , salary. Write a program to update the salary to 50000 whose post is “Manager”.
13. Short Notes:
  - i. Types and roles of JDBC drivers
  - ii. Executing SQL queries from Java
  - iii. JDBC-ODBC bridge
  - iv. Java native Driver
  - v. ResultSet

## Introduction :

A **database** is an integrated collection of data. A **Database Management System (DBMS)**; provides mechanisms for storing , organizing , retrieving and modifying data. Today's popular database management systems are relational database systems. **SQL**(Structured Query Language) is the international standard language used to query and manipulate relational data.

**JDBC** stands for Java Database Connectivity , is an API(Application Programming Language) to connect and execute the query with the database ( Which defines how a client may access a database). It is a part of JavaSE (Java Standard Edition). JDBC API uses JDBC drivers ; enables Java applications to connect to a database in a particular DBMS and allows you to retrieve and manipulate database data.



*Figure: Java application Connection to database*

## Why should use JDBC?

Before JDBC API, ODBC API was used to connect and execute the query with database. But, ODBC API uses ODBC driver which is written in C language (i.e. platform dependent and unsecured). As you know that Java language is platform independent , that's why Java has defined its own API (JDBC API) that uses JDBC drivers (written in Java language).

## Some SQL statements and clauses:

1. **SELECT** – Retrieves data from one or more tables.

**Syntax:**

**SELECT \* FROM table\_name; //select all column values**

**SELECT column1,column2, ..... FROM table\_name; //select particular column values**

2. **INSERT** – Insert rows into a specified table. (Table Name: *it\_4<sup>th</sup>\_tbl*)

**Syntax:**

**INSERT INTO table\_name(column1,column2,...) VALUES(value1,value2,...);**

**INSERT INTO *it\_4<sup>th</sup>\_tbl* (*s\_first\_name*, *s\_address*, *s\_age*) Values( 'Ram', 'Bharatpur',18);**

3. **UPDATE** – Update rows data in a specified table.

**Syntax:**

**UPDATE table\_name SET column1 = value1, column2 = value2, ... WHERE criteria;**

**UPDATE *it\_4<sup>th</sup>\_tbl* SET *s\_address* = "Kathmandu" WHERE *s\_first\_name* = "Ram" AND *s\_age* > 19;**

4. **DELETE** – Delete rows from a specified table.

**Syntax:**

**DELETE FROM table\_name WHERE criteria;**

**DELETE FROM *it\_4<sup>th</sup>\_tbl* WHERE *s\_name* = "Hari";**

5. **WHERE clauses** – is necessary to locate rows in a database that satisfy certain selection criteria . Only rows that satisfy the selection criteria (formally called predicates) are selected.

**Syntax:**

```
SELECT * FROM employee_tbl WHERE e_salary > 15000;
UPDATE employee_tbl SET e_salary WHERE e_duration = 5;
SELECT e_name, e_salary, e_duration FROM employee_tbl WHERE e_name LIKE 'R%';
```

6. **ORDER BY** – is used to sort the data returned by a query in ascending(**ASC**) or descending(**DESC**) order. Ascending order is a default order. The ORDER BY clause could be used in conjunction with the SELECT statement.

**Syntax:**

```
SELECT expressions FROM table_name [WHERE conditions] ORDER BY expression [ASC |
DESC];
SELECT * FROM it_4th_tbl ORDER BY name; //default ASCENDING order
SELECT * FROM it_4th_tbl ORDER BY name ASC; //also ASCENDING order
SELECT * FROM it_4th_tbl ORDER BY name DESC; //also DESCENDING order
```

7. **GROUP BY** – is used to organize data that have the same attribute values. Usually, used with the SELECT statement. Place GROUP BY after WHERE clause and before ORDER BY clause. We can often use GROUP BY clause in collaboration with aggregate functions like **SUM**, **AVG**, **MIN**, **MAX** and **COUNT** to produce summary reports from the database. Always used in conjunction with the **SELECT** clause, is grouped query, and returns a single row for each grouped object.

**Syntax:**

```
SELECT column_name, function(column_name) FROM table_name WHERE conditions
GROUP BY column_name;
SELECT staff_position, AVG(staff_salary) AS Salary FROM college_employee GROUP BY
staff_position ORDER BY staff_name DESC;
```

8. **LIKE** – is a logical operator in the Structured Query Language. Used in the **WHERE** clause with the **SELECT**, **UPDATE**, and **DELETE** Statement. It filters the records from the columns based on the pattern specified in the SQL query. Two wildcard characters that are used either in conjunction or independently with the SQL LIKE operator:

- i. **% (percent sign)** – matches zero, one or more than one character.
- ii. **\_ (underscore sign)** – matches only one or a single

**Syntax:**

```
SELECT column_name1 FROM table_name WHERE column_name LIKE pattern ;
```

```
SELECT * FROM college_employee WHERE staff_name LIKE 'S%'; //filter staff_name start
with 'S'
```

```
SELECT * FROM college_employee WHERE staff_name LIKE '%h'; //filter staff_name ends
with 'h'
```

```
SELECT * FROM college_employee WHERE staff_name LIKE 'S%h'; //filter staff_name start
with 'S' and ends with 'h'
```

```
SELECT * FROM college_employee WHERE staff_name LIKE '_a%h'; //filter staff_name
contains 'a' at second position and ends with 'h'
```

9. **JOIN** – Join helps to combine two or more tables. The SQL JOIN clause takes record from two or more tables in a database and combines it together.

**ANSI standard SQL** defines five types of JOIN:

1. **Inner Join** – joining of two or more table on common field between them, also known as simple join and is the most common type of join. Return common record only.  
**SELECT** staff\_id, staff\_name,staff\_salary **FROM** emp\_detail ed,emp\_payment ep  
**WHERE** ed.id = ep.staff\_id;
2. **Left outer join** – **DO YOURSELF !!!**
3. **Right outer join** – **DO YOURSELF !!!**
4. **Full outer join , and** – **DO YOURSELF !!!**
5. **cross join** – **DO YOURSELF !!!**

## JDBC-Drivers

A **JDBC driver** is a software component enabling a Java application to interact with a database. To connect with individual databases, JDBC (the Java Database Connectivity API) requires drivers for each database (You can use any sort of database for example MySQL, ORACLE, MongoDB, SQL server etc but required respective JDBC Driver).

There are **four** types of JDBC Drivers:

### Type 1-Driver: JDBC-ODBC Bridge Driver

- ODBC driver is used to connect to the database.
- The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls.
- The driver is platform-dependent as it makes use of ODBC which in turn depends on native libraries of that operating system where the JVM is running.
- This driver needs other installation of dependencies i.e. ODBC must be installed on the computer having the driver and database must support ODBC driver.

#### Advantages:

- easy to use.
- can be easily connected to any database.

#### Disadvantages:

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.
- Oracle does not support the JDBC-ODBC Bridge from Java 8.

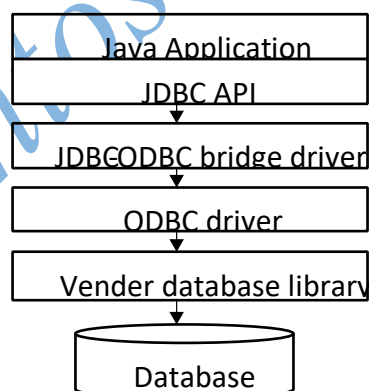


Figure: Type 1 Driver

### Type 2-Driver: Native-API driver

- The Native API driver uses the client-side libraries of the database.
- The driver converts JDBC method calls into native calls of the database API. For example: Oracle OCI driver is a type 2 driver.

#### Advantage:

- performance upgraded than JDBC-ODBC bridge driver.

#### Disadvantage:

- The Native driver needs to be installed on the each client machine.
- The Vendor client library needs to be installed on client machine.

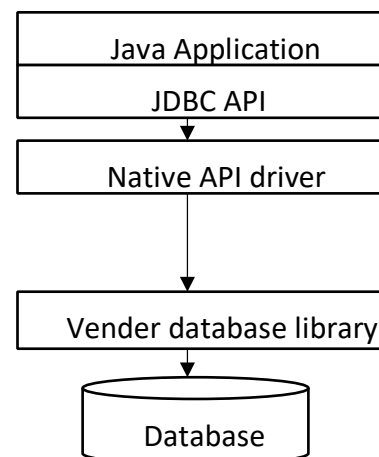


Figure: Type 2 Driver

**Type 3-Driver: Network Protocol Driver (middleware driver)**

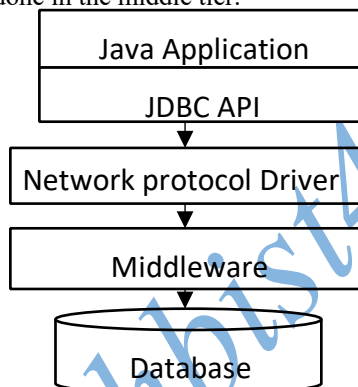
- The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol.
- It is fully written in java.
- The protocol conversion logic resides not at the client, but in the middle-tier.
- The type 3 driver is platform-independent as the platform-related differences are taken care of by the middleware.
- The same client-side JDBC driver may be used for multiple databases and depend on the number of database configured for middleware supported.

**Advantages:**

- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

**Disadvantages:**

- Network support is required on client machine.
- Requires database-specific coding to be done in the middle tier.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.



*Figure: Type 3-Driver*

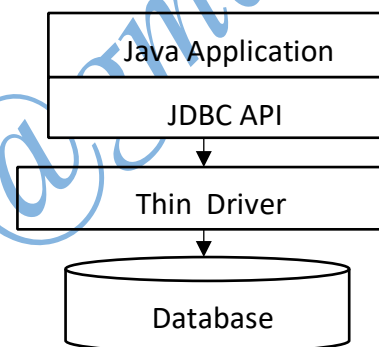
**Type 4-Driver: Database-Protocol driver/Thin Driver(Pure Java driver)**

- The thin driver converts JDBC calls directly into the vendor-specific database protocol.
- That is why it is known as thin driver. It is fully written in Java language.
- As the database protocol is vendor specific, the JDBC client requires separate drivers, usually vendor supplied, to connect to different types of databases.

**Advantages:**

- Completely implemented in Java to achieve platform independence.
- These drivers don't translate the requests into an intermediary format (such as ODBC).
- No software is required at client side or server side.

**Disadvantages:** • Drivers depend on the Database.



*Figure: Type 4-Driver*

## How to connect Java Application to any database ?

There are 5 steps to connect any java application with the database using JDBC. These steps are as follows:

- **Register the Driver class** – The **forName()** method of **Class** class is used to register the driver class. This method is used to dynamically load the driver class.
- **Create connection** – The **getConnection()** method of DriverManager class is used to establish connection with the database.
- **Create statement** – The **createStatement()** method of Connection interface is used to create statement. The object of statement is responsible to execute queries with the database.
- **Execute queries** – The **executeQuery()** method of Statement interface is used to execute queries to the database. This method returns the object of **ResultSet** that can be used to get all the records of a table.

- Similarly, *executeUpdate()* method of Statement interface is used to execute queries related to **INSERT, UPDATE, ALTER, DELETE, DROP** statement.
- **Close connection** – By closing connection object statement and **ResultSet** will be closed automatically. The *close()* method of Connection interface is used to close the connection.

Now here is the example to connect Java Application with MySQL database. Before to connect MySQL database we need to add *mysql connector / jar file (provides relative driver)* download from to the java application. To connect Java application with the MySQL database, we need to follow following steps. Also we need to know following information's for the mysql database:

1. **Driver class(Register Driver ):** The driver class for the mysql database is **com.mysql.cj.jdbc.Driver**.
2. **Connection URL:** The connection URL for the mysql database(database provided by apache server using xampp here) is **jdbc:mysql://localhost:3306/indreni** where jdbc is the API, mysql is the database, localhost is the server name on which mysql is running, we may also use IP address, 3306 is the port number and **indreni** is the database name. We may use any database, in such case, we need to replace the **indreni** with your database name.
3. **Username:** The default username for the mysql database is **root**.
4. **Password:** It is the password given by the user at the time of installing the mysql database. In this example, we are not going to use any password.

#### Program to create database named : indreni

*/\* filename: StudentRecord.java \*/*

```
package DatabaseExample;
import java.sql.Connection;
import java.sql.Statement;

import java.sql.DriverManager;

import java.sql.SQLException;
public class StudentRecord {
 private Connection con;
 private Statement stm;
 StudentRecord(){
 try{
 //register driver
 Class.forName("com.mysql.cj.jdbc.Driver");
 //connection established to MySQL DBMS using getConnection(...) of DriverManager class
 con = DriverManager.getConnection("jdbc:mysql://localhost:3306","root","");
 //createStatement() creates a statement, that means to send SQL query to database for execution
 stm = con.createStatement();
 } catch (ClassNotFoundException | SQLException ce) {
 System.out.println("Exception Occurred:"+ce.getMessage());
 }
 }
 public void trySqlStatements(){
 try{
 //define query to create database name indreni , if there is no such named database exist
 String queryToCreateDatabase = "CREATE DATABASE IF NOT EXISTS indreni";
```



```

//excute above query i.e. queryToCreateDatabase and return either 1 i.e. success OR 0 i.e. unsuccess
int r=stm.executeUpdate(queryToCreateDatabase);
System.out.println(r);
boolean t = (r==1);
System.out.println(t);
if(t){
 System.out.println("Success...");
}else{
 System.out.println("Unsuccessful.Already Exist...");
}
} catch(SQLException se){
 System.out.println("Exception Occurred:"+se.getMessage());
}
}
}
public static void main(String[] args){
 StudentRecord sr = new StudentRecord();
 sr.trySqlStatements();
}
}

```

**Program to create table (named: *tbl\_ind*) within *indreni* database**

```

/* StudentRecord.java */
package DatabaseExample;
import java.sql.Connection;
import java.sql.Statement;

import java.sql.DriverManager;

import java.sql.SQLException;
public class StudentRecord {
 private Connection con;
 private Statement stm;
 StudentRecord(){
 try{
 //register driver
 Class.forName("com.mysql.cj.jdbc.Driver");
 //connection established to connect indreni database within MySQL DBMS using getConnection(...)
 of DriverManager class
 con = DriverManager.getConnection("jdbc:mysql://localhost:3306/indreni","root","");
 //createStatement() creates a statement, that means to send SQL query to database for execution
 stm = con.createStatement();
 } catch(ClassNotFoundException | SQLException ce){
 System.out.println("Exception Occurred:"+ce.getMessage());
 }
 }
 public void trySqlStatements(){
 try{
 //define query to create table name tbl_ind
 String queryToCreateTable = "CREATE TABLE tbl_ind(id INT(5) PRIMARY KEY
 AUTO_INCREMENT,firstName VARCHAR(30),secondName VARCHAR(30),address VARCHAR(30))";
 //excute above query i.e. queryToCreateTable and return 0 (Table created successfully without
 affecting any table row)
 //Otherwise Exception occur
 int r=stm.executeUpdate(queryToCreateTable);
 System.out.println(r);
 boolean t = (r==0);

```

```

 System.out.println(t);
 if(t){
 System.out.println("Table Created Successfully ...");
 }
 } catch(SQLException se){
 System.out.println("Exception Occurred:"+se.getMessage());
 }
}
}
public static void main(String[] args){
 StudentRecord sr = new StudentRecord();
 sr.trySqlStatements();
}
}

```

### Program to Insert record in table named : tbl\_ind

*/\*StudentRecord.java\*/*

```

package DatabaseExample;
import java.sql.Connection;
import java.sql.Statement;

```

```

import java.sql.DriverManager;

```

```

import java.sql.SQLException;

```

```

public class StudentRecord {
 private Connection con;
 private Statement stm;
 StudentRecord(){

```

```

 try{

```

*//register driver*

```

 Class.forName("com.mysql.cj.jdbc.Driver");

```

*//connection established to connect indreni database within MySQL DBMS using getConnection(...) of*

*DriverManager class*

```

 con = DriverManager.getConnection("jdbc:mysql://localhost:3306/indreni ","root","");

```

*//createStatement() creates a statement, that means to send SQL query to database for execution*

```

 stm = con.createStatement();

```

```

 } catch(ClassNotFoundException | SQLException ce){

```

```

 System.out.println("Exception Occurred:"+ce.getMessage());

```

```

 }

```

```

 }

```

```

 public void trySqlStatements(){

```

```

 try{

```

*//define query to insert record in table named tbl\_ind*

```

 String queryToInsertRecord = "INSERT INTO tbl_ind(firstName,secondName,address)

```

```

VALUES('Ram','Bahadur','Lalitpur')";

```

*//execute above query i.e. queryToInsertRecord and return 1 (One row affected)*

*//Otherwise Exception occur*

```

 int r=stm.executeUpdate(queryToInsertRecord);

```

```

 System.out.println(r);

```

```

 boolean t = (r==1);

```

```

 System.out.println(t);

```

```

 if(t){

```

```

 System.out.println("Record Inserted Successfully ...");

```

```

 }

```

```

 con.close();

```

```

 } catch(SQLException se){

```

```

 System.out.println("Exception Occurred:"+se.getMessage());

```



```

 }
}
public static void main(String[] args){
 StudentRecord sr = new StudentRecord();
 sr.trySqlStatements();
}
}

```

### Example 1: Read / Fetch /Get record from table *tbl\_ind*



This is the screenshot that shows the database name *indreni* and table name *tbl\_ind* having four field(id, firstName, secondName & address).

Java Program that illustrate Java Application connection with MySQL database. Netbeans as an IDE with platform independent MySQL driver(*com.mysql.cj.jdbc.Driver*).

step-by-step to connect java application with MySQL database management system that connect "*indreni*" database and fetch records from "*tbl\_ind*" table.

1. Main Program: Here class name is **JavaDatabaseConnection** where (**DBconnection**) been executed.

*/\*JavaDatabaseConnection.java\*/*

```

package javadatabaseconnection;
public class JavaDatabaseConnection {
 public static void main(String[] args) {
 DBconnection dbc=new DBconnection(); //create object(here dbc)
 dbc.getData();
 }
}

```

2. Database connection and query execution in different class.

*/\*DBconnection.java\*/*

```

package javadatabaseconnection;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
//import java.sql.*; This statement import all sql interfaces,class,methods,etc..
public class DBconnection {
 private Connection con;
}

```

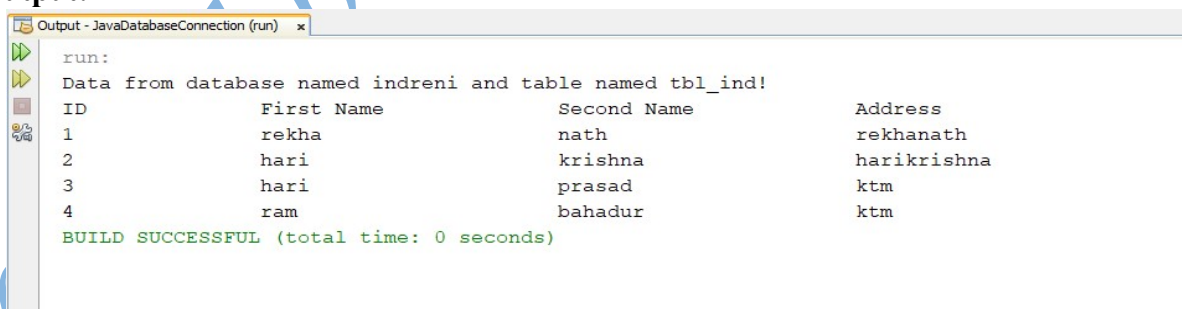
```

private Statement st;
private ResultSet rst;
public DBconnection(){
 try{
 Class.forName("com.mysql.cj.jdbc.Driver"); //driver registration
 //connection to database
 con=DriverManager.getConnection("jdbc:mysql://localhost:3306/indreni","root","");
 st=con.createStatement(); //creating statement
 }catch(Exception ex){
 System.out.println("Not connected! (Error Occured:)" +ex);
 }
}
public void getData(){
 try{
 String query="select * from tbl_ind";
 rst=st.executeQuery(query); //executing QUERY
 System.out.println("Data from database named indreni and table named tbl_ind!");
 System.out.println("ID"+"\\t\\t"+"First Name"+"\\t\\t"+"Second Name"+"\\t\\t"+"Address");

 //Here while(rst.next()) help to reach all the fetched records from the table(tbl_ind)
 while(rst.next()){
 //following "id", "firstName", "secondName", "address"
 int id=rst.getInt("id");
 String firstName=rst.getString("firstName");
 String secondName=rst.getString("secondName");
 String address=rst.getString("address");
 System.out.println(id+"\\t\\t"+firstName+"\\t\\t"+secondName+"\\t\\t"+address);
 }
 }catch(Exception ex){
 System.out.println("Errors:" +ex);
 }
}
}

```

### Output:

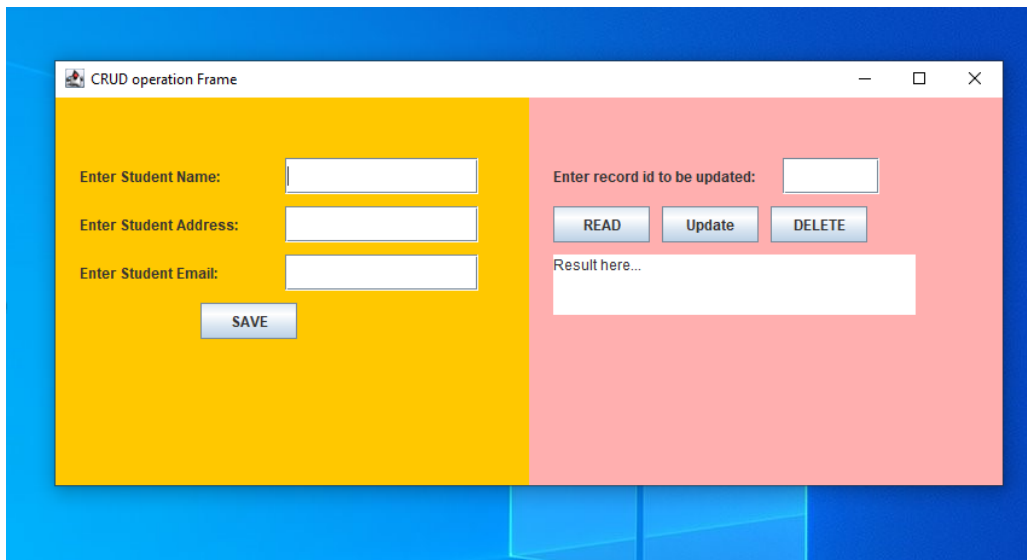


```

run:
Data from database named indreni and table named tbl_ind!
ID First Name Second Name Address
1 rekha nath rekhanath
2 hari krishna harikrishna
3 hari prasad ktm
4 ram bahadur ktm
BUILD SUCCESSFUL (total time: 0 seconds)

```

**Example 2:** CRUD operation(C – Create records(i.e. **INSERT** statement), **R** – Read records(i.e. **SELECT** statement), **U** – Update records(i.e. **UPDATE** statement), **D** – Delete records(i.e. **DELETE** statement) ). You see all following user interface that shows some labels, text field, button and one text area through which handle CURD operation.



**Program:**

**/\*CRUDOperationExample.java\*/**

```
package crudoperationexample;
public class CRUDOperationExample {
 public static void main(String[] args) {
 new ApplicationFrame();
 }
}
```

**/\*ApplicationFrame.java\*/**

```
package crudoperationexample;
```

```
import java.awt.Color;
import java.awt.GridLayout;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextArea;
import javax.swing.JTextField;
```

```
public class ApplicationFrame extends JFrame {
 JPanel recordInsertPanel, recordUpdatePanel;
 JLabel nameLabel, addressLabel, emailLabel, idLabel;
 JTextArea result;
 JTextField nameField, addressField, emailField, idField;
 JButton insert, delete, update, read;
```

```
public ApplicationFrame(){
```

```
 //left panel to insert record
```

```
 recordInsertPanel = new JPanel();
```

```
 nameLabel = new JLabel("Enter Student Name:");
```

```
 nameLabel.setBounds(20,50,160,30);
```

```
 recordInsertPanel.add(nameLabel);
```

```

nameField = new JTextField();
nameField.setBounds(190,50,160,30);
recordInsertPanel.add(nameField);

addressLabel = new JLabel("Enter Student Address:");
addressLabel.setBounds(20,90,160,30);
recordInsertPanel.add(addressLabel);

addressField = new JTextField();
addressField.setBounds(190,90,160,30);
recordInsertPanel.add(addressField);

emailLabel = new JLabel("Enter Student Email:");
emailLabel.setBounds(20,130,160,30);
recordInsertPanel.add(emailLabel);

emailField = new JTextField();
emailField.setBounds(190,130,160,30);
recordInsertPanel.add(emailField);

insert = new JButton("SAVE");
insert.setBounds(120,170,80,30);
recordInsertPanel.add(insert);

recordInsertPanel.setBackground(Color.ORANGE);
recordInsertPanel.setLayout(null);
add(recordInsertPanel);

//right panel, to read,delete and update record
recordUpdatePanel = new JPanel();

idLabel = new JLabel("Enter record id to be updated:");
idLabel.setBounds(20,50,180,30);
recordUpdatePanel.add(idLabel);

idField = new JTextField();
idField.setBounds(210,50,80,30);
recordUpdatePanel.add(idField);

read = new JButton("READ");
read.setBounds(20,90,80,30);
recordUpdatePanel.add(read);

update = new JButton("Update");
update.setBounds(110,90,80,30);
recordUpdatePanel.add(update);

delete = new JButton("DELETE");
delete.setBounds(200,90,80,30);
recordUpdatePanel.add(delete);

```

```

recordUpdatePanel.setBackground(Color.pink);
recordUpdatePanel.setLayout(null);
add(recordUpdatePanel);

result = new JTextArea("Result here...");
result.setBounds(20,130,300,50);
result.setEditable(false);
recordUpdatePanel.add(result);

//let's handle event outside the class
DatabaseHandler handler = new DatabaseHandler(this);
insert.addActionListener(handler);
read.addActionListener(handler);
update.addActionListener(handler);
delete.addActionListener(handler);

setTitle("CRUD operation Frame");
setSize(800,360);
setLayout(new GridLayout());
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setVisible(true);
}
}

```

**/\*DatabaseHandler.java\*/**

```
package crudoperationexample;
```

```

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
import javax.swing.JOptionPane;

```

```
public class DatabaseHandler implements ActionListener{
```

```

 Connection connect;
 Statement stm;
 ResultSet resultSet;

```

```

 ApplicationFrame af;
 public DatabaseHandler(ApplicationFrame af){
 this.af = af;
 }

```

```
@Override
```

```

public void actionPerformed(ActionEvent ae){
 connectDatabase();
 int getId;
 try{

```

```
//get input from text field
```

```

 String name = af.nameField.getText();
 String adrs = af.addressField.getText();
 }
}

```

```

String email = af.emailField.getText();
// to insert record when SAVE button clicked
if(ae.getSource().equals(af.insert)){
 String query = "INSERT INTO it_4th_student(name,address,email)
VALUES('"+name+"','"+adrs+"','"+email+"')";
 stm.executeUpdate(query);
 af.result.setText("One record inserted to table(it_4th_student).");
 connect.close();
}
// read/retrieve/fetch record when READ button is clicked
// must pass available id , in id text field
if(ae.getSource().equals(af.read)){
 getId = Integer.parseInt(af.idField.getText());
 String query = "SELECT * FROM it_4th_student Where id='"+getId+"'";
 resultSet = stm.executeQuery(query);
 while(resultSet.next()){
 String sname = resultSet.getString("name");
 String saddress = resultSet.getString("address");
 String semail = resultSet.getString("email");

 String record = "Name:"+sname+"\nAddress:"+saddress+"\nEmail:"+semail;
 af.result.setText(record);
 connect.close();
 }
}
// update record when UPDATE button is clicked
//must pass available id value
if(ae.getSource().equals(af.update)){
 getId = Integer.parseInt(af.idField.getText());
 String upName = JOptionPane.showInputDialog("Updated Name:");
 String upAdrs = JOptionPane.showInputDialog("Updated Address:");
 String upEmail = JOptionPane.showInputDialog("Updated Email:");
 String query = "UPDATE it_4th_student SET name= '"+upName+"', address='"+upAdrs+"',
email='"+upEmail+"' WHERE id='"+getId+"'";
 stm.executeUpdate(query);
 af.result.setText("One record updated with name "+upName);
 connect.close();
}
// delete record when DELETE button is clicked
// must pass available id value
if(ae.getSource().equals(af.delete)){
 getId = Integer.parseInt(af.idField.getText());
 String query = "DELETE FROM it_4th_student WHERE id='"+getId+"'";
 stm.executeUpdate(query);
 af.result.setText("Record with id:"+getId+" deleted .");
 connect.close();
}
}
} catch (Exception e){
 System.out.println(e);
 e.printStackTrace();
}
}
}

```

```

public void connectDatabase(){
 try{
 //define user,url,password
 String url = "jdbc:mysql://localhost:3306/it_4th";
 String user = "root";
 String password = "";

 Class.forName("com.mysql.cj.jdbc.Driver"); //mysql driver registration
 connect = DriverManager.getConnection(url,user,password); //connection to database(it_4th)
 stm = connect.createStatement(); //create statement to execute sql query
 if(connect != null){
 System.out.println("Database connection success ...");
 }
 connect.close();
 }catch(Exception e){
 System.out.println(e);
 }
}
}

```

### Output:

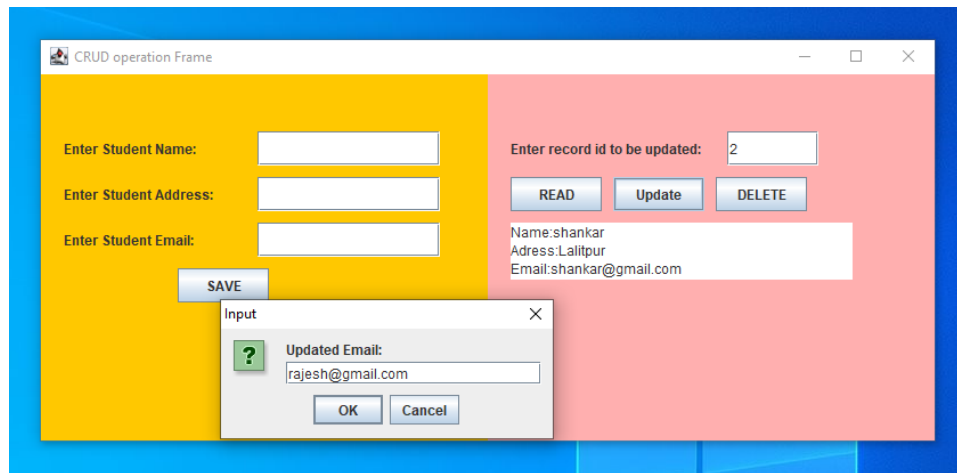
Insert data to table(it\_4<sup>th</sup>\_student) when clicked on “SAVE” button.

The screenshot shows a window titled "CRUD operation Frame". On the left, there are three text input fields: "Enter Student Name:" with "Ganesh", "Enter Student Address:" with "Pokhara", and "Enter Student Email:" with "ganesh@pokhara". Below these is a "SAVE" button. On the right, there is a section for "Enter record id to be updated:" with an empty text field and three buttons: "READ", "Update", and "DELETE". A message box on the right displays the text: "One record inserted to table(it\_4th\_student)".

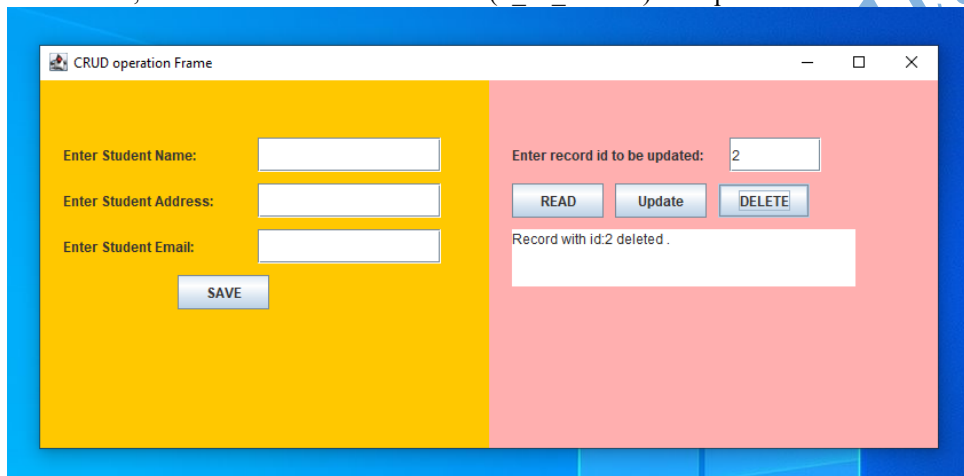
On click of “READ”, read data or fetch record from table (it\_4<sup>th</sup>\_student) as id passed from text field.

The screenshot shows the same "CRUD operation Frame" window. The "Enter Student Name:" field is empty. The "Enter Student Address:" field is empty. The "Enter Student Email:" field is empty. The "SAVE" button is still present. The "Enter record id to be updated:" field now contains the value "2". The "READ", "Update", and "DELETE" buttons are still present. The message box on the right displays the text: "Name:shankar", "Adress:Lalitpur", and "Email:shankar@gmail.com".

Update data or record from table (it\_4<sup>th</sup>\_student) as id passed from text field. On click of “Update” button , there will appear some input dialogs one after another for name, address and email.



On click of “DELETE” , delete data or record from table (it 4<sup>th</sup> student) as id passed from text field.



**Example 3:** A database “*testdb*” contains a table “*Student*” with fields id, name and address. Write a program that asks user to enter the information and save them in the database. The program should prompt the user to press y/n to either continue or exit the program after every successful entry.

#### Program:

**/\*DatabaseApplication.java\*/**

```
package databaseapplication;
import DBConnection.*;
public class DatabaseApplication {
 public static void main(String[] args) {
 new StudentRecordFrame();
 }
}
```

**/\*StudentRecordFrame.java\*/**

```
package DBConnection;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;
```



```

public class StudentRecordFrame extends JFrame {
 JLabel nlabel,alabel;
 JTextField ntxt,atxt;
 JButton save;
 public StudentRecordFrame(){
 setSize(400,360);
 nlabel = new JLabel("Enter student name:");
 nlabel.setBounds(20,20,120,20);
 add(nlabel);

 ntxt = new JTextField();
 ntxt.setBounds(190,20,120,20);
 add(ntxt);

 alabel = new JLabel("Enter student address:");
 alabel.setBounds(20,50,160,20);
 add(alabel);

 atxt = new JTextField();
 atxt.setBounds(190,50,120,20);
 add(atxt);

 save = new JButton("SAVE");
 save.setBounds(50,80,70,20);
 add(save);

 Handler handler = new Handler(this);
 save.addActionListener(handler);

 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 setLayout(null);
 setVisible(true);
 }
}

```

**/\* Handler .java\*/**

```

package DBConnection;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import javax.swing.JOptionPane;
public class Handler implements ActionListener{
 StudentRecordFrame srf;
 public Handler(StudentRecordFrame srf){
 this.srf = srf;
 }
 @Override
 public void actionPerformed(ActionEvent ae){
 String option;

```

```

String sName = srf.ntxt.getText();
String sAddress = srf.atxt.getText();

System.out.println("Name:"+sName+"\nAddress:"+sAddress);
try{
 Class.forName("com.mysql.cj.jdbc.Driver");
 Connection connect = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb","root","");
 Statement stm = connect.createStatement();
 String sql = "INSERT INTO Student(name,address) VALUES('"+sName+"','"+sAddress+"')";
 stm.executeUpdate(sql);
 option = JOptionPane.showInputDialog("Press y/n to either continue / exit:");
 if(option.equals("y")){
 srf.ntxt.setText("");
 srf.atxt.setText("");
 }else{ //input other than y will exit program
 srf.setVisible(false);
 srf.dispose();
 }
}catch(Exception e){
 System.out.println(e);
}
}
}

```

## DriverManager class

The DriverManager class acts as an interface between user and drivers. It keeps track of the drivers that are available and handles establishing a connection between a database and the appropriate driver. The DriverManager class maintains a list of Driver classes that have registered themselves by calling the method **DriverManager.registerDriver()**.

### Useful methods of DriverManager class :

| Method                                                                                 | Description                                                                                   |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1) public static void registerDriver(Driver driver):                                   | It is used to register the given driver with DriverManager.                                   |
| 2) public static void deregisterDriver(Driver driver):                                 | It is used to deregister the given driver (drop the driver from the list) with DriverManager. |
| 3) public static Connection getConnection(String url):                                 | It is used to establish the connection with the specified url.                                |
| 4) public static Connection getConnection(String url,String userName,String password): | It is used to establish the connection with the specified url, username and password.         |

### Prepared Statement:

- The PreparedStatement interface is a sub-interface of Statement. It is used to execute parameterized query.
- Improves performance:** The performance of the application will be faster if you use PreparedStatement interface because query is compiled only once.

- The ***prepareStatement()*** method of Connection interface is used to return the object of PreparedStatement.

### Methods of PreparedStatement interface

The important methods of **PreparedStatement** interface are given below:

| Method                                              | Description                                                                  |
|-----------------------------------------------------|------------------------------------------------------------------------------|
| public void setInt(int paramIndex, int value)       | sets the integer value to the given parameter index.                         |
| public void setString(int paramIndex, String value) | sets the String value to the given parameter index.                          |
| public void setFloat(int paramIndex, float value)   | sets the float value to the given parameter index.                           |
| public void setDouble(int paramIndex, double value) | sets the double value to the given parameter index.                          |
| public int executeUpdate()                          | executes the query. It is used for create, drop, insert, update, delete etc. |
| public ResultSet executeQuery()                     | executes the select query. It returns an instance of ResultSet.              |

**Example 1:** CRUD operation using PreparedStatement interface.

**/\*CRUDOperationExample.java\*/**

```
package crudoperationexample;
public class CRUDOperationExample {
 public static void main(String[] args) {
 PreparedStatementExample pse = new PreparedStatementExample();
 pse.InsertRecord();
 pse.ReadRecord();
 pse.UpdateRecord();
 pse.DeleteRecord();
 }
}
```

**/\*PreparedStatementExample.java\*/**

```
package crudoperationexample;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class PreparedStatementExample {
 Connection connect;
 PreparedStatement stm;
 ResultSet result;
```

```

public PreparedStatementExample(){
 try{
 //define user,url,password
 String url = "jdbc:mysql://localhost:3306/it_4th";
 String user = "root";
 String password = "";

 Class.forName("com.mysql.cj.jdbc.Driver"); //mysql driver registration
 connect = DriverManager.getConnection(url,user,password); //connection to database(it_4th)
 }catch(Exception e){
 System.out.println(e);
 }
}

//insert record to table it_4th_student
public void InsertRecord(){
 try{
 String query = "INSERT INTO it_4th_student(name,address,email) VALUES(?,?,?)";
 stm = connect.prepareStatement(query);
 stm.setString(1,"Gaule");
 stm.setString(2, "Dharan");
 stm.setString(3,"gaule@gmail.com");
 int i = stm.executeUpdate();
 System.out.println(i+" record inserted.");
 }catch(Exception e){
 System.out.println(e);
 }
}

//read/fetch/retrieve record from table it_4th_student
public void ReadRecord(){
 try{
 String query = "SELECT * FROM it_4th_student WHERE name=?";
 stm = connect.prepareStatement(query);
 stm.setString(1,"Gopal");
 result = stm.executeQuery();
 System.out.println("Fetched record:");
 System.out.println("ID:\t\tName:\t\tAddress:\t\tEmail:");
 while(result.next()){
 int id = result.getInt("id");
 String name = result.getString("name");
 String address = result.getString("address");
 String email = result.getString("email");
 System.out.println(id+"\t\t"+name+"\t\t"+address+"\t\t"+email+"\t\t");
 }
 }catch(Exception e){
 System.out.println(e);
 }
}

//update record
public void UpdateRecord(){
 try{
 String query = "UPDATE it_4th_student SET name=?,email=? WHERE id=?";
 }
}

```

```

 stm = connect.prepareStatement(query);
 stm.setString(1,"Gita");
 stm.setString(2,"gita@gmail.com");
 stm.setInt(3,6);
 int i = stm.executeUpdate();
 System.out.println(i+" record updated.");
 }catch(Exception e){
 System.out.println(e);
 }
}

//delete record from table it_4th_student
public void DeleteRecord(){
 try{
 String query = "DELETE FROM it_4th_student WHERE id=?";
 stm = connect.prepareStatement(query);
 stm.setInt(1,1);
 int i = stm.executeUpdate();
 System.out.println(i+" record deleted.");
 }catch(Exception e){
 System.out.println(e);
 }
}
}

```

```

Output
CRUDOperationExample (run) x CRUDOperationExample (run) #2 x
run:
1 record inserted.
Fetched record:
ID: Name: Address: Email:
5 Gopal Kailali gopal@gmail.com
8 Gopal Dharan gopal12@gmail.com
9 Gopal Dharan gopal12@gmail.com
1 record updated.
1 record deleted.
BUILD SUCCESSFUL (total time: 0 seconds)

```

## ResultSet and ResultSetMetaData:

**ResultSet**(*java.sql.ResultSet*) is an interface defined under *java.sql* package. When **Statement** method *executeQuery()* executes a query and returns a **ResultSet** object. Which means **ResultSet** enable a program to manipulate query results. **ResultSet** method *next* positions the **ResultSet** cursor to the row and returns true if the row exists; otherwise, it returns false. This method must be called to begin processing a **ResultSet** because the cursor is initially positioned before the first row. **ResultSet** row and column numbers start at 1. Each **Statement** object can open only one **ResultSet** at a time. When a **Statement** returns a new **ResultSet**, the **Statement** closes the prior **ResultSet**. **ResultSet** get methods typically receive as an argument either a column number(as an int) or a column name (as a String) indicating which column's value to obtain. For example: *getInt("column\_name")*, *getString("column\_name")*, *getFloat("column\_name")*, *getObject("column\_name")*, .... .

**ResultSetMetaData**(*java.sql.ResultSetMetaData*) obtains the **ResultSet** meta data, which describes a **ResultSet**'s content. Programs can use metadata programmatically to obtain information about the **ResultSet** column names and types. Where *getColumnCount()* retrieves

the number of ResultSet column's , *getColumnType()* returns column's type and *getColumnName()* display column name.

**Example:**

**/\*DatabaseExample.java\*/**

```
package databaseexample;
public class DatabaseExample {
 public static void main(String[] args) {
 new UsingResultSetMetaData();
 }
}
```

**/\*UsingResultSetMetaData.java\*/**

```
package databaseexample;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.Statement;
```

```
public class UsingResultSetMetaData {
 Connection connect;
 Statement stm;
 ResultSet rs;
 ResultSetMetaData rsMD;
 public UsingResultSetMetaData(){
 try{
 //define user,url,password
 String url = "jdbc:mysql://localhost:3306/it_4th";
 String user = "root";
 String password = "";

 Class.forName("com.mysql.cj.jdbc.Driver"); //mysql driver registration
 connect = DriverManager.getConnection(url,user,password); //connection to database(it_4th)
 stm = connect.createStatement(); //create statement to execute sql query
 String query = "SELECT * FROM it_4th_student";
 rs= stm.executeQuery(query);
 rsMD = rs.getMetaData();
 //count column number
 int columnNumber = rsMD.getColumnCount();
 System.out.printf("i_4th_student table have:%d column\n's.\n",columnNumber);
 //display the names of the columns
 for(int i=1;i<= columnNumber;i++){
 System.out.printf("%-8s\t",rsMD.getColumnName(i));
 }
 System.out.println();
 // display query results
 while(rs.next()){
 for(int i=1;i<=columnNumber;i++){
 System.out.printf("%-8s\t",rs.getObject(i));
 }
 System.out.println();
 }
 }
 }
}
```

```

 }
 connect.close();

} catch (Exception e) {
 System.out.println(e);
}
}
}

```

### Output:



```

run:
i_4th_student table have:4 column's.
id name address email
3 Harish Haridwar harish@yahoo.com
4 Ganesh Pokhara ganesh@pokhara
5 Gopal Kailali gopal@gmail.com
6 Gita Kailali gita@gmail.com
9 Gopal Dharan gopal12@gmail.com
10 Gaule Dharan gaule@gmail.com
11 ramesh mauriphanta ramesh@gmail.com
BUILD SUCCESSFUL (total time: 0 seconds)

```

**Example :** A database “testdb” contains a table “employee” with some records having id, name, post , salary. Write a program to update the salary to 50000 whose post is “Manager”.

#### /\*UpdateEmployeeSalaryExample.java\*/

```

package MoreDatabaseExample;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class UpdateEmployeeSalaryExample {
 Connection connect;
 Statement stm;
 public static void main(String[] args){
 UpdateEmployeeSalaryExample upd = new UpdateEmployeeSalaryExample();
 upd.connectDatabase();
 upd.updateSalary();
 }
 public void connectDatabase(){
 try{
 String url = "jdbc:mysql://localhost:3306/testdb";
 String user = "root";
 String pass = "";
 Class.forName("com.mysql.cj.jdbc.Driver");
 connect= DriverManager.getConnection(url,user,pass);
 stm = connect.createStatement();
 } catch (Exception e){
 System.out.println(e);
 }
 }
}

```

```

 }
}
public void updateSalary(){
 try{
 String sql = "UPDATE employee SET salary=50000 WHERE post='Manager'";
 int result = stm.executeUpdate(sql); //results 1 if success
 if(result == 1){
 System.out.println("Record updated successfully.");
 }
 }catch(Exception e){
 System.out.println(e);
 }
}
}
}

```

**Execute following menu based statement using sql statement:**

1. Create Database
2. Create Table with field (id,name,gender,address)
3. Fetch all record from specified table
4. Exit

Request database name and table name from user at runtime. If not found throw exception.

***/\*MenuBasedStatement.java\*/***

```

package menubased;
import java.util.Scanner;
import java.sql.*;
public class MenuBasedStatement {
 static Connection con; //instance of Connection interface (confusion)
 static Statement stm; // instance of Statement interface (confusion)
 static ResultSet rst; // instance of ResultSet interface (confusion)
 // define object of Scanner class for user input
 static Scanner scn = new Scanner(System.in);
 //path to access mysql dbms i.e. url
 static String url = "jdbc:mysql://localhost:3306";
 static String user = "root"; // user name defined
 static String pass = ""; //right now password is empty
 //above variables are static type
 public MenuBasedStatement() //default constructor
 {
 try{
 //connecting type-4 driver(MySQL) to Java application
 Class.forName("com.mysql.cj.jdbc.Driver");
 //ClassNotFoundException will be thrown if Driver class not found
 }catch(ClassNotFoundException ce){
 //returns Exception, Description , cause , message etc
 ce.printStackTrace();
 }
 }
 static void toCreateDatabase(){
 try{
 //get connection to MySQL DBMS
 con = DriverManager.getConnection(url,user,pass);
 stm = con.createStatement(); // create statement
 System.out.println("Enter database name:"); //print user statement

```



```

String db_name = scn.nextLine(); //user input
// Query to create database. Database name defined by user
String createDb = "CREATE DATABASE "+db_name;
// execute sql query using executeUpdate method of statement i.e. stm
stm.executeUpdate(createDb);
//success statement with database name in console
System.out.println("New Database Successfully created:"+db_name);
}catch(SQLException se){
 System.out.println(se.getMessage()); //Exception Handling
}
}
static void toCreateTable(){
 try{
 System.out.println("Enter database name:");
 String db_name = scn.nextLine(); //Database name by user
//get connection to user defined database
 con = DriverManager.getConnection(url+"/"+db_name,user,pass);
 stm = con.createStatement(); //create statement

 System.out.println("Enter table name:");
 String tbl_name = scn.nextLine(); // Table name by user
//createTbl string variable hold Query to create table named by user
//with fields (id,name,gender,address)
 String createTbl = "CREATE TABLE "+tbl_name;
 createTbl += "(id INT(4) PRIMARY KEY AUTO_INCREMENT,";
 createTbl += "name VARCHAR(30),gender VARCHAR(10),address VARCHAR(30))";
 stm.executeUpdate(createTbl); //execute create statement of sql
//success statement with table name in console
 System.out.println("New Table successfully created :"+tbl_name);
 }catch(SQLException se){
 se.printStackTrace();
 }
}
static void toInsertRecord(){
 try{
 System.out.println("Enter database name:");
 String db_name = scn.nextLine();
//get connection to user defined database
 con = DriverManager.getConnection(url+"/"+db_name,user,pass);
 stm = con.createStatement(); //create statement
 System.out.println("Enter table name:");
 String tbl_name = scn.nextLine(); // Table name by user
 System.out.println("Enter name:");
 String name = scn.nextLine(); //name by user
 System.out.println("Enter gender:");
 String gender = scn.nextLine(); // gender by user
 System.out.println("Enter address:");
 String address = scn.nextLine(); //address by user
 String insertRecord = "INSERT INTO ";
 insertRecord += tbl_name+"(name,gender,address)";
 insertRecord += "VALUES('"+name+"','"+gender+"','"+address+"')";
 stm.executeUpdate(insertRecord);
 }
}

```

```

 System.out.println("One Record Inserted Successfully.");
 }catch(SQLException se){
 System.out.println(se.getMessage());
 }
}

static void toFetchRecord(){
 try{
 System.out.println("Enter database name:");
 String db_name = scn.nextLine();
 con = DriverManager.getConnection(url+"/"+db_name,user,pass);
 stm = con.createStatement(); //create statement
 System.out.println("Enter table name:");
 String tbl_name = scn.nextLine();
 //fetch record from user defined table
 String fetchRecord = "SELECT * FROM "+tbl_name;
 //return all rows as object to rst
 rst = stm.executeQuery(fetchRecord);
 System.out.println("ID\t\tName\t\tGender\t\tAddress");
 while(rst.next()){
 int id = rst.getInt("id");
 String name = rst.getString("name");
 String gender = rst.getString("gender");
 String address = rst.getString("address");
 System.out.println(id+"\t\t"+name+"\t\t"+gender+"\t\t"+address);
 }
 }catch(SQLException se){
 System.out.println(se.getMessage());
 }
}

public static void main(String[] args){
 int choice;
 new MenuBasedStatement(); //constructor call
 while(true){
 System.out.println("\n*****Menu*****");
 System.out.println("1.Create Database");
 System.out.println("2.Create Table");
 System.out.println("3.Insert Record");
 System.out.println("4.Fetch Record");
 System.out.println("5.Exit program");
 System.out.println("*****\n");
 System.out.println("Enter your choice");
 choice = scn.nextInt();
 //to ignore user input / or to break choice sequence
 scn.nextLine();
 switch(choice){
 case 1:
 toCreateDatabase();
 break;
 case 2:
 toCreateTable();
 break;

```

```

 case 3:
 toInsertRecord();
 break;
 case 4:
 toFetchRecord();
 break;
 case 5:
 System.exit(0);
 break;
 default:
 System.out.println("Try again.....");
 break;
 }
}
}
}

```

### Output:

```

****Menu****
1.Create Database
2.Create Table
3.Insert Record
4.Fetch Record
5.Exit program

```

```

Enter your choice
1
Enter database name:
test_db
New Database Successfully created:test_db

```

```

****Menu****
1.Create Database
2.Create Table
3.Insert Record
4.Fetch Record
5.Exit program

```

```

Enter your choice
2
Enter database name:
test_db
Enter table name:
student_tbl
New Table successfully created :student_tbl

```

```

****Menu****
1.Create Database
2.Create Table
3.Insert Record
4.Fetch Record
5.Exit program

```

```

Enter your choice
3
Enter database name:
test_db
Enter table name:
student_tbl
Enter name:
Keshav Bist
Enter gender:
Male
Enter address:
Kanchanpur
One Record Inserted Successfully.

```

```

****Menu****
1.Create Database
2.Create Table
3.Insert Record
4.Fetch Record
5.Exit program

```

```

Enter your choice
4
Enter database name:
test_db
Enter table name:
student_tbl
ID Name Gender Address
1 Keshav Bist Male Kanchanpur

```

### Java Login Application:

This is simple Java swing application for login without validation. So validate login credential for more reliable. (IDE: netbeans)

**/\*LoginApplication.java\*/**

```
package loginapplication;
```

```

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;

public class LoginApplication extends JFrame{
 JTextField un_tf,psw_tft;
 JLabel un_label,psw_label;
 JButton lgn;
 LoginApplication(){
 JFrame.setDefaultLookAndFeelDecorated(true);

 un_label = new JLabel("Username:");
 un_label.setBounds(40,70,80,30);

 un_tf = new JTextField();
 un_tf.setBounds(140,70,180,30);

 psw_label = new JLabel("Password:");
 psw_label.setBounds(40,120,80,30);

 psw_tft = new JTextField();
 psw_tft.setBounds(140,120,180,30);

 lgn = new JButton("Login");
 lgn.setBounds(160,170,160,30);
 DataHandler handler = new DataHandler(this);
 lgn.addActionListener(handler);

 add(un_label);
 add(psw_label);
 add(un_tf);
 add(psw_tft);
 add(lgn);

 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 setTitle("Simple Login Application");
 setSize(400,360);
 setLayout(null);
 setVisible(true);
 }
 public static void main(String[] args) {
 new LoginApplication();
 }
}

```

**/\*DataHandler.java\*/**

```

package loginapplication;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.Connection;
import java.sql.DriverManager;

```

```

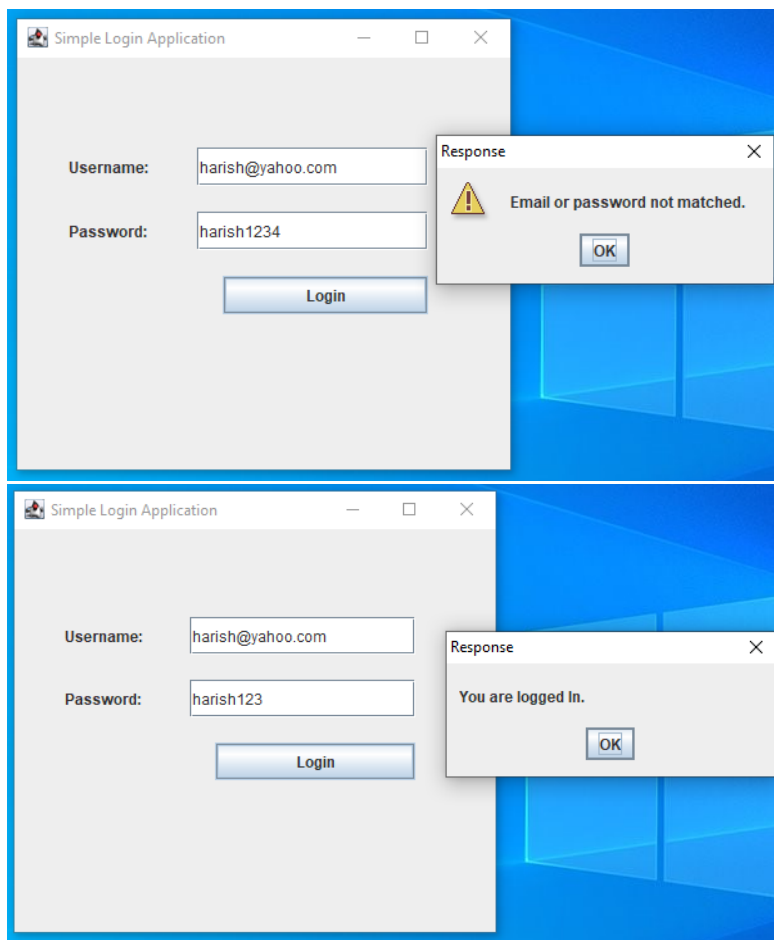
import java.sql.SQLException;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import javax.swing.JOptionPane;

public class DataHandler implements ActionListener{
 LoginApplication lapp;
 Connection connect;
 PreparedStatement pst;
 ResultSet result;
 DataHandler(LoginApplication lapp){
 this.lapp = lapp;
 }
 @Override
 public void actionPerformed(ActionEvent ae){
 String url = "jdbc:mysql://localhost:3306/it_4th";
 String user = "root";
 String password = "";

 String user_name = lapp.un_tf.getText();
 String pass = lapp.psw_tft.getText();
 try{
 //first register mysql driver
 Class.forName("com.mysql.cj.jdbc.Driver");
 //establish connection to database
 connect = DriverManager.getConnection(url,user,password);
 }catch(ClassNotFoundException | SQLException e){
 System.out.println("Unable to connect database."+e.getMessage());
 e.printStackTrace();
 }
 try{
 String sql = "SELECT email,password FROM it_4th_student WHERE email=? AND password=?";
 pst = connect.prepareStatement(sql);
 pst.setString(1, user_name);
 pst.setString(2,pass);
 result = pst.executeQuery();
 if(result.next()){
 JOptionPane.showMessageDialog(lapp, "You are logged In.", "Response",JOptionPane.PLAIN_MESSAGE);
 }else{
 JOptionPane.showMessageDialog(lapp, "Email or password not matched.",
 "Response",JOptionPane.WARNING_MESSAGE);
 }
 }catch(Exception e){
 e.printStackTrace();
 }
 }
}

```

**Output:**



## Scrollable and Updatable ResultSets

Whenever we execute **Statement** method *executeQuery()* that execute query and return **ResultSet** object i.e. **ResultSet** enable a program to manipulate query results only in forward direction (like 1<sup>st</sup> row, 2<sup>nd</sup> row, 3<sup>rd</sup> row, ....., last row). Such type of **ResultSet** is known as **non-updatable** and **non-scrollable ResultSet**. In default **ResultSet** specify **TYPE\_FORWARD\_ONLY** and **CONCUR\_READ\_ONLY** constant.

So, for Updatable and Scrollable **ResultSet** we must use **TYPE\_SCROLL\_SENSITIVE** / **TYPE\_SCROLL\_INSENSITIVE** and **CONCUR\_UPDATABLE** constants present in **ResultSet** Interface. Type Scroll constants allow to scroll in either direction or to move the cursor to a particular row.

**TYPE\_SCROLL\_SENSITIVE** - reflect changes made while it is still open.

**TYPE\_SCROLL\_INSENSITIVE** – doesn't reflect changes made while it is still open.

**CONCUR\_UPDATABLE** – allow to update rows

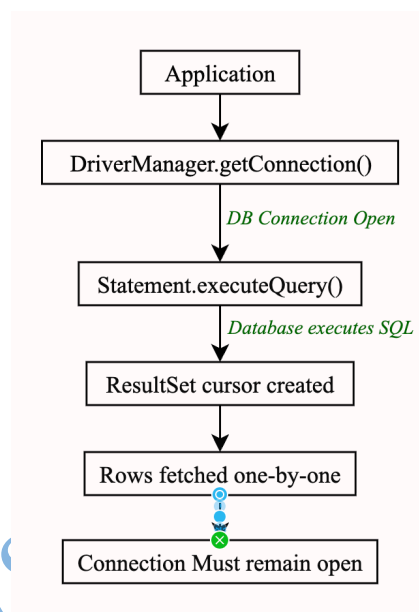
### Commonly used Methods of **ResultSet** interface:

1. **public boolean next()** – used to move the cursor to the one row next from the current position.
2. **public boolean previous()** – used to move the cursor to the one row previous from the current position.
3. **public boolean first()** – used to move the cursor to the first row of the **ResultSet** object.

4. public boolean last() – used to move the cursor to the last row of the ResultSet object.
5. public boolean absolute(int row) – used to move the cursor to the specified row number in the ResultSet object.
6. public boolean relative(int row) – used to move the cursor to relative row in the ResultSet object. It may be positive or negative.
7. public boolean moveToInsertRow() – makes the ResultSet object to hold the record.
8. public boolean insertRow() – to insert data in ResultSet object i.e. to database table.
9. public boolean deleteRow() – delete the record from database table permanently.
10. public boolean updateRow() – update content of ResultSet object.

## ResultSet vs RowSet

**ResultSet:** Already defined above. Whereas ResultSet needs database connection or live until records are operated(or displayed). You can't close connection before records processed.

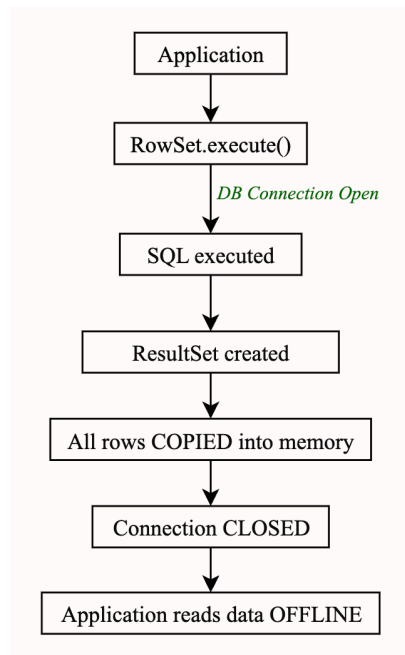


**Figure:** ResultSet flow

**RowSet:** A RowSet is an object that encapsulates a set of rows from either Java Database Connectivity (JDBC) result sets or tabular data sources. RowSet is an advanced version of ResultSet that provides more flexible, Java friendly, and often disconnected data handling. Support component-based development models like JavaBeans. Provides standard set of properties and an event notification mechanism.

Types of RowSet:

1. CachedRowSet(mostly used and we preferred too)
2. JdbcRowSet
3. WebRowSet
4. FilteredRowSet
5. JoinRowSet



### Example:

```

package jdbcexamples;
import javax.sql.rowset.CachedRowSet;
import javax.sql.rowset.RowSetProvider;
public class RecordsWithRowSets {
 public static void main(String[] args) {
 try{
 CachedRowSet crs = RowSetProvider.newFactory().createCachedRowSet();
 crs.setUrl("jdbc:mysql://localhost:3306/sw_it_db_2082");
 crs.setUsername("root");
 crs.setPassword("");

 crs.setCommand("Select * from sw_students_regiter");
 crs.execute();

 while(crs.next()){
 int id = crs.getInt("id");
 String name = crs.getString("name");
 String address = crs.getString("address");
 System.out.printf("\nID:%d\nName:%s\nAddress:%s\n",id,name,address);
 }
 }catch(Exception e){
 e.printStackTrace();
 }
 }
}

```

### Steps:

1. Create CachedRowSet
2. Set Connection Details
3. Set SQL command
4. Execute (Fetch and disconnect Database connection)



5. Process data (Connection already closed)

**Execution Flow:**

1. RowSet opens DB connection
2. Executes SQL query
3. Gets ResultSet from DB
4. Copies ALL rows into memory
5. Stores metadata + row values
6. Closes DB connection automatically

**Java Regular Expression: do yourself.....**

santoshbist4385@gmail.com

## Chapter 9: Servlet and Java Server Page(Prefer class note....)

santoshbist4385@gmail.com

## Chapter 10: Advanced Topics in Java

### Overview Of ORM

Object-Relational Mapping (ORM) is a programming technique in software development that provides / allows to interact with a relational database using object-oriented programming instead of writing raw SQL queries. That means ORM acts as a bridge between Object-Oriented Languages (Java, Python, JavaScript, etc) and Relational Databases (MySQL, PostgreSQL, Oracle).

Let's see, how ORM maps:

Tables → Classes  
Rows → Objects  
Columns → Attributes

### Types of ORM

1. Active Record Pattern – object directly interacts with database
2. Data Mapper Pattern – separate layer between object and Database

### Popular ORM tools

1. Java – Hibernate ORM
2. JavaScript / Node.js – Sequelize ORM, TypeORM, Prisma ORM
3. Python – Django ORM, SQLAlchemy

### Advantages of ORM

1. Reduces development time
2. Easy to switch database
3. Cleaner and maintainable code
4. Prevents sql injection
5. Faster CRUD operations

### Disadvantages of ORM

1. Slower than raw SQL for complex queries. Performance overhead too.
2. Need to understand ORM behavior
3. Joins, optimization become tricky.
4. Hard to debug sometimes (Not so flexible)

### Hibernate

Hibernate is a an open-source Object-Relational Mapping (ORM) framework for Java, designed to map Java classes to relational database tables. Eliminates the need for manual JDBC code, simplifying database operations and allowing developers to work with Java objects instead of SQL queries. Also handle result set automatically, provides an easier alternative to writing JDBC code by managing connection pooling and SQL generation.

### Key Hibernate architectural components and features:

- Hibernate ORM – maps Java objects to database tables, providing database independence through SQL dialects.
- HQL (Hibernate Query Language): An object-oriented querying language.
- Caching Mechanism: Includes first-level (session) and second-level caching for improved performance.
- Hibernate Data Repositories: A simplified, Jakarta Data-based programming model.
- Hibernate Search & Reactive: Specialized tools for advanced search and non-blocking database interaction

#### **Components of Hibernate:**

- SessionFactory → creates sessions(heavy, created once)
- Session → main interface to interact with Database
- Transaction → ensures data consistency
- Configuration → loads database settings (*hibernate.cfg.xml*)
- Entity class → java class mapped to table

#### **Advantages of Hibernate:**

- No need to write SQL – reduces boilerplate code
- Database Independence – works with any database(relational)
- Automatic Table Creation
- Caching Mechanism – improves performance (1<sup>st</sup> and 2<sup>nd</sup> level cache)
- Object-Oriented Approach – clean and maintainable code
- Security – prevents SQL injection

#### **Disadvantages of Hibernate:**

- Performance Overhead – slower than JDBC for complex queries
- Learning Curve – many configurations and concepts
- Hidden SQL – hard to debug generated queries
- Memory consumption – uses cache, higher memory usage
- Not Ideal for complex queries – joins and analytics queries are harder

### **Web Framework Introduction**

(do yourself.....)

#### **Basics of Spring Boot**

An open-source Java framework that simplifies the process of building stand-alone, production-grade applications. Builds on Spring Framework, that automates much of the manual configuration traditionally required. Helps to build Java applications quickly, reduce configuration (no complex XML) and run apps easily (embedded server).

#### **Core Features:**

- Auto-Configuration – automatically configures your application based on the JAR dependencies you add. Example, adding a database driver to your classpath allows Spring Boot to automatically set up a data source.
- Starter Dependencies – These are "curated" sets of dependency descriptors that you can include in your application. Common starters include *spring-boot-starter-web* for RESTful APIs and *spring-boot-starter-data-jpa* for database access.

- Embedded Servers – embeds external servers like Apache Tomcat, Jetty(optional).
- Actuator – provides production-ready features like health checks , metrics, and application auditing to help monitor your app in real-time.

#### Application Architecture:

- Typically follows a layered architecture to separate concerns:
- Controller layer – handles incoming HTTP requests and maps them to specific methods.
- Service layer – contains the core business logic and coordinates data between other layers.
- Repository Layer – interacts with the database using Spring Data JPA to perform CRUD (Create, Read, Update, Delete) operations.
- Model/Entity Layer – represents the data structure , often mapping directly to database tables.

### Concurrency and Multithreading in Java

Concurrently means happening or existing at the same time. When two tasks are operating concurrently, we mean that they're both making progress at once(Not necessarily at the same exact time).

Example: Prepare Dinner(One cook)

- No concurrency  
Cook rice -> then cook curry -> then make salad
- Concurrency  
Start rice -> while boiling -> chop salad -> stir curry -> back to rice. (All task progress together, known as concurrency)

**Concurrency** occurs in single processor computer by switching tasks execution rapidly in presence of Operating System; until early 2000s. Doing small portion of each before moving on to the next, so that all tasks keep processing. Now a day computer are available with multi-CPU cores, muti-GPU cores and super power and intelligence. Which means they perform multiple tasks simultaneously, called parallelism and is a subset of concurrency.

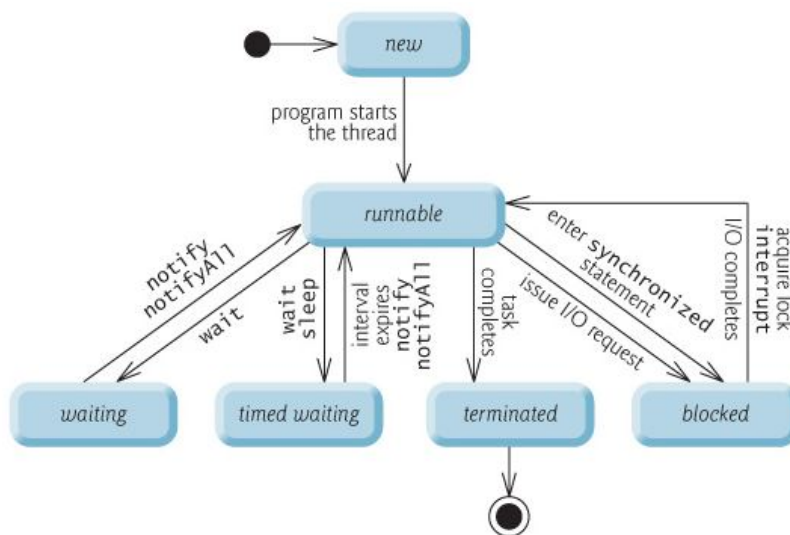
In Java language and APIs are responsible for concurrency. *Multithreading* through java helps to execute concurrently. **Multithreading** is the capability to execute multiple *threads*, each with its own method-call stack and program counter and share application resources like memory and file handles.

(Running multiple threads inside one program at the same time – called multithreading. Thread is a lightweight unit of execution inside a process.)

Example: Prepare Dinner(Three cooks)

- Cook1 -> cooks -> rice
- Cook2 -> cooks ->curry
- Cook3 -> prepare -> salad

#### Thread States and Life cycle:



## Design Patterns

Design Patterns are not code, but a template or blueprint for solving problems. They represent industry standard best practices for object-oriented programming to ensure code is reusable, flexible and maintainable. Using patterns provides a shared, battle-tested vocabulary that improves code reliability and system flexibility.

Benefits, using Design Patterns:

- Reusable solutions
- Improves code structures
- Makes code scalable & maintainable
- Follows best practices

Types of Design Patterns:

Mainly divided into 3 categories,

1. Creational – Deal with object creation. Examples, Singleton Pattern, Factory Method, Abstract Factory, Builder and Prototype.
2. Structural – Deal with class/object structure. Examples, Adapter Pattern, Decorator Pattern, Proxy Pattern, Facade Pattern, Composite, Bridge, Flyweight.
3. Behavioral – Deal with communication between objects. Examples, Observer Pattern, Strategy pattern, Command Pattern, State Pattern , etc.

## Singleton Pattern

Singleton Pattern says that just "define a class that has only one instance and provides a global point of access to it". And that single object can be used by all other classes.

**Example:** Instantiate single object to display student records.

/\* Filename: SingletonPatternExample.java\*/

```

package designpatternexample;
class StudentRecord{
 private static StudentRecord student_instance;
 private StudentRecord(){}
 public static StudentRecord getInstance(){
 if(student_instance == null){
 student_instance = new StudentRecord();
 }
 return student_instance;
 }
 public void displayRecord(String name){
 System.out.println("Name:"+name);
 }
}
public class SingletonPatternExample {
 public static void main(String[] args) {
 StudentRecord sr = StudentRecord.getInstance();
 sr.displayRecord("Ram");
 sr.displayRecord("Hari");
 }
}

```

### Factory Method

Creates objects without exposing creation logic. Allows the sub-classes to choose the type of objects to create.

Example:

```

/*Filename: FactoryMethodPatternExample.java*/
package designpatternexample; // Define the package name
// Interface Vehicle defining a common method for all vehicles
interface Vehicle {
 void startWith(); // Abstract method to describe how the vehicle starts
}
// Concrete class Bike implementing Vehicle interface
class Bike implements Vehicle {
 @Override
 public void startWith() {
 // Implementation of startWith for Bike
 System.out.println("Start with key or kick.");
 }
}
// Concrete class Car implementing Vehicle interface
class Car implements Vehicle {
 @Override
 public void startWith() {
 // Implementation of startWith for Car
 System.out.println("Start with key.");
 }
}
// Factory class responsible for creating Vehicle objects
class HandleFactory {
 // Factory method: returns an instance of a Vehicle based on type

```

```

public Vehicle getInstance(String type) {
 if (type == null) return null; // If type is null, return nothing
 // If type is "bike", create and return Bike object
 if (type.equalsIgnoreCase("bike")) {
 return new Bike();
 }
 // If type is "car", create and return Car object
 else if (type.equalsIgnoreCase("car")) {
 return new Car();
 }
 // If type is neither "bike" nor "car", return null
 return null;
}
}
// Main class to demonstrate Factory Method pattern
public class FactoryMethodPatternExample {
 public static void main(String[] args) {
 // Create a factory object that will create vehicles
 HandleFactory factory = new HandleFactory();
 // Request a Bike object from the factory
 Vehicle v1 = factory.getInstance("bike");
 v1.startWith(); // Call method to show how Bike starts
 // Request a Car object from the factory
 Vehicle v2 = factory.getInstance("car");
 v2.startWith(); // Call method to show how Car starts
 }
}

```

Why return type is interface (Vehicle) in **HandleFactory** class?

Because of following reasons:

- Means the method can return any object that implements Vehicle
- The client doesn't need to know the exact concrete class (bike or Cat)
- Benefits; loose coupling, polymorphism and extensibility .

**Abstract Factory Pattern (do yourself...)**



## How to setup environment?

### Java Environment Setup

1. First, verify that it is already installed or not. Download if not installed.
2. Download Latest stable JDK version  
(<https://www.oracle.com/java/technologies/downloads/#java21>)
3. Install JDK
4. Setup Path (required in window)
5. Install IDE to perform programming (we used NetBeans as IDE, can use anyone)  
(<https://netbeans.apache.org/front/main/download/nb29/>)

### How to create project in NetBeans?

1. Open/Run NetBeans
2. Go To file -> New Project OR Press ctrl+shift+N (window) OR command +shift + N (macos)
3. Select Java with Ant -> Java Application
4. Now enter project name, select location and folder.
5. Click Finish button.

### JavaFx Environment Setup(Window)

1. Download JavaFx (<https://www.oracle.com/java/technologies/downloads/javafx/#javafx25-windows> or mac or linux)
2. Unzip JavaFx
3. Create normal project (Go to file -> New Project -> Java with Ant -> Java Application)
4. Add JavaFx SDK
  - i. Right click project
  - ii. Select properties -> Libraries->Compile
  - iii. Now add JavaFx Jar's (click: Add JAR/Folder)
  - iv. Navigate to JavaFX lib folder
  - v. Select ALL files
  - vi. Confirm(click: Apply)
5. Set Run VM options(very important) – (--module-path=/Users/.... /Downloads/javafx-sdk-25.0.2/lib --add-modules=javafx.controls,javafx.fxml)
6. Save/Apply

Note: If you are unable to setup JavaFx environment as above, please follow other resources that fit to you device.